

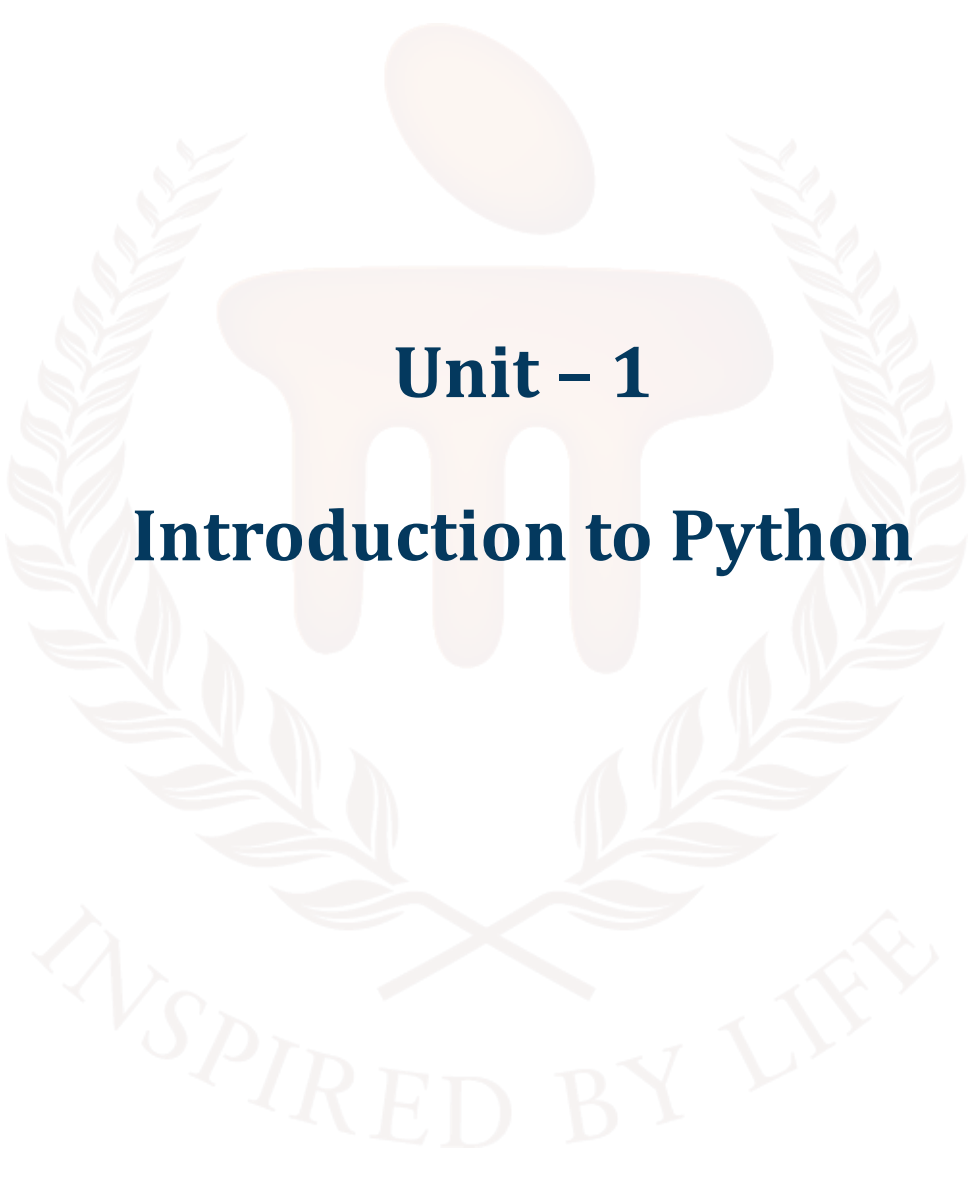
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 1

Introduction to Python

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	4
1.1	Learning Objectives	-	-	
2	Getting Started With Python	-	-	
2.1	Overview of Python (History, Features, Advantages)	-	-	
2.2	Installing Python (Windows, macOS, Linux)	-	-	
2.3	Setting Up the Python Environment (IDLE, VS Code, Jupyter Notebook, PyCharm)	-	-	
2.4	Running Python Programs	-	1	
3	Understanding Python Basics	-	-	
3.1	Python Syntax and Indentation	-	-	
3.2	Python Comments and Documentation Strings	-	-	
3.3	Variables and Data Types in Python	-	-	
3.4	Basic Input and Output Functions	-	2	
4	Applications Of Python In The Real World	-	-	
4.1	Why Python is Popular in Industry	-	-	
4.2	Applications in Data Science, AI, Web Development, and Automation	-	-	
4.3	Python in Education and Research	-	-	
4.4	Future Scope of Python Programming	-	3	
5	Summary	-	-	33
6	Glossary	-	-	34
7	Terminal Questions	-	-	35
8	Answers	-	-	
8.1	Self-Assessment Answers	-	-	
8.2	Terminal Questions Answers	-	-	
9	References	-	-	38

1. INTRODUCTION

In this unit, learners will explore the fundamentals of Python programming, starting from installation to real-world applications. They will first understand Python's origin, key features, and advantages, followed by detailed guidance on how to install Python on various operating systems and set up environments like IDLE, VS Code, Jupyter Notebook, and PyCharm. Learners will also learn how to run Python programs using both interactive and script modes.

The unit then focuses on Python basics, such as its clean syntax, use of indentation, and the role of comments and docstrings. Students will become familiar with variables, different data types, and how to take input and display output using Python.

Finally, the unit highlights Python's diverse real-world applications across domains such as data science, AI, web development, and automation. Learners will also understand Python's rising relevance in education and research, along with its promising future scope in the evolving tech industry.

1.1. Learning Objectives

- Explain the significance of Python as a beginner-friendly yet powerful programming language.
- Describe the history, design philosophy, and features that make Python widely adopted across industries.
- Identify the different application areas of Python such as web development, data science, AI, automation, and scripting.
- Install Python on various operating systems and configure environment settings like system path.
- Recognise the role of Integrated Development Environments (IDEs) and editors such as IDLE, VS Code, and PyCharm in Python programming.



2. GETTING STARTED WITH PYTHON

Python has grown to be one of the most influential programming languages in the world. Whether you are stepping into programming for the first time or already have experience with other languages, Python provides a smooth entry into the world of coding. It is valued for its simplicity, yet it is powerful enough to build large-scale systems, scientific applications, and advanced artificial intelligence solutions. In this section, we will explore the basics of Python, beginning with its background and features, moving through installation and environment setup, and finally learning how to run Python programs. By the end, you will have a solid foundation to begin writing and executing your own Python code.

2.1 Overview of Python (History, Features, Advantages)

History of Python

Python was created by Guido van Rossum in the late 1980s and was officially released in 1991. The inspiration behind Python was to design a language that prioritized readability and reduced the complexity programmers often face. Unlike many other languages that focus heavily on performance, Python was designed to make programming more accessible and enjoyable. Guido named the language “Python” after the comedy group Monty Python, reflecting the light-hearted nature he envisioned for programming.

Over the years, Python has gone through multiple versions. Python 2, released in 2000, became extremely popular but had limitations in design. In 2008, Python 3 was introduced to address those shortcomings and to modernize the language. Today, Python 3 is the standard version, while Python 2 has been officially retired. The continuous updates and contributions from a large developer community have ensured Python’s growth as one of the most widely used programming languages in the world.

Features of Python

Several unique features distinguish Python from other programming languages:

- **Readable and simple syntax:** Python code is often described as being close to natural English, making it easier for beginners to understand.
- **Interpreted language:** Python does not require compilation. The interpreter reads and executes the code line by line.

- **Dynamic typing:** Variables do not require explicit data type declarations; Python assigns types automatically at runtime.
- **Extensive standard library:** Python comes with a large library of built-in modules for tasks such as file handling, math, networking, and web development.
- **Cross-platform support:** Python programs can run on Windows, macOS, and Linux with minimal changes.
- **Object-oriented and functional programming:** Python supports both paradigms, giving flexibility in coding style.
- **Large ecosystem of third-party libraries:** From data analysis (Pandas, NumPy) to machine learning (TensorFlow, PyTorch), Python has libraries for nearly every domain.

Advantages of Python

Python's widespread popularity is not accidental. Its advantages are evident in both education and industry:

- **Beginner-friendly:** The language lowers the barrier to entry for new learners.
- **Versatile applications:** From automation scripts to web servers and artificial intelligence systems, Python adapts to a wide range of tasks.
- **Strong community:** Millions of developers contribute tutorials, libraries, and solutions, making help easy to find.
- **Rapid development:** Its simplicity reduces the time required to develop and test applications.
- **Integration capabilities:** Python can integrate with other languages like C, C++, and Java, as well as databases and cloud platforms.

Together, these features and advantages make Python one of the most practical and future-oriented programming languages.

2.2 Installing Python (Windows, macOS, Linux)

Before writing Python programs, the language must be installed on your system. Fortunately, Python is free and open-source, available on the official Python website (<https://www.python.org/>). Let us go through the installation process on different operating systems.

Installing Python on Windows

1. Visit the official Python website and download the latest version for Windows.

2. Run the installer file. In the first window, make sure to select “Add Python to PATH” before clicking install. This allows Python commands to be recognized in the command prompt.
3. Choose either “Install Now” (recommended for beginners) or “Customize Installation” if you want to adjust settings.
4. Once the installation completes, open the Command Prompt and type `python --version` to confirm that Python has been installed successfully.

Installing Python on macOS

Many macOS systems come with Python pre-installed, though often it is an older version. To install the latest version:

1. Download the installer from the official website.
2. Open the .pkg file and follow the installation instructions.
3. After installation, open the Terminal and type `python3 --version` to check the installation. macOS uses the command `python3` instead of `python`.

Alternatively, users can install Python on macOS using Homebrew, a package manager, with the command:

```
brew install python
```

Installing Python on Linux

Most Linux distributions also come with Python pre-installed. To check, type `python3 --version` in the terminal. If not installed or if an update is needed, you can use the package manager for your distribution. For example, on Ubuntu:

```
sudo apt-get update
```

```
sudo apt-get install python3
```

After installation, confirm with `python3 --version`.

No matter the operating system, installation is straightforward and takes only a few minutes.

2.3 Setting Up the Python Environment (IDLE, VS Code, Jupyter Notebook, PyCharm)

After installation, the next step is setting up an environment where you can write and test Python code. Different environments cater to different needs.

IDLE

IDLE (Integrated Development and Learning Environment) is included with Python installations. It provides a simple editor and interactive Shell for running Python code. IDLE is lightweight and suitable for beginners who are just starting out.

Visual Studio Code (VS Code)

VS Code is a free, open-source code editor from Microsoft. With the Python extension, VS Code provides syntax highlighting, debugging tools, and integration with version control systems like Git. It is lightweight but powerful, making it popular among developers.

Jupyter Notebook

Jupyter Notebook is widely used in data science and research. It allows users to write code in blocks (cells) and see the output immediately below each cell. Jupyter also supports adding text, equations, and images, making it ideal for creating interactive reports, tutorials, and experiments. It is installed via the `pip install notebook` command or as part of the Anaconda distribution.

PyCharm

PyCharm, developed by JetBrains, is a professional integrated development environment designed specifically for Python. It includes advanced features such as project management, intelligent code completion, debugging, testing, and integration with web frameworks and databases. The community edition is free, while the professional edition offers more advanced features.

Choosing the right environment depends on your needs. Beginners may start with IDLE, while advanced users working on larger projects often prefer VS Code or PyCharm. For research and data analysis, Jupyter Notebook is the go-to tool.

2.4 Running Python Programs

After installing Python and setting up an environment, the next step is learning how to run Python programs. There are two common modes of execution: interactive mode (Shell) and script mode.

Using Python Shell (Interactive Mode)

The Python Shell provides a quick way to run commands line by line. To open it, simply type `python` or `python3` in the terminal or command prompt. You will see the `>>>` prompt, which indicates that Python is ready to accept commands.

Example:

```
>>> print("Hello, World!")
```

Hello, World!

```
>>> 2 + 3
```

5

```
>>> "Python".upper()
```

'PYTHON'

In interactive mode, you get immediate feedback, making it excellent for testing small pieces of code, trying out functions, or learning syntax.

Writing and Executing Python Scripts (Script Mode)

For larger programs, it is better to write code in a file, also called a script. A script is saved with the .py extension.

Steps to write and execute a Python script:

1. Open a text editor or IDE (such as VS Code or PyCharm).
2. Write the code, for example:
3. `# my_first_program.py`
4. `print("Welcome to Python programming!")`
5. Save the file with a .py extension.
6. Open the terminal or command prompt, navigate to the file location, and type:
7. `python my_first_program.py`
8. The program will run, and the output will appear in the terminal.

Script mode is preferred for projects, automation, and applications since the code can be reused and modified easily. Unlike the Shell, scripts remain stored permanently and can be shared with others.

SELF-ASSESSMENT QUESTIONS – 1

Multiple Choose Questions

- | | |
|---|---|
| 1 | Who created Python?
a) Dennis Ritchie
b) Guido van Rossum
c) James Gosling
d) Bjarne Stroustrup |
|---|---|

2	Which of the following is used to open the Python interactive mode? a) python run b) python3 or python c) run py d) execute python
3	What is the file extension for Python scripts? a) .exe b) .txt c) .py d) .script
4	Which tool is most commonly used in data science for interactive coding and visualization? a) IDLE b) PyCharm c) Jupyter Notebook d) Eclipse
5	Python is best described as: a) Compiled and strongly typed b) Interpreted and dynamically typed c) Low-level and machine dependent d) None of the above

3. UNDERSTANDING PYTHON BASICS

Learning a programming language always begins with the foundations, and in Python these foundations are both simple and practical. Python was designed with readability and clarity in mind, which means that beginners can focus on the actual logic of programming instead of struggling with confusing syntax rules. Before building advanced applications or exploring specialized libraries, it is important to understand the basics of how Python code is structured, how comments and documentation are written, how variables and data types are handled, and how basic input and output functions operate. Mastering these essentials ensures that every learner builds a strong platform to progress smoothly in the journey of programming.

3.1 Python Syntax and Indentation

When learning any programming language, one of the first things to understand is its syntax. Syntax refers to the rules and structure that dictate how code should be written so that the computer can interpret it correctly. Just as human languages have grammar rules that must be followed to form meaningful sentences, programming languages have syntax rules that must be followed to form valid instructions. If the syntax is incorrect, the interpreter or compiler will generate an error, preventing the program from running.

Python, unlike many other languages, is known for having clean and minimal syntax. It was deliberately designed to prioritize readability and reduce unnecessary complexity. For example, languages like Java or C use curly braces `{}` to define blocks of code and semicolons `;` to end statements. Python removes such clutter. Instead, it uses indentation (spaces or tabs at the beginning of lines) to indicate the structure of the code. This makes Python code look neat, organized, and easy to understand, even for someone new to programming.

Importance of Indentation

In many programming languages, indentation is used only to improve readability, and the compiler ignores it. For instance, in C++ or Java, you could technically write all of your code in a single line, and it would still run as long as the braces `{}` are in the right places. This is not the case in Python. In Python, indentation is mandatory. Without proper indentation, the interpreter will throw an error, and the program will not execute.

Consider the following example:

```
if 5 > 2:
```

```
    print("Five is greater than two")
```

Here, the `print()` statement is indented with four spaces. This tells the Python interpreter that the `print` statement belongs to the `if` block. Without this indentation, Python will not know which statements are supposed to be executed if the condition is true.

Logic:

- The condition `5 > 2` is checked by the interpreter. Since it evaluates to `True`, Python executes the next indented line.
- The indentation under the `if` tells Python that `print("Five is greater than two")` belongs to the block controlled by the `if`.
- If the condition were false, Python would skip the indented block.
- Key point: Indentation is what signals that the `print()` belongs to the `if` block.

If the same code is written without indentation:

```
if 5 > 2:
```

```
print("Five is greater than two") # Error
```

The interpreter will immediately raise an `IndentationError`. This shows how critical indentation is in Python. The language forces developers to write code that is visually structured, which improves readability and reduces errors caused by poorly formatted code.

Consistency in Indentation

Python allows developers to use either spaces or tabs for indentation, but mixing them within the same block is not allowed. The Python community strongly recommends using four spaces per indentation level. Most code editors and IDEs automatically handle this, but programmers must remain consistent throughout their code.

For example:

```
for i in range(5):
```

```
    print(i)
```

```
    if i % 2 == 0:
```

```
print("Even number")
```

This code is properly indented and will run successfully. If you mistakenly mix spaces and tabs in the same block, it may still look aligned visually, but the interpreter will raise an error. Thus, Python not only enforces indentation but also enforces consistency.

Logic

- The loop for `i in range(5)`: generates the sequence [0, 1, 2, 3, 4] and runs once for each value.
- On each iteration, the value is assigned to the variable `i`.
- The first indented line `print(i)` displays the current value of `i`.
- The next statement `if i % 2 == 0`: checks if `i` is divisible by 2.
- The `%` operator gives the remainder. If the remainder is 0, the number is even.
- When the condition is true, the further indented statement `print("Even number")` executes.
- If the condition is false, Python skips that print statement and continues with the loop.
- The indentation levels show hierarchy: `print(i)` belongs to the for loop, and `print("Even number")` belongs specifically to the if block.

Output confirms the logic:

0

Even number

1

2

Even number

3

4

Line Breaks and Statements

Python is designed to be simple, and as a rule, it expects one statement per line. This makes the code easy to read and debug. However, there are situations where a programmer may want to place multiple statements on a single line. Python allows this by separating statements with a semicolon `;`.

For example:

```
x = 10; y = 20; print(x + y)
```

This works, but it is not considered good practice. The Python philosophy values readability, and writing each statement on a separate line is always preferred.

Sometimes, a statement may be too long to fit on one line. In such cases, Python allows line continuation using a backslash (\).

Example:

```
total = 100 + 200 + 300 + \  
    400 + 500 + 600  
print(total)
```

Here, the backslash tells Python that the statement continues on the next line. Without it, the interpreter would treat the second line as a new statement and throw an error. This feature is useful when writing long mathematical expressions or complex function calls.

Logic:

- Normally, Python expects a statement to end at the end of a line.
- The backslash (\) at the end of the first line tells Python: "This statement continues on the next line."
- The interpreter joins both lines into a single statement: `total = 100 + 200 + 300 + 400 + 500 + 600`.
- Finally, the result of this arithmetic operation is stored in the variable `total`.

Key point: Backslashes let you break long statements across multiple lines for readability.

Case Sensitivity

Python is a case-sensitive language. This means that variable names, function names, and identifiers written with different letter cases are considered separate and distinct.

For example:

```
Name = "Alice"
```

```
name = "Bob"
```

```
NAME = "Charlie"
```

```
print(Name) # Output: Alice
```

```
print(name) # Output: Bob
```

```
print(NAME) # Output: Charlie
```

In this example, Name, name, and NAME are treated as three different variables. This behavior requires programmers to be consistent in how they write identifiers. Using inconsistent case can lead to bugs that are sometimes hard to spot. A common best practice is to follow naming conventions such as snake_case for variables (student_name) and CamelCase for class names (StudentName).

Logic:

- Python treats identifiers (Name, name, NAME) differently based on capitalization.
- Name, name, and NAME are stored in memory as three separate variables.
- The print() function retrieves the exact variable requested.

Key point: Variable names must be written exactly as declared, otherwise you'll get errors like NameError.

Keywords and Identifiers

Python has a list of reserved words, called keywords, that cannot be used as identifiers. Keywords are special words that have predefined meanings in the language. Examples include if, else, while, for, class, def, True, False, None, and many more. Using these as variable names will result in errors.

To see all keywords in your version of Python, you can run:

```
import keyword  
  
print(keyword.kwlist)
```

Identifiers are the names given to variables, functions, classes, or other objects created by the programmer. Python identifiers must follow these rules:

1. They must start with a letter or underscore (_).
2. They cannot start with a digit.
3. They can contain letters, digits, and underscores.
4. They cannot be the same as a keyword.

Examples of valid identifiers:

- student

- `_value`
- `total_marks`
- `number2`

Examples of invalid identifiers:

- `2number` (starts with a digit)
- `class` (keyword)
- `total marks` (space not allowed)

Proper naming of identifiers improves code readability and avoids conflicts with system-defined names.

Python's Philosophy of Readability

All of these syntax rules point back to Python's core philosophy: code should be easy to read, understand, and maintain. The forced indentation ensures that developers write structured programs, while the avoidance of extra symbols reduces clutter. The case sensitivity enforces discipline in naming, and the reserved keywords prevent confusion by giving special meaning to certain terms.

This philosophy is summarized in the famous "Zen of Python," which can be displayed in the interpreter by typing:

```
import this
```

It contains guiding principles such as:

- "Beautiful is better than ugly."
- "Simple is better than complex."
- "Readability counts."

Practical Examples of Syntax and Indentation

Let us look at a few more examples to see how Python's syntax works in practice.

Example 1: Using indentation in loops

```
for i in range(3):
```

```
    print("Outer loop:", i)
```

```
    for j in range(2):
```

```
print(" Inner loop:", j)
```

Here, the indentation clearly shows which print statements belong to the outer loop and which belong to the inner loop.

Logic:

- The outer loop runs 3 times, with i taking values 0, 1, and 2.
- For each value of i, the first print() runs once.
- Inside the outer loop, there is another loop (for j in range(2):) which runs 2 times, with j taking values 0 and 1.
- The second print() is indented further, meaning it belongs to the inner loop.
- So the inner loop executes for each cycle of the outer loop.

Key point: Indentation makes it clear which code belongs to which loop.

Example 2: Conditional blocks

```
x = 15
```

```
if x > 10:
```

```
    print("x is greater than 10")
```

```
    if x > 20:
```

```
        print("x is also greater than 20")
```

```
    else:
```

```
        print("x is not greater than 20")
```

The structure is easy to read because indentation visually separates the blocks.

Logic:

- First, the interpreter checks if $x > 10$:. Since $x = 15$, this is True, so the indented block executes.
- Inside that block, there is another if checking whether $x > 20$.
- Since 15 is not greater than 20, this condition is False, so Python executes the else: block instead.
- Therefore, the output will be:
- x is greater than 10

- x is not greater than 20

Key point: Indentation allows multiple levels of logic (nested decisions) to be clearly represented.

Example 3: Functions

```
def greet(name):
```

```
    print(f"Hello, {name}!")
```

```
    print("Welcome to Python basics.")
```

```
greet("Alice")
```

Indentation here shows that both `print()` statements belong to the function `greet()`.

Logic:

- The keyword `def` defines a function named `greet` that takes one parameter (`name`).
- The two `print()` statements are indented, meaning they belong to the function. They will only run when the function is called.
- The call `greet("Alice")` passes "Alice" into the function, replacing the placeholder `name`.
- So the output is:
- Hello, Alice!
- Welcome to Python basics.

Key point: Indentation defines the body of the function. Without it, Python wouldn't know which statements belong inside the function.

3.2 Python Comments and Documentation Strings

Comments and documentation are essential parts of programming because they make code understandable not just for the computer but also for humans. Python provides clear ways to include comments and docstrings.

Single-Line Comments

A single-line comment begins with the hash symbol `#`. Anything written after `#` is ignored by the Python interpreter.

Example:

This is a single-line comment

```
print("Hello World") # This prints a message
```

Logic:

1. The # symbol tells Python to ignore everything written after it on the same line.
2. The first line (# This is a single-line comment) is meant only for humans reading the code—it does not affect the program's output.
3. The second line prints "Hello World".
4. The comment written after the code (# This prints a message) is again ignored by the interpreter but explains what the code does.

Output:

Hello World

Multi-Line Comments

Python does not have a specific syntax for multi-line comments, but multiple # symbols can be used on consecutive lines.

```
# This is line one of a comment
```

```
# This is line two
```

```
# This is line three
```

Another common practice is to use triple quotes ''' ... ''' or """ ... """ for block comments, although these are technically treated as strings.

Logic:

1. Each line starts with a #.
2. Python skips all of them when executing the program.
3. Programmers use this style to write explanations that take up multiple lines.
4. Although it doesn't affect output, it improves readability and makes the purpose of the code clear.

Documentation Strings (Docstrings)

Docstrings are used to describe the purpose of a function, class, or module. They are enclosed in triple quotes and placed immediately after the definition line. Unlike regular comments, docstrings are accessible at runtime through the `__doc__` attribute.

Example:

```
def add(a, b):  
    """This function adds two numbers and returns the result."""  
    return a + b  
print(add.__doc__)
```

Output:

This function adds two numbers and returns the result.

Docstrings are vital for writing self-explanatory code, especially in larger projects or shared modules. They make collaboration easier and improve maintainability.

Logic:

1. The function `add` is defined with two parameters `a` and `b`.
2. Inside the function, the triple-quoted string is a docstring that describes what the function does.
3. The function returns the sum of `a` and `b`.
4. Unlike comments, the docstring is stored as metadata and can be accessed at runtime using `add.__doc__`.
5. The `print(add.__doc__)` displays the description written in the docstring.

3.3 Variables and Data Types in Python

At the heart of programming lies the concept of variables. A variable is essentially a container that stores data values. In Python, variables are extremely flexible compared to many other languages.

Defining Variables

Variables in Python are created the moment you assign a value to them. No explicit declaration is needed.

```
x = 10
```

```
y = "Hello"
```

```
z = 3.14
```

Here, x is an integer, y is a string, and z is a floating-point number. Python automatically determines the data type of a variable based on the assigned value.

Logic:

1. x = 10 assigns the integer value 10 to the variable x.
2. y = "Hello" assigns a string value to the variable y.
3. z = 3.14 assigns a floating-point number to z.
4. Python automatically detects the type of each variable based on the value—no need for manual type declaration.

Dynamic Typing

Python uses dynamic typing, which means you do not have to declare the data type of a variable in advance. A variable can even change its type during execution:

```
a = 5    # Integer
```

```
a = "World" # Now a string
```

Logic:

1. Initially, the variable a stores the integer 5.
2. Later, the same variable a is reassigned to the string "World".
3. Python allows this because it uses dynamic typing, meaning a variable's type can change at runtime.
4. At first, type(a) would return <class 'int'>, but after reassignment, it would return <class 'str'>.

Variable Naming Rules

- Must start with a letter or underscore.
- Cannot start with a digit.
- Can contain letters, digits, and underscores.

- Cannot use reserved keywords.

Data Types in Python

Python provides several built-in data types:

- Numeric types: int (integers), float (floating-point numbers), complex (complex numbers).
- Text type: str (string).
- Sequence types: list, tuple, range.
- Mapping type: dict (dictionary).
- Set types: set, frozenset.
- Boolean type: bool (True or False).
- Binary types: bytes, bytearray, memoryview.

Examples:

```
a = 100      # int
b = 3.14     # float
c = "Python" # str
d = [1, 2, 3] # list
e = (4, 5, 6) # tuple
f = {"name": "Alex"} # dict
g = True     # bool
```

Python's flexibility with data types makes it possible to write concise programs without worrying about strict type declarations.

Logic:

1. a is assigned an integer value.
2. b stores a decimal number, making it a float.
3. c stores a text string.
4. d is a list (a collection that can hold multiple values).
5. e is a tuple, which is similar to a list but immutable.
6. f is a dictionary that maps keys to values ("name" → "Alex").
7. g is a boolean value representing True (logical truth).

This demonstrates Python's built-in support for multiple data types.

3.4 Basic Input and Output Functions

Programs become more useful when they interact with users. Python provides built-in functions for both input and output operations.

Output Using print()

The print() function displays output on the screen. It can handle strings, numbers, and even multiple items at once.

```
print("Welcome to Python")
```

```
print("The sum of 2 and 3 is:", 2 + 3)
```

The function automatically inserts spaces between items and ends output with a newline unless specified otherwise.

Logic:

1. The first line simply prints the string "Welcome to Python".
2. The second line demonstrates that print() can output multiple items separated by commas.
3. $2 + 3$ is an arithmetic expression, which is evaluated to 5.
4. The output shows both the string and the computed result.

Output:

Welcome to Python

The sum of 2 and 3 is: 5

Formatting Output

Python supports string formatting for creating more readable output.

Example using f-strings (Python 3.6+):

```
name = "Alice"
```

```
age = 25
```

```
print(f"My name is {name} and I am {age} years old.")
```

Output:

My name is Alice and I am 25 years old.

Logic:

1. Variables name and age store "Alice" and 25.
2. The f before the string tells Python to use f-string formatting.
3. Inside the curly braces {}, variable values are inserted directly into the string.
4. This creates a dynamic, formatted string without needing manual concatenation.

Output:

My name is Alice and I am 25 years old.

Input Using input()

The input() function allows the program to take input from the user. It always returns data as a string, so conversion is needed for numbers.

Example:

```
name = input("Enter your name: ")
age = int(input("Enter your age: "))
print(f"Hello {name}, you are {age} years old.")
```

If the user enters "Sam" and "30," the program outputs:

Hello Sam, you are 30 years old.

Logic:

1. The first line asks the user to enter their name. Whatever the user types is stored as a string in name.
2. The second line asks for the age. Since input() always returns a string, the int() function converts it into an integer.
3. The final print() uses f-string formatting to display a personalized greeting.
4. If the user types Sam and 30, the program builds the sentence accordingly.

Output:

Hello Sam, you are 30 years old.

Importance of Input and Output

Programming is not just about writing instructions for a computer to execute; it is about building systems that interact with the world around them. This interaction happens primarily through input

and output operations. Input allows a program to receive information from users, files, sensors, or other systems, while output enables the program to display, transmit, or store results. Without input and output, a program would remain isolated, processing only hardcoded values and producing results that no one can access. The ability to communicate with users and external systems is what makes software practical and valuable in real-world scenarios.

Input as the Gateway to Interaction

Input is the process of providing data to a program so it can perform tasks based on variable information. In Python and many other languages, the simplest form of input comes from the user typing values during program execution. For example, asking a user to type their name or age creates a personalized interaction. Unlike static programs that work with predefined data, programs that accept input can behave dynamically, meaning they respond differently depending on the data provided.

Consider an example of an online calculator. Instead of only working with preset numbers, the calculator allows the user to type any two values, and it produces the result accordingly. This flexibility is made possible by input functions. Similarly, in banking software, input allows a customer to enter their account number and PIN. Without this, the program could not identify individual customers or respond to specific requests.

Input is not limited to keyboard data. It can come from files, databases, web forms, or even real-time sensors in systems such as smart devices. For instance, weather monitoring systems take input from sensors measuring temperature, humidity, and air pressure, and then process that data to forecast weather conditions. This highlights how essential input is in transforming raw data into meaningful applications.

Output as the Bridge to Communication

Output is equally important because it allows a program to communicate results back to users or other systems. In its simplest form, output may appear on a screen as text. For example, printing a greeting message after the user enters their name makes the interaction meaningful. Without output, the program might still process the input but the user would never know the result.

Output can take many forms: text displayed on a console, graphical results shown in charts, data saved in files, or even signals sent to external devices. In modern applications, output often goes beyond simple text. E-commerce websites, for instance, output personalized product recommendations after

analyzing customer input data. Automated banking systems display balance information, transaction histories, and confirmation messages to guide customers in their activities.

The combination of input and output is what enables personalization. When a user provides data, and the program outputs tailored responses, the experience becomes more engaging and useful. This is why input and output are often referred to as the foundation of human-computer interaction.

Dynamic Behavior Through Input and Output

The real power of input and output lies in enabling dynamic program behavior. A static program, where the developer hardcodes values, is limited in scope. But when input functions are used, the program becomes adaptable. It can handle a variety of scenarios based on the data received. Output complements this by showing results that reflect the current context rather than fixed values.

Take an online exam system as an example. Input functions allow students to enter answers, while output functions provide feedback, display scores, and sometimes even suggest topics for improvement. Similarly, in online food delivery applications, input includes customer choices such as selected items and delivery address, while output provides order confirmations, estimated delivery times, and receipts. These applications would be impossible without well-structured input and output operations.

Automation and Practical Applications

Another critical importance of input and output is their role in automation. Programs designed to handle repetitive tasks rely on input data and output results without requiring manual effort each time. For example, payroll systems take employee details, working hours, and pay rates as input, then automatically calculate salaries and generate pay slips as output.

In research, scientific experiments often involve large volumes of input data collected from instruments, which the program processes and then outputs as graphs, statistical models, or predictions. In this way, input and output not only improve efficiency but also make systems scalable to handle large tasks.

SELF-ASSESSMENT QUESTIONS – 2

Fill in the blanks:	
6	In Python, _____ is used to indicate code blocks instead of curly braces
7	The symbol used for writing single-line comments in Python is _____.
8	Variables in Python are created at the moment of _____.
9	The function used to take user input in Python is _____.
10	In Python, the function used to display output on the screen is _____.



4. APPLICATIONS OF PYTHON IN THE REAL WORLD

Python has moved far beyond being just another programming language. It is now one of the most important tools shaping industries, research, and education worldwide. Its simplicity, readability, and adaptability have made it a preferred language for a wide range of applications. From powering global websites to driving artificial intelligence breakthroughs, Python is everywhere. In this chapter, we will explore why Python is so popular in industry, examine its use cases in data science, AI, web development, and automation, look at its growing role in education and research, and finally, discuss its future scope.

4.1 Why Python is Popular in Industry

Python's rise to popularity in the industry is not accidental; it is the result of specific qualities that make it stand out.

Simplicity and Readability

One of the strongest reasons for Python's adoption is its simple syntax. Python code often looks close to natural English, which makes it easier for developers to learn and write. This reduces the time needed for training and enables teams to collaborate effectively.

Open Source and Free

Python is free to use, modify, and distribute. This makes it accessible to startups, students, and enterprises without financial barriers. The open-source nature also ensures that the language evolves quickly, supported by a massive global community.

Extensive Libraries and Frameworks

Another factor that drives Python's popularity is its rich library ecosystem. Libraries like NumPy, Pandas, TensorFlow, Flask, and Django provide ready-to-use tools for data analysis, artificial intelligence, and web development. This reduces development time significantly, allowing companies to focus on solving problems instead of reinventing solutions.

Cross-Platform Compatibility

Python programs can run on Windows, macOS, Linux, and even mobile operating systems with minimal adjustments. This compatibility reduces development complexity and ensures wider application.

Strong Community Support

Python has one of the largest developer communities in the world. This means that solutions to problems are easy to find, and beginners have access to tutorials, forums, and open-source projects to learn from. Companies benefit because hiring Python developers is easier due to the availability of talent.

Integration with Other Technologies

Python can integrate smoothly with other programming languages and platforms. It can interact with databases, communicate with APIs, and even work with lower-level languages like C and C++. This flexibility makes it an ideal tool for multi-layered applications.

Overall, Python's popularity in industry comes from its ability to balance ease of use with powerful functionality, making it suitable for both beginners and professionals.

4.2 Applications in Data Science, AI, Web Development, and Automation

Python is not confined to one domain. It has proven itself in multiple industries and technical fields.

Python in Data Science

Data science involves analyzing and interpreting large amounts of data to make better decisions. Python has become the leading language in this field due to its wide set of libraries and ease of handling data.

- Pandas is used for data manipulation and cleaning.
- NumPy provides numerical and mathematical functions for large datasets.
- Matplotlib and Seaborn are powerful visualization libraries.
- Scikit-learn enables machine learning model development.

For example, businesses use Python to analyze customer behavior, track sales patterns, and optimize supply chains. In healthcare, Python helps analyze medical images and predict disease risks using patient data.

Python in Artificial Intelligence (AI) and Machine Learning

AI and machine learning require handling massive datasets and building complex algorithms. Python is the first choice in this area because of its simple syntax and specialized libraries.

- TensorFlow and PyTorch are leading frameworks for deep learning.
- Keras allows quick building of neural networks.
- NLTK and spaCy help with natural language processing (NLP).

Real-world examples include voice assistants like Alexa and Siri, fraud detection systems in banking, and recommendation systems on platforms like Netflix and Amazon. Python acts as the backbone for developing, training, and deploying these intelligent systems.

Python in Web Development

Python also has a strong presence in web development. It is used to create dynamic websites, manage backend systems, and build APIs.

- Django is a high-level framework that encourages fast development and clean code.
- Flask is a lightweight framework suitable for smaller projects or microservices.
- FastAPI is gaining popularity for building APIs quickly and efficiently.

Well-known companies such as Instagram, Spotify, and Dropbox have used Python for their web services. Its frameworks allow developers to focus on business logic while handling security, scalability, and performance.

Python in Automation

Python is also called the “Swiss army knife” of automation. With just a few lines of code, tasks that usually require manual effort can be automated.

- Automating file operations such as renaming or moving files.
- Writing scripts to scrape data from websites.
- Automating repetitive office tasks like sending emails or generating reports.
- Testing software automatically with frameworks such as Selenium.

Organizations save time and reduce errors by using Python automation scripts, which directly improves efficiency and productivity.

In short, Python’s applications in data science, AI, web development, and automation highlight its versatility across diverse domains.

4.3 Python in Education and Research

Python has also made a significant mark in education and academic research.

Python in Education

Python is now taught in schools, colleges, and universities around the world as a first programming language. Its beginner-friendly nature makes it ideal for introducing programming concepts. Unlike languages with complex syntax, Python allows students to focus on problem-solving rather than getting stuck on technical details.

Many educational boards have included Python in their curriculum, replacing older languages such as C++ or Java. The reason is clear: Python prepares students for the future job market, as it is widely used in professional environments.

Python in Research

Researchers across domains—whether physics, biology, economics, or social sciences—are using Python for simulations, statistical analysis, and modeling. Its popularity in research comes from three major factors:

1. **Accessibility** – It is free, easy to install, and supported on all major platforms.
2. **Flexibility** – Researchers can use Python for quick prototypes and then scale up to complex projects.
3. **Visualization** – Tools like Matplotlib and Plotly make it easy to visualize experimental data.

For example, climate scientists use Python to model weather changes, biologists use it to analyze DNA sequences, and economists apply it to forecast financial markets.

Python has become the common ground between computer science and other disciplines, enabling collaboration and innovation.

4.4 Future Scope of Python Programming

Python's current dominance raises a natural question: what does the future hold? The signs suggest that Python will continue to grow and expand.

Continued Demand in Data Science and AI

With industries increasingly adopting AI and machine learning, Python will remain at the center of innovation. As more organizations rely on data-driven strategies, demand for Python skills will keep rising.

Expansion into New Fields

Python is already being used in emerging technologies such as blockchain, Internet of Things (IoT), and robotics. Its adaptability ensures it will be at the forefront as these fields expand.

Role in Cloud Computing and DevOps

Cloud platforms like AWS, Azure, and Google Cloud support Python for automation, scripting, and application deployment. Similarly, DevOps tools integrate Python for managing infrastructure. As companies shift to cloud-based operations, Python's importance will increase further.

Growing Educational Importance

Python's adoption in schools and universities ensures that the next generation of programmers will already be fluent in it. This creates a sustainable cycle where industry demand and educational supply reinforce each other.

Community and Ecosystem Growth

The Python community continues to release new frameworks and libraries. With each passing year, Python becomes more capable, addressing new challenges and reducing development time even further.

In summary, Python is not just relevant today but is also a language with a secure and promising future.

SELF-ASSESSMENT QUESTIONS – 3

Match of Following			
	Column A		Column B
11	Pandas	a)	Framework for deep learning
12	Django	b)	Data manipulation and analysis
13	TensorFlow	c)	Web development framework
14	Jupyter Notebook	d)	Interactive environment for coding and visualization
15	Selenium	e)	Automation testing tool

5. SUMMARY

- Python is a powerful, high-level programming language valued for its simplicity, readability, and wide range of applications.
- Learners are introduced to Python's history, features, advantages, and why it is widely adopted across industries.
- Installing Python is straightforward on Windows, macOS, and Linux, with tools like IDLE, VS Code, PyCharm, and Jupyter supporting development.
- Python programs can be run in two ways: using the interactive Shell for quick testing or script mode for reusable programs.
- Python enforces indentation as part of its syntax, making code clean, structured, and error-free.
- Comments (#) and docstrings (""" """) are essential for documentation, improving readability and collaboration.
- Variables are created dynamically in Python, and data types include numbers, strings, lists, tuples, sets, dictionaries, and booleans.
- Input and output functions enable user interaction, allowing programs to accept data, display results, and behave dynamically.
- Python finds applications in diverse areas such as data science, artificial intelligence, machine learning, web development, and automation.
- In education and research, Python is used for teaching, simulations, modeling, and data analysis across multiple disciplines.
- Python's future is strong, with growing roles in cloud computing, IoT, robotics, and advanced AI solutions.
- Self-assessment questions and terminal exercises consolidate learning, while the glossary and references provide additional support for revision.

6. GLOSSARY

Python Introduction	- Python is a high-level, interpreted programming language known for its simple syntax, readability, and flexibility, making it useful for beginners and professionals alike..
Overview of Python	- Developed by Guido van Rossum, Python emphasizes clean code and readability, supports multiple paradigms, and is widely applied in industries such as AI, data science, and automation.
Python Environments	- Beginners often use IDLE (included with Python), while developers prefer VS Code, PyCharm, or Jupyter Notebook for larger projects and data-driven tasks.
Running Programs	- Python supports two modes: interactive mode (Python Shell) for quick testing and script mode (saved .py files) for reusable and structured programming.
Syntax and Indentation	- Python enforces indentation to define code blocks, ensuring clarity and readability. It is case-sensitive and uses keywords with strict naming rules for identifiers.
Comments and Docstrings	- Single-line comments (<code>#</code>), multi-line comments, and docstrings (<code>"""</code> <code>"""</code>) improve code readability, provide explanations, and support documentation at runtime.
Variables and Data Types	- Variables in Python are dynamically typed, meaning their type is determined by the assigned value. Core data types include integers, floats, strings, lists, tuples, dictionaries, sets, and booleans.
Input and Output	- Input functions allow user interaction, while output functions display results. Together, they form the backbone of dynamic programs, personalization, and automation.

7. TERMINAL QUESTIONS

1. Explain the history of Python and discuss the key features that make it one of the most popular programming languages today.
2. Describe the process of installing Python on Windows, macOS, and Linux. Why is adding Python to the system path important?
3. Compare the Python interactive Shell and script execution modes. Provide examples where each mode is more suitable.
4. Discuss the importance of indentation in Python. How does Python's approach differ from other programming languages such as C or Java?
5. Differentiate between comments and documentation strings in Python. Provide examples to illustrate their uses.
6. Define variables in Python. How does dynamic typing work, and what are the advantages and potential drawbacks of this feature?
7. Explain Python's built-in data types with examples. How do lists, tuples, and dictionaries differ in terms of mutability and usage?
8. Discuss the role of input and output functions in making programs interactive. Illustrate with examples of formatted output and user input.
9. Identify and explain at least four major real-world applications of Python in fields such as data science, artificial intelligence, web development, and automation.
10. Evaluate the future scope of Python programming. Why will Python continue to remain relevant in education, research, and industry?

8. ANSWERS

8.1. Self-Assessment Answers:

1. b) Guido van Rossum
2. b) python3 or python
3. c) .py
4. c) Jupyter Notebook
5. b) Interpreted and dynamically typed
6. Indentation
7. # (hash symbol)
8. Assignment
9. input()
10. print()
11. Pandas → b) Data manipulation and analysis
12. Django → c) Web development framework
13. TensorFlow → a) Framework for deep learning
14. Jupyter Notebook → d) Interactive environment for coding and visualization
15. Selenium → e) Automation testing tool

8.2. Terminal Question Answers

Answer 1: Python was developed by Guido van Rossum and released in 1991. Its simplicity, readability, dynamic typing, cross-platform support, and rich libraries make it widely adopted in both academia and industry

Answer 2: Python can be installed via its official website. On Windows, users should select “Add to PATH” during installation. macOS and Linux users can use package managers. Adding Python to the system path allows easy access to Python from any command line terminal

Answer 3: The interactive mode lets users test code one line at a time, ideal for debugging or learning. Script mode is used for writing larger programs saved as .py files and run together, making it suitable for real-world applications.

Answer 4: Python uses indentation to define code blocks, unlike C or Java which use braces. This improves readability and enforces structure. Improper indentation causes errors, making it an integral part of Python syntax.

Answer 5: Comments, marked by #, explain code but aren't executed. Docstrings, enclosed in triple quotes, describe modules, classes, or functions and are accessible through `help()`. Both improve code clarity and documentation.

Answer 6: Variables are created by assigning values directly. With dynamic typing, the variable type is determined at runtime. It allows flexibility but can lead to runtime type errors if misused.

Answer 7: Python includes built-in types like `int`, `float`, `str`, `list`, `tuple`, and `dict`. Lists are mutable, tuples are immutable, and dictionaries store key-value pairs. Each is suited to different use cases depending on the need for modification or access speed.

Answer 8: The `input()` function reads user input, while `print()` outputs data. These functions make programs interactive. For example, formatted output using `print(f"Name: {name}")` enhances clarity and user experience.

Answer 9: Python is widely used in data science for analysis, in AI using libraries like TensorFlow, in web development through Django, and for automation with scripts. Its versatility makes it suitable across domains.

Answer 10: Python's future is strong due to its simplicity, active community, and wide application in emerging technologies like AI, IoT, and robotics. It remains a top choice for education, research, and commercial software.

9. REFERENCES

- Python Software Foundation. Python Official Documentation. Available at: <https://docs.python.org/3/>
- Python Software Foundation. Python Downloads. Available at: <https://www.python.org/downloads/>
- Van Rossum, G., & Drake, F. L. (2009). The Python Language Reference Manual. Network Theory Ltd.
- Lutz, M. (2013). Learning Python (5th Edition). O'Reilly Media.
- Sweigart, A. (2015). Automate the Boring Stuff with Python. No Starch Press.
- McKinney, W. (2017). Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython (2nd Edition). O'Reilly Media.
- Downey, A. (2015). Think Python: How to Think Like a Computer Scientist (2nd Edition). Green Tea Press.
- JetBrains. PyCharm Documentation. Available at: <https://www.jetbrains.com/pycharm/>
- Microsoft. Visual Studio Code Documentation. Available at: <https://code.visualstudio.com/docs>
- Project Jupyter. Jupyter Notebook Documentation. Available at: <https://jupyter.org/documentation>

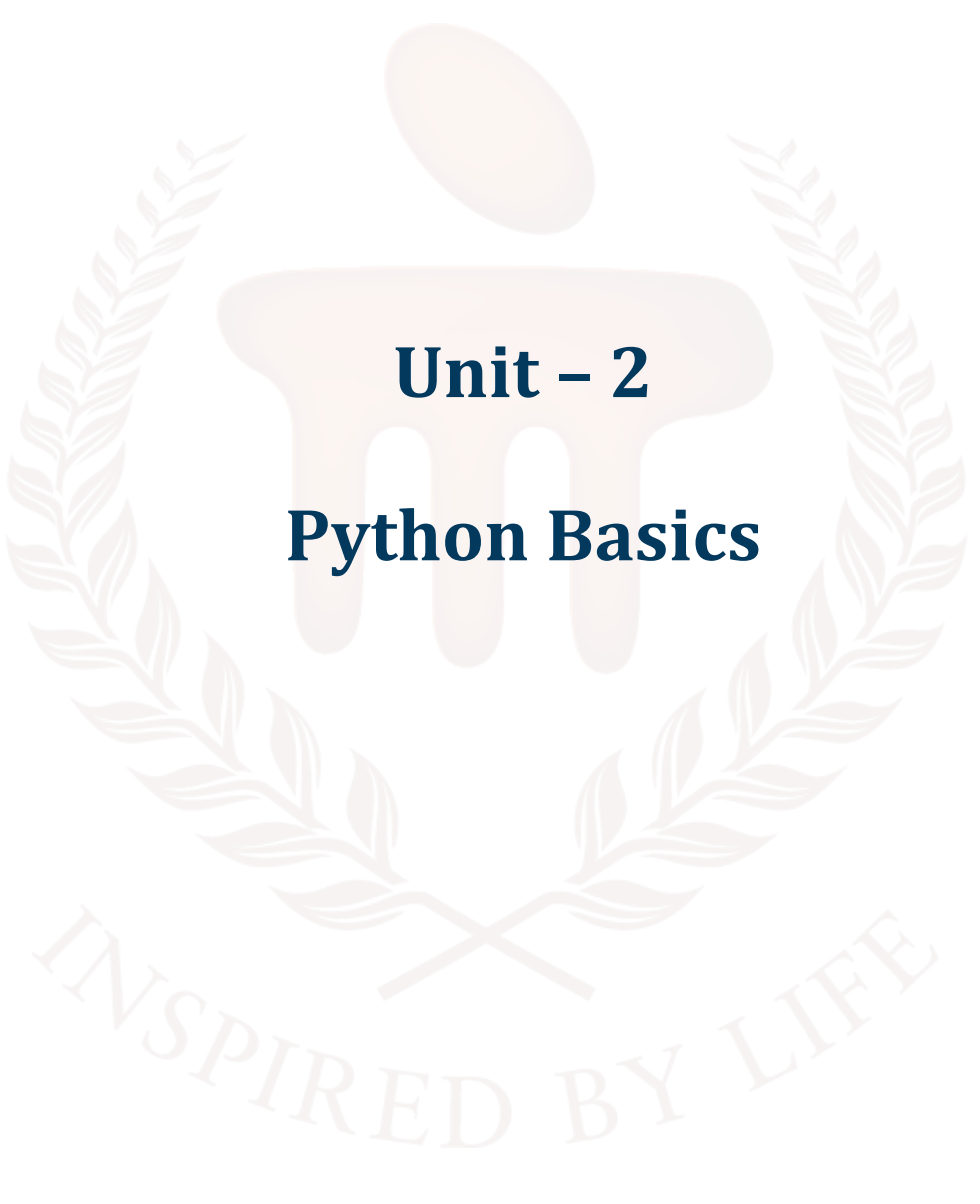
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 2

Python Basics

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 - 6
1.1	Learning Objectives	-	-	
2	Expressions In Python	-	-	7 - 17
2.1	Definition of Expressions	-	-	
2.2	Literals and Constants	-	-	
2.3	Simple and Compound Expressions	-	-	
2.4	Evaluating Expressions in Python	-	1	
3	Data Types In Python	-	-	18-28
3.1	Overview of Built-in Data Types	-	-	
3.2	Type Conversion (Implicit and Explicit Casting)	-	-	
3.3	Checking and Identifying Data Types	-	2	
4	Variables In Python	-	-	29-34
4.1	Concept of Variables	-	-	
4.2	Rules for Naming Variables	-	-	
4.3	Variable Assignment and Reassignment	-	-	
4.4	Constants and Best Practices	-	-	
4.5	Memory Management and References in Python	-	3	
5	Operators In Python	-	-	35-39
5.1	Arithmetic Operators (+, -, *, /, %, //, **)	-	-	
5.2	Relational/Comparison Operators (==, !=, >, <, >=, <=)	-	-	
5.3	Logical Operators (and, or, not)	-	-	
5.4	Assignment Operators (=, +=, -=, *=, etc.)	-	-	
5.5	Bitwise Operators (&, , ^, ~, <<, >>)	-	-	
5.6	Membership Operators (in, not in)	-	-	

	5.7	Identity Operators (is, is not)	-	-	
	5.8	Operator Precedence and Associativity	-	4	
6		Summary	-	-	40
7		Glossary	-	-	41
8		Terminal Questions	-	-	42
9		Answers	-	-	
	9.1	Self-Assessment Answers	-	-	43-45
	9.2	Terminal Questions Answers	-	-	
10		References	-	-	46

1. INTRODUCTION

In this unit, learners will explore the essential building blocks of Python programming, which form the foundation for writing effective and logical code. Python is one of the most popular programming languages today, celebrated for its simplicity, readability, and adaptability across diverse domains. Its syntax closely resembles natural English, making it an ideal starting point for beginners. As learners progress through this unit, they will understand the role of core elements such as expressions, data types, variables, and operators in shaping meaningful Python programs.

The unit begins by introducing expressions, which combine values, variables, and operators to generate results—ranging from simple calculations like $3 + 5$ to more complex logic. Learners will discover how expressions drive data processing and decision-making within programs. Next, they will understand data types, including integers, floats, strings, booleans, lists, tuples, and dictionaries, and how each data type is suited for specific computational needs. Recognising and applying the correct data type will help learners avoid runtime errors and ensure accurate outcomes.

Further, the unit focuses on variables, which act as named storage for data, enhancing the readability and reusability of code. Through examples like $x = 10$, learners will grasp how variables simplify programming by allowing dynamic value manipulation. The topic of operators will then be explored, showcasing arithmetic, logical, comparison, and assignment operators—each enabling precise control over data and flow within a program.

By the end of this unit, learners will be able to construct functional Python code using these basic constructs. They will develop the ability to write clean, well-structured programs and lay a strong foundation for advancing into more complex topics like conditional statements, functions, and object-oriented design.

1.1. Learning Objectives

- Explain the concept of expressions
- Identify Python data types such as integers, floats, strings, booleans, and collections,
- Demonstrate how to declare and use variables to store and manipulate values in Python programs.
- Apply different types of operators to perform operations on data and variables.
- Write simple Python programs combining expressions, variables, data types, and operators to solve basic problems.
- Analyse errors caused by incompatible data types or incorrect operator usage
- Develop a foundation for advanced Python concepts by building confidence in handling Python's core building blocks effectively



2. EXPRESSIONS IN PYTHON

Expressions are one of the most important parts of programming in Python. They represent the pieces of code that produce values and make programs meaningful. Without expressions, a program would simply contain lines of text that do nothing. By combining values, variables, and operators, expressions allow us to perform calculations, evaluate logic, and manipulate data. This chapter introduces the idea of expressions step by step, beginning with their definition, moving to literals and constants, then exploring simple and compound expressions, and finally understanding how Python evaluates them. At the end, you will find self-assessment questions in the form of multiple-choice items to test your understanding

2.1 Definition of Expressions

In programming, an expression is a combination of values, variables, operators, and function calls that Python interprets and reduces to a single value. For example, when you write:

```
3 + 4
```

Python interprets this as an expression. It evaluates it and produces the result 7. Similarly, if you write:

```
x * 5
```

and if $x = 2$, then Python evaluates the expression as 10.

The key idea is that expressions are always about producing a value. They can be simple, like a single number 42, or complex, like $((a + b) * c) / d$.

In natural language, you can think of an expression like a mathematical sentence. Just as “2 + 3” in math means “add two and three to get five,” an expression in Python tells the computer to compute a value.

There is also an important distinction between expressions and statements.

- An expression is something that produces a value.
- A statement is an instruction that performs an action.

For example:

`y = 10` # This is a statement (assignment) containing an expression on the right-hand side.

The 10 is an expression, while `y = 10` as a whole is a statement.

Understanding expressions is crucial because most programming tasks—whether calculations, decisions, or transformations—depend on writing correct expressions.

2.2 Literals and Constants

To understand expressions more deeply, we need to study literals and constants, since these are often the building blocks of expressions.

Literals

A literal is a raw value directly written in the code. For example:

- 5 → integer literal
- 3.14 → floating-point literal
- "Hello" → string literal
- True → boolean literal

These values are not stored in variables at the start; they are written as they are. For example, in:

```
area = 3.14 * r * r
```

Here, 3.14 is a floating-point literal.

Python supports several kinds of literals:

- **Numeric literals** → integers (7), floats (2.5), complex numbers (3 + 4j)
- **String literals** → "Python", 'Hello', """"This is a long string""""
- **Boolean literals** → True, False
- **Special literal** → None, which represents no value

Constants

In programming, constants are values that should remain unchanged throughout the execution of a program. They are important because they add clarity, consistency, and safety to code. While many languages like Java, C++, or C# provide explicit syntax for declaring constants (e.g., using `final` or `const` keywords), Python does not have such built-in support. However, Python developers use naming conventions and disciplined practices to indicate which variables should be treated as constants.

What Are Constants?

A constant is essentially a value assigned to a variable name that is not meant to be modified later.

For instance:

PI = 3.14159

GRAVITY = 9.8

Here, PI and GRAVITY are constants. While Python technically allows us to reassign them (e.g., PI = 22/7), doing so is discouraged. By convention, constants are written in UPPERCASE letters, often with underscores separating words (MAX_SPEED, DEFAULT_TIMEOUT). This convention signals to other developers that these values should be treated as fixed.

Why Use Constants?

1. Readability and Clarity

Constants make the code easier to read and understand. If you see `2 * PI * radius`, you immediately know this formula refers to a circle's circumference. On the other hand, `2 * 3.14159 * radius` may look confusing and repetitive.

2. Maintainability

Suppose you used the value 3.14159 directly in multiple places across your program. If you later decide to use a more precise value of π (say, 3.141592653), you would have to replace it everywhere. With a constant, you only update it in one place, reducing errors.

3. Error Prevention

Constants prevent accidental changes to important values. For example, if you mistakenly assign a new value to GRAVITY, your entire physics calculation system could break. Treating GRAVITY as a constant ensures you avoid such pitfalls.

Literals vs. Constants

It's important to differentiate between literals and constants.

- Literals are raw fixed values directly written in code, such as 42, "Hello", or 3.14.
- Constants are symbolic names given to such values to provide meaning and reuse, e.g., MAX_USERS = 42.

In short, literals are the actual data, while constants are labels we give them for clarity and consistency.

Real-World Examples

1. Mathematics and Science Applications

- `PI = 3.14159`
- `SPEED_OF_LIGHT = 3.0e8`
- `PLANCK_CONSTANT = 6.626e-34`

These values rarely change in real-world contexts, making them perfect candidates for constants.

2. Configuration Settings

- `DEFAULT_TIMEOUT = 30 (seconds)`
- `MAX_RETRIES = 5`
- `API_URL = "https://api.example.com/v1"`

By using constants for such settings, the program becomes easier to configure and update.

3. Game Development

- `SCREEN_WIDTH = 800`
- `SCREEN_HEIGHT = 600`
- `MAX_LIVES = 3`

Constants make it easier to tweak game settings without searching through thousands of lines of code.

Best Practices for Constants in Python

- **Use Uppercase Naming:** Always write constant names in uppercase letters to signal immutability.
- **Group Constants Together:** Keep all constants in one place, often at the top of the file or in a separate `constants.py` module.
- **Do Not Reassign:** While Python won't stop you from reassigning a constant, it's good discipline to avoid doing so.
- **Use Constants for Magic Numbers:** Replace unexplained numbers in code with named constants. For example:
 - `TAX_RATE = 0.18`
 - `total = price + (price * TAX_RATE)`

This is much clearer than:

```
total = price + (price * 0.18)
```

2.3 Simple and Compound Expressions

Expressions are at the very heart of programming in Python. They allow us to work with values, variables, and operators to perform calculations, manipulate text, or process data. Broadly, expressions can be simple or compound. Understanding this distinction helps beginners write clean code and lays the foundation for solving larger problems. Let's carefully explore both, starting with simple expressions and then moving on to compound ones.

What is an Expression?

In programming, an expression is a piece of code that produces a value. For example:

`5 + 3`

This is an expression because Python evaluates it and produces the result 8. Similarly:

`"Hello" + "World"`

is an expression because it evaluates to "HelloWorld". Whether it involves numbers, text, or variables, an expression always has some output.

Simple Expressions

A simple expression is the most basic form of an expression. It contains only a single literal value or a single variable name. There are no operators or function calls in simple expressions.

Some examples:

1. Numeric literal

`7`

Here, the value is just 7. It evaluates to itself.

2. Variable reference

`x`

If `x` has been defined earlier, say `x = 10`, then using `x` in an expression evaluates to 10.

3. String literal

`"Hello"`

This expression directly represents the text "Hello".

4. Boolean literal

True

A Boolean literal evaluates to the logical truth value True.

Why Are Simple Expressions Important?

Although they look basic, simple expressions are building blocks. More complex operations always start with these. For instance, in the compound expression $(3 + 5) * 2$, the simple expressions are 3, 5, and 2. Without these atomic pieces, no compound structure could exist.

Simple expressions also make code readable and direct. For example, $PI = 3.14159$ represents a constant, and every time you use PI , you are working with a simple expression.

Compound Expressions

A compound expression is formed when two or more values, variables, or expressions are combined using operators or functions. Instead of evaluating directly to itself, a compound expression must be worked out step by step until a final result is produced.

Some common examples:

1. Arithmetic Expressions

2. $(3 + 5) * 2$

- $3 + 5$ evaluates to 8.
- Then $8 * 2$ gives 16.

This is a compound expression because it involves multiple operators.

3. Variable-based Expressions

4. $x + y - z$

Suppose $x = 10$, $y = 5$, and $z = 2$. Substituting values, we get $10 + 5 - 2$, which evaluates to 13.

5. String Concatenation

6. `name + " is learning Python"`

If `name = "Alice"`, then this expression becomes "Alice is learning Python". The `+` operator combines text pieces.

Function Calls in Compound Expressions

Python allows function calls inside expressions, and this often produces more powerful compound structures. For example:

```
len("Python") + 5
```

Step by step:

- `len("Python")` returns 6 because the word "Python" has 6 letters.
- Adding 5 to that gives 11.

This compound expression demonstrates how built-in functions can participate in evaluation.

Nesting of Compound Expressions

Compound expressions can be nested, meaning one expression can sit inside another.

Example:

```
(2 + (3 * (4 + 1)))
```

Breaking it down:

- $(4 + 1) \rightarrow 5$
- $3 * 5 \rightarrow 15$
- $2 + 15 \rightarrow 17$

While nesting is possible, too much of it can make code hard to read. A best practice is to keep expressions short and assign intermediate results to variables.

For instance, instead of writing:

```
final_result = ((a + b) * (c - d)) / (len(name) + max(numbers))
```

It is clearer to break it into smaller steps:

```
part1 = a + b
```

```
part2 = c - d
```

```
part3 = len(name) + max(numbers)
```

```
final_result = (part1 * part2) / part3
```

This way, the logic remains understandable for both you and other developers.

Comparing Simple and Compound Expressions

Aspect	Simple Expression	Compound Expression
Structure	Single value or variable	Combination of values, variables, and operators
Examples	7, x, "Hello"	$(3 + 5) * 2$, $x + y - z$
Evaluation	Direct (evaluates to itself or stored value)	Step-by-step calculation required
Use in programs	Building blocks, clarity	Performing actual computations, logic, and tasks

Importance of Expressions in Programming

1. Foundation of Logic

Every decision or calculation in a program comes down to expressions. Whether it's `if x > 10:` or `total = price * quantity`, expressions define the logic.

2. Flexibility

Expressions can combine numbers, strings, lists, or even objects. This allows Python to solve varied problems, from math to text processing.

3. Scalability

Simple expressions scale into compound ones. As programs grow, this layering enables developers to write sophisticated code while still understanding it step by step.

Best Practices

- **Start Simple:** Write smaller expressions and test them before building larger compound ones.
- **Prioritize Readability:** Avoid deeply nested expressions. Use parentheses or intermediate variables.
- **Use Constants and Variables Wisely:** Replace repeated numbers or strings with named variables or constants.
- **Combine with Functions:** Use built-in functions like `len()`, `max()`, or `sum()` to enrich your compound expressions.

2.4 Evaluating Expressions in Python

When Python encounters an expression, it follows strict rules to evaluate it. These rules are based on operator precedence and associativity.

Operator Precedence

Operator precedence determines which operations are performed first. **For example:**

```
result = 3 + 5 * 2
```

Python first performs multiplication ($5 * 2 = 10$), then addition ($3 + 10 = 13$).

The order of precedence (from highest to lowest) includes:

1. Parentheses `()`
2. Exponentiation `**`
3. Unary operators `(+x, -x, ~x)`
4. Multiplication, division, floor division, modulus `(*, /, //, %)`
5. Addition and subtraction `(+, -)`
6. Comparisons `(==, !=, <, >, <=, >=)`
7. Logical operators `(not, and, or)`
8. Assignment operators `(=, +=, -= etc.)`

Using parentheses `()` is always recommended to make the order explicit and avoid confusion.

Associativity

When two operators of the same precedence appear, Python decides based on associativity. For example:

- `*` and `/` are left-associative: $100 / 10 * 2 \rightarrow (100 / 10) * 2 = 20$.
- Exponentiation `**` is right-associative: $2 ** 3 ** 2 \rightarrow 2 ** (3 ** 2) = 2 ** 9 = 512$.

Type Rules

Expressions must also respect data type rules. For instance:

```
5 + "Hello"
```

will result in an error because integers and strings cannot be added. However,

```
"Hi " + "there"
```


works fine because both are strings.

Practical Example

Suppose we want to calculate the total bill in a restaurant with a tax and tip:

```
meal = 500
```

```
tax = meal * 0.1
```

```
tip = meal * 0.05
```

```
total = meal + tax + tip
```

Here, each line contains expressions. Python evaluates them in order, respecting precedence. The result is a meaningful calculation for the total bill.

Evaluating expressions correctly is at the heart of making Python programs useful in real-world scenarios.

SELF-ASSESSMENT QUESTIONS – 1

Multiple Choose Questions

- | | |
|---|--|
| 1 | Which of the following is an expression in Python?

a) <code>x = 10</code>

b) <code>3 + 7</code>

c) <code>print("Hello")</code>

d) Both b and c |
| 2 | What is the output of the expression <code>2 + 3 * 4</code> in Python?

a) 20

b) 14

c) 24

d) 9 |
| 3 | Which of these is NOT a literal in Python?

a) 42 |

	b) "Python" c) True d) PI
4	. In the expression <code>2 ** 3 ** 2</code>, how does Python evaluate it? a) <code>(2 ** 3) ** 2</code> b) <code>2 ** (3 ** 2)</code> c) Error d) None of the above
5	What will happen if you try to evaluate <code>"10" + 5</code> in Python? a) It will give 15. b) It will give "105". c) It will raise a TypeError. d) It will return None



3. DATA TYPES IN PYTHON

Data types are the backbone of every programming language, and Python is no exception. When we talk about data types, we are essentially describing the kind of value stored in memory and the operations that can be performed on it. Python is known for being dynamically typed, which means that you do not need to declare the type of a variable explicitly—Python interprets and assigns it automatically at runtime.

Understanding data types is essential because they dictate how data is stored, how it behaves during operations, and how it interacts with other pieces of data. For instance, adding two integers results in a mathematical sum, whereas adding two strings concatenates them into a larger string. In this chapter, we will carefully examine Python's built-in data types, type conversion, methods for checking data types, and finally test your knowledge with self-assessment questions.

3.1 Overview of Built-in Data Types

Python provides a rich set of built-in data types that are divided into categories such as numeric, sequence, mapping, set, boolean, and special types like `NoneType`.

3.1.1 Numeric Types

Numbers are one of the most fundamental data types in any programming language. Python provides built-in support for three numeric types: integers (`int`), floating-point numbers (`float`), and complex numbers (`complex`). These types cover most numerical operations required in mathematics, science, and everyday programming tasks. Let's look at each type in detail.

1. Integers (`int`)

An integer represents whole numbers without any fractional or decimal component. They can be positive, negative, or zero. Examples include:

```
a = 10
```

```
b = -25
```

```
c = 0
```

One of Python's major advantages is that integers can be arbitrarily large. Unlike some other languages (like C or Java) where integers are limited by fixed sizes (e.g., 32-bit or 64-bit), Python's integers grow as large as the available memory allows. For example:

```
big_number = 123456789012345678901234567890  
  
print(big_number)
```

This makes Python especially powerful for tasks like cryptography, scientific computation, or finance, where large integers are common.

2. Floating-Point Numbers (float)

A float represents real numbers that contain a decimal point or are expressed in scientific notation. They are useful when precision with fractions is needed. Examples:

```
pi = 3.14159  
temperature = -5.4  
e = 2.7e3 # 2700.0 in scientific notation
```

Floats can represent both very small and very large values, making them suitable for scientific calculations. However, they are stored internally in binary floating-point format, which can lead to tiny rounding errors. For example:

```
print(0.1 + 0.2) # Outputs 0.30000000000000004
```

This behavior is normal in many programming languages and arises from how computers approximate decimal fractions in binary. For most applications, these errors are negligible, but in finance or sensitive calculations, developers often use the decimal module for higher precision.

3. Complex Numbers (complex)

Python also has built-in support for complex numbers, which are widely used in fields like engineering, physics, and signal processing. A complex number has two parts:

- A real part
- An imaginary part, represented with the suffix `j`

Example:

```
z = 3 + 4j  
  
print(z.real) # 3.0
```

```
print(z.imag) # 4.0
```

Python allows direct arithmetic with complex numbers:

```
z1 = 2 + 3j
```

```
z2 = 1 - 4j
```

```
print(z1 + z2) # (3 - 1j)
```

```
print(z1 * z2) # (14 - 5j)
```

This feature makes Python stand out compared to many languages that don't have native complex number support.

Why Numeric Types Matter

- Integers are essential for counting, indexing, or whole-number calculations.
- Floats handle continuous values like distances, measurements, or percentages.
- Complex numbers open doors to advanced problem-solving in technical domains.

Together, these three numeric types make Python versatile for everything from simple math to scientific computing.

3.1.2 Sequence Types

Python provides a rich collection of data types that allow developers to work with ordered and unordered collections of data. Among these, sequences and mappings are the most frequently used. Sequences are ordered collections where elements are stored in a specific order, while mappings store data as key-value pairs. Understanding these types is essential because they form the backbone of Python programming in data manipulation, storage, and retrieval.

1. Sequence Types

A sequence in Python is an ordered collection that supports operations like indexing, slicing, iteration, and membership testing. Python offers several built-in sequence types: strings (str), lists (list), tuples (tuple), and ranges (range).

a) String (str)

Strings are sequences of Unicode characters. They can be written using single quotes ('), double quotes ("), or triple quotes (''' or ''') for multi-line text.

Examples:

```
name = "Python"

sentence = 'Learning is fun'

multiline = """This is
a multi-line
string."""
```

Key features of strings:

- **Immutable:** Once created, the contents of a string cannot be changed.
- **Indexing:** Access individual characters using their position.
- **Slicing:** Extract substrings using [:] notation.
- **Operators:** + for concatenation and * for repetition.

Example:

```
print(name[0])  # 'P'
print(name[1:4]) # 'yth'
print(name * 2)  # 'PythonPython'
```

Strings are widely used for handling text, making them one of the most important sequence types.

b) List (list)

Lists are mutable, ordered collections that can store items of different data types. They are written in square brackets.

Example:

```
fruits = ["apple", "banana", "cherry", 5, 3.14]
```

Key features:

- **Mutable:** Elements can be added, removed, or modified.
- **Versatile:** Lists can store mixed data types.
- **Methods:** Support powerful operations like `append()`, `remove()`, `sort()`, `extend()`, etc.

Example:

```
fruits.append("mango")

print(fruits) # ['apple', 'banana', 'cherry', 5, 3.14, 'mango']
```

Lists are often used when data needs to be changed or updated frequently.

c) Tuple (tuple)

Tuples are immutable, ordered collections, written using parentheses. Because they cannot be modified after creation, tuples are ideal when data should remain constant.

Example:

```
coordinates = (10.5, 20.3)
person = ("Alice", 30, "Engineer")
print(person[0]) # 'Alice'
```

Advantages of tuples:

- Safer than lists when immutability is required.
- Can be used as keys in dictionaries (since they are immutable).
- Useful for grouping related data.

d) Range (range)

A range represents a sequence of numbers, commonly used in loops or generating numeric sequences.

Basic usage:

```
for i in range(5):
    print(i)
```

Output:

```
0 1 2 3 4
```

Custom ranges can be created with start, stop, and step values:

```
for i in range(2, 10, 2):
    print(i) # 2, 4, 6, 8
```

Ranges are memory-efficient because they generate numbers on demand rather than storing them all at once.

2. Mapping Type: Dictionary (dict)

Unlike sequences, a dictionary stores data in key–value pairs. Keys must be unique and immutable (like strings, numbers, or tuples), while values can be of any type. Dictionaries are written using curly braces {}.

Example:

```
student = {
    "name": "John",
    "age": 21,
    "marks": [85, 90, 78]
}

print(student["name"]) # John
```

Key features of dictionaries:

- Fast lookups: Data is accessed using keys, not indexes.
- Dynamic: New key–value pairs can be added or updated easily.
- Methods: Include get(), keys(), values(), items(), and update().

Use cases:

- Storing structured data (like student records).
- Representing objects in real-world scenarios.
- Fast retrieval in applications like databases, APIs, or caching

3. Sequences vs. Mappings

Feature	Sequence (str, list, tuple, range)	Mapping (dict)
Order	Ordered	Unordered (Python 3.7+ maintains insertion order)
Access method	By index or slicing	By key
Mutability	Some mutable (list), some immutable (str, tuple, range)	Mutable
Use case	Store ordered collections	Store labeled, associative data

3.1.4 Set Types (set, frozenset)

Python offers a variety of data types that make it powerful and flexible for different applications. Among these, sets, Boolean values, and NoneType hold a special place because they handle uniqueness, decision-making, and absence of value. Additionally, Python supports type conversion and type checking, which are essential when working with dynamic or mixed data. Let's explore each of these concepts in detail.

Sets in Python

A set is an unordered collection of unique elements. Unlike lists or tuples, sets automatically eliminate duplicate values, making them ideal when uniqueness matters.

Mutable Set (set)

Sets are defined using curly braces {} or with the set() constructor:

```
numbers = {1, 2, 3, 3, 4}

print(numbers) # {1, 2, 3, 4}
```

Here, the duplicate 3 is automatically removed. Sets are mutable, which means we can add or remove elements:

```
numbers.add(5)

numbers.remove(2)

print(numbers) # {1, 3, 4, 5}
```

Immutable Set (frozenset)

Python also provides frozenset, which is an immutable version of a set. Once created, its elements cannot be modified:

```
fs = frozenset([1, 2, 3])

# fs.add(4) → Error, since frozenset is immutable
```

Set Operations

Sets support several mathematical operations:

- Union (| or .union()) – combines elements from two sets.

- Intersection (& or .intersection()) – finds common elements.
- Difference (- or .difference()) – finds elements in one set but not the other.
- Symmetric Difference (^) – elements in either set but not in both.

Example:

```
a = {1, 2, 3}
b = {3, 4, 5}
print(a | b) # {1, 2, 3, 4, 5}
print(a & b) # {3}
print(a - b) # {1, 2}
```

These operations make sets very useful in tasks like data analysis, duplicate removal, and membership testing.

3.1.5 Boolean Type (bool)

The Boolean type in Python has only two possible values: True and False. These values often come from comparisons or logical operations.

Example:

```
x = 10
y = 20
print(x < y) # True
print(x == y) # False
```

Booleans play a central role in decision-making through if, while, and other control structures:

```
is_logged_in = True

if is_logged_in:
    print("Access granted")
else:
    print("Access denied")
```

Logical operators like and, or, and not extend the power of Boolean expressions.

3.1.6 NoneType

NoneType represents the absence of a value, and Python has exactly one such object: None. It is often used as a default value or to indicate that a function did not return anything meaningful.

Example:

```
result = None
```

```
if result is None:
```

```
    print("No value assigned")
```

Functions frequently use None to signal that no result or no data is available:

```
def find_item(items, target):
```

```
    for i in items:
```

```
        if i == target:
```

```
            return i
```

```
    return None
```

```
print(find_item([1, 2, 3], 5)) # None
```

3.2 Type Conversion (Implicit and Explicit Casting)

When working with data, you often need to change one type into another. Python supports both implicit and explicit conversion.

a) Implicit Conversion (Type Promotion)

Python automatically converts one type to another when it is safe. This usually happens in arithmetic operations where a smaller type is promoted to a larger one:

```
a = 5    # int
```

```
b = 2.5  # float
```

```
c = a + b # int is promoted to float
```

```
print(c)  # 7.5
```

```
print(type(c)) # <class 'float'>
```

Here, Python safely converts the integer `a` to a float before performing addition.

b) Explicit Conversion (Type Casting)

Sometimes, the programmer must explicitly convert data from one type to another using built-in functions:

- `int()` → convert to integer
- `float()` → convert to floating-point
- `str()` → convert to string
- `list()`, `tuple()`, `set()` → convert to respective collections

Example:

```
num_str = "100"  
  
num_int = int(num_str)  
  
print(num_int + 50) # 150
```

However, invalid conversions raise errors:

```
int("hello") # ValueError
```

Thus, explicit conversion must be used carefully.

3.3 Checking and Identifying Data Types

Python is dynamically typed, meaning variables do not need explicit type declarations. However, it is often necessary to confirm a variable's type. Python provides two functions:

a) `type()`

The `type()` function returns the class of the given object:

```
x = 10  
  
print(type(x)) # <class 'int'>
```

b) `isinstance()`

The `isinstance()` function checks whether an object belongs to a specific type (or tuple of types):

```
x = 3.5
```

```
print(isinstance(x, float)) # True
```

```
print(isinstance(x, int)) # False
```

This is particularly useful in large programs where user inputs may vary, and type validation becomes necessary.

SELF-ASSESSMENT QUESTIONS - 2

Fill in the blanks:	
6	The three numeric data types in Python are _____, _____, and _____.
7	Strings in Python are _____ (mutable/immutable).
8	A collection of key-value pairs is represented by the _____ data type.
9	The function used to check the type of a variable in Python is _____.
10	The immutable version of a set is called _____.

4. VARIABLES IN PYTHON

Variables are one of the most fundamental concepts in Python and programming in general. They allow us to store data, give it a name, and manipulate it throughout a program. Without variables, we would need to work directly with raw values every time, making code repetitive, confusing, and hard to maintain. Variables provide a way to attach meaning to data and make our programs more flexible and human-readable.

In Python, variables are dynamic, meaning that their type does not need to be declared beforehand. The interpreter decides the type at runtime based on the value assigned. This flexibility sets Python apart from many strongly typed languages like C or Java, where you must specify the type explicitly.

In this chapter, we will explore the concept of variables, rules for naming them, assignment and reassignment, constants, best practices, and the way Python manages memory for variables.

4.1 Concept of Variables

At its core, a variable is a name that refers to a value stored in memory. Think of a variable as a labeled container or a pointer that tells Python where to find the data.

For example:

```
x = 10
```

```
y = "Python"
```

```
z = 3.14
```

Here:

- x is a variable referring to the integer value 10.
- y refers to the string "Python".
- z refers to the floating-point number 3.14.

Unlike mathematics, where variables are placeholders for unknowns, in programming, variables are labels pointing to known data.

Python variables are dynamic. If you assign an integer to a variable and later assign a string, Python adjusts the type automatically.

```
a = 5
```



```
print(a)    # 5  
  
a = "Hello"  
  
print(a)    # Hello
```

This flexibility makes coding in Python faster but also requires careful handling to avoid confusion.

4.2 Rules for Naming Variables

Python has specific rules for naming variables to avoid errors and maintain clarity.

1. Start with a letter or underscore
 - Valid: name, _value
 - Invalid: 1variable (cannot start with a digit)
2. Contain only letters, digits, and underscores
 - Valid: user_name, age1
 - Invalid: user-name, age@
3. Case sensitive
 - age and Age are two different variables.
4. Avoid reserved keywords

Words like if, else, for, class, True, None, etc., cannot be used as variable names. You can check reserved words using:

5. `import keyword`
6. `print(keyword.kwlist)`
7. Meaningful names

It is good practice to use descriptive names that indicate the purpose of the variable. For example, `student_score` is more meaningful than `s1`.

Examples:

```
student_name = "Alice" # valid  
  
_marks = 90           # valid  
  
2nd_student = "Bob"   # invalid
```

Following these rules ensures clean and maintainable code.

4.3 Variable Assignment and Reassignment

Assignment

Assigning a value to a variable in Python is simple, using the = operator:

```
x = 100
```

```
name = "John"
```

Python also supports multiple assignments in one line:

```
a, b, c = 1, 2, 3
```

```
print(a, b, c) # 1 2 3
```

You can assign the same value to multiple variables:

```
x = y = z = 0
```

Reassignment

Reassignment means updating a variable with a new value:

```
x = 10
```

```
x = 20
```

```
print(x) # 20
```

Because Python is dynamically typed, you can also reassign a variable with a different type:

```
x = 10
```

```
x = "Hello"
```

```
print(x) # Hello
```

Swapping variables

Python allows elegant swapping without a temporary variable:

```
a, b = 5, 10
```

```
a, b = b, a
```

```
print(a, b) # 10 5
```

4.4 Constants and Best Practices

Although Python does not have built-in constant support, programmers use conventions to define constants. Typically, constants are written in uppercase letters with underscores:

```
PI = 3.14159
```

```
GRAVITY = 9.8
```

```
MAX_USERS = 100
```

Even though constants can technically be changed, good practice avoids modifying them after definition.

Best Practices for Variables and Constants

1. Use meaningful names to make code self-explanatory.
2. Use lowercase letters with underscores for variables (`student_age`), uppercase for constants (`MAX_LIMIT`).
3. Keep variables within the smallest possible scope to reduce errors.
4. Initialize variables before using them.
5. Avoid reusing the same variable for unrelated purposes.

Example of poor naming:

```
x = 100
```

```
y = 200
```

Better naming:

```
item_price = 100
```

```
item_quantity = 200
```

These best practices improve readability and collaboration.

4.5 Memory Management and References in Python

Understanding memory management is crucial for grasping how variables work in Python.

References

In Python, variables do not store values directly; they store references to objects in memory. When you assign a value, Python creates an object in memory and the variable points to it.

```
x = 10
```

```
y = x
```

Both x and y point to the same object 10. If you check:

```
print(id(x), id(y))
```

you will see they have the same memory reference.

Mutable vs Immutable Types

1. Immutable types: integers, floats, strings, tuples. Reassigning them creates a new object.
2. `a = 5`
3. `b = a`
4. `a = a + 1`
5. `print(a, b) # 6 5`

Here, a points to a new object, while b still points to the old one.

- Mutable types: lists, dictionaries, sets. Modifying them changes the object in memory, affecting all references.
- `list1 = [1, 2, 3]`
- `list2 = list1`
- `list1.append(4)`
- `print(list2) # [1, 2, 3, 4]`

Garbage Collection

Python automatically manages memory using garbage collection. If an object has no references pointing to it, it is removed from memory.

```
x = [1, 2, 3]
```

```
y = x
```

```
del x
```

```
print(y) # [1, 2, 3]
```

The list remains because y still refers to it. Once all references are gone, Python frees the memory.

This reference-based memory management is powerful but requires awareness, especially when working with mutable objects.

SELF-ASSESSMENT QUESTIONS – 3

True/False	
11	Variables in Python must be declared with their data type before assignment. (T/F)
12	The variable names studentAge and StudentAge refer to the same variable. (T/F)
13	Python allows assigning values to multiple variables in one line. (T/F).
14	Constants in Python cannot be changed once defined. (T/F)
15	In Python, variables store references to objects in memory, not the actual values. (T/F)

5. OPERATORS IN PYTHON

Operators are the special symbols or keywords in Python that are used to perform specific operations on values and variables. They act as the foundation for expressions and are crucial for carrying out calculations, comparisons, and logical decisions in a program. For example, the + operator can add two numbers, while the == operator checks whether two values are equal.

Python provides a wide range of operators that can be classified into different categories based on their purpose.

5.1 Arithmetic Operators (+, -, *, /, %, //, **)

Arithmetic operators are used to perform mathematical calculations.

1. Addition (+) → Adds two values.

- a = 10
- b = 5
- print(a + b) # 15

2. Subtraction (-) → Subtracts the right operand from the left.

- print(a - b) # 5

3. Multiplication (*) → Multiplies values.

- print(a * b) # 50

4. Division (/) → Divides values and returns a float.

- print(a / b) # 2.0

5. Modulus (%) → Returns the remainder.

- print(a % b) # 0
- print(10 % 3) # 1

6. Floor Division (//) → Returns the quotient without decimals.

- print(10 // 3) # 3

7. Exponentiation () → Raises the left operand to the power of the right operand.**

- print(2 ** 3) # 8

These operators are frequently used in numerical computations, simulations, and real-world calculations like billing systems or scientific tasks.

5.2 Relational/Comparison Operators (==, !=, >, <, >=, <=)

Comparison operators compare values and return True or False.

1. Equal (==):

- `print(5 == 5) # True`
- `print(5 == 6) # False`

2. Not Equal (!=):

- `print(10 != 7) # True`

3. Greater Than (>):

- `print(8 > 3) # True`

4. Less Than (<):

- `print(3 < 2) # False`

5. Greater or Equal (>=):

- `print(5 >= 5) # True`

6. Less or Equal (<=):

- `print(4 <= 6) # True`

Comparison operators are widely used in conditional statements, loops, and decision-making.

5.3 Logical Operators (and, or, not)

Logical operators are used to combine conditional statements. They work with boolean values (True or False).

1. and → Returns True only if both conditions are true.

- `print(5 > 2 and 10 > 3) # True`
- `print(5 > 2 and 10 < 3) # False`

2. or → Returns True if at least one condition is true.

- `print(5 > 2 or 10 < 3) # True`

3. not → Reverses the truth value.

- `print(not (5 > 2)) # False`

Logical operators are commonly used in authentication systems, filters, and decision-making algorithms.

5.4 Assignment Operators (=, +=, -=, *=, etc.)

Assignment operators assign values to variables, often combined with arithmetic operations.

1. = (Simple assignment):

- `x = 10`

2. += (Add and assign):

- `x += 5` # `x = x + 5`

3. -= (Subtract and assign):

- `x -= 3` # `x = x - 3`

4. *= (Multiply and assign):

- `x *= 2`

5. /= (Divide and assign):

- `x /= 4`

6. //= (Floor divide and assign), %= (Modulus and assign), = (Exponentiate and assign) also exist.

These operators save time and make code cleaner, especially in loops.

5.5 Bitwise Operators (&, |, ^, ~, <<, >>)

Bitwise operators work at the binary level, manipulating bits of integers.

1. & (AND) → Performs bitwise AND.

- `print(6 & 3)` # 2

2. | (OR) → Performs bitwise OR.

- `print(6 | 3)` # 7

3. ^ (XOR) → Performs bitwise exclusive OR.

- `print(6 ^ 3)` # 5

4. ~ (NOT) → Inverts bits.

- `print(~6)` # -7

5. << (Left shift) → Shifts bits left.

- `print(6 << 1)` # 12

6. >> (Right shift) → Shifts bits right.

- `print(6 >> 1)` # 3

Bitwise operators are heavily used in low-level programming, cryptography, compression, and optimisation.

5.6 Membership Operators (in, not in)

These operators check for membership in sequences like strings, lists, tuples, sets, or dictionaries.

1. in:

- `print("a" in "apple") # True`

2. not in:

- `print(5 not in [1, 2, 3]) # True`

They are widely used in searching and filtering tasks.

5.7 Identity Operators (is, is not)

Identity operators compare memory locations of objects, not values.

1. is: Returns True if two variables point to the same object.

- `x = [1, 2, 3]`
- `y = x`

2. print(x is y) # True

- `is not`: Returns True if they are different objects.

3. z = [1, 2, 3]

- `print(x is not z) # True` (different objects, even if values match)

5.8 Operator Precedence and Associativity

When multiple operators appear in an expression, Python follows precedence rules to decide the order.

Precedence order (high to low):

1. Parentheses `()`
2. Exponentiation `**`
3. Unary `+`, `-`, `~`
4. Multiplication `*`, Division `/`, Floor Division `//`, Modulus `%`
5. Addition `+`, Subtraction `-`
6. Bitwise shifts `<<`, `>>`
7. Bitwise AND `&`
8. Bitwise XOR `^`
9. Bitwise OR `|`
10. Comparison operators `(==, !=, <, >, <=, >=)`

11. Logical operators (not, and, or)
12. Assignment operators
13. Membership (in, not in) and Identity (is, is not)

Associativity

- Left to right for most operators.
- Right to left for exponentiation (**).

Example:

```
print(2 ** 3 ** 2) # 512 (evaluated as 2 ** (3 ** 2))
```

SELF-ASSESSMENT QUESTIONS – 4

Column A

Match The Following	
16	%
17	==
18	And
19	+=
20	is

Column B

Match The Following	
16	Comparison operator
17	Logical AND operator
18	Identity operator
19	Compound assignment operator
20	Modulus operator

6. SUMMARY

- **Expressions in Python** are combinations of values, variables, and operators that Python evaluates to produce results.
- **Literals and Constants** provide fixed values; literals are directly written values like 5 or "hello", while constants are named values written in uppercase.
- **Simple expressions** involve a single value/variable, while compound expressions combine operators, functions, or multiple values.
- **Evaluation of expressions** depends on Python's rules of precedence and associativity, ensuring consistent results.
- **Built-in Data Types** in Python include numeric, sequence, mapping, set, boolean, and NoneType, each serving unique storage and computational roles.
- **Type Conversion** allows conversion between data types, either implicitly (automatic) or explicitly (manual casting).
- **Checking data types** is done using `type()` and `isinstance()` for debugging and validation.
- Variables act as names pointing to values in memory, with flexible assignment and reassignment rules.
- **Operators in Python** include arithmetic, relational, logical, assignment, bitwise, membership, and identity operators for a wide variety of tasks.
- **Operator precedence and associativity** ensure expressions are evaluated in a predictable order, with parentheses available to override defaults

7. GLOSSARY

Expression	- : A combination of values, variables, and operators that produces a result.
Literal	- A fixed raw value, e.g., 42, "Python".
Constant	- A variable intended to remain unchanged, usually in uppercase (PI = 3.14).
Simple Expression	- An expression with one value or variable, e.g., 7.
Compound Expression	- An expression with multiple operators or function calls, e.g., (a + b) * c.
Data Type	- A category of values that defines storage and operations (int, float, str, etc.).
Type Casting	- Conversion of one data type into another.
Variable	- A named reference to a value stored in memory.
Operator	- A symbol performing an operation, e.g., +, ==.
Precedence	- The order in which operators are applied in an expression.

8. TERMINAL QUESTIONS

1. What is an expression in Python, and how does it differ from a statement?
2. Differentiate between literals and constants with examples.
3. Explain simple and compound expressions with examples.
4. What are Python's built-in numeric data types, and where are they used?
5. Differentiate between mutable and immutable data types with examples.
6. What is the difference between implicit and explicit type conversion?
7. State the rules for naming variables in Python.
8. What is the difference between `is` and `==` operators in Python?
9. Explain the role of assignment operators with examples.
10. What is operator precedence, and why is it important in Python?

9. ANSWERS

9.1. Self-Assessment Answers:

1. d) Both b and c
2. b) 14
3. d) PI
4. b) $2 ** (3 ** 2)$
5. c) It will raise a `TypeError`
6. int, float, complex
7. immutable
8. dict (dictionary)
9. `type()`
10. `frozenset`
11. False
12. False
13. True
14. False (They can technically be changed, but by convention should not be.)
15. True
16. $1 \rightarrow e$ (Modulus operator)
17. $2 \rightarrow a$ (Comparison operator)
18. $3 \rightarrow b$ (Logical AND operator)
19. $4 \rightarrow d$ (Compound assignment operator)
20. $5 \rightarrow c$ (Identity operator)

9.2. Terminal Question Answers

Answer 1: An expression in Python is a valid combination of values, variables, and operators that produces a result. For example, $3 + 5$ evaluates to 8. A statement, on the other hand, is a complete instruction that Python executes, such as `x = 10` or a for loop. Expressions always return a value, while statements may perform actions like assignment or flow control. Importantly, expressions can exist inside statements, but not every statement is an expression.

Answer 2: Literals are raw, fixed values directly written in code, such as 42 (integer), 3.14 (float), or "Python" (string). Constants, however, are variables meant to store values that should not change during program execution. Python does not enforce constants, but by convention, they are written in uppercase like `PI = 3.14159`. While literals are inherently unchangeable, constants can technically be reassigned but should remain unchanged to maintain clarity and consistency in programs.

Answer 3: A simple expression contains a single value or variable, such as 5, "Hello", or x. These evaluate directly to themselves. A compound expression combines values, variables, and operators to form a more complex result. For example, `(3 + 5) * 2` or `len("Python") + 4` are compound expressions. Python evaluates them according to operator precedence and associativity rules. Simple expressions are basic building blocks, while compound ones allow complex calculations and logic in programming.

Answer 4: Python provides three numeric data types: int, float, and complex. Integers represent whole numbers like 100 or -25. Floats handle numbers with decimal points such as 3.14, often used in financial and scientific calculations. Complex numbers include both real and imaginary parts, written as `a + bj`, for example `3 + 4j`. They are mainly applied in engineering and advanced mathematics. Together, these numeric data types allow Python to support everything from basic arithmetic to complex simulations.

Answer 5: Mutable data types can be changed after creation, while immutable ones cannot. Examples of mutable types include lists, dictionaries, and sets—you can add, remove, or modify elements in place. For instance, `lst = [1, 2]; lst.append(3)` modifies the list directly. Immutable types include integers, strings, and tuples. For example, modifying a string creates a new object: `s = "Hi"; s = s + " there"`. Understanding mutability helps developers write safer and more predictable code.

Answer 6: Implicit type conversion, or type promotion, occurs automatically when Python converts one type into another for safe operations. For example, in `5 + 2.5`, the integer 5 is promoted to a float, and the result is 7.5. Explicit type conversion, or type casting, requires the programmer to manually convert using functions like `int()`, `float()`, or `str()`. For example, `int("100")` converts the string "100" into an integer. Explicit conversion provides control, while implicit makes operations seamless.

Answer 7: Variable naming in Python must follow strict rules. Names should begin with a letter or underscore, never with a digit. They may only contain letters, digits, and underscores. Variables are case-sensitive, so `age` and `Age` are different identifiers. Reserved keywords such as `for`, `class`, or `True` cannot be used as variable names. Developers should also choose meaningful names like

student_score instead of ss. Following these rules ensures readability, avoids errors, and maintains consistent coding standards.

Answer 8: The == operator compares the values of two objects, while the is operator checks if two variables point to the same object in memory. For example, two separate lists [1, 2, 3] may have equal values (== returns True) but reside at different memory locations (is returns False). In contrast, immutable objects like small integers may share the same memory reference, making both is and == return True. Correct usage prevents subtle logic errors in programs.

Answer 9: Assignment operators are used to assign values to variables and can also combine arithmetic operations with assignment. The simple = operator assigns a value, like x = 10. Compound assignment operators update and reassign the variable in one step: x += 5 is equivalent to x = x + 5, x *= 2 doubles the value, and x //= 3 performs floor division. These operators simplify code, reduce redundancy, and are especially useful in iterative or loop-based operations.

Answer 10: Operator precedence defines the order in which operations are carried out in an expression. For example, in 3 + 5 * 2, multiplication has higher precedence than addition, so the result is 13 instead of 16. Associativity rules resolve cases where two operators share the same precedence, such as evaluating + and - from left to right. Understanding precedence ensures accurate calculations. Programmers can use parentheses () to make evaluation explicit and avoid ambiguity.

10. REFERENCES

- Python Software Foundation. Python 3 Documentation – Data Types. Available at: <https://docs.python.org/3/library/stdtypes.html>
- Python Software Foundation. Python 3 Documentation – Expressions and Operators. Available at: <https://docs.python.org/3/reference/expressions.html>
- Lutz, M. (2013). Learning Python (5th Edition). O'Reilly Media.
- Downey, A. (2015). Think Python: How to Think Like a Computer Scientist (2nd Edition). Green Tea Press.
- Sweigart, A. (2019). Automate the Boring Stuff with Python (2nd Edition). No Starch Press.
- Zelle, J. (2009). Python Programming: An Introduction to Computer Science (2nd Edition). Franklin, Beedle & Associates.
- Guttag, J. (2016). Introduction to Computation and Programming Using Python (2nd Edition). MIT Press.
- Martelli, A. et al. (2017). Python in a Nutshell (3rd Edition). O'Reilly Media.
- Shaw, Z. A. (2017). Learn Python the Hard Way (3rd Edition). Addison-Wesley.
- Beazley, D. (2009). Python Essential Reference (4th Edition). Addison-Wesley.

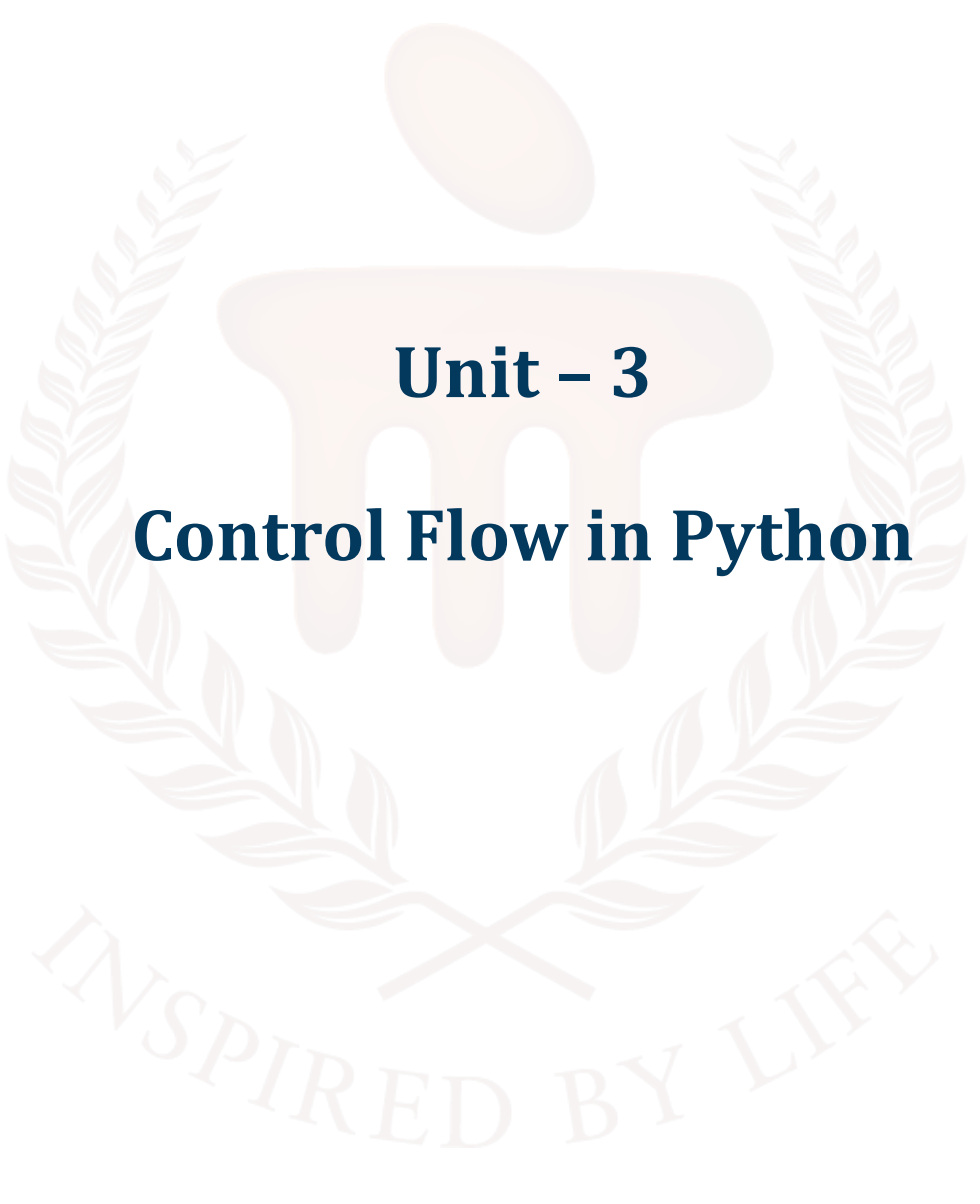
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 3

Control Flow in Python

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 – 6
1.1	Learning Objectives	-	-	
2	Conditional Statements In Python	-	-	7 – 32
2.1	The Concept of Decision Making in Programming	-	-	
2.2	The if Statement	-	-	
2.3	if...else Statement	-	-	
2.4	Nested if Statements	-	-	
2.5	if...elif...else Ladder	-	-	
2.6	Practical Examples of Conditional Statements	-	1	
3	Looping Constructs In Python	-	-	33-42
3.1	Importance of Loops in Programming	-	-	
3.2	for Loop and Its Syntax	-	-	
3.3	while Loop and Its Syntax	-	-	
3.4	Loop Control Statements	-	-	
3.5	Nested Loops and Practical Use Cases	-	-	
3.6	Iteration with Built-in Functions	-	-	
3.7	else Statement with Loops	-	2	
4	Combining Conditional And Looping Constructs	-	-	43-48
4.1	Using Conditional Statements Inside Loops	-	-	
4.2	Loop Optimisation and Efficiency	-	-	
4.3	Real-World Examples	-	3	
5	Summary	-	-	49
6	Glossary	-	-	50
7	Terminal Questions	-	-	51
8	Answers	-	-	52-54

	8.1	Self-Assessment Answers	-	-	
	8.2	Terminal Questions Answers	-	-	
9		References	-	-	55



1. INTRODUCTION

In this unit, learners will explore the fundamental concept of control flow in Python, which determines how a program's statements are executed based on conditions and logic. They will understand that, without control flow, Python would simply execute code sequentially, limiting flexibility. Through this unit, learners will gain insight into how control flow empowers programs to make decisions, repeat actions, and exit loops when specific conditions are met. By learning conditional statements like `if`, `elif`, and `else`, students will see how programs can choose between different actions based on logical evaluations—for instance, verifying if a bank account has sufficient balance before processing a transaction.

Additionally, learners will explore looping constructs such as `for` and `while` loops, which allow for the repeated execution of code blocks until a condition is fulfilled. These constructs help avoid code repetition and make it easier to automate tasks such as iterating through a list of items, processing records, or performing calculations over large datasets. The unit also highlights how conditionals and loops often work together, enabling developers to write more powerful, efficient, and intelligent code.

By the end of this unit, learners will understand how control flow contributes to problem-solving and algorithm development in Python. They will be able to create programs that are not only efficient and readable but also capable of responding dynamically to various real-world scenarios. Mastery of control flow constructs lays the groundwork for more advanced topics like data structures, file handling, and object-oriented programming. This unit helps learners build a strong foundation in Python, enhancing both their programming logic and their ability to develop applications that are practical, responsive, and scalable in nature.

1.1. Learning Objectives

- Explain the importance of control flow.
- Use conditional statements (if, elif, else) to guide decision-making in Python programs.
- Apply looping constructs (for and while) to perform repetitive tasks efficiently.
- Implement real-world scenarios using nested conditionals and nested loops.
- Differentiate between entry-controlled (while) and count-controlled (for) loops.



2. CONDITIONAL STATEMENTS IN PYTHON

2.1 The Concept of Decision Making in Programming

When you start learning programming, one of the most important realisations is that computers follow instructions in a very literal way. By default, a program executes statements one after another in a sequence. While this is useful for simple tasks, most real-world problems cannot be solved by executing everything in the same order every time. Programs must be able to make decisions—to select one path of execution when certain conditions are true, and another path when those conditions are false. This ability is what makes a program intelligent, flexible, and useful in practice.

Decision making in programming means evaluating one or more conditions and then executing a block of code depending on the result of those conditions. In Python, these conditions are generally Boolean expressions, which return either True or False. For example, when you check if a number is greater than another, Python evaluates the statement and produces a truth value. Based on this truth value, your program can take one path or another.

To understand this more clearly, consider a simple real-life example: when you use an ATM, the machine must decide whether to allow your withdrawal. It checks conditions such as:

- Is the entered PIN correct?
- Is the balance sufficient?
- Does the requested amount meet the withdrawal rules?

If all these conditions evaluate to True, then the ATM dispenses cash. If even one of them evaluates to False, the ATM must follow a different path (for example, displaying an error message). Without conditional logic, such decision making would not be possible.

In Python, conditions are expressed using comparison operators (`==`, `!=`, `<`, `>`, `<=`, `>=`) and logical operators (`and`, `or`, `not`). Together, they allow you to form complex rules that control the flow of execution.

Here is a small example:

```
temperature = 36.8
```

```
if temperature >= 37.5:
```

```
    print("High temperature detected")
```

code logic:

1. A variable temperature is assigned the value 36.8.
2. The condition temperature ≥ 37.5 checks whether the temperature is at least 37.5.
3. Since 36.8 is less than 37.5, the condition evaluates to False.
4. Because the condition is False, the indented print statement is skipped. Nothing is printed.

If the value of temperature were changed to 38.0, the condition would become True, and the message “High temperature detected” would appear. This shows how decision making allows programs to react to data and not just execute instructions blindly.

In practice, decision making is essential in almost every application:

- Websites use conditions to check if a user is logged in before showing personalized pages.
- Online shopping platforms use conditions to apply discounts if a coupon code is valid.
- Security systems use conditions to allow or deny access depending on credentials.

By mastering decision making, you gain the power to build programs that respond intelligently to inputs and real-world situations.

2.2 The if Statement

The if statement is the simplest and most fundamental conditional construct in Python. It allows you to check whether a condition is true, and if it is, Python executes a block of code. If the condition is false, Python skips that block and moves on to the next instruction in the program.

Think of the if statement as a gatekeeper. The condition acts as the key: if the key fits (the condition is true), you pass through and execute the code inside. If the key does not fit (the condition is false), the gate stays closed and the code is ignored.

Syntax of the if statement

if condition:

block of code

- The keyword if introduces the condition.
- The condition must be a Boolean expression (something that evaluates to True or False).
- A colon (:) is required at the end of the condition.
- The block of code that runs when the condition is true must be indented (typically 4 spaces in Python).

Example 1: Checking if a number is positive

```
num = 10
```

```
if num > 0:
```

```
    print("The number is positive")
```

logic:

1. The variable num is assigned the value 10.
2. The condition num > 0 is evaluated. Since 10 is greater than 0, the condition is True.
3. Because the condition is True, Python executes the indented code and prints:
4. The number is positive
5. If num had been -5, the condition num > 0 would be False, and the program would skip the print statement, resulting in no output.

This shows how the if statement allows your program to run code only when a specific condition is satisfied.

Example 2: Checking if a list is not empty

```
items = ["pen", "book"]
```

```
if items:
```

```
    print("The shopping cart has items")
```

logic:

1. A list called items is created with two elements: "pen" and "book".
2. In Python, non-empty lists evaluate to True, while empty lists evaluate to False.
3. Since items is non-empty, the condition inside the if statement is considered True.
4. Therefore, the program prints:
5. The shopping cart has items
6. If items = [] (an empty list), the condition would be False, and the message would not print.

This example demonstrates the concept of truthy and falsy values in Python, where not just booleans but also numbers, strings, lists, and other objects can determine the flow of execution.

Example 3: Guarding against division by zero

```
numerator = 20
```

```
denominator = 0

if denominator != 0:

    result = numerator / denominator

    print("Result:", result)

print("Operation complete")
```

logic:

1. The numerator is 20, and the denominator is 0.
2. The condition `denominator != 0` checks whether the denominator is not equal to zero.
3. Since the denominator is 0, the condition evaluates to False.
4. The block inside the if statement (division and print) is skipped, so no division occurs.
5. The program continues to the final line and prints:
6. Operation complete

Without the if statement, this code would have caused a runtime error (`ZeroDivisionError`). The if statement prevents the program from crashing by checking the condition first. This is known as a guard clause—it protects the program from invalid operations.

Key Points to Remember

1. The if statement executes its block only when the condition is true.
2. Indentation is crucial in Python. If you do not indent properly, you will get errors.
3. Any expression that evaluates to True or False can be used as a condition.
4. Non-Boolean values also have truth values in Python:
 - Numbers: 0 is False, any non-zero number is True.
 - Strings: empty string "" is False, non-empty strings are True.
 - Lists, tuples, dictionaries, sets: empty ones are False, non-empty ones are True.
5. If the condition is false, Python simply skips the block and continues with the rest of the program.

The if statement is the foundation of all other conditional statements in Python. Once you understand it, you can move to more complex forms like if...else, nested if, and the if...elif...else ladder.

2.3 if...else Statement

The if...else statement in Python is used when there are two possible paths of execution. The program checks a condition:

- If the condition is True, the code inside the if block runs.
- If the condition is False, the code inside the else block runs instead.

This is different from a simple if statement, where the code only runs if the condition is true and does nothing otherwise. With if...else, you ensure that one of two blocks will always execute.

Think of if...else like a two-way switch. If the condition is satisfied, the switch directs the flow down the first path (if block). If not, it automatically goes down the second path (else block).

Syntax of if...else

if condition:

```
# code block when condition is True
```

else:

```
# code block when condition is False
```

Example 1: Even or Odd number check

```
num = 7
```

```
if num % 2 == 0:
```

```
    print("Even number")
```

```
else:
```

```
    print("Odd number")
```

Step-by-step explanation:

1. The variable num is given the value 7.
2. The condition num % 2 == 0 checks if the remainder when dividing by 2 is zero.
 - For 7, the remainder is 1, so the condition is False.
3. Because the condition is False, the program skips the if block and executes the else block.
4. The output is:
5. Odd number

If num were 12, the condition would be True, and the program would print Even number.

This is a very common use of if...else: handling binary (two-option) situations.

Example 2: Age-based access

```
age = 15
```

```
if age >= 18:
```

```
    print("Access granted")
```

```
else:
```

```
    print("Access denied")
```

Step-by-step explanation:

1. The program checks whether age is greater than or equal to 18.
2. For age 15, the condition age >= 18 evaluates to False.
3. Therefore, the else block runs, and the output is:
4. Access denied
5. If age were 20, the condition would be True, and the output would be:
6. Access granted

This shows how an if...else decision can be used to enforce rules, such as restricting underage users from accessing certain services.

Example 3: Checking if a number is positive or negative

```
num = -4
```

```
if num >= 0:
```

```
    print("The number is non-negative")
```

```
else:
```

```
    print("The number is negative")
```

Step-by-step explanation:

1. The value of num is -4.
2. The condition num >= 0 is evaluated. Since -4 is less than 0, the condition is False.
3. Python skips the if block and executes the else block.
4. Output:

5. The number is negative

If num = 0, the condition would be True, and it would print The number is non-negative.

Example 4: Login authentication (basic simulation)

```
username = "admin"
```

```
password = "1234"
```

```
if username == "admin" and password == "1234":
```

```
    print("Login successful")
```

```
else:
```

```
    print("Login failed")
```

Step-by-step explanation:

1. The variables username and password are compared to stored credentials.
2. The condition checks whether both match exactly.
3. If both values are correct, the condition is True, and the message Login successful is displayed.
4. Otherwise, the else block runs, printing Login failed.

This demonstrates how conditions and logical operators (and in this case) can be combined with if...else to perform real-world tasks.

Example 5: Using conditional expression (short form of if...else)

Python also supports a ternary-like conditional expression, which allows you to assign a value based on a condition in a single line:

```
num = 8
```

```
result = "Even" if num % 2 == 0 else "Odd"
```

```
print(result)
```

Explanation:

1. The expression before the if is returned if the condition is True.
2. The expression after else is returned if the condition is False.
3. Since $8 \% 2 == 0$ is True, the variable result is assigned "Even".
4. Output:
5. Even

While this is shorter, it is recommended for simple cases only. For more complex logic, a full if...else block improves readability.

Key Points to Remember

1. if...else ensures that one of two possible paths will always execute.
2. The condition must evaluate to True or False.
3. You can combine conditions using logical operators like and, or, and not.
4. Indentation matters—make sure both the if and else blocks are indented correctly.
5. The else block is optional, but if present, it must always be paired with an if.

The if...else structure is extremely powerful for handling binary choices. However, real-world problems often involve multiple possibilities, not just two. For such cases, Python provides nested if statements and the if...elif...else ladder, which we will discuss next.

2.4 Nested if Statements

A nested if statement means putting one if statement inside another. The inner if is only checked if the outer if condition is true. This allows you to build hierarchical decision-making structures where one test leads to another.

Think of it like a security check at an airport:

1. First, you must show your ticket to enter the terminal (outer condition).
2. Once inside, you go through the baggage screening (inner condition).
3. If you pass baggage screening, you proceed to identity verification (another nested check).

At each stage, the next check only happens if you cleared the previous one.

Syntax of Nested if

if condition1:

 if condition2:

 # code block when both condition1 and condition2 are True

 else:

 # code block when condition1 is True but condition2 is False

else:

 # code block when condition1 is False

Example 1: Grading system

```
score = 85
```

```
if score >= 40:
```

```
    print("Pass")
```

```
    if score >= 75:
```

```
        print("Distinction")
```

```
    elif score >= 60:
```

```
        print("First class")
```

```
    elif score >= 50:
```

```
        print("Second class")
```

```
    else:
```

```
        print("Third class")
```

```
else:
```

```
    print("Fail")
```

logic:

1. The program first checks if `score >= 40`. Since the score is 85, this condition is True, so the outer block runs.
2. Inside the outer block, another series of `if...elif...else` statements classify the student's grade.
 - Since 85 is greater than or equal to 75, the first condition inside is True, and it prints:
 - Pass
 - Distinction
3. If the score had been 35, the outer `if` would be False, and the program would skip all inner checks, printing only:
4. Fail

This example shows how nested `if` ensures that grade classification is only considered if the student has already passed.

Example 2: Online shopping discount eligibility

```
cart_value = 1200
```

```
membership = True

if cart_value >= 1000:

    if membership:

        print("You get a 20% discount")

    else:

        print("You get a 10% discount")

else:

    print("No discount available")
```

logic:

1. The program first checks whether the `cart_value` is at least 1000.
 - For 1200, the condition is True, so the program enters the outer block.
2. Inside, it checks whether the customer has membership.
 - Since `membership = True`, the program executes the first inner branch and prints:
 - You get a 20% discount
3. If the customer did not have membership, it would print:
4. You get a 10% discount
5. If the cart value were less than 1000, the outer else would trigger:
6. No discount available

This example illustrates how nested if handles multi-level decisions, where one condition must be true before checking the next.

Example 3: ATM withdrawal rules

```
balance = 2500

amount = 1500

daily_limit = 2000

if amount <= daily_limit:

    if amount <= balance:

        print("Transaction approved")

    else:
```

```
print("Insufficient balance")
```

else:

```
print("Exceeds daily withdrawal limit")
```

logic:

1. The outer condition checks whether the withdrawal amount is within the daily limit.
 - For 1500, the condition is True, so the program continues inside.
2. The inner condition checks whether the account balance is enough.
 - Since the balance is 2500, which is greater than 1500, the condition is True.
3. The program prints:
4. Transaction approved
5. If the amount was 3000, the outer condition would be False, and it would directly print:
6. Exceeds daily withdrawal limit

This scenario shows how nesting ensures rules are checked in order of importance.

Example 4: Login system with two checks

```
username = "alice"
```

```
password = "mypassword"
```

```
stored_username = "alice"
```

```
stored_password = "mypassword"
```

```
if username == stored_username:
```

```
    if password == stored_password:
```

```
        print("Login successful")
```

```
    else:
```

```
        print("Incorrect password")
```

```
else:
```

```
    print("Unknown username")
```

logic:

1. The program first checks if the username matches the stored username.

2. If it matches, the program then checks if the password is also correct.
3. If both conditions are true, the program prints:
4. Login successful
5. If the username is wrong, the outer else executes directly without even checking the password.

This makes the login process secure, because checking the password only makes sense if the username is valid.

Important Points about Nested if

1. You can place one if inside another as deeply as needed, but too much nesting makes code hard to read.
2. Nested if is useful when one condition depends on another (like eligibility checks).
3. If conditions are independent alternatives, an if...elif...else ladder (discussed in the next section) is usually better.
4. Always maintain proper indentation so it is clear which block belongs to which condition.

Nested if statements help structure multi-step decision making, but for many real-world problems, you often have multiple parallel conditions rather than hierarchical ones. To handle that more cleanly, Python provides the if...elif...else ladder, which we will now explore.

2.5 if...elif...else Ladder

The if...elif...else ladder is one of the most powerful and commonly used structures in Python for decision-making. It is used when there are multiple possible conditions, and you want to execute exactly one block of code based on which condition is true.

With a simple if...else, you can only choose between two outcomes. But in the real world, you often need to choose from many. For example:

- Deciding a grade (A, B, C, D, F) based on marks.
- Categorising age groups (child, teenager, adult, senior).
- Classifying air quality index into multiple levels.

In such cases, an if...elif...else ladder is the most appropriate solution.

Syntax of if...elif...else

if condition1:

 # code block 1

elif condition2:

 # code block 2

elif condition3:

 # code block 3

...

else:

 # code block when none of the above are True

- The program checks condition1 first. If it is True, block 1 executes, and the rest are ignored.
- If condition1 is False, the program checks condition2. If that is True, block 2 executes, and the ladder stops.
- This continues down the chain until one condition is True.
- If none of the conditions are True, the else block (if present) is executed.

The key idea is that only the first matching block runs.

Example 1: Student grading system

marks = 72

if marks >= 90:

 grade = "A"

elif marks >= 75:

 grade = "B"

elif marks >= 60:

 grade = "C"

elif marks >= 40:

 grade = "D"

else:

 grade = "F"

print("Grade:", grade)

Step-by-step explanation:

1. The variable marks has the value 72.
2. The first condition checks marks ≥ 90 . This is False, so Python moves to the next condition.
3. The second condition checks marks ≥ 75 . This is also False.
4. The third condition checks marks ≥ 60 . Since 72 is greater than 60, this condition is True.
5. The block under this condition executes, assigning "C" to grade.
6. The ladder stops at the first True condition, so the program does not check further.
7. The output is:
8. Grade: C

If the marks were 38, none of the conditions would be True, and the program would run the else block, assigning grade "F".

Example 2: Air Quality Index classification

```
aqi = 185
```

```
if aqi <= 50:
```

```
    category = "Good"
```

```
elif aqi <= 100:
```

```
    category = "Satisfactory"
```

```
elif aqi <= 200:
```

```
    category = "Moderate"
```

```
elif aqi <= 300:
```

```
    category = "Poor"
```

```
elif aqi <= 400:
```

```
    category = "Very Poor"
```

```
else:
```

```
    category = "Severe"
```

```
print("Air Quality Category:", category)
```

Step-by-step explanation:

1. The value of aqi is 185.
2. Python checks conditions from top to bottom.
 - aqi <= 50 → False
 - aqi <= 100 → False
 - aqi <= 200 → True
3. Since the third condition is True, the program sets category to "Moderate".
4. The program skips all remaining conditions because one branch has already been executed.
5. Output:
6. Air Quality Category: Moderate

This is an example of why ordering matters in an if...elif...else ladder. If the conditions were placed in the wrong order, the result might not match the real-world logic.

Example 3: Fee calculation based on user type

```
user_type = "guest"

if user_type == "premium":
    fee = 0
elif user_type == "student":
    fee = 100
elif user_type == "guest":
    fee = 200
else:
    fee = 300

print("Fee to pay:", fee)
```

Step-by-step explanation:

1. The value of user_type is "guest".
2. The program checks each condition:
 - user_type == "premium" → False
 - user_type == "student" → False
 - user_type == "guest" → True
3. The program executes this branch and sets fee = 200.

4. Since a branch has been executed, no further conditions are checked.
5. Output:
6. Fee to pay: 200

If the value of `user_type` were something unexpected like "visitor", none of the conditions would match, and the program would use the else branch, assigning `fee = 300`.

Example 4: Temperature classification

```
temperature = 34
if temperature < 0:
    print("Freezing")
elif temperature < 20:
    print("Cold")
elif temperature < 30:
    print("Warm")
elif temperature < 40:
    print("Hot")
else:
    print("Very Hot")
```

Step-by-step explanation:

1. The program evaluates the conditions one by one.
2. Since 34 is not less than 0, 20, or 30, the first three conditions are False.
3. The fourth condition `temperature < 40` is True.
4. The program executes that branch and prints:
5. Hot

This example shows how ranges can be expressed clearly with an `if...elif...else` ladder.

Example 5: Using multiple conditions with logical operators

```
marks = 68
attendance = 85
```

```
if marks >= 90 and attendance >= 80:
```

```
    result = "Excellent"
```

```
elif marks >= 60 and attendance >= 75:
```

```
    result = "Good"
```

```
elif marks >= 40 and attendance >= 60:
```

```
    result = "Satisfactory"
```

```
else:
```

```
    result = "Needs Improvement"
```

```
print(result)
```

Step-by-step explanation:

1. The program checks both marks and attendance together using and.
2. With marks = 68 and attendance = 85:
 - The first condition requires marks ≥ 90 , which is False.
 - The second condition requires marks ≥ 60 and attendance ≥ 75 , which is True.
3. The program executes the second block and assigns "Good".
4. Output:
5. Good

This demonstrates how if...elif...else can incorporate logical operators for more complex decision-making.

Key Points to Remember

1. The ladder checks conditions top to bottom. Once a True condition is found, the rest are skipped.
2. Ordering of conditions matters. Place the most restrictive or highest-priority conditions first.
3. Use elif for additional conditions, not multiple ifs. Using multiple ifs would check all conditions even after one matches, which is inefficient.
4. Always include an else block as a safety net when possible, to handle unexpected inputs.
5. Use if...elif...else when there are multiple mutually exclusive possibilities.

The if...elif...else ladder is often the best choice for structured decision-making. It keeps the code cleaner compared to deeply nested ifs and makes the logic easier to read.

2.6 Practical Examples of Conditional Statements

Up to this point, you have learned about if, if...else, nested if, and the if...elif...else ladder. To really understand these concepts, it is important to see how they apply to real-world situations. In this section, we will go through several practical examples that show how conditional statements help programs make meaningful decisions.

Example 1: Password Strength Checker

```
password = "Hello123!"  
  
if len(password) < 8:  
    print("Password too short")  
  
elif password.isalpha():  
    print("Weak password – only letters")  
  
elif password.isdigit():  
    print("Weak password – only digits")  
  
elif password.isalnum():  
    print("Moderate password – letters and digits only")  
  
else:  
    print("Strong password")
```

logic:

1. First, the program checks the length of the password. If it is less than 8, it is immediately marked as too short.
2. If the length is fine, the program checks if the password has only letters (isalpha). If so, it is weak.
3. If it has only digits (isdigit), it is also weak.
4. If it contains both letters and numbers but no special characters (isalnum), it is moderate.
5. If none of the above apply, the password must have a combination of letters, numbers, and special characters, so it is strong.

This example shows how conditions can be layered to create rules for security validation.

Example 2: Online Shopping Discount

```
cart_value = 850  
  
is_member = True  
  
if cart_value >= 1000:  
    discount = 20  
  
elif is_member and cart_value >= 500:  
    discount = 10  
  
else:  
    discount = 0  
  
print("Discount applied:", discount, "%")
```

logic:

1. If the cart value is 1000 or more, the customer gets the highest discount of 20%.
2. If the customer is a member and the cart value is at least 500, a 10% discount is applied.
3. If neither condition applies, no discount is given.
4. With `cart_value = 850` and `is_member = True`, the second condition matches, so the discount is 10%.

This demonstrates combining logical operators (and) with conditionals to reflect real-world policies.

Example 3: ATM Withdrawal Validation

```
balance = 2000  
  
amount = 1800  
  
daily_limit = 1500  
  
if amount <= 0:  
    print("Invalid amount")  
  
elif amount % 100 != 0:  
    print("Amount must be in multiples of 100")
```



```
elif amount > daily_limit:
```

```
    print("Exceeds daily withdrawal limit")
```

```
elif amount > balance:
```

```
    print("Insufficient balance")
```

```
else:
```

```
    balance -= amount
```

```
    print("Withdrawal successful")
```

```
    print("Remaining balance:", balance)
```

logic:

1. The program checks the amount in a safe order:
 - It must be positive.
 - It must be a multiple of 100 (as ATMs usually work with such denominations).
 - It must not exceed the daily limit.
 - It must not exceed the account balance.
2. Only if all checks are passed does the withdrawal succeed, and the balance is updated.
3. With amount = 1800 and daily_limit = 1500, the third condition fails, so it prints:
4. Exceeds daily withdrawal limit

This shows how validation checks are structured carefully to prevent errors and ensure safety.

Example 4: Student Exam Result

```
marks = 64
```

```
if marks >= 90:
```

```
    result = "Outstanding"
```

```
elif marks >= 75:
```

```
    result = "Excellent"
```

```
elif marks >= 60:
```

```
    result = "Very Good"
```

```
elif marks >= 40:
```

```
    result = "Pass"

else:

    result = "Fail"

print("Result:", result)
```

logic:

1. Marks are Categorised into performance levels.
2. The ladder checks from highest to lowest, ensuring that only one condition matches.
3. With marks = 64, the third condition is True (marks $\geq$ 60), so the result is "Very Good".

This example shows how conditionals are useful for grading and classification problems.

Example 5: File Path Validation

```
import os

path = "data.txt"

if os.path.exists(path):
    if os.path.isfile(path):
        print("It is a file")
    elif os.path.isdir(path):
        print("It is a directory")
    else:
        print("It exists but is neither a file nor a directory")
else:
    print("Path not found")
```

logic:

1. The program uses the os module to check if a given path exists.
2. If it exists, the inner if checks whether it is a file or a directory.
3. If it is neither, a different message is shown.
4. If the path does not exist, the outer else runs.

This shows how nested if statements are useful when conditions must be checked in steps.

Example 6: Ticket Pricing System

```
age = 65
```

```
if age < 5:
```

```
    price = 0
```

```
elif age < 18:
```

```
    price = 100
```

```
elif age < 60:
```

```
    price = 200
```

```
else:
```

```
    price = 150
```

```
print("Ticket Price:", price)
```

logic:

1. Children under 5 travel free.
2. Children under 18 pay 100.
3. Adults under 60 pay 200.
4. Seniors (60 and above) pay 150.
5. With age = 65, the final else block applies, giving a price of 150.

This is a very practical use of the if...elif...else ladder in transport or event management systems.

Example 7: Risk Level Assessment

```
temperature = 38.5
```

```
symptoms = True
```

```
if symptoms:
```

```
    if temperature >= 38.0:
```

```
        print("High Risk")
```

```
    else:
```

```
        print("Moderate Risk")
```

else:

```
print("Low Risk")
```

logic:

1. The program first checks whether symptoms are present.
2. If symptoms exist, it then checks the temperature.
 - If it is 38.0 or higher, the risk is high.
 - Otherwise, it is moderate.
3. If there are no symptoms, the risk is automatically low.

This example demonstrates nested decision making in health assessments.

Example 8: Electricity Bill Calculation

```
units = 250
```

```
if units <= 100:
```

```
    bill = units * 5
```

```
elif units <= 200:
```

```
    bill = (100 * 5) + (units - 100) * 7
```

```
else:
```

```
    bill = (100 * 5) + (100 * 7) + (units - 200) * 10
```

```
print("Total Bill:", bill)
```

logic:

1. For the first 100 units, the cost is 5 per unit.
2. For the next 100 units, the cost is 7 per unit.
3. Any additional units are charged at 10 per unit.
4. With units = 250, the calculation goes to the third block:
 - First 100 → 500
 - Next 100 → 700
 - Remaining 50 → 500
 - Total = 1700
5. Output:

6. Total Bill: 1700

This demonstrates tiered conditions often seen in utility billing systems.

Key Learning from Practical Examples

1. Conditionals are at the heart of real-world applications like billing, authentication, grading, and validation.
2. The order of conditions matters a lot—write them from most restrictive or highest priority downwards.
3. Combining logical operators (and, or, not) increases flexibility.
4. Nested conditionals are useful when one check depends on another.
5. Use an else block wherever possible to handle unexpected cases gracefully.

SELF-ASSESSMENT QUESTIONS – 1

Multiple Choose Questions	
1	Which of the following statements about the if statement in Python is correct? a) The block of code under if executes only when the condition is True. b) The block of code under if executes even if the condition is False. c) The block of code under if always executes once. d) The if statement does not require indentation
2	What will be the output of the following code? <pre>x = 15 if x % 2 == 0: print("Even") else: print("Odd")</pre> a) Even b) Odd c) Error d) No output
3	In a nested if statement, when does the inner if condition get checked? a) Always, regardless of the outer condition.

	b) Only if the outer if condition is True. c) Only if the outer if condition is False. d) Only if there is no else block
4	Which of the following is the correct way to write an if...elif...else ladder? a) <pre>if condition1: # code elif condition2: # code else: # code</pre> b) <pre>if condition1: # code else if condition2: # code else: # code</pre> c) <pre>if condition1: # code elseif condition2: # code else: # code</pre> d) All of the above
5	Consider the following code: <pre>age = 20 if age < 13:</pre>

```
print("Child")  
elif age < 20:  
    print("Teenager")  
else:  
    print("Adult")  
a) Child  
b) Teenager  
c) Adult  
d) No output
```



3. LOOPING CONSTRUCTS IN PYTHON

3.1 Importance of Loops in Programming

Programming is all about giving computers a set of instructions. Sometimes, the same instruction needs to be repeated multiple times. For example, suppose you want to print numbers from 1 to 100. If you write `print(1)`, `print(2)`, and so on until `print(100)`, it would be repetitive, time-consuming, and prone to errors. Loops solve this problem.

A loop is a structure in programming that allows us to execute a block of code multiple times without writing it repeatedly. Instead, the program checks a condition or goes through a sequence and automatically repeats the required steps.

The importance of loops in programming can be summarized as follows:

1. Automation – repetitive tasks are handled automatically.
2. Efficiency – reduces the amount of code written, making programs shorter and easier to maintain.
3. Flexibility – works with different types of data like lists, strings, or numerical ranges.
4. Problem Solving – helps in implementing algorithms such as searching, sorting, or mathematical calculations.

For instance, if a teacher wants to calculate the average marks of 50 students, loops allow the program to read each student's score, add them, and then calculate the average without repeating the same instructions 50 times.

3.2 for Loop and Its Syntax

The for loop in Python is used when we want to iterate over a sequence such as a list, tuple, string, or range of numbers. It goes through each item in the sequence, one by one, until all items are covered.

Syntax of for Loop

for variable in sequence:

code block

- variable is a temporary name given to each element of the sequence during the loop.
- sequence is the collection of items (like a list or string).
- The code block under the loop runs once for every element.

Iterating Over Sequences

1. Lists

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:
```

```
    print(fruit)
```

Logic :

- Python starts with the first item "apple" and assigns it to fruit.
- It prints "apple".
- Then it moves to "banana" and "cherry".
- The loop ends after covering all elements.

2. Tuples

```
numbers = (10, 20, 30)
```

```
for num in numbers:
```

```
    print(num)
```

Here, the loop extracts each number from the tuple and prints it.

3. Strings

```
word = "PYTHON"
```

```
for char in word:
```

```
    print(char)
```

Each character of the string "PYTHON" is taken one by one and printed.

4. Range

```
for i in range(1, 6):
```

```
    print(i)
```

Logic:

- The function range(1, 6) generates numbers from 1 up to 5 (6 is excluded).

- The loop prints numbers 1, 2, 3, 4, 5.

Nested for Loops

A nested loop means having one loop inside another. This is useful when dealing with multiple levels of data, like printing a multiplication table.

```
for i in range(1, 4):  
    for j in range(1, 4):  
        print(i, j)
```

Logic:

- Outer loop runs with $i = 1$, then the inner loop executes fully with $j = 1, 2, 3$.
- Then $i = 2$, inner loop repeats again.
- Finally $i = 3$, inner loop runs once more.
- The result is pairs of numbers from (1,1) up to (3,3).

3.3 while Loop and Its Syntax

The while loop is used when we do not know beforehand how many times we need to repeat the code. Instead, the loop runs until a condition becomes false.

Syntax of while Loop

while condition:

```
# code block
```

- The loop continues as long as condition is True.
- Once it becomes False, the loop stops.

Loop with Condition

```
count = 1  
  
while count <= 5:  
    print("Count is:", count)  
    count += 1
```

Logic:

- Initially count = 1, condition count <= 5 is True.
- Prints "Count is: 1" and increases count by 1.
- The loop continues until count becomes 6, at which point the condition fails.

Infinite Loops

An infinite loop happens when the condition never becomes False. This can crash programs if not handled carefully.

while True:

```
    print("This loop never ends!")
```

Here, the condition True is always satisfied, so the loop never stops unless we manually interrupt the program (Ctrl+C).

Infinite loops are sometimes useful, such as when writing programs that constantly wait for user input.

3.4 Loop Control Statements

Sometimes, we need to control the behaviour of loops. Python provides three special statements for this purpose: break, continue, and pass.

break Statement

The break statement is used to stop the loop immediately, even if the condition or sequence is not fully completed.

```
for num in range(1, 10):
```

```
    if num == 5:
```

```
        break
```

```
    print(num)
```

Logic:

- Loop starts from 1.
- When num becomes 5, the break statement stops the loop.
- Output will be numbers 1 to 4 only.

continue Statement

The continue statement skips the current iteration and moves to the next one.

```
for num in range(1, 6):
```

```
    if num == 3:
```

```
        continue
```

```
    print(num)
```

Logic:

- Loop prints 1, 2.
- When num is 3, continue skips printing.
- Loop resumes with 4 and 5.
- Output is 1, 2, 4, 5.

pass Statement

The pass statement is a placeholder that does nothing. It is used when the syntax requires a statement but we don't want any action yet.

```
for num in range(1, 4):
```

```
    if num == 2:
```

```
        pass
```

```
    print(num)
```

Logic:

- When num is 2, nothing happens, but the loop continues normally.
- Output is 1, 2, 3.

3.5 Nested Loops and Practical Use Cases

Nested loops are powerful tools. They help in solving problems where multiple dimensions or repeated checks are needed.

Example: Multiplication Table

```
for i in range(1, 4):
```

```
    for j in range(1, 4):
```

```
print(i * j, end=" ")
```

```
print()
```

Logic:

- Outer loop chooses a row number.
- Inner loop multiplies that row number with each column number.
- `end=" "` ensures results appear side by side, while the outer `print()` moves to the next line.

Output:

```
1 2 3
```

```
2 4 6
```

```
3 6 9
```

Use Cases

1. Generating grids for games (like tic-tac-toe).
2. Processing matrices in mathematics.
3. Comparing elements across multiple lists.
4. Printing star patterns in programming exercises.

3.6 Iteration with Built-in Functions

Python provides built-in functions that simplify looping.

enumerate()

The `enumerate()` function is useful when we want both the index and the value while looping.

```
fruits = ["apple", "banana", "cherry"]
```

```
for index, fruit in enumerate(fruits):
```

```
    print(index, fruit)
```

Logic:

- `enumerate()` gives both the position (index) and the actual element.
- Output will be:

- 0 apple
- 1 banana
- 2 cherry

zip()

The zip() function allows looping over two or more sequences at the same time.

```
names = ["Alice", "Bob", "Charlie"]
```

```
scores = [85, 90, 95]
```

```
for name, score in zip(names, scores):
```

```
    print(name, "scored", score)
```

Logic:

- Pairs the first element of names with the first element of scores, and so on.
- Output:
- Alice scored 85
- Bob scored 90
- Charlie scored 95

This is especially useful when dealing with related datasets.

3.7 else Statement with Loops

In Python, loops (both for and while) can have an optional else part. This may look unusual at first, because many languages use else only with if. In loops, the else block is executed after the loop finishes its normal execution. The important point is that the else part runs only if the loop is not terminated by a break statement.

i.e.,

- If the loop completes all its iterations → else block runs.
- If the loop is stopped early using break → else block is skipped.

Syntax with for loop

```
for item in sequence:
```

```
    # loop body
```


if condition:

break

else:

this runs only if break never happened

Syntax with while loop

while condition:

loop body

if condition_to_stop_early:

break

else:

runs when while condition becomes False naturally

Example 1: Searching in a list using for-else

```
numbers = [10, 20, 30, 40, 50]
```

```
key = 30
```

```
for num in numbers:
```

```
    if num == key:
```

```
        print("Number found:", num)
```

```
        break
```

```
    else:
```

```
        print("Number not found")
```

Logic:

Loop checks each element in the list.

If key is found, it prints “Number found” and break stops the loop.

Because of break, else is skipped.

If the loop finishes without finding key, break is never executed, so else prints “Number not found”.

Example 2: while-else without break

```
count = 1

while count <= 3:

    print("Attempt", count)

    count += 1

else:

    print("All attempts completed")
```

Logic:

The while loop runs for count = 1, 2, 3.

When count becomes 4, the condition count <= 3 is False, so the loop ends normally.

Since there is no break, the else block runs and prints “All attempts completed”.

Example 3: while-else with break

```
count = 1

while count <= 3:

    print("Attempt", count)

    if count == 2:

        print("Stopping early")

        break

    count += 1

else:

    print("All attempts completed")
```

Here, when count becomes 2, the message “Stopping early” is printed and break stops the loop. Because the loop did not finish naturally, the else part does not execute.

This else with loops feature is very useful in search problems, where we want to perform one action if an item is found inside the loop, and a different action if it is not found after checking all elements.

SELF-ASSESSMENT QUESTIONS – 2

Fill in the blanks:	
6	The ____ loop is used to iterate over a sequence like a list or string.
7	A loop that never ends is called an ____ loop
8	The statement used to exit a loop immediately is called ____.
9	To get both index and value during iteration, we use the function ____.
10	A loop inside another loop is known as a ____ loop.



4. COMBINING CONDITIONAL AND LOOPING CONSTRUCTS

4.1 Using Conditional Statements Inside Loops

In Python programming, loops help us repeat tasks, while conditionals allow decision-making. When we combine them, we get more powerful programs. By placing if, if-else, or nested if statements inside loops, we can control what happens in each repetition. This combination lets us apply rules while iterating through data.

Example 1: Printing Only Even Numbers

```
for num in range(1, 11):
```

```
    if num % 2 == 0:
```

```
        print(num, "is Even")
```

Logic :

- Loop goes through numbers 1 to 10.
- The if condition checks if a number is divisible by 2.
- If yes, it prints the number as even.
- Odd numbers are skipped.

Example 2: Categorising Scores

```
scores = [35, 70, 90, 55]
```

```
for score in scores:
```

```
    if score >= 60:
```

```
        print(score, "Pass")
```

```
    else:
```

```
        print(score, "Fail")
```

Logic:

- Loop picks each score.
- Conditional checks if it is at least 60.
- Prints “Pass” if True, otherwise prints “Fail.”

This pattern is widely used in analysing datasets, validating input, or filtering required values.

4.2 Loop Optimisation and Efficiency

Loops are powerful but can become inefficient if not written carefully. Loop Optimisation means writing loops in a way that reduces time and memory usage, making the program faster.

Common Optimisation Techniques

1. Reduce Repeated Computations

Avoid recalculating values inside the loop that remain constant.

Less efficient

```
for i in range(1000):
```

```
    result = 5 ** 2
```

Optimised

```
result = 5 ** 2
```

```
for i in range(1000):
```

```
    pass
```

Here, the calculation $5 ** 2$ is done only once instead of 1000 times.

2. Use Built-in Functions

Python’s built-ins (sum(), max(), etc.) are faster than writing manual loops.

Manual sum

```
total = 0
```

```
for num in range(1, 101):
```

```
    total += num
```

```
print(total)
```

Optimised

```
print(sum(range(1, 101)))
```

3. Break Early When Possible

If you already found what you need, use `break` to exit the loop.

```
numbers = [10, 20, 30, 40]
```

```
for n in numbers:
```

```
    if n == 30:
```

```
        print("Found 30")
```

```
        break
```

4. Avoid Unnecessary Nested Loops

Nested loops increase complexity. Instead, try dictionary lookups or set operations when possible.

Why Optimisation Matters?

In small scripts, Optimisation may not seem important. But in real-world applications (like analysing millions of data records or running simulations), efficient loops save both time and computing power.

4.3 Real-World Examples

Now let us look at some practical uses of combining conditional and looping constructs.

1. Pattern Printing

Pattern problems are classic exercises in programming that combine loops and conditions.

Example: Right-Angled Triangle of Stars

```
rows = 5
```

```
for i in range(1, rows + 1):
```

```
    for j in range(1, i + 1):
```

```
        print("*", end=" ")
```

```
    print()
```

Logic:

- Outer loop controls the number of rows.
- Inner loop prints stars for each row.
- `end=" "` keeps stars on the same line.
- Conditional control isn't explicitly visible here, but the loop range itself acts as a condition to restrict star counts.

Output:

```
*  
* *  
* * *  
* * * *  
* * * * *
```

2. Menu-Driven Programs

In many applications, users are presented with menus. Loops ensure the program keeps running until the user decides to exit, and conditionals check which option the user selects.

while True:

```
print("Menu:")  
print("1. Add")  
print("2. Subtract")  
print("3. Exit")  
choice = int(input("Enter choice: "))  
if choice == 1:  
    print("Addition selected")  
elif choice == 2:  
    print("Subtraction selected")  
elif choice == 3:  
    print("Exiting...")
```



```
break
```

```
else:
```

```
    print("Invalid choice")
```

Logic:

- The loop repeats, showing the menu continuously.
- if-elif-else checks which option is chosen.
- The break statement ends the loop when the user selects 3.

This structure is the backbone of many console-based applications.

3. Summation Problems

Loops combined with conditions help solve mathematical problems like summing specific numbers.

Example: Sum of Even Numbers from 1 to 20

```
total = 0
```

```
for num in range(1, 21):
```

```
    if num % 2 == 0:
```

```
        total += num
```

```
print("Sum of even numbers:", total)
```

Logic:

- Loop checks numbers 1 to 20.
- Condition selects only even numbers.
- Each even number is added to total.
- Final result is the sum of all even numbers.

This technique is extended in finance, data analytics, and scientific calculations.

SELF-ASSESSMENT QUESTIONS – 3

Match the items in Column A with the correct description in Column B

Column A

Match The Following		
Column A		Column B
11	Using if inside a loop.	a) Exiting the loop as soon as the required condition is met
12	break statement	b) Combining stars or numbers in structured designs using loops
13	Pattern printing	c) Avoiding repeated work to make loops faster
14	Menu-driven program	d) Applying decisions on each item during iteration
15	Loop Optimisation	e) Presenting user choices and repeating until exit.

5. SUMMARY

- Conditional statements in Python allow decision-making based on logical conditions.
- The if statement executes a block only when the condition is True.
- The if...else structure enables programs to handle both True and False outcomes.
- Nested if statements allow multiple layers of decisions.
- The if...elif...else ladder handles multiple possible conditions.
- Loops (for, while) help automate repetitive tasks efficiently.
- The for loop iterates through sequences like lists, tuples, strings, and ranges.
- The while loop runs as long as its condition remains True.
- Loop control statements (break, continue, pass) modify the normal flow of loops.
- Nested loops are useful for handling multi-dimensional problems like patterns and matrices.
- Built-in functions like enumerate() and zip() simplify iteration.
- Combining loops with conditionals enables powerful solutions, such as menu-driven programs and summation problems.

6. GLOSSARY

Conditional Statement	- A programming structure that executes different blocks of code depending on whether a condition is True or False.
if Statement	- The simplest conditional in Python, executing a block only when a condition is True.
for Loop	- A loop that iterates over items in a sequence such as a list, tuple, or string.
while Loop	- A loop that runs repeatedly as long as a specified condition remains True.
break Statement	- Used to exit a loop before it has finished all iterations.
continue Statement	- Skips the current iteration and moves to the next loop cycle.
pass Statement	- A placeholder that does nothing, used when a statement is syntactically required but no action is needed.
Nested Loop	- A loop inside another loop, useful for patterns, matrices, and grid operations.
enumerate()	- A built-in function that allows iteration over a sequence with both index and value.
zip()	- A built-in function that pairs elements from two or more sequences.

7. TERMINAL QUESTIONS

1. What is the purpose of conditional statements in programming?
2. Differentiate between if and if...else statements.
3. Explain the use of nested if statements with an example.
4. What is the difference between a for loop and a while loop?
5. How does the break statement affect loops?
6. How does the continue statement work inside a loop?
7. What is the role of the pass statement in loops?
8. Why are nested loops important in practical programming?
9. How does the zip() function help in iteration?
10. Give a real-world example where conditionals and loops are applied together.

8. ANSWERS

8.1. Self-Assessment Answers:

1. a) The block of code under if executes only when the condition is True.
2. b) Odd
3. b) Only if the outer if condition is True.
4. a) (with elif)
5. c) Adult
6. for
7. infinite
8. break
9. enumerate()
10. nested
11. 1 → d) Applying decisions on each item during iteration
12. 2 → a) Exiting the loop as soon as the required condition is met
13. 3 → b) Combining stars or numbers in structured designs using loops
14. 4 → e) Presenting user choices and repeating until exit
15. 5 → c) Avoiding repeated work to make loops faster

8.2. Terminal Question Answers

Answer 1. Conditional statements guide program decisions by executing different code blocks depending on logical conditions. For instance, they allow validating user input, processing only valid entries, or rejecting invalid ones. Without conditionals, programs would run sequentially without flexibility. They form the basis of decision-making systems in software, ensuring tailored responses to varying inputs and making programs interactive, logical, and practical in real-world problem-solving.

Answer 2: The if statement checks a condition and executes a block if True, otherwise skips it. The if...else adds flexibility by handling both outcomes. For example, if age $\geq$ 18 prints “Adult”, but with if...else, it can print “Adult” if True and “Minor” if False. Thus, if alone works for one condition, while if...else ensures alternative paths for both possibilities, giving a complete decision-making mechanism in programs.

Answer 3: Nested if statements allow deeper decision-making where one condition leads to another check. For example, checking grades: if score ≥ 50 , then inside that condition check if score ≥ 75 for “Distinction,” else “Pass.” If the outer condition is False, the inner one is not evaluated. This structure is useful in multi-level decisions, such as eligibility checks, form validations, or Categorising data into multiple levels based on combined conditions.

Answer 4: A for loop is sequence-based and iterates over lists, tuples, strings, or ranges when the number of iterations is known. A while loop is condition-based and continues running until the condition becomes False, suitable when repetitions are unknown beforehand. For example, a for loop prints numbers 1 to 10, while a while loop keeps asking for user input until the correct password is entered. Thus, both serve different repetition scenarios.

Answer 5: The break statement immediately ends the loop, even if there are remaining iterations. It is particularly useful when the desired result is found early, preventing unnecessary processing. For example, in searching for a number within a list, once the number is found, break exits the loop. This saves execution time and makes programs more efficient by avoiding redundant work after the condition of interest has already been satisfied.

Answer 6: The continue statement skips the current iteration of a loop and moves directly to the next cycle. This is useful when certain elements should not be processed but the loop should continue. For example, printing numbers from 1 to 10 but skipping 5 requires using continue when the number equals 5. It ensures the loop runs normally while excluding specific cases. This provides selective control in repetitive tasks.

Answer 7: The pass statement acts as a placeholder that performs no action. It is often used when a code block is required syntactically but the developer does not want to execute any command at that point. For example, in a loop checking values, if a condition is still being designed, pass allows the program to run without errors. It helps during program development or when leaving sections intentionally blank for future updates.

Answer 8: Nested loops are vital for handling problems involving multiple dimensions or repeated checks within checks. For example, generating multiplication tables uses an outer loop for rows and an inner loop for columns. Similarly, nested loops are used in processing matrices, building grids in games, or printing patterns with stars. They extend the power of looping by enabling complex repetitive tasks that involve layered structures, multidimensional data, or structured outputs.

Answer 9: The `zip()` function pairs elements from two or more sequences, allowing parallel iteration. For example, combining student names and marks into pairs using `zip(names, marks)` simplifies processing both lists together. Instead of managing indexes manually, `zip()` directly provides related elements. This is useful in scenarios such as merging datasets, displaying related information, or iterating over multiple input sources simultaneously. It improves readability, reduces code complexity, and makes loops more efficient.

Answer 10: A common real-world example is an online shopping system where a loop iterates over items in a shopping cart. Inside the loop, conditionals check stock availability, apply discounts, or calculate shipping charges. The loop ensures every product is processed, while the conditions ensure correct decisions are applied to each case. Without this combination, the program would either repeat unnecessarily or fail to handle specific conditions, making shopping systems unreliable or inefficient.

9. REFERENCES

- Python Software Foundation. Python Official Documentation. Available at: <https://docs.python.org/3/>
- Lutz, M. (2013). Learning Python (5th Edition). O'Reilly Media.
- Sweigart, A. (2019). Automate the Boring Stuff with Python (2nd Edition). No Starch Press.
- Zelle, J. (2009). Python Programming: An Introduction to Computer Science (2nd Edition). Franklin, Beedle & Associates.
- Guttag, J. (2016). Introduction to Computation and Programming Using Python (2nd Edition). MIT Press.
- Downey, A. (2015). Think Python: How to Think Like a Computer Scientist (2nd Edition). Green Tea Press.
- Beazley, D. (2009). Python Essential Reference (4th Edition). Addison-Wesley.
- Martelli, A. et al. (2005). Python in a Nutshell (2nd Edition). O'Reilly Media.
- Shaw, Z. A. (2017). Learn Python the Hard Way (3rd Edition). Addison-Wesley.
- Han, J., Kamber, M., & Pei, J. (2012). Data Mining: Concepts and Techniques (3rd Edition). Morgan Kaufmann.

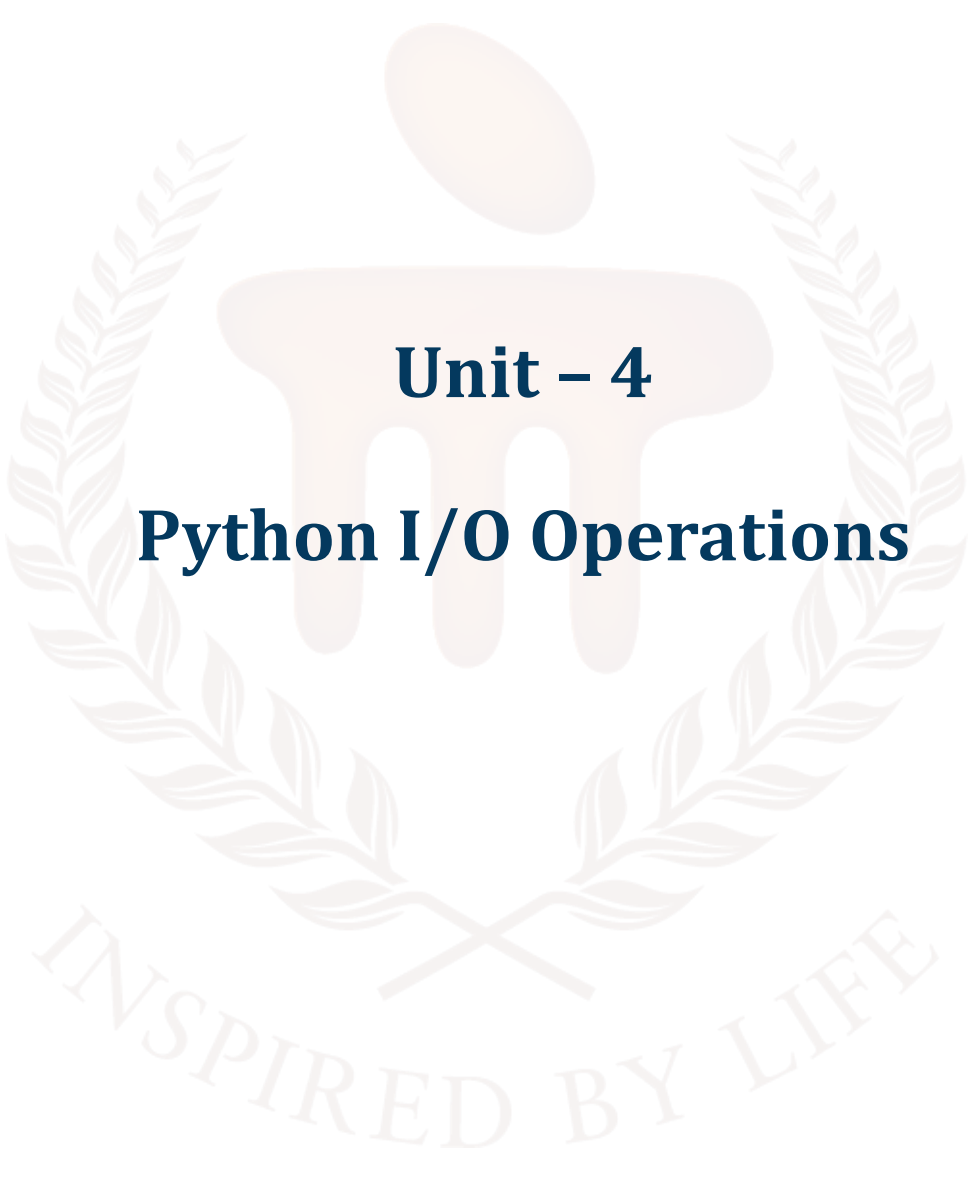
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 4

Python I/O Operations

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 - 6
1.1	Learning Objectives	-	-	
2	Standard Input And Output In Python	-	-	7 - 15
2.1	Overview of I/O Operations	-	-	
2.2	Using print() for Output	-	-	
2.3	Taking Input with input()	-	-	
2.4	Command Line Arguments (Brief Introduction with sys.argv)	-	1	
3	File Handling In Python (File I/O)	-	-	16-30
3.1	Need for File Handling in Programs	-	-	
3.2	Opening and Closing Files (open(), close())	-	-	
3.3	Reading from Files (read(), readline(), readlines())	-	-	
3.4	Writing to Files (write(), writelines())	-	-	
3.5	File Modes (r, w, a, r+, w+, etc.)	-	-	
3.6	Working with Text Files vs Binary Files	-	-	
3.7	Using with Statement for File Handling (Context Managers)	-	-	
3.8	File Pointer Operations (tell(), seek())	-	-	
3.9	Error Handling in File Operations (Exceptions)	-	2	
4	Advanced File I/O Operations	-	-	31-40
4.1	Handling CSV Files with csv Module	-	-	
4.2	JSON File Operations with json Module	-	-	
4.3	Working with OS Module for File Management (os, shutil)	-	-	
4.4	Real-World Use Cases	-	3	

5	Summary		-	-	41
6	Glossary		-	-	42
7	Terminal Questions		-	-	43
8	Answers		-	-	44-46
	8.1	Self-Assessment Answers	-	-	
	8.2	Terminal Questions Answers	-	-	
9	References		-	-	47



1. INTRODUCTION

This unit introduces the fundamental concepts of data mining, its evolution, step-by-step processes, and real-world applications. By the end of this unit, In this unit, learners will explore the importance and advanced techniques of file handling in Python, a critical skill for developing real-world applications. They will understand how Python allows programs to interact with persistent data—data that remains even after the program ends—through various file I/O operations. While basic read and write operations form the foundation, the unit will move beyond these to introduce more complex capabilities that modern applications demand.

Learners will work with CSV (Comma-Separated Values) files, which are widely used for importing and exporting tabular data between Python and tools like spreadsheets or databases. Using Python's built-in `csv` module, students will get hands-on experience with tools such as `reader`, `writer`, `DictReader`, and `DictWriter` that make structured data manipulation efficient and intuitive. They will also explore JSON (JavaScript Object Notation), a key format for data exchange in APIs and configuration files. Through the `json` module, they will learn to convert Python objects to JSON and vice versa, allowing seamless interaction with hierarchical data structures.

Beyond working with data formats, learners will understand how to interact directly with the file system using the `os` module to create, delete, or rename files and directories. The `shutil` module will help them perform higher-level operations like copying and moving files—useful in automating tasks such as backups and deployments.

Finally, the unit emphasises the use of context managers and error handling to ensure safe and reliable file operations. Learners will see how real-world tasks like logging, configuration management, and data pipelines depend on strong file handling skills. By the end of the unit, students will be equipped to build robust applications that manage persistent data with precision and control.

1.1. Learning Objectives

- Explain the need for advanced file I/O operations in modern applications.
- Discuss how to handle CSV files using Python's csv module for structured tabular data.
- Explore JSON operations with the json module for hierarchical data storage and exchange.
- Apply the shutil module to perform high-level operations like copying and moving files.
- Differentiate between CSV and JSON formats, recognizing use cases for each.
- Analyse real-world scenarios where advanced file I/O operations are essential (logs, configs, imports/exports).
- Implement robust error handling in file operations using exception handling techniques



2. STANDARD INPUT AND OUTPUT IN PYTHON

2.1 Overview of I/O Operations

In programming, one of the most fundamental concepts is Input/Output (I/O) operations. The abbreviation I/O stands for input and output, which together describe the way a program interacts with the external world. A program on its own is just a set of instructions written for the computer to follow. However, without some form of input and output, those instructions would run in isolation and serve very little purpose. Input provides the data a program needs in order to work, while output allows it to share results or responses. In other words, I/O is the bridge that connects users and systems to the internal logic of a program.

Input Operations

Input refers to the process of providing data to a program. This data may come from the user directly, from a file stored on the computer, from sensors in an embedded system, or even from a remote server over the internet. For example, when you type your name into a form, you are giving input to the program handling that form. Input is what allows programs to adapt to changing needs and avoid producing static or hardcoded results.

In Python, the most common method of collecting input is through the `input()` function. This function pauses the execution of the program, waits for the user to type something, and then returns that value as a string. **For example:**

```
name = input("Enter your name: ")  
print("Hello,", name)
```

In this simple case, the program takes the user's name as input and then stores it in a variable for further use. Without input, the program would not know what to display back, and every execution would be identical.

Output Operations

Output, on the other hand, is the process of displaying or sending information back to the user or another system. The most common form of output is displaying text on the screen, but it could also include writing to files, printing documents, sending data to a database, or even transmitting information to another device.

In Python, the standard way to produce output is by using the `print()` function. This function can display messages, variable values, or results of computations on the console. For instance:

```
x = 10
y = 20
print("The sum is:", x + y)
```

Here, the program produces output that communicates the result of the computation to the user. Without output, the program would perform its operations silently, leaving the user unaware of the outcome.

Advanced I/O in Python

While `input()` and `print()` are essential for basic I/O tasks, real-world programs often require more advanced forms of data exchange. Python provides several such mechanisms:

1. **File Handling:** Programs can read from and write to files. For example, reading a text file containing sales data or writing a report to a CSV file. File I/O is critical for persistent data storage.
2. **Command-Line Arguments:** Python programs can accept arguments directly from the command line. This is often used in automation scripts or tools where user interaction is minimal.
3. **Formatted Output:** Sometimes, the output needs to be presented in a structured and readable way. Python offers features like f-strings and formatting methods that make output more user-friendly.
4. **Networking and APIs:** Input and output can also involve sending or receiving data over networks, such as making API calls to fetch information or posting data to a server.

Why Are I/O Operations Important?

I/O operations are crucial because they transform a program from being just an algorithm into a tool that people can actually use. There are several reasons why I/O is central to programming:

- **Interactivity:** Programs become interactive when they can receive input from users and respond accordingly. A calculator program, for instance, would be pointless without the ability to input numbers and see the results.

- **Customization:** Input allows users to customize the behavior of programs. For example, word processors let users input text, which the program then formats and displays.
- **Real-World Applications:** Almost every practical application requires I/O. Consider an ATM system. It takes input in the form of PIN numbers, account details, and transaction amounts, and it provides output such as balance information, confirmation receipts, or warnings. Similarly, billing software accepts product codes or service details as input and generates invoices as output.
- **Data Persistence and Sharing:** Input and output make it possible to store information for future use or share it across platforms. File systems, databases, and cloud storage all rely heavily on I/O.

Illustrative Example: ATM System

An ATM program is a clear example of the importance of I/O. When a customer inserts their card, the machine prompts for a PIN—this is input. The ATM then displays the account balance—this is output. If the customer requests a withdrawal, the entered amount is input, and the confirmation message or receipt is output. Without these I/O processes, the ATM would not be able to function, as it would have no way of receiving instructions or delivering results.

2.2 Using print() for Output

In Python, the `print()` function is one of the most frequently used tools for producing output. It allows a program to communicate with the user by sending text, numbers, or the results of operations to the console. For beginners, `print()` is usually the very first function they encounter, and even advanced developers rely on it for quick debugging or user interaction.

Basic Syntax of `print()`

The general form of the function is:

```
print(object1, object2, ..., sep=' ', end='\n')
```

- `object1, object2, ...`: These represent the values, variables, or expressions that need to be displayed. Multiple items can be passed at once.
- `sep`: This parameter defines the separator between values. By default, a single space " " is used.
- `end`: This defines what should be added at the end of the output. By default, `print()` appends a newline character (`\n`), meaning every call to `print()` starts on a new line.

1. Simple Printing

```
print("Hello, World!")
```

Logic: The string "Hello, World!" is passed as an argument to the function. Python sends this to the standard output (console). This is the classic first program for learners and demonstrates how easily Python handles text output.

2. Printing Multiple Values

```
print("Python", "is", "fun")
```

Output:

Python is fun

Logic: By default, when multiple values are passed, Python separates them using a space (sep=' '). This avoids the need to manually insert spaces when printing multiple objects.

3. Formatting Output

The true power of print() lies in its flexibility to format the output.

a) Using sep

```
print("2025", "09", "08", sep="-")
```

Output:

2025-09-08

Logic: The sep parameter changes the separator. Instead of spaces, a hyphen - is used, making it ideal for structured outputs like dates or file paths.

b) Using end

```
print("Hello", end=" ")
```

```
print("World")
```

Output:

Hello World

Logic: Normally, print() adds a newline after execution. Here, end=" " overrides the default behavior, so the second print() continues on the same line. This is useful for progress indicators or inline outputs.

c) f-strings (Formatted String Literals)

Introduced in Python 3.6, f-strings simplify string formatting.

```
name = "Alice"
age = 25
print(f"My name is {name} and I am {age} years old.")
```

Output:

My name is Alice and I am 25 years old.

Logic: The f before the string enables placeholders {}. Python replaces them with the values of the corresponding variables or expressions. This method is concise, readable, and widely used in modern Python programs.

d) format() Method

Before f-strings, the .format() method was the preferred choice.

```
name = "Bob"
marks = 92
print("Student: {}, Marks: {}".format(name, marks))
```

Output:

Student: Bob, Marks: 92

Logic: The curly braces {} act as placeholders. The format() method fills them with values provided in order.

You can also specify positions:

```
print("First: {1}, Second: {0}".format("X", "Y"))
```

Output:

First: Y, Second: X

This flexibility allows developers to control the arrangement of output dynamically.

Practical Importance of print()

- The print() function is not only about displaying text. It plays a vital role in:

- User Interaction: Showing results, confirmations, or instructions.
- Debugging: Printing variable values or program states while testing.
- Formatting Reports: Creating clean and structured outputs for logs, tables, or summaries.

For example, when displaying a summary of exam results, `print()` can format data into a neat structure, making the information easy to read.

2.3 Taking Input with `input()`

The `input()` function allows the program to receive data from the user. Whatever the user types is treated as a string by default.

Syntax

```
variable = input("Message to user: ")
```

- "Message to user" is optional but helps guide the user.
- The value entered is stored as a string in variable.

1. Reading Strings

```
name = input("Enter your name: ")  
print("Welcome,", name)
```

Logic:

- The program pauses and waits for the user to type.
- Suppose the user types Alice.
- The variable name stores "Alice".
- Then the program prints Welcome, Alice.

2. Type Casting Input Values

Since `input()` always returns a string, we need to convert (cast) input to other data types when required.

Integer Input

```
age = int(input("Enter your age: "))  
print("Next year you will be", age + 1)
```

Logic:

- The user types something like 20.
- `int()` converts it from string "20" to number 20.
- Adding 1 gives 21.

Float Input

```
height = float(input("Enter your height in meters: "))  
print("Height recorded:", height)
```

Logic:

- User types 1.75.
- Stored as 1.75 (float).

Multiple Inputs (using split)

```
x, y = input("Enter two numbers separated by space: ").split()  
x, y = int(x), int(y)  
print("Sum is:", x + y)
```

Logic:

- User enters 5 10.
- `.split()` separates them into "5" and "10".
- `int()` converts them to numbers.
- Prints Sum is: 15.

2.4 Command Line Arguments (Brief Introduction with `sys.argv`)

Besides interactive input, Python can also take input directly from the command line when running a program. This is done using the `sys` module, specifically the list `sys.argv`.

- `sys.argv` → A list containing arguments passed to the program.
- `sys.argv[0]` → The script name.
- `sys.argv[1], sys.argv[2], ...` → The arguments entered after the script name.

Example

Suppose we save a program as `example.py`:

```
import sys

print("Script name:", sys.argv[0])

print("First argument:", sys.argv[1])

print("Second argument:", sys.argv[2])
```

If we run it from the terminal:

```
python example.py Alice 25
```

Output:

Script name: example.py

First argument: Alice

Second argument: 25

Logic:

- The command line passes arguments as strings.
- "Alice" and "25" are stored in `sys.argv[1]` and `sys.argv[2]`.
- This is useful for automation, where input is provided at execution instead of typing during runtime.

SELF-ASSESSMENT QUESTIONS – 1

Multiple Choose Questions

1	What does the <code>print()</code> function do by default after printing text? a) Ends with a tab b) Ends with a newline c) Ends with a space d) Ends with nothing
2	Which function in Python is used for standard input? a) <code>scanf()</code> b) <code>get()</code> c) <code>input()</code> d) <code>read()</code>
3	What is the data type of the value returned by <code>input()</code> ?

	a) int b) str c) float d) depends on input
4	What will be the output of the following code? <pre>print("A", "B", "C", sep="-", end="*")</pre> a) A B C * b) A-B-C* c) A-B-C d) ABC*
5	If we run a script using <code>python program.py hello 123</code>, what will <code>sys.argv[2]</code> contain? a) program.py b) hello c) 123 d) Error



3. FILE HANDLING IN PYTHON (FILE I/O)

3.1 Need for File Handling in Programs

When we write and run Python programs, most of the data that the program works with resides only in the computer's memory (RAM). This memory is temporary and volatile. As soon as the program ends or the system shuts down, all the data stored in RAM disappears. While this temporary data storage works fine for small, one-time calculations, it is not enough for real-world applications where information needs to be preserved beyond the lifetime of the program.

To overcome this limitation, we use file handling. File handling is the process of creating, reading, writing, and managing files stored on a computer's disk. Unlike RAM, the disk provides permanent storage, which ensures that information is retained even after the program finishes execution.

Why Do We Need File Handling?

1. Permanent Data Storage

One of the most important needs for file handling is permanence. Imagine a situation where you design a program to take user details—such as name, email, and phone number—but once the program closes, that information is lost. In real life, this would make the program useless because users expect their information to be stored for future use. File handling allows you to save this data into text files, CSV files, or databases, so that it can be retrieved and updated later.

For example, a student grading program must not lose records of marks once the program ends. With file handling, the grades can be written to a file, ensuring that the next time the program runs, those grades can be accessed and analyzed.

2. Reading and Processing Large Datasets

Modern applications often deal with massive amounts of data. Examples include financial transactions, sensor readings from IoT devices, or large scientific datasets. It would be impractical to manually input such huge volumes of data every time the program runs. Instead, data is stored in files such as text files, CSVs (comma-separated values), JSON files, or binary files.

File handling allows programs to directly read these files, process them, and even update them if needed. For instance, data scientists often use Python to read CSV files containing millions of

records and perform data analysis. Without file handling, working with such large datasets would be nearly impossible.

3. Data Exchange Between Systems

Another key reason for file handling is data exchange. Files make it possible for different programs, written in different languages or running on different systems, to share information easily. For example, one program may generate a log file that records transactions, and another program may analyze those logs for errors or trends.

Similarly, configuration files allow applications to share settings across different runs or even across machines. For example, a cloud service may export data in JSON format, which can be read by Python programs, Java programs, or web applications alike. Thus, file handling provides a universal method for storing and exchanging information.

4. Automation of Workflows

Many real-world applications rely on automation. Programs often need to generate reports, maintain logs, or read configuration files without requiring manual user input every time. File handling enables this automation.

- A script may generate a daily sales report in CSV format and store it in a directory.
- Another script may read that file the next day and update the company's central database.
- System administrators use log files to track performance or identify security breaches automatically.

Without file handling, every run of a program would require re-entering information, making automation impossible.

Example Scenario: Student Grading System

Let's consider the case of a student grading program. Suppose a teacher uses a Python program to calculate the average score of students in an exam. If the program only stores this information in memory, all results will be lost once the program stops. The next time the teacher runs the program, they would have to re-enter the data again, wasting time and effort.

By introducing file handling, the program can save each student's name, marks, and grade into a text or CSV file. When the program is reopened, it can read the file, load all student data, and continue from where it left off. This makes the program practical, reliable, and efficient.

Additional Real-World Examples

- **ATM Machines:** When you withdraw cash, the transaction is recorded in a file or database so that your account balance remains updated permanently.
- **Billing Software:** A shop's billing system writes receipts and daily transaction reports into files. These files serve as a permanent record for auditing and analysis.
- **Web Browsers:** Browsers save cookies and cache in files, which allow you to remain logged in to websites or quickly load frequently visited pages.
- **Scientific Research:** Large amounts of experimental data are stored in files for later analysis and reproducibility.

3.2 Opening and Closing Files (`open()`, `close()`)

File handling is one of the most important aspects of programming because it allows information to persist beyond the lifetime of a program. In Python, the very first step of working with a file is to open it, and the last step is to close it. Between these two steps, we can perform operations such as reading data, writing data, or appending information. Without opening, we cannot access the file; without closing, we risk losing data or locking valuable system resources.

Syntax of Opening a File

```
file_object = open("filename", "mode")
```

- "filename" → The name of the file, along with its extension (.txt, .csv, etc.). If the file is in another directory, the path must be specified, such as "C:/Users/Student/Documents/data.txt".
- "mode" → Defines how the file will be used:
 - "r" → Read (default mode).
 - "w" → Write (creates a new file or overwrites an existing one).
 - "a" → Append (adds new content at the end of the file without removing the existing data).
 - "b" → Binary mode (for images, audio, etc.).
 - "+" → Update mode (read and write together).
- file_object → A variable that serves as a handle to the file. All operations (read, write, etc.) will be done through this object.

Example:

```
f = open("notes.txt", "r")
```

This statement opens the file notes.txt in read mode and assigns the file object to variable f.

Why Do We Need Modes?

Modes are necessary because they tell Python what our intention is. For example:

- If you only want to look at the contents of a file, "r" is safe.
- If you want to write new information but don't care about existing content, "w" works.
- If you want to keep old content and just add more, "a" is the right choice.

Without specifying modes clearly, data could be lost or operations may fail.

Closing Files

Once operations are complete, it is essential to close the file using:

```
file_object.close()
```

Closing ensures that:

1. Data is properly saved: When writing to files, the information is often first stored in a temporary memory buffer. Closing guarantees that this buffer is flushed and written permanently to disk.
2. System resources are released: Files use system resources such as file descriptors. Keeping files open unnecessarily may slow down the system or cause errors if too many files are left open.
3. Good programming practice: Closing files explicitly shows clear intent and keeps the program reliable.

Basic Example

```
f = open("example.txt", "w")    # Open file in write mode
```

```
f.write("Hello, File Handling") # Write text into the file
```

```
f.close()                      # Close the file
```

Logic:

- The file "example.txt" is opened in write mode. If it doesn't exist, Python creates it.
- The string "Hello, File Handling" is written into the file.

- The file is closed to ensure the data is safely stored on disk.

Reading Example

```
f = open("example.txt", "r") # Open file in read mode
content = f.read()          # Read entire file
print(content)              # Display content
f.close()                   # Close the file
```

Here, the file is opened for reading, its contents are stored in the variable `content`, and then printed. The file is closed after use.

Append Example

```
f = open("example.txt", "a")
f.write("\nThis is an appended line.")
f.close()
```

This program reopens "example.txt" in append mode and adds a new line at the end without removing the original content.

Risks of Not Closing Files

- **Unsaved Data:** If you forget to close a file after writing, some data may remain in the buffer and never get saved.
- **Resource Leaks:** Open files consume operating system resources. A program that opens many files without closing them may crash or slow down the system.
- **File Locking Issues:** On some systems, a file may remain locked if not closed properly, preventing other programs from using it.

Thus, closing files is not optional—it is a necessity for safe and efficient programming.

Using with Statement for Safety

A common mistake is forgetting to call `.close()`. To solve this, Python provides the `with` statement, which automatically closes the file once the block of code is executed.

with open("example.txt", "w") as f:

```
    f.write("This file is handled safely.")
```

Logic:

- The with keyword opens the file and assigns it to variable f.
- Once the block is finished, Python automatically closes the file, even if an error occurs inside the block.
- This ensures safer and cleaner code.

Practical Scenario

Imagine a student grading program. Each time a student's grade is calculated, it must be stored in a file for future reference.

Writing grades

```
f = open("grades.txt", "w")  
f.write("Alice: 85\n")  
f.write("Bob: 92\n")  
f.close()
```

Reading grades later

```
f = open("grades.txt", "r")  
print(f.read())  
f.close()
```

The first part writes grades into the file and closes it. The second part reopens the file and reads the stored results. If the file were not closed after writing, some grades might not be stored, making the program unreliable.

3.3 Reading from Files (read(), readline(), readlines())

Python provides different methods for reading files.

1. read()

Reads the entire content of a file as a single string.

```
f = open("sample.txt", "r")
```

```
content = f.read()

print(content)

f.close()
```

Logic:

- Opens file in read mode.
- `f.read()` reads everything at once.
- Useful for small files.

2. readline()

Reads one line at a time.

```
f = open("sample.txt", "r")

line1 = f.readline()

line2 = f.readline()

print(line1, line2)

f.close()
```

Logic:

- Reads first line into `line1`.
- Reads second line into `line2`.
- Efficient for processing line-by-line (like logs).

3. readlines()

Reads all lines into a list, where each line is an element.

```
f = open("sample.txt", "r")

lines = f.readlines()

for line in lines:

    print(line.strip())

f.close()
```

Logic:

- Stores lines in a list.
- Loop prints each line after removing `\n`.

3.4 Writing to Files (`write()`, `writelines()`)

1. `write()`

Writes a string into the file.

```
f = open("output.txt", "w")  
f.write("Python File Handling Example\n")  
f.write("Second Line")  
f.close()
```

Logic:

- File is opened in write mode (`w`).
- Two strings are written.

2. `writelines()`

Writes a list of strings to the file.

```
f = open("list_output.txt", "w")  
lines = ["Line 1\n", "Line 2\n", "Line 3\n"]  
f.writelines(lines)  
f.close()
```

Logic:

- Each element of the list is written as a line.
- `\n` ensures line breaks.

3.5 File Modes (`r`, `w`, `a`, `r+`, `w+`, etc.)

When working with files in Python, it is not enough to simply open them. We must also tell Python how we want to interact with the file—whether we plan to read its contents, write new data, append extra information, or work with binary data like images and audio. This is where file modes come in. File modes are special string arguments passed to the `open()` function that define the purpose of

opening the file. Choosing the correct mode is important, as the wrong one could overwrite important data or cause errors.

The general syntax remains:

```
file_object = open("filename", "mode")
```

Here, "filename" is the file name (with extension and path if necessary), and "mode" is one of the options we will explore.

1. "r" → Read Mode (Default)

- Opens an existing file for reading.
- If the file does not exist, Python throws a `FileNotFoundError`.
- This is the safest mode when you only want to look at file contents without modifying them.

Example:

```
f = open("data.txt", "r")
content = f.read()
print(content)
f.close()
```

Here, "data.txt" must already exist. Its contents are read and displayed.

2. "w" → Write Mode

- Opens a file for writing.
- If the file does not exist, Python creates it.
- If the file exists, its contents are erased before writing.

Example:

```
f = open("report.txt", "w")
f.write("This is a fresh report.\n")
f.close()
```

If "report.txt" already had data, it will be completely replaced by "This is a fresh report.". Thus, "w" should be used with caution.

3. "a" → Append Mode

- Opens a file for writing but does not erase existing data.
- New data is added to the end of the file.
- If the file does not exist, Python creates it.

Example:

```
f = open("log.txt", "a")  
f.write("New log entry\n")  
f.close()
```

If "log.txt" already has previous log entries, they remain intact. The new line is added at the end. Append mode is useful for logs, records, and journals.

4. "r+" → Read and Write Mode

- Opens a file for both reading and writing.
- The file must already exist, or Python raises an error.
- The cursor is placed at the beginning, so writing can overwrite existing data.

Example:

```
f = open("data.txt", "r+")  
print(f.read())    # Read file contents  
f.seek(0)          # Move cursor back to start  
f.write("Updated")  # Overwrites starting part  
f.close()
```

Here, "Updated" replaces the first part of the file, while the rest remains.

5. "w+" → Write and Read Mode

- Opens a file for both writing and reading.
- If the file exists, it is completely overwritten.
- If the file does not exist, it is created.

Example:

```
f = open("temp.txt", "w+")
```

```
f.write("Temporary data\n")  
  
f.seek(0)  
  
print(f.read())  
  
f.close()
```

This mode is helpful when you want to create or reset a file but still need to read back what was written.

6. "a+" → Append and Read Mode

- Opens a file for both reading and appending.
- If the file does not exist, it is created.
- Data is always added to the end, but you can still read existing content.

Example:

```
f = open("history.txt", "a+")  
  
f.write("Session started\n")  
  
f.seek(0)  
  
print(f.read())  
  
f.close()
```

This mode is useful for maintaining logs or history while still allowing access to the older data.

7. "rb" → Read Binary

- Opens a file in binary mode for reading.
- Useful for files like images, audio, videos, or executables where data is not text but raw bytes.

Example:

```
f = open("image.jpg", "rb")  
  
bytes_data = f.read(20) # Reads first 20 bytes  
  
print(bytes_data)  
  
f.close()
```

Here, the program reads raw byte data rather than human-readable text.

8. "wb" → Write Binary

- Opens a file in binary mode for writing.
- If the file exists, it is overwritten.
- If it does not exist, it is created.
- Used when writing non-text files, such as saving images or serializing objects.

Example:

```
f = open("output.bin", "wb")  
f.write(b'\x41\x42\x43') # Writing bytes (A, B, C)  
f.close()
```

This writes three bytes corresponding to ASCII values of "ABC".

Importance of Choosing Correct File Modes

1. Preventing Data Loss: Using "w" instead of "a" can erase valuable data.
2. Ensuring Compatibility: Reading binary files with "r" instead of "rb" may cause corruption.
3. Flexibility: Modes like "r+" and "a+" allow combined operations, reducing the need to open files multiple times.

Real-World Example

Imagine a student grading system:

- "w" is used at the beginning of the semester to create a fresh grade file.
- "a" is used each time a new assignment score is recorded.
- "r" is used when teachers want to review student scores.
- "a+" can be used to both read previous grades and add new ones in the same program.

This careful choice of file modes ensures that grades are never accidentally deleted while still allowing updates.

Summary Table of File Modes

Mode	Description	File Existence Requirement	Overwrites Data	Appends Data
r	Read only	Must exist	No	No
w	Write only	Created if missing	Yes	No
a	Append only	Created if missing	No	Yes
r+	Read + Write	Must exist	Partial	Possible
w+	Write + Read	Created if missing	Yes	No
a+	Append + Read	Created if missing	No	
rb	Read Binary	Must exist	No	No
wb	Write Binary	Created if missing	Yes	No

3.6 Working with Text Files vs Binary Files

Text Files

- Store human-readable data (e.g., .txt, .csv).
- Encoding like UTF-8 is used.
- Example: "Hello World\n".

Binary Files

- Store data in raw bytes (e.g., images, audio, executables).
- Not human-readable.
- Must be opened in "rb" or "wb" mode.

Example: Reading Binary File

```
f = open("image.jpg", "rb")

data = f.read(10) # Read first 10 bytes

print(data)
```

```
f.close()
```

3.7 Using with Statement for File Handling (Context Managers)

Instead of manually closing files, Python provides the with statement. It automatically closes the file when operations are done.

```
with open("data.txt", "r") as f:
```

```
    content = f.read()
```

```
    print(content)
```

Logic:

- File is opened and assigned to f.
- After the block ends, file closes automatically.
- Reduces chances of forgetting close().

3.8 File Pointer Operations (tell(), seek())

When reading or writing, Python maintains a file pointer (current position).

tell()

Returns current position of pointer.

```
f = open("sample.txt", "r")
```

```
print(f.tell())
```

```
f.close()
```

seek()

Moves pointer to a given position.

```
f = open("sample.txt", "r")
```

```
f.seek(5) # Move to 6th byte
```

```
print(f.read())
```

```
f.close()
```

Logic:

- `tell()` shows the pointer's current index.
- `seek(n)` moves to the *n*th byte.

3.9 Error Handling in File Operations (Exceptions)

File operations may fail if:

- File does not exist.
- Permission is denied.
- File is in wrong format.

To avoid program crashes, use try-except.

try:

```
f = open("nonexistent.txt", "r")
content = f.read()
f.close()
```

except `FileNotFoundError`:

```
print("File not found. Please check filename.")
```

Logic:

- Attempts to open a non-existing file.
- `FileNotFoundError` is caught.
- Program continues without crashing.

SELF-ASSESSMENT QUESTIONS – 2

Fill in the blanks:	
6	The function used to open a file in Python is ____.
7	The default file mode is ____.
8	The function that reads one line at a time is ____.
9	To ensure a file closes automatically, we use the ____ statement.
10	The function used to move the file pointer is ____.

4. ADVANCED FILE I/O OPERATIONS

File handling in Python goes beyond basic read and write operations. Modern applications demand structured data handling (CSV, JSON), system-level file management, and integration with real-world workflows like logs or configuration files. Python provides specialized modules (csv, json, os, shutil) to handle these tasks efficiently.

4.1 Handling CSV Files with csv Module

Data is at the heart of almost every modern application, and one of the simplest yet widely used formats for storing and sharing structured data is the CSV (Comma-Separated Values) file. A CSV file stores information in a tabular format where each row represents a record and each column represents a field. Unlike complex database systems, CSV files are human-readable, lightweight, and supported across programming languages and platforms, making them an excellent choice for data exchange.

Structure of a CSV File

Consider an example students.csv:

Name,Age,Marks

Alice,20,85

Bob,21,90

Charlie,22,78

- The first row usually contains column names or headers (Name, Age, Marks).
- Each subsequent row represents a student record.
- Values are separated by commas (though other delimiters like ; or \t can also be used).

Python provides a built-in csv module to read, write, and process CSV files efficiently. Let us explore its different functionalities step by step.

Reading CSV Files

Using csv.reader()

The csv.reader() class is the most basic way to read CSV files. It treats each line as a row and each comma-separated value as a list element.

Example:

```
import csv

with open("students.csv", "r") as file:

    reader = csv.reader(file)

    for row in reader:

        print(row)
```

Logic:

- The file students.csv is opened in read mode.
- csv.reader(file) returns each row as a list of strings.
- Iterating through reader prints rows like:

```
['Name', 'Age', 'Marks']
```

```
['Alice', '20', '85']
```

```
['Bob', '21', '90']
```

```
['Charlie', '22', '78']
```

This shows how CSV content is mapped into Python lists, making it easy to process row by row.

Writing CSV Files**Using csv.writer()**

The csv.writer() class lets us create CSV files by writing rows one at a time.

Example:

```
import csv

with open("output.csv", "w", newline="") as file:

    writer = csv.writer(file)

    writer.writerow(["Name", "Age", "Marks"])

    writer.writerow(["David", 23, 88])

    writer.writerow(["Eva", 21, 92])
```

Logic:

- The file is opened in write mode.
- `writerow()` is used to write a single row at a time.
- The resulting `output.csv` will contain:

Name,Age,Marks

David,23,88

Eva,21,92

This is useful for exporting reports or generating new datasets.

Reading CSV Files as Dictionaries

While lists are simple, they are not always the most convenient way to work with CSV data. Accessing values by index (`row[0]`, `row[1]`) is less intuitive. The `DictReader` class solves this by allowing us to access values using column names.

Example:

```
import csv
```

```
with open("students.csv", "r") as file:
```

```
    reader = csv.DictReader(file)
```

```
    for row in reader:
```

```
        print(row["Name"], "scored", row["Marks"])
```

Output:

Alice scored 85

Bob scored 90

Charlie scored 78

Logic:

- `DictReader` automatically maps each row into a dictionary where keys are column headers and values are data entries.
- This makes code cleaner and more understandable, especially for large datasets.

Writing CSV Files as Dictionaries

Just like reading with dictionaries, Python allows writing data as dictionaries using DictWriter. This is very convenient when dealing with structured data stored in dictionaries or JSON-like formats.

Example:

```
import csv

data = [
    {"Name": "John", "Age": 24, "Marks": 80},
    {"Name": "Lily", "Age": 22, "Marks": 89}
]

with open("students_dict.csv", "w", newline="") as file:
    fieldnames = ["Name", "Age", "Marks"]
    writer = csv.DictWriter(file, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(data)
```

Logic:

- fieldnames define the column headers.
- writeheader() writes the header row into the file.
- writerows() writes each dictionary in the list as a row.
- The output file looks like this:

Name,Age,Marks

John,24,80

Lily,22,89

Handling Special Cases

CSV files in real-world applications are not always straightforward. They may have different delimiters, missing values, or special characters. The csv module provides parameters to handle these:

1. Custom Delimiter

```
2. reader = csv.reader(file, delimiter=";")
```

This reads files where values are separated by semicolons instead of commas.

```
3. Quoting Special Characters
```

```
4. writer = csv.writer(file, quoting=csv.QUOTE_ALL)
```

This ensures that all values are enclosed in quotes, useful when fields contain commas.

```
5. Skipping Headers
```

Sometimes you may want to skip headers during reading:

```
6. next(reader) # Skips the first row
```

Real-World Applications of CSV Handling

CSV handling is not just an academic concept; it is widely used in everyday scenarios:

1. **Data Analysis:** Data scientists import large datasets in CSV format for analysis in tools like Pandas.
2. **Business Applications:** Sales reports, billing information, or employee data are often stored in CSV files.
3. **Web Applications:** Many web services provide CSV downloads (for example, exporting contacts from Gmail or transaction history from a bank).
4. **Automation:** Python scripts can automatically generate CSV files as reports or parse logs into CSV for further use.

Best Practices

1. **Always use with statement:** This ensures files are automatically closed, even if errors occur.
2. **with open("file.csv", "r") as f:**
 1. `reader = csv.reader(f)`
 2. **Use `newline=""` when writing:** Prevents extra blank lines in CSVs on Windows.
3. **Validate Data:** When reading CSV files from external sources, validate values before using them.
4. **Combine with Other Libraries:** For large-scale data analysis, Python's pandas library builds on top of csv for advanced operations like filtering, grouping, and merging.

4.2 JSON File Operations with json Module

JSON (JavaScript Object Notation) is widely used for structured data exchange in APIs, configuration, and databases. JSON resembles Python dictionaries.

Example JSON (config.json):

```
{  
    "username": "admin",  
    "theme": "dark",  
    "auto_save": true  
}
```

Python provides the json module for JSON handling.

Reading JSON

```
import json  
  
with open("config.json", "r") as file:  
    data = json.load(file)  
    print(data["username"], "uses", data["theme"], "theme")
```

Output:

admin uses dark theme

Logic:

- json.load(file) parses JSON into a Python dictionary.
- Access values using keys.

Writing JSON

```
import json  
  
data = {"name": "Alice", "age": 25, "is_student": False}  
  
with open("output.json", "w") as file:
```

```
json.dump(data, file, indent=4)
```

Logic:

- `json.dump()` writes Python dictionary to file in JSON format.
- `indent=4` makes the file human-readable.

Converting Between JSON and Python Strings

- `json.loads(string)` → Convert JSON string to Python object.
- `json.dumps(object)` → Convert Python object to JSON string.

Example:

```
import json

person = '{"name": "Bob", "age": 30}'

data = json.loads(person)

print(data["name"])

json_string = json.dumps(data)

print(json_string)
```

4.3 Working with OS Module for File Management (os, shutil)

Python's `os` and `shutil` modules allow interacting with the file system.

Common os Module Functions

```
import os
```

Current working directory

```
print(os.getcwd())
```

Create directory

```
os.mkdir("new_folder")
```

List files in directory

```
print(os.listdir("."))
```

Remove file

```
os.remove("output.csv")
```

Rename file

```
os.rename("old.txt", "new.txt")
```

shutil Module

The shutil module helps with high-level file operations like copying and moving.

```
import shutil
```

Copy file

```
shutil.copy("data.txt", "backup_data.txt")
```

Move file

```
shutil.move("backup_data.txt", "archive/backup_data.txt")
```

Why Use OS/SHUTIL?

- Automate file backups.
- Organize logs.
- Manage large datasets.

4.4 Real-World Use Cases

Advanced File I/O is used in practical applications across industries.

1. Log Files

Applications write logs to track errors, performance, and user activity.

with open("app.log", "a") as log:

```
log.write("User logged in\n")
```

2. Data Import/Export

CSV and JSON are standard formats for sharing data. Example: exporting sales reports.

```
import csv
```

```
sales = [{"Product", "Quantity"}, ["Laptop", 5], ["Phone", 10]]
```

with open("sales.csv", "w", newline="") as file:

```
writer = csv.writer(file)
```

```
writer.writerows(sales)
```

3. Config Files

Applications store user preferences in JSON.

```
import json
```

```
config = {"theme": "light", "font": "Arial"}
```

with open("settings.json", "w") as file:

```
json.dump(config, file)
```

SELF-ASSESSMENT QUESTIONS – 3

Column A

Match the following		
Column A		Column B
11	csv.reader()	a) Creates a new directory
12	json.dump()	b) Reads CSV rows as dictionaries
13	os.mkdir()	c) Writes Python dictionary to JSON file
14	shutil.copy()	d) Reads CSV file row by row
15	DictReader	e) Copies file from one location to another

5. SUMMARY

- Python provides specialized modules like csv, json, os, and shutil to handle advanced file operations.
- The csv module allows reading and writing tabular data efficiently using reader, writer, DictReader, and DictWriter.
- The json module is widely used for structured data exchange, configuration files, and API responses.
- JSON operations in Python include load, dump, loads, and dumps for converting between Python objects and JSON.
- The os module helps interact with the operating system by creating, renaming, or deleting files and directories.
- The shutil module enables higher-level operations such as copying and moving files between directories.
- CSV is best suited for tabular data, while JSON handles hierarchical and nested data structures effectively.
- Log files, configuration files, and data export/import are practical real-world use cases of file handling.
- Using context managers (with statement) ensures safe file operations by automatically closing files after use.
- Proper exception handling is essential in file I/O to prevent crashes due to missing files, permission errors, or format issues..

6. GLOSSARY

CSV (Comma-Separated Values)	- A file format storing tabular data where fields are separated by commas.
JSON (JavaScript Object Notation)	- A lightweight, human-readable format for representing structured data.
os module	- Python module used for interacting with the operating system's file and directory structures.
shutil module	- : A Python module providing higher-level file operations like copy and move.
DictReader	- A class in the csv module that reads CSV rows as dictionaries.
Serialization	- Converting data structures into formats that can be stored or transmitted.
Context Manager	- A Python construct (with statement) ensuring safe acquisition and release of resources.
Log File	- A text file storing sequential system or application events for monitoring and debugging.
Configuration File	- A file storing application settings such as user preferences and operational modes.
Exception Handling	- The process of managing runtime errors in programs using try-except blocks.

7. TERMINAL QUESTIONS

1. What is the role of the csv module in Python, and how does DictReader differ from csv.reader()?
2. Explain the importance of the json module in modern applications and provide an example of reading a JSON file.
3. Differentiate between json.dump() and json.dumps() with suitable examples.
4. How does the os module assist in managing directories and files programmatically?
5. Describe the functionalities provided by the shutil module in file management.
6. Compare CSV and JSON file formats, highlighting their advantages and limitations.
7. Explain how log files are created and used in Python programs.
8. Why are JSON files preferred for configuration data over CSV files?
9. Write a use case where both os and shutil modules are used together for automation.
10. Discuss the significance of exception handling in advanced file I/O operations.

8. ANSWERS

8.1. Self-Assessment Answers:

1. b) Ends with a newline
2. c) input()
3. b) str
4. b) A-B-C*
5. c) 123
6. open()
7. "r" (read mode)
8. readline()
9. with
10. seek()
11. 1 → d) Reads CSV file row by row
12. 2 → c) Writes Python dictionary to JSON file
13. 3 → a) Creates a new directory
14. 4 → e) Copies file from one location to another
15. 5 → b) Reads CSV rows as dictionaries

8.2. Terminal Question Answers

Answer 1: The csv module enables Python programs to read and write tabular data stored in CSV format. `csv.reader()` reads rows as lists, making data accessible by index. In contrast, `csv.DictReader()` treats the first row as headers and provides each row as a dictionary, allowing column access by names. This makes DictReader particularly useful in structured data handling, where field names are known and meaningful, reducing index-based errors and improving readability of code

Answer 2: The json module is crucial in modern applications because most web APIs and configuration files use JSON as the preferred data format. It supports structured and nested data, making it suitable for exchanging information between systems. For example, reading a JSON file with `json.load(file)` converts the file content into a Python dictionary. Developers can then access values using keys, enabling efficient configuration handling, API response parsing, and storage of structured application settings across diverse environments.

Answer 3: The `json.dump()` function writes Python objects directly into a JSON file, ensuring data is stored persistently. On the other hand, `json.dumps()` converts Python objects into a JSON-formatted string, which can be transmitted or logged without writing to a file. For instance, using `json.dump(data, file)` saves data permanently, while `json.dumps(data)` might be used in sending structured data over a network connection. Both functions ensure interoperability, with `dump` focused on files and `dumps` on strings

Answer 4: The `os` module provides functions that allow Python programs to interact directly with the operating system's file system. Developers can create, rename, or delete directories, check the current working directory, and list available files. It automates repetitive tasks such as managing large datasets or organizing logs. By offering these functionalities programmatically, `os` supports robust, cross-platform file handling, making Python scripts flexible and suitable for automation tasks in real-world applications requiring system-level interaction

Answer 5: The `shutil` module extends Python's file handling capabilities by enabling high-level operations like copying, moving, and deleting files and directories. For example, `shutil.copy()` duplicates files, while `shutil.move()` relocates them across directories. These functions make backup creation and dataset management much easier. Unlike the `os` module, which deals with low-level file system interactions, `shutil` focuses on simplifying more complex operations. It is widely used for managing project directories, automating file backups, and preparing structured data pipelines.

Answer 6: CSV files are simple, human-readable, and suitable for tabular data like spreadsheets. However, they cannot represent nested structures. JSON files, in contrast, support hierarchical and nested data, making them ideal for modern applications like APIs and configurations. CSV is preferred for importing/exporting data to tools like Excel, while JSON is favored in programming environments. Both formats are lightweight, but JSON is more versatile, whereas CSV remains efficient and popular in flat data structures

Answer 7: Log files are used in Python applications to record system events, errors, or user activities for monitoring and debugging purposes. They are created by writing messages to a file in append mode so that previous entries are preserved. For example, with `open("app.log", "a")` as `log`: allows writing log entries continuously. Logs are vital in large-scale applications because they provide detailed histories, making it easier to trace issues, analyze patterns, and ensure accountability in workflows.

Answer 8: JSON files are preferred for configuration data because they support hierarchical and nested structures, unlike CSV files which are limited to tabular data. Applications often require storing settings like themes, user credentials, and nested preferences, which JSON handles naturally. Additionally, JSON integrates seamlessly with Python dictionaries, enabling easy reading and modification. CSV lacks this flexibility. Hence, JSON is the standard for configuration files in software applications, allowing complex yet readable storage of program settings.

Answer 9: A practical use case combining both `os` and `shutil` is automated file backup. For example, the `os` module can be used to check if a backup directory exists, and create it if missing. Then, the `shutil.copy()` function can duplicate important files into this backup directory. This workflow ensures structured organization, fault tolerance, and ease of recovery in case of system failures. Together, these modules allow automation of data safety tasks in production environments.

Answer 10: Exception handling in advanced file I/O operations is critical because operations often fail due to missing files, incorrect permissions, or corrupted data. Without error handling, such failures can crash programs. By using `try-except` blocks, programmers can catch specific errors like `FileNotFoundError` and provide meaningful error messages. This ensures program stability, better user experience, and fault tolerance. Effective exception handling is particularly important in enterprise applications where continuous availability and reliability are essential.

9. REFERENCES

- Python Official Documentation – csv Module
- Python Official Documentation – json Module
- Python Official Documentation – os Module
- Python Official Documentation – shutil Module
- Mark Lutz, Learning Python, O'Reilly Media, 5th Edition.
- Eric Matthes, Python Crash Course, No Starch Press, 2nd Edition.
- Al Sweigart, Automate the Boring Stuff with Python, No Starch Press, 2nd Edition.
- Luciano Ramalho, Fluent Python, O'Reilly Media, 2nd Edition.
- John Zelle, Python Programming: An Introduction to Computer Science, Franklin Beedle & Associates.
- Paul Barry, Head First Python, O'Reilly Media, 2nd Edition.

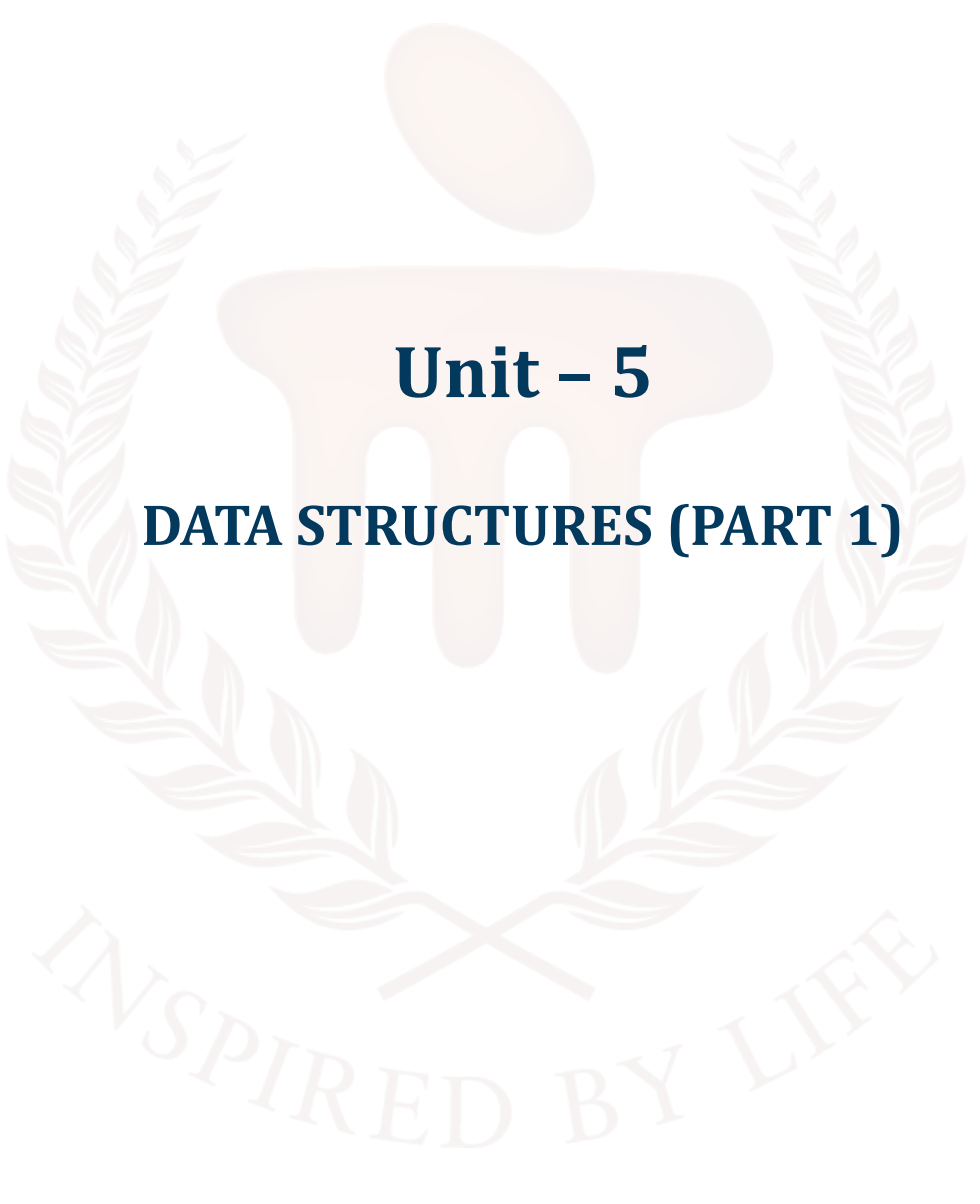
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 5

DATA STRUCTURES (PART 1)

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5 - 6
1.1	Learning Objectives	-	-	
2	Basics Of Lists In Python	-	1	7 – 12
2.1	Definition and Characteristics of Lists	-	-	
2.2	Creating and Initializing Lists	-	-	
2.3	Indexing and Accessing List Elements	-	-	
2.4	Lists vs Other Data Structures	-	-	
3	List Operations	-	2	13-27
3.1	Adding Elements (append(), insert(), extend())	-	-	
3.2	Removing Elements (remove(), pop(), clear())	-	-	
3.3	Updating Elements (Reassignment, Slices)	-	-	
3.4	Searching in Lists (in, not in, index(), count())	-	-	
3.5	Sorting and Reversing Lists (sort(), sorted(), reverse())	-	-	
3.6	Common Built-in Functions (len(), min(), max(), sum())	-	-	28-39
4	List Slicing In Python	-	3	
4.1	Understanding Slicing Syntax (list[start:end:step])	-	-	
4.2	Positive Indexing in Slices	-	-	
4.3	Negative Indexing in Slices	-	-	
4.4	Advanced Slicing Patterns (Skipping Elements, Reversing Lists)	-	-	
4.5	Nested Lists and Multidimensional Slicing	-	-	
4.6	Practical Use Cases of Slicing (Sub-lists, Copying)	-	-	
5	List Comprehension In Python	-	4	40-47

	5.1	Introduction to Comprehensions	-	-	
	5.2	Syntax and Structure of List Comprehension	-	-	
	5.3	Conditional List Comprehensions (if, if-else)	-	-	
	5.4	Nested List Comprehensions	-	-	
	5.5	Performance Benefits of List Comprehensions	-	-	
	5.6	Real-World Applications (Filtering Data, Transformations)	-	-	
6	Summary		-	-	48
7	Glossary		-	-	49
8	Terminal Questions		-	-	50
9	Answers		-	-	
	9.1	Self-Assessment Answers	-	-	51-52
	9.2	Terminal Question Answers	-	-	
10	References		-	-	53

1. INTRODUCTION

In this unit, learners will explore the fundamentals of Python lists, one of the most powerful and flexible data structures in Python programming. They will understand how lists act as ordered collections that can store multiple data items of different types. The unit begins by explaining the definition and characteristics of lists, followed by ways to create and initialise lists using various techniques. Learners will also understand indexing and accessing elements, and how lists differ from other data structures such as tuples and arrays.

Next, learners will study list operations, which form the core of Python programming. They will explore how to add, remove, update, and search elements efficiently using built-in methods like `append()`, `insert()`, `remove()`, and `index()`. The section also introduces techniques for sorting, reversing, and applying common built-in functions such as `len()`, `min()`, and `sum()` for quick computations.

They focus on list slicing, where learners will understand the slicing syntax, positive and negative indexing, and advanced slicing patterns used for copying or reversing lists. They will also learn to handle nested and multidimensional lists, which are useful in data processing applications.

Finally, learners will gain hands-on experience with list comprehensions, a concise and Pythonic way of creating lists. This includes understanding syntax, conditional expressions, nesting, and performance benefits. Real-world applications such as data filtering and transformation will highlight the practical power of list comprehensions.

1.1. Learning Objectives

- **Explain** the concept and properties of lists in Python.
- **Discuss** how to Create and manipulate lists using basic operations (add, remove, update, search).
- **Apply** slicing techniques to access and modify portions of lists.
- **Construct** lists efficiently using list comprehensions.
- **Demonstrate** practical applications of lists in solving everyday programming problems



2. BASICS OF LISTS IN PYTHON

Python is a powerful programming language, and one of its greatest strengths lies in the simplicity and flexibility of its data structures. Among these, the list is one of the most important and commonly used. Lists are widely used in almost every program, from small scripts to large applications, because they make it easy to group and manipulate data. Understanding lists is fundamental for mastering Python and for building more advanced concepts later, such as algorithms, machine learning workflows, and database operations.

In this section, we will explore the definition, properties, and operations of lists in detail. We will also compare them with other related structures like tuples and arrays. By the end, you will have a strong grasp of how lists work and why they are so essential in programming.

2.1 Definition and Characteristics of Lists

A list in Python is one of the most versatile and widely used data structures. It acts as an ordered collection of items, which means that elements are arranged in a sequence and can be retrieved using their position. The most attractive feature of Python lists is their flexibility. Unlike arrays in languages such as C or Java, where every element must be of the same type, lists in Python allow a mix of different data types. For example, a single list may hold integers, strings, floating-point values, and even another list. This makes them highly practical for real-world tasks where data is rarely uniform.

To understand lists better, let us look at their core characteristics in detail.

1. Ordered

Lists preserve the order in which items are added. If you create a list like `fruits = ["apple", "banana", "cherry"]`, the first element will always be "apple" unless explicitly changed. This property is important when working with data where sequence matters, such as processing steps in a workflow or maintaining chronological records.

2. Mutable

Unlike immutable structures such as tuples or strings, lists are mutable, meaning they can be altered after creation. You can add elements with methods like `.append()` or `.insert()`, remove elements with `.remove()` or `.pop()`, and update existing items by directly assigning a new value to an index. For instance:


```
numbers = [10, 20, 30]
numbers[1] = 25 # Updates second element
print(numbers) # Output: [10, 25, 30]
```

This property makes lists very powerful, especially in applications where data frequently changes.

3. Heterogeneous

Python lists can store mixed data types within a single structure. For example:

```
mixed = [1, "apple", 3.14, [2, 4, 6]]
```

Here, the list contains an integer, a string, a floating-point number, and another list (nested list). This heterogeneous nature makes lists extremely useful in situations where different types of information must be grouped together, such as student details containing names, roll numbers, and marks.

4. Dynamic Size

Lists in Python are not restricted to a fixed size. You can start with an empty list and keep adding items as needed. Similarly, items can be removed, shrinking the list's size dynamically. This feature makes lists more flexible than arrays in languages like C, where the size must be defined during declaration.

For example:

```
data = []
data.append(5)
data.append(10)
print(data) # Output: [5, 10]
```

The list grows automatically as items are added.

5. Index-Based Access

Each element in a list is identified by an index. Python uses zero-based indexing, meaning the first element has index 0, the second has index 1, and so on. Negative indexing is also supported, where -1 refers to the last element.

For example:

```
colors = ["red", "blue", "green"]
```

```
print(colors[0]) # Output: red
print(colors[-1]) # Output: green
```

This allows efficient and direct access to elements, making lists convenient for tasks where frequent lookups are required.

Example:

```
# Defining a list
my_list = [10, "Python", 3.5, True]
print(my_list)    # Output: [10, 'Python', 3.5, True]
print(type(my_list)) # Output: <class 'list'>
```

Logic: Here, we created a list that holds different types of elements—an integer, a string, a float, and a boolean. The `type()` function confirms that it is recognized as a list in Python.

2.2 Creating and Initializing Lists

There are multiple ways to create a list in Python.

1. Using Square Brackets

This is the most common method.

```
fruits = ["apple", "banana", "cherry"]
print(fruits) # Output: ['apple', 'banana', 'cherry']
```

2. Using the `list()` Constructor

```
numbers = list((1, 2, 3, 4))
print(numbers) # Output: [1, 2, 3, 4]
```

Logic: Here, we used the `list()` function to convert a tuple into a list.

3. Creating an Empty List

```
empty_list = []
print(empty_list) # Output: []
```

4. Using List Comprehension

List comprehension provides a compact way to generate lists.

```
squares = [x**2 for x in range(5)]
```

```
print(squares) # Output: [0, 1, 4, 9, 16]
```

Logic: The list comprehension `[x**2 for x in range(5)]` iterates from 0 to 4 and stores the square of each number in the list.

2.3 Indexing and Accessing List Elements

Lists are indexed, which means each element can be referred to by its position.

Positive Indexing

Python uses zero-based indexing, meaning the first element has index 0.

```
animals = ["cat", "dog", "lion", "tiger"]
```

```
print(animals[0]) # Output: cat
```

```
print(animals[2]) # Output: lion
```

Negative Indexing

Python also allows negative indexing, where -1 refers to the last element, -2 to the second last, and so on.

```
print(animals[-1]) # Output: tiger
```

```
print(animals[-2]) # Output: lion
```

Slicing

Slicing helps retrieve a portion of the list.

```
print(animals[1:3]) # Output: ['dog', 'lion']
```

```
print(animals[:2]) # Output: ['cat', 'dog']
```

```
print(animals[2:]) # Output: ['lion', 'tiger']
```

Logic:

- `animals[1:3]` means from index 1 up to (but not including) index 3.
- `animals[:2]` means from start till index 1.

- `animals[2:]` means from index 2 till the end.

2.4 Lists vs Other Data Structures

Python offers several data structures. Lists are versatile, but sometimes other structures like tuples or arrays are more suitable.

Lists vs Tuples

- **Mutability:** Lists are mutable, tuples are immutable.
- **Performance:** Tuples are faster for read-only operations.
- **Syntax:** Lists use `[]`, tuples use `()`.

Example:

```
my_list = [1, 2, 3]
my_tuple = (1, 2, 3)
my_list[0] = 10 # Allowed
# my_tuple[0] = 10 # Error: tuples are immutable
```

Logic: Lists allow modifications, while tuples do not.

Lists vs Arrays

Python has an array module, but lists are more commonly used.

- **Lists** can hold mixed data types.
- **Arrays** (from array module) store only one type, which makes them more memory efficient for numerical operations.

Example using array:

```
import array
arr = array.array('i', [1, 2, 3, 4])
print(arr[0]) # Output: 1
```

Logic: Here 'i' specifies that the array holds integers. Unlike lists, arrays cannot hold strings mixed with numbers..

Self-Assessment Questions-1

Multiple Choose Questions	
1	Which of the following statements about Python lists is true? a) Lists are immutable b) Lists can hold mixed data types c) Lists do not maintain order d) Lists cannot be resized
2	What will be the output of the following code? <pre>a = [1, 2, 3, 4] print(a[-2])</pre> a) 2 b) 3 c) 4 d) Error
3	Which of the following creates an empty list? a) empty = () b) empty = {} c) empty = [] d) empty = list(0)
4	Which of the following is not a difference between a list and a tuple? a) Lists are mutable, tuples are immutable b) Lists use square brackets, tuples use round brackets c) Lists are faster than tuples for read-only operations d) Tuples are generally more memory-efficient
5	Which of the following will generate a list of squares from 0 to 9? a) [x*2 for x in range(10)] b) [x**2 for x in range(10)] c) (x**2 for x in range(10)) d) {x**2 for x in range(10)}

3. LIST OPERATIONS

Python lists are one of the most powerful and flexible data structures in the language, and they form the foundation for handling collections of data. A list allows us to store multiple items, which may be integers, strings, floating-point values, or even other lists, in a single variable. What makes lists especially useful is their mutable nature. Being mutable means that the contents of a list can be changed at any time—elements can be added, removed, or updated without creating a new list. This characteristic sets lists apart from immutable structures like tuples and makes them indispensable in situations where data is expected to grow, shrink, or evolve during program execution.

To ensure programmers can work with lists efficiently, Python provides a wide range of operations that simplify handling and manipulation. These operations are designed to make lists both user-friendly and computationally efficient. For example, methods like `append()` and `insert()` help in adding elements to a list, while `remove()` and `pop()` allow unwanted items to be deleted. Updating existing elements is as simple as reassigning a value at a particular index, and slicing provides a clean way to modify a range of items in one step. Searching is supported through operators such as `in` and `not in`, or methods like `index()` and `count()`, which make it easy to locate elements or verify their presence.

Beyond basic insertion and deletion, lists also support powerful operations such as sorting and reversing. The `sort()` method, along with the built-in `sorted()` function, helps arrange data in ascending or descending order, which is critical in many algorithmic and real-world applications. Reversing the order of elements is also straightforward using `reverse()`. Furthermore, Python's built-in functions like `len()`, `min()`, `max()`, and `sum()` provide quick insights into the size, smallest element, largest element, and total of numeric lists respectively.

3.1 Adding Elements (`append()`, `insert()`, `extend()`)

Definition

- **`append()`** → Adds a single element to the end of the list. Useful for growing lists one item at a time.
- **`insert()`** → Adds an element at a specified index. Existing elements shift to the right to make space.

- **extend()** → Adds multiple elements from another list or iterable, effectively merging them into the original list.
- These methods cover most scenarios of list growth: `append()` for sequential additions, `insert()` for controlled placement, and `extend()` for combining datasets.
- Together, they ensure flexibility and efficiency in handling dynamic collections in Python.

Explanation

When working with lists, you often need to add more items. Python provides multiple ways:

1. If you only need to add one element at the end, use **append()**.
2. If you need to add at a specific position, use **insert()**.
3. If you want to combine two lists or add multiple items, use **extend()**.

Code Example 1: `append()`

```
fruits = ["apple", "banana", "cherry"]
fruits.append("mango")
print(fruits)
```

Output:

```
['apple', 'banana', 'cherry', 'mango']
```

Logic:

The `append()` function always adds the element to the end of the list. Here, "mango" was added as the last item.

Code Example 2: `insert()`

```
fruits = ["apple", "banana", "cherry"]
fruits.insert(1, "grape")
print(fruits)
```

Output:

```
['apple', 'grape', 'banana', 'cherry']
```


Logic:

The `insert()` method takes two arguments → (index, element). Here, "grape" is inserted at index 1 (second position).

Code Example 3: extend()

```
fruits = ["apple", "banana"]
extra_fruits = ["mango", "cherry", "orange"]
fruits.extend(extra_fruits)
print(fruits)
```

Output:

```
['apple', 'banana', 'mango', 'cherry', 'orange']
```

Logic:

The `extend()` function takes an iterable (like another list, tuple, or string) and adds all its elements one by one to the current list.

3.2 Removing Elements (`remove()`, `pop()`, `clear()`)

Definition

- **`remove()`** → Removes the first occurrence of the given element.
- **`pop()`** → Removes an element by index (default: last element).
- **`clear()`** → Removes all elements from the list.

Explanation

Removing elements is as important as adding them. Depending on the requirement:

1. Use **`remove()`** if you know the element's value.
2. Use **`pop()`** if you know the index or just want the last element removed.
3. Use **`clear()`** if you want to empty the entire list.

Code Example 1: remove()

```
fruits = ["apple", "banana", "cherry", "banana"]
fruits.remove("banana")
```

```
print(fruits)
```

Output:

```
['apple', 'cherry', 'banana']
```

Logic:

`remove("banana")` deletes the first occurrence of "banana". The second "banana" remains.

Code Example 2: pop()

```
fruits = ["apple", "banana", "cherry"]  
removed_item = fruits.pop(1)  
print(fruits)  
print("Removed:", removed_item)
```

Output:

```
['apple', 'cherry']  
Removed: banana
```

Logic:

`pop(1)` removes the element at index 1 and also returns it, allowing you to store it.

Code Example 3: clear()

```
fruits = ["apple", "banana", "cherry"]  
fruits.clear()  
print(fruits)
```

Output:

```
[]
```

Logic:

The `clear()` method completely empties the list, leaving an empty `[]`.

3.3 Updating Elements (Reassignment, Slices)

Definition

- Lists are mutable, meaning you can directly update elements by reassigning values.
- You can also update multiple values at once using slicing.

Code Example 1: Reassignment

```
fruits = ["apple", "banana", "cherry"]  
fruits[1] = "grape"  
print(fruits)
```

Output:

```
['apple', 'grape', 'cherry']
```

Logic:

We reassigned the element at index 1 from "banana" to "grape".

Code Example 2: Slices

```
fruits = ["apple", "banana", "cherry", "mango"]  
fruits[1:3] = ["grape", "orange"]  
print(fruits)
```

Output:

```
['apple', 'grape', 'orange', 'mango']
```

Logic:

The slice [1:3] includes index 1 and 2, so both "banana" and "cherry" are replaced.

3.4 Searching in Lists (in, not in, index(), count())

Definition

- in / not in → Checks if an element exists in the list.
- index() → Returns the index of the first occurrence.
- count() → Counts how many times an element appears.

Code Example 1: in, not in

```
fruits = ["apple", "banana", "cherry"]  
print("apple" in fruits)  
print("grape" not in fruits)
```

Output:

```
True  
True
```

Logic:

"apple" in fruits returns True since "apple" is present. "grape" not in fruits is also True since "grape" is absent.

Code Example 2: index() and count()

```
fruits = ["apple", "banana", "cherry", "banana"]  
print(fruits.index("banana"))  
print(fruits.count("banana"))
```

Output:

```
1  
2
```

Logic:

- index("banana") → returns 1 (first occurrence).

- `count("banana")` → returns 2 since "banana" occurs twice.

3.5 Sorting and Reversing Lists (`sort()`, `sorted()`, `reverse()`)

Definition

- `sort()` → Sorts the list in-place (ascending by default).
- `sorted()` → Returns a new sorted list, leaving the original unchanged.
- `reverse()` → Reverses the list order.

Code Example 1: `sort()`

```
numbers = [4, 1, 7, 3, 9]
numbers.sort()
print(numbers)
```

Output:

```
[1, 3, 4, 7, 9]
```

Logic:

`sort()` rearranges the list in ascending order by default.

Code Example 2: `sorted()`

```
numbers = [4, 1, 7, 3, 9]
sorted_numbers = sorted(numbers, reverse=True)
print(sorted_numbers)
print(numbers)
```

Output:

```
[9, 7, 4, 3, 1]
[4, 1, 7, 3, 9]
```

Logic:

- `sorted()` returns a new list, sorted in descending order (due to `reverse=True`).

- The original list remains unchanged.

Code Example 3: reverse()

```
fruits = ["apple", "banana", "cherry"]  
fruits.reverse()  
print(fruits)
```

Output:

```
['cherry', 'banana', 'apple']
```

Logic:

reverse() flips the list order.

3.6 Common Built-in Functions (len(), min(), max(), sum())

Definition

- **len()** → Returns number of elements.
- **min()** → Returns smallest element.
- **max()** → Returns largest element.
- **sum()** → Returns total sum of numeric elements.

Code Example

```
numbers = [5, 10, 15, 20]  
print(len(numbers))  
print(min(numbers))  
print(max(numbers))  
print(sum(numbers))
```

Output:

```
4  
5
```

20

50

Logic:

- **len(numbers)** → 4 items.
- **min(numbers)** → smallest is 5.
- **max(numbers)** → largest is 20.
- **sum(numbers)** → total is 50.

Example 1: Student Marks Management System**Problem Explanation**

Suppose you are managing a list of students' marks in a Python program. You need to:

1. Add new marks (append, insert, extend).
2. Remove incorrect marks (remove, pop, clear).
3. Update a student's marks (reassignment, slicing).
4. Search for specific marks (in, not in, index, count).
5. Sort and reverse the list.
6. Use built-in functions like len(), min(), max(), sum() to get useful statistics.

Code**# Initial list of student marks**

```
marks = [85, 90, 78, 92]
```

Adding Elements

```
marks.append(88)           # append new mark at end
marks.insert(2, 95)        # insert at index 2
marks.extend([70, 60])     # extend with multiple values
print("After Adding:", marks)
```


Removing Elements

```
marks.remove(70)          # remove a specific mark
removed = marks.pop(1)     # pop by index (index 1 = second element)
print("Removed value:", removed)
marks_copy = marks.copy()
marks_copy.clear()        # clear all from copy
print("After Removing:", marks)
```

Updating Elements

```
marks[0] = 89             # reassign first element
marks[2:4] = [80, 82]     # update a slice
print("After Updating:", marks)
```

Searching in List

```
print("Is 90 in marks?", 90 in marks)
print("Count of 88:", marks.count(88))
print("Index of 82:", marks.index(82))
```

Sorting and Reversing

```
marks.sort()
print("Sorted Marks:", marks)
marks.reverse()
print("Reversed Marks:", marks)
```

Common Built-in Functions

```
print("Total Students:", len(marks))
print("Highest Marks:", max(marks))
print("Lowest Marks:", min(marks))
print("Sum of Marks:", sum(marks))
```

Output

After Adding: [85, 95, 90, 78, 92, 88, 70, 60]

Removed value: 95

After Removing: [89, 90, 80, 82, 92, 88, 60]

Is 90 in marks? True

Count of 88: 1

Index of 82: 3

Sorted Marks: [60, 80, 82, 88, 89, 90, 92]

Reversed Marks: [92, 90, 89, 88, 82, 80, 60]

Total Students: 7

Highest Marks: 92

Lowest Marks: 60

Sum of Marks: 581

Logic Explanation

1. Adding:

- `append(88)` adds 88 at the end.
- `insert(2,95)` adds 95 at position 2.
- `extend([70,60])` merges new marks.

2. Removing:

- `remove(70)` deletes the first 70.
- `pop(1)` removes and returns the second element.
- `clear()` empties the list copy.

3. Updating:

- `marks[0] = 89` updates the first student's mark.
- `marks[2:4] = [80,82]` updates multiple values at once.

4. Searching:

- 90 in marks checks membership.
- count(88) finds how many times 88 appears.
- index(82) returns the position of 82.

5. **Sorting/Reversing:**

- sort() arranges marks in ascending order.
- reverse() flips the list order.

6. **Built-in Functions:**

- len() gives total students.
- max() and min() give highest and lowest marks.
- sum() gives total marks scored.

Example 2: Shopping Cart Operations

Problem Explanation

Imagine an e-commerce app where you maintain a shopping cart list. The cart should support:

1. Adding new products (append, insert, extend).
2. Removing items (remove, pop, clear).
3. Updating product names.
4. Searching for specific items.
5. Sorting and reversing product names.
6. Using built-in functions on prices.

Code

Initial shopping cart with product names

```
cart = ["Laptop", "Phone", "Tablet"]
```

Adding Elements

```
cart.append("Headphones")      # add at end
cart.insert(1, "Smartwatch")    # insert at index
cart.extend(["Charger", "Keyboard"]) # add multiple items
```

```
print("Cart after Adding:", cart)
```

Removing Elements

```
cart.remove("Charger")          # remove by value
removed_item = cart.pop(2)      # remove by index
print("Removed item:", removed_item)
cart_copy = cart.copy()
cart_copy.clear()              # clear copy for demonstration
print("Cart after Removing:", cart)
```

Updating Elements

```
cart[0] = "Gaming Laptop"      # update single element
cart[1:3] = ["iPhone", "iPad"] # update slice
print("Cart after Updating:", cart)
```

Searching

```
print("Is 'iPad' in cart?", "iPad" in cart)
print("Index of iPhone:", cart.index("iPhone"))
print("Count of Tablet:", cart.count("Tablet"))
```

Sorting and Reversing

```
cart.sort()
print("Sorted Cart:", cart)
cart.reverse()
print("Reversed Cart:", cart)
```

Built-in Functions with Prices

```
prices = [1000, 800, 1200, 500, 300]
```

```
print("Total Items:", len(prices))  
print("Cheapest Item:", min(prices))  
print("Most Expensive Item:", max(prices))  
print("Total Bill:", sum(prices))
```

Output

Cart after Adding: ['Laptop', 'Smartwatch', 'Phone', 'Tablet', 'Headphones', 'Charger', 'Keyboard']

Removed item: Phone

Cart after Removing: ['Laptop', 'Smartwatch', 'Tablet', 'Headphones', 'Keyboard']

Cart after Updating: ['Gaming Laptop', 'iPhone', 'iPad', 'Headphones', 'Keyboard']

Is 'iPad' in cart? True

Index of iPhone: 1

Count of Tablet: 0

Sorted Cart: ['Gaming Laptop', 'Headphones', 'Keyboard', 'iPad', 'iPhone']

Reversed Cart: ['iPhone', 'iPad', 'Keyboard', 'Headphones', 'Gaming Laptop']

Total Items: 5

Cheapest Item: 300

Most Expensive Item: 1200

Total Bill: 3800

Logic Explanation

1. **Adding:** Cart grows with new items using append, insert, and extend.
2. **Removing:** remove deletes "Charger", pop(2) deletes the 3rd item (Phone), clear() empties a copy.
3. **Updating:** We replaced "Laptop" with "Gaming Laptop" and updated a slice with "iPhone" and "iPad".
4. **Searching:** Membership operators and methods help locate items (index, count).
5. **Sorting/Reversing:** Items are alphabetically sorted, then reversed for the opposite order.

6. **Built-in Functions:** Using prices, we computed item count, cheapest, most expensive, and total bill.

SELF-ASSESSMENT QUESTIONS – 2

Fill in the blanks:	
6	The method used to add an element at the end of a list is _____.
7	To remove the last element of a list, we use _____.
8	The method _____ returns the number of times a value occurs in a list.
9	The built-in function _____ is used to calculate the total of list elements.
10	To reverse a list in place, we use the _____ method.

4. LIST SLICING IN PYTHON

Slicing in Python is a technique that allows programmers to access a specific portion of a list instead of working with the entire collection. It is based on the concept of selecting a range of elements by specifying the starting index, the ending index, and an optional step value. The general form of slicing is written as `list[start:end:step]`, where the start index is inclusive, the end index is exclusive, and the step determines how the traversal takes place. If the start index is left empty, Python begins slicing from the beginning of the list, and if the end index is not provided, it continues until the last element. Similarly, leaving the step blank assumes a default step of one. Slicing also supports negative indexing, which allows elements to be accessed from the end of the list, making it possible to extract sections in reverse order. This makes slicing not only powerful but also highly flexible for tasks like subsetting data, reversing sequences, or skipping values at regular intervals. Unlike loops, slicing provides a concise and efficient way to extract parts of a list without writing multiple lines of code, which is why it is frequently used in both simple scripts and complex data analysis tasks.

4.1 Understanding Slicing Syntax (`list[start:end:step]`)

Definition and Explanation

Slicing in Python allows us to access a portion of a list without writing explicit loops. Instead of iterating through elements one by one, Python provides a clean and efficient slicing operator `[start:end:step]`. This operator lets us extract specific sections of a list based on starting index, ending index, and step size.

- `start` → The index from where slicing begins (inclusive).
- `end` → The index where slicing ends (exclusive).
- `step` → The interval between elements (default is 1).

If any of these are left empty, Python assumes default values:

- `start = 0` (beginning of the list)
- `end = len(list)` (till the end of the list)
- `step = 1` (move element by element)

This makes slicing flexible and very powerful.

Code Example

Example of slicing

```
numbers = [10, 20, 30, 40, 50, 60, 70]
```

Slice from index 1 to 5

```
subset = numbers[1:5]
```

```
print(subset) # Output: [20, 30, 40, 50]
```

Slice with step

```
step_slice = numbers[0:6:2]
```

```
print(step_slice) # Output: [10, 30, 50]
```

Logic Explanation

- In `numbers[1:5]`, Python starts from index 1 (value = 20) and goes up to index 5 (value = 60), but does not include it.
- In `numbers[0:6:2]`, it starts at index 0 (value = 10), stops at index 6 (value = 70 but excluded), and selects every 2nd element.

This simple syntax makes slicing both time-saving and readable, avoiding the need for manual loops.

4.2 Positive Indexing in Slices

Definition and Explanation

Python lists use zero-based indexing. Positive indexing means counting from left to right, where the first element has index 0, the second element has index 1, and so on. Slicing with positive indexes is the most straightforward way to extract elements.

Code Example

```
fruits = ["apple", "banana", "cherry", "date", "mango"]
```

Slice first three elements

```
first_three = fruits[0:3]
```

```
print(first_three) # Output: ['apple', 'banana', 'cherry']
```

Slice middle part

```
middle = fruits[1:4]
```

```
print(middle) # Output: ['banana', 'cherry', 'date']
```

Logic Explanation

- In `fruits[0:3]`, slicing starts from index 0 ("apple") and goes till index 3 but excludes it.
- In `fruits[1:4]`, it starts at "banana" and stops before index 4, giving "banana", "cherry", "date".

Positive indexing is simple and useful when we know the positions of elements from the start of the list.

4.3 Negative Indexing in Slices

Definition and Explanation

Negative indexing allows counting elements from the end of the list. The last element has index -1, the second-last has index -2, and so on. This is useful when we want to access the tail portion of the list without calculating its exact length.

Code Example

```
letters = ['a', 'b', 'c', 'd', 'e', 'f']
```

Slice last 3 elements

```
last_three = letters[-3:]
```

```
print(last_three) # Output: ['d', 'e', 'f']
```

Slice from -5 to -2

```
sub_part = letters[-5:-2]
```

```
print(sub_part) # Output: ['b', 'c', 'd']
```

Logic Explanation

- In letters[-3:], Python starts from the 3rd element from the end ('d') and goes till the last element.
- In letters[-5:-2], it starts from the 5th element from the end ('b') and stops just before index -2 ('e').

Negative indexing makes slicing more dynamic, especially when list sizes are not fixed.

4.4 Advanced Slicing Patterns (Skipping Elements, Reversing Lists)

Definition and Explanation

The real strength of slicing comes from using the step parameter creatively. By adjusting step, we can skip elements or even reverse the list without writing loops.

- **Skipping Elements:** By using a step greater than 1, we can select every 2nd, 3rd, or nth element from the list.
- **Reversing a List:** By using a negative step (-1), Python walks backward through the list, effectively reversing it.

These patterns are especially useful in tasks such as sampling data, reversing sequences, or skipping over unwanted elements.

Code Example: Skipping Elements

```
nums = [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

Select every 2nd element

```
evens = nums[::2]
```

```
print(evens) # Output: [1, 3, 5, 7, 9]
```

Select every 3rd element

```
thirds = nums[::3]
```

```
print(thirds) # Output: [1, 4, 7]
```

Logic Explanation

- In `nums[::2]`, the step is 2, so Python picks elements at indexes 0, 2, 4, 6, 8.
- In `nums[::3]`, the step is 3, so indexes 0, 3, 6 are selected.

This allows us to filter out elements at regular intervals quickly.

Code Example: Reversing Lists

```
nums = [10, 20, 30, 40, 50]
```

Reverse the entire list

```
reversed_list = nums[::-1]
```

```
print(reversed_list) # Output: [50, 40, 30, 20, 10]
```

Reverse only a portion

```
reverse_partial = nums[4:1:-1]
```

```
print(reverse_partial) # Output: [50, 40, 30]
```

Logic Explanation

- In `nums[::-1]`, the step is -1, so Python starts from the end and moves backward.
- In `nums[4:1:-1]`, slicing begins at index 4 (50) and moves backward until just before index 1.

This technique is especially handy in algorithms that need quick reversal (like checking palindromes or reversing arrays).

4.5 Nested Lists and Multidimensional Slicing

Definition and Explanation

Lists in Python can contain other lists inside them. These are called nested lists and are often used to represent matrices, tables, or multidimensional data.

Slicing nested lists works by applying slicing rules to each list individually. To access specific elements or sub-lists, we combine indexing and slicing together.

Code Example: Nested Lists

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]
```

Extract the first row

```
row1 = matrix[0]  
print(row1) # Output: [1, 2, 3]
```

Extract first two rows

```
rows = matrix[0:2]  
print(rows) # Output: [[1, 2, 3], [4, 5, 6]]
```

Logic Explanation

- `matrix[0]` extracts the entire first sub-list.
- `matrix[0:2]` slices the outer list, giving the first two rows.

Code Example: Multidimensional Slicing

Extract a column (2nd element of each row)

```
second_column = [row[1] for row in matrix]
```

```
print(second_column) # Output: [2, 5, 8]
```

Extract sub-matrix

```
sub_matrix = [row[0:2] for row in matrix[1:3]]
```

```
print(sub_matrix) # Output: [[4, 5], [7, 8]]
```

Logic Explanation

- In `[row[1] for row in matrix]`, we iterate over each row and extract the element at index 1. This simulates column slicing.
- In `[row[0:2] for row in matrix[1:3]]`, slicing is applied twice:
 - `matrix[1:3]` selects the last two rows.
 - `row[0:2]` then slices each row to pick only the first two columns.

This is how Python can mimic 2D slicing similar to NumPy arrays.

4.6 Practical Use Cases of Slicing (Sub-lists, Copying)

1. Extracting Sub-lists

Slicing allows breaking a list into smaller, meaningful parts.

Example: Splitting a list of student scores into halves.

```
scores = [45, 67, 89, 72, 56, 91, 84, 77]
```

First half

```
first_half = scores[:4]
```

```
print(first_half) # [45, 67, 89, 72]
```

Second half

```
second_half = scores[4:]  
  
print(second_half) # [56, 91, 84, 77]
```

Logic: [:4] gets everything before index 4, while [4:] gets everything from index 4 to the end.

2. Copying Lists

A very common use of slicing is to create shallow copies.

```
original = [1, 2, 3, 4, 5]  
copy_list = original[:]  
  
print(copy_list)    # [1, 2, 3, 4, 5]  
print(original == copy_list) # True  
print(original is copy_list) # False
```

Logic:

- original[:] creates a new list with the same contents.
- == checks if the values are equal.
- is checks if both refer to the same object in memory.

Thus, slicing ensures we get a copy without altering the original list.

3. Removing or Selecting Patterns

We can slice lists to filter patterns (e.g., alternate elements, odd/even positions).

Extract odd-indexed elements

```
nums = [10, 20, 30, 40, 50, 60]  
odd_positions = nums[1::2]  
  
print(odd_positions) # [20, 40, 60]
```

Logic:

Here, we start at index 1 (20) and pick every 2nd element → indices [1, 3, 5].

4. Palindrome Checking

```
word = "radar"
```

```
if word == word[::-1]:
```

```
    print("Palindrome")
```

```
else:
```

```
    print("Not Palindrome")
```

Logic: word[::-1] reverses the string (or list). If reversed equals original, it's a palindrome.

Example 1: Student Score Analysis

Problem:

A class has 12 students, and their test scores are stored in a list. We need to:

1. Extract top half and bottom half of students (sub-lists).
2. Pick every alternate student's score (skipping elements).
3. Get the last 3 scores using negative indexing.
4. Reverse the entire list.
5. Copy the list to ensure original data is safe.
6. Create a sub-matrix of grouped scores for project teams (nested slicing).

Code Example

Scores of 12 students

```
scores = [55, 67, 72, 81, 64, 90, 77, 83, 59, 68, 74, 88]
```

1. Top half and bottom half

```
top_half = scores[:6]    # First 6 students
```

```
bottom_half = scores[6:] # Last 6 students
```

2. Every alternate student's score

```
alternate = scores[::2]
```

3. Last 3 scores

```
last_three = scores[-3:]
```

4. Reverse all scores

```
reversed_scores = scores[::-1]
```

5. Copy list

```
copied_scores = scores[:]
```

6. Nested slicing - divide into project groups of 4

```
groups = [scores[i:i+4] for i in range(0, len(scores), 4)]
```

Logic Explanation

- `scores[:6]` → takes first 6 scores using positive indexing.
- `scores[6:]` → from index 6 till end → bottom half.
- `scores[::2]` → step = 2, selecting alternate students.
- `scores[-3:]` → last 3 scores via negative indexing.
- `scores[::-1]` → reverse using step = -1.
- `scores[:]` → full copy of the list.
- `[scores[i:i+4] for i in range(0, len(scores), 4)]` → creates 3 groups of 4 students each (nested slicing pattern).

This one example covers sub-lists, skipping, negative indexing, reversing, copying, and nested slicing.

Example 2: Movie Database Operations

Problem:

A movie database stores film titles in a list. We need to:

1. Extract a slice of movies released between index positions 2 to 6.
2. Get only every 3rd movie (skipping).
3. Select the last 5 movies using negative slicing.
4. Reverse the movie list to display latest first.
5. Make a copy before making changes.
6. Organize movies into rows and columns (nested lists) like a catalog grid.

Code Example

```
movies = [  
    "Inception", "Interstellar", "The Dark Knight", "Memento", "Dunkirk",  
    "Tenet", "Prestige", "Batman Begins", "Oppenheimer", "Insomnia"  
]
```

1. Slice between index 2 and 6

```
subset = movies[2:7] # The Dark Knight → Tenet
```

2. Every 3rd movie

```
every_third = movies[::3]
```

3. Last 5 movies

```
last_five = movies[-5:]
```

4. Reverse list (latest first)

```
reversed_list = movies[::-1]
```

5. Copy before modifying

```
backup = movies[:]
```

6. Nested slicing - arrange in rows of 3

```
catalog = [movies[i:i+3] for i in range(0, len(movies), 3)]
```

Logic Explanation

- `movies[2:7]` → slicing from index 2 to 6.
- `movies[::3]` → picks every 3rd movie.
- `movies[-5:]` → last 5 movies using negative indexing.
- `movies[::-1]` → reversed to show latest releases first.
- `movies[:]` → backup list so edits won't affect original.
- `[movies[i:i+3] for i in range(0, len(movies), 3)]` → nested slicing to create 2D grid:

```
[['Inception', 'Interstellar', 'The Dark Knight'],  
 ['Memento', 'Dunkirk', 'Tenet'],  
 ['Prestige', 'Batman Begins', 'Oppenheimer'],  
 ['Insomnia']].
```

Self-Assessment Questions-3

True/False	
11	The slice numbers[2:5] will include the element at index 5.
12	Negative indexing in slicing allows us to count from the end of the list.
13	The slice list[::-1] creates a reversed version of the list.
14	Using list[:] creates a reference to the same list, not a copy.
15	Nested lists can be sliced just like one-dimensional lists, but slicing must be applied carefully at each level.

5. LIST COMPREHENSION IN PYTHON

List comprehension is a compact and elegant method in Python for creating new lists by applying an expression to each item of an existing iterable. It combines looping and conditional filtering into a single readable statement, making the code shorter and more expressive compared to traditional approaches using for loops. The structure of a list comprehension follows the pattern [expression for item in iterable if condition], where the expression defines what transformation will be applied, the item refers to each element being processed, and the condition, which is optional, determines whether the element should be included in the resulting list. List comprehension is not only concise but also computationally efficient, as it avoids the repeated function calls involved in methods like `append()`. It can handle conditions using both `if` and `if-else` constructs, and even supports nested comprehensions when working with multi-dimensional data structures. In practice, list comprehensions are widely used for filtering values, transforming datasets, flattening nested collections, and preparing input for further computation. Their combination of simplicity, clarity, and performance benefits makes them one of the most powerful features of Python's list operations, encouraging developers to write clean and Pythonic code.

5.1 Introduction to Comprehensions

In Python, working with collections like lists is a daily task. Programmers often need to create new lists, filter values, or transform existing data into a different format. Traditionally, this was done using loops like `for` or `while`, combined with conditional statements. While this works, it can sometimes lead to longer code that is harder to read.

List comprehension provides a short, elegant, and Pythonic way of creating new lists from existing sequences or iterables. Instead of writing multiple lines of code, a single line can achieve the same goal.

Definition:

List comprehension is a compact way of generating new lists by applying an expression to each item in an iterable.

Why it matters:

- Saves time by reducing the number of lines of code.
- Improves readability (once you're familiar with the syntax).
- Allows filtering and transformation in one step.

Example without list comprehension:

```
squares = []  
for i in range(1, 6):  
    squares.append(i * i)  
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

With list comprehension:

```
squares = [i * i for i in range(1, 6)]  
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

Logic:

- In the first example, we used a loop and `append()`.
- In the second, we directly wrote the logic in one line.
- Both outputs are the same, but the second is shorter and clearer.

5.2 Syntax and Structure of List Comprehension

The general syntax of a list comprehension is:

```
[expression for item in iterable if condition]
```

- `expression` → the operation or value you want to store in the new list.
- `item` → the variable that takes values from the iterable (like `for i in ...`).

- iterable → the sequence or object you are looping over (list, string, range, etc.).
- if condition → (optional) a filter to decide whether to include the item.

Basic Example:

```
numbers = [1, 2, 3, 4, 5]
doubles = [x * 2 for x in numbers]
print(doubles)
```

Output:

[2, 4, 6, 8, 10]

Logic:

- For each element x in numbers, multiply it by 2.
- Store the result in a new list doubles.

5.3 Conditional List Comprehensions (if, if-else)

One of the strongest features of list comprehension is the ability to filter elements using conditions.

1. With if condition

Only elements that meet the condition are included.

```
numbers = [1, 2, 3, 4, 5, 6]
evens = [x for x in numbers if x % 2 == 0]
print(evens)
```

Output:

[2, 4, 6]

Logic:

- Loop through numbers 1 to 6.
- Only include those divisible by 2 ($x \% 2 == 0$).

2. With if-else

We can apply a transformation depending on the condition.

```
numbers = [1, 2, 3, 4, 5]
labels = ["even" if x % 2 == 0 else "odd" for x in numbers]
print(labels)
```

Output:

```
['odd', 'even', 'odd', 'even', 'odd']
```

Logic:

- Each number is checked.
- If divisible by 2 → label as "even", otherwise "odd".

5.4 Nested List Comprehensions

Sometimes, we need multiple loops inside a comprehension. This is called nested list comprehension.

Example 1: Creating pairs

```
pairs = [(x, y) for x in [1, 2, 3] for y in [10, 20, 30]]
print(pairs)
```

Output:

```
[(1, 10), (1, 20), (1, 30), (2, 10), (2, 20), (2, 30), (3, 10), (3, 20), (3, 30)]
```

Logic:

- First loop: x takes values 1, 2, 3.
- Second loop: y takes values 10, 20, 30.
- Each combination of (x, y) is added to the list.

Example 2: Flattening a matrix

```
matrix = [[1, 2], [3, 4], [5, 6]]  
  
flattened = [num for row in matrix for num in row]  
  
print(flattened)
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

Logic:

- Outer loop goes row by row.
- Inner loop extracts each number.
- Final result is a flat list.

5.5 Performance Benefits of List Comprehensions

List comprehensions are not just concise, but also faster compared to traditional loops. This is because:

1. They are optimized in C at the Python interpreter level.
2. They reduce overhead by avoiding repeated calls to methods like `append()`.

Performance Test Example:

```
import time  
  
# Using for loop  
  
start = time.time()  
  
squares = []  
  
for i in range(1, 1000000):  
    squares.append(i * i)  
  
end = time.time()  
  
print("For loop time:", end - start)
```

Using list comprehension

```
start = time.time()

squares = [i * i for i in range(1, 1000000)]

end = time.time()

print("List comprehension time:", end - start)
```

Expected Output (times will vary):

For loop time: 0.45s

List comprehension time: 0.31s

Logic:

- Both produce the same list.
- Comprehension runs faster because it executes internally in optimized bytecode.

5.6 Real-World Applications (Filtering Data, Transformations)

List comprehensions are widely used in real-world scenarios.

1. Filtering Data

Suppose we want to filter out names that start with "A":

```
names = ["Alice", "Bob", "Angela", "Mark", "Ankit"]

filtered = [name for name in names if name.startswith("A")]

print(filtered)
```

Output:

```
['Alice', 'Angela', 'Ankit']
```

2. Transformations

Convert temperatures from Celsius to Fahrenheit:

```
celsius = [0, 10, 20, 30, 40]

fahrenheit = [(temp * 9/5) + 32 for temp in celsius]
```

```
print(fahrenheit)
```

Output:

```
[32.0, 50.0, 68.0, 86.0, 104.0]
```

3. Extracting Specific Values

Get all email domains from a list of emails:

```
emails = ["alex@gmail.com", "maria@yahoo.com", "john@outlook.com"]
```

```
domains = [email.split("@")[1] for email in emails]
```

```
print(domains)
```

Output:

```
['gmail.com', 'yahoo.com', 'outlook.com']
```

4. Flattening Nested Data

Useful in machine learning data preprocessing:

```
dataset = [[1, 2], [3, 4], [5, 6]]
```

```
flattened = [num for row in dataset for num in row]
```

```
print(flattened)
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

5. Working with Dictionaries

Create a dictionary from two lists:

```
keys = ["name", "age", "city"]
```

```
values = ["Alice", 25, "New York"]
```

```
dictionary = {k: v for k, v in zip(keys, values)}
```

```
print(dictionary)
```

Output:

```
{'name': 'Alice', 'age': 25, 'city': 'New York'}
```

SELF-ASSESSMENT QUESTIONS – 4

Column A

Match The Following		
Column A		Column B
16	<code>[x**2 for x in range(5)]</code>	a) Creates pairs of numbers as tuples.
17	<code>[x for x in range(10) if x % 2 == 0]</code>	b) Returns labels for even/odd numbers.
18	<code>["even" if x % 2 == 0 else "odd" for x in range(5)]</code>	c) Generates squares of numbers.
19	<code>[(x, y) for x in [1,2] for y in [3,4]]</code>	d) Flattens a nested list.
20	<code>[num for row in [[1,2],[3,4]] for num in row]</code>	e) Selects only even numbers from a range.

6. SUMMARY

- Lists in Python are versatile, ordered, and mutable collections that can store heterogeneous data types.
- They support multiple operations like adding, removing, updating, searching, and sorting, making them highly flexible for day-to-day programming.
- Indexing allows direct access to elements, while negative indexing simplifies retrieval from the end of a list.
- Slicing enables extracting sublists using the syntax `list[start:end:step]`, supporting both positive and negative indexing.
- Advanced slicing patterns allow skipping elements, reversing lists, and copying efficiently without loops.
- Nested lists enable multidimensional data storage, and slicing works on them to retrieve specific rows or columns.
- List comprehensions provide a compact, Pythonic way to create new lists by combining loops, expressions, and conditions.
- They support conditional and nested structures, making data filtering and transformations concise.
- Lists, along with slicing and comprehensions, form the foundation for handling data efficiently in Python applications.

7. GLOSSARY

List	- A mutable, ordered Python data structure that stores multiple elements.
Tuple	- An immutable, ordered collection in Python often used for fixed data.
Indexing	- The method of accessing elements of a list by their position.
Negative Indexing	- Refers to counting elements from the end of a list, starting at -1.
Slicing	- A technique to extract a range of elements from a list using [start:end:step].
Append	- A list method that adds a single element to the end of the list.
Pop	- A list method that removes and returns an element at a specified index.
List Comprehension	- A compact syntax for creating lists from existing iterables.
Nested List	- A list containing other lists as elements, supporting multidimensional data.
Iterable	- Any Python object capable of returning its members one at a time, such as lists, tuples, or strings.

8. TERMINAL QUESTIONS

1. What are the key characteristics of Python lists?
2. How can you create and initialize a list in Python?
3. Explain the difference between lists and tuples with examples.
4. Describe three methods of adding elements to a list.
5. How does the `remove()` method differ from the `pop()` method in lists?
6. What is slicing in Python lists? Give an example.
7. How does negative indexing work in slicing?
8. Define list comprehension and explain its general syntax.
9. What are the performance benefits of using list comprehensions over loops?
10. Provide two real-world applications where list comprehensions are commonly used.

9. ANSWERS

9.1. Self-Assessment Answers:

1. b) Lists can hold mixed data types
2. b) 3 (since -2 refers to the second last element)
3. c) empty = []
4. c) Lists are faster than tuples for read-only operations (incorrect; tuples are faster)
5. b) [x**2 for x in range(10)]
6. append()
7. pop()
8. count()
9. sum()
10. reverse()
11. False
12. True
13. True
14. False
15. True
16. → c) Generates squares of numbers.
17. → e) Selects only even numbers from a range.
18. → b) Returns labels for even/odd numbers.
19. → a) Creates pairs of numbers as tuples.
20. → d) Flattens a nested list.

9.2. Terminal Question Answers

Answer 1: Python lists are ordered, mutable collections that can store multiple types of data including numbers, strings, and even other lists. They maintain insertion order, allow duplicate elements, and provide powerful methods for modification and iteration.

Answer 2: A list can be created using square brackets, for example `my_list = [1, 2, 3]`. Lists can also be initialized as empty using `[]` or `list()`. They can hold heterogeneous values such as `[10, "hello", 3.14]`, which makes them very flexible.

Answer 3: Lists are mutable, meaning elements can be changed after creation, while tuples are immutable. For example, `my_list[0] = 20` updates a list, but attempting the same with a tuple raises an error. Lists are slower than tuples in read-only contexts due to mutability.

Answer 4: The `append()` method adds a single element to the end of a list, `insert()` places an element at a specified position, and `extend()` appends all items from another iterable to the current list. Together, they make lists highly dynamic.

Answer 5: The `remove()` method deletes the first occurrence of a specified value, while `pop()` removes an element at a given index and returns it. If no index is specified, `pop()` removes the last element. Both methods alter the original list.

Answer 6: Slicing is a technique to access parts of a list using `[start:end:step]`. For example, `numbers = [1, 2, 3, 4, 5]`; `print(numbers[1:4])` outputs `[2, 3, 4]`. It provides an efficient way to extract sublists without writing loops.

Answer 7: Negative indexing counts elements from the end of the list. For example, `numbers[-1]` returns the last element. When combined with slicing, `numbers[::-1]` reverses a list. This feature simplifies accessing data from the tail of collections.

Answer 8: List comprehension is a concise syntax to create lists using an expression followed by a loop and an optional condition. Its general structure is `[expression for item in iterable if condition]`. It makes code shorter, readable, and efficient.

Answer 9: List comprehensions run faster than traditional loops because they are implemented in optimized C bytecode, reducing overhead from repeated method calls like `append()`. They also minimize boilerplate code, improving readability while maintaining efficiency.

Answer 10: List comprehensions are commonly used in data filtering (e.g., extracting only even numbers from a dataset) and data transformation (e.g., converting temperatures from Celsius to Fahrenheit). They are also applied in machine learning preprocessing tasks.

10. REFERENCES

- Sweigart, A. (2020). Automate the Boring Stuff with Python (2nd ed.). No Starch Press. Retrieved from <https://automatetheboringstuff.com/>
- Van Rossum, G., & Drake, F. L. (2023). The Python Language Reference (Version 3.11). Python Software Foundation. Retrieved from <https://docs.python.org/3/library/stdtypes.html#mapping-types-dict>
- Real Python. (2023). Python Dictionaries: A Comprehensive Guide. Retrieved from <https://realpython.com/python-dicts/>
- Programiz. (2024). Python Dictionary (With Examples). Retrieved from <https://www.programiz.com/python-programming/dictionary>

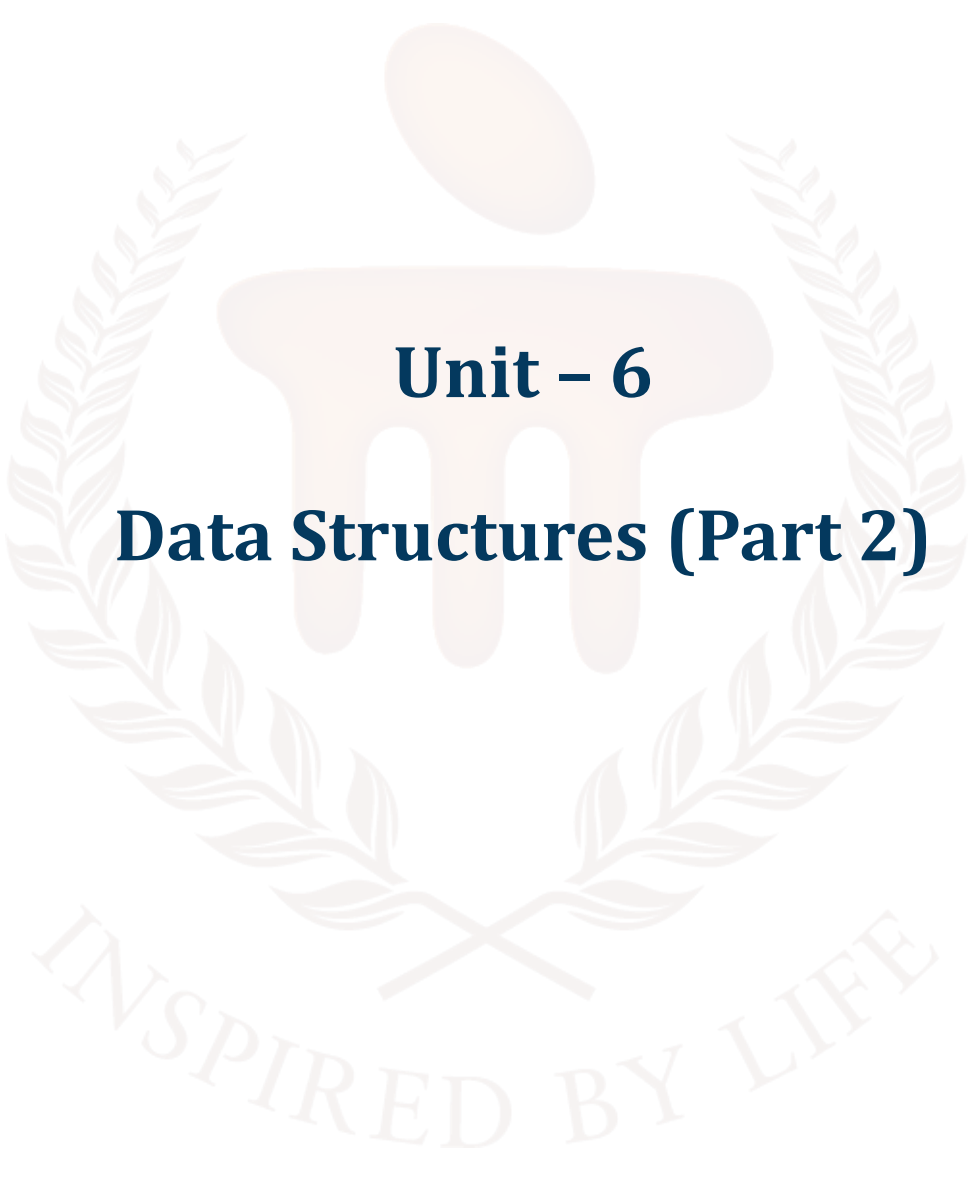
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 6

Data Structures (Part 2)

TABLE OF CONTENTS

SL No	Topic	Fig No / Table / Graph	SAQ / Activity	Page No
1	Introduction	-	-	5-6
1.1	Learning Objectives	-	-	
2	Basics Of Sets In Python	-	1	7-24
2.1	Definition and Characteristics of Sets	-	-	
2.2	Creating and Initialising Sets	-	-	
2.3	Properties of Sets (Unordered, Unique Elements, Mutable)	-	-	
2.4	Accessing Elements in a Set (Iteration, Membership Test)	-	-	
2.5	Difference Between Sets, Lists, and Tuples	-	-	
2.6	Worked Examples	-	-	
3	Set Operations In Python	-	2	25-38
3.1	Adding and Removing Elements	-	-	
3.2	Mathematical Set Operations	-	-	
3.3	Subset and Superset Operations	-	-	
3.4	Frozen Sets – Immutable Sets	-	-	
3.5	Built-in Functions with Sets	-	-	
3.6	Practical Applications of Sets	-	-	
3.7	Worked Examples	-	-	
4	Summary	-	-	39
5	Glossary	-	-	40
6	Terminal Questions	-	-	41
7	Answers	-	-	42-45
7.1	Self-Assessment Answers	-	-	
7.2	Terminal Questions Answers	-	-	

8	References	-	-	46
---	----------------------------	---	---	----



1. INTRODUCTION

In this unit, learners will explore one of the most powerful data structures in Python — Sets. Sets are essential for handling collections of unique items efficiently, making them a key concept in data analysis, programming logic, and real-world problem-solving. Learners will first understand the definition and characteristics of sets, including their unordered nature, ability to store only unique elements, and their mutable property. They will learn various ways to create and initialise sets using curly braces or the `set()` constructor, and explore how Python ensures that no duplicate elements exist within them.

Next, learners will understand how to access elements using iteration and membership tests (in keyword) and distinguish sets from other Python structures like lists and tuples through practical examples. This section also includes worked examples and self-assessment MCQs to strengthen conceptual understanding.

The second part focuses on Set Operations in Python, where learners will gain hands-on experience with Python's built-in methods for adding and removing elements. Functions such as `add()`, `update()`, `remove()`, `discard()`, `pop()`, and `clear()` will be explained through practical demonstrations. The concept of mathematical set operations — including `union()`, `intersection()`, `difference()`, and `symmetric_difference()` — will show how Python seamlessly models real-world mathematical relationships between data groups.

Learners will then explore subset and superset operations using `issubset()` and `issuperset()` to compare datasets effectively. The concept of frozen sets will also be introduced to understand immutable forms of sets that are useful when data must remain unchanged. Built-in functions like `len()`, `max()`, `min()`, and `sum()` will be applied to sets to perform quick analytical operations.

Finally, the unit connects theory with practice through real-world applications of sets such as duplicate removal in data cleaning, membership testing for efficient searches, and detecting commonality across datasets, which are widely used in fields like machine learning, cybersecurity, and database management.

By the end of this unit, learners will understand how sets simplify data handling through unique storage, fast lookup, and mathematical operations, making them indispensable tools for efficient and clean Python programming.

1.1. Learning Objectives

- Explain the concept and characteristics of sets as a data structure.
- Differentiate sets from other data structures such as lists and arrays.
- Apply common set operations: union, intersection, difference, and subset testing.
- Use sets to remove duplicates and test membership efficiently.
- Relate set operations to real-world problem-solving scenarios



2. BASICS OF SETS IN PYTHON

Python's set is one of the most important built-in data structures. It is designed for situations where uniqueness of elements and fast membership testing matter more than order or positional access. Sets are directly inspired by the mathematical concept of sets, which are collections of distinct items. In mathematics, a set of numbers like $\{1, 2, 3\}$ means each number is present only once, and the order does not matter. Python carries this philosophy into programming, with additional features suited for real-world coding tasks.

In everyday programming, you encounter “set-shaped” problems frequently. For example:

- Removing duplicates from a large list of user IDs.
- Checking if a particular item is already in a collection (such as verifying if a user has logged in today).
- Computing overlaps between two groups (e.g., students enrolled in both “Python” and “SQL” courses).
- Filtering items quickly from a dataset.

Python implements sets using a hash table internally. This design means that checking whether an element is present ($x \in s$) is very fast—on average, it takes constant time $O(1)$. This performance is much better than scanning through a list, which takes linear time $O(n)$. However, because sets rely on hashing, only hashable objects can be stored inside them. Hashable objects are those with a fixed, stable hash value throughout their lifetime, such as integers, strings, or tuples containing only hashable elements.

2.1 Definition and Characteristics of Sets

Definition:

A set in Python is an unordered collection of unique, hashable elements.

- Unordered means the elements do not have fixed positions or indices.
- Unique means duplicates are automatically removed.
- Hashable means each element must have an immutable identity with a stable hash value.

1. Uniqueness of Elements

Sets automatically prevent duplicates. If you try to add an element that already exists, Python silently ignores it and stores only one copy.

```
s = {2, 2, 3, 3, 3}
```

```
print(s) # {2, 3}
```

Logic: When inserting, Python computes the element's hash value and determines the correct "bucket" in the hash table. If an equal element (based on both hash and equality comparison `__eq__`) is already present, Python skips the insertion. This makes sets a natural tool for deduplication.

Practical Example: Removing duplicate customer emails.

```
emails = ["a@test.com", "b@test.com", "a@test.com"]
```

```
unique_emails = set(emails)
```

```
print(unique_emails) # {'a@test.com', 'b@test.com'}
```

2. Unordered (No Indices)

Unlike lists or tuples, sets are conceptually unordered. Although modern Python (3.7+) preserves insertion order for dictionaries and sets at the implementation level, you must not rely on this behavior for **logic**. **Attempting to access by index fails:**

```
s = {10, 20, 30}
```

```
# print(s[0]) # TypeError: 'set' object is not subscriptable
```

If you want an ordered version, you can sort the set into a list:

```
print(sorted(s)) # [10, 20, 30]
```

This ensures consistent order when required for display or further computation.

3. Mutable Container; Immutable Elements

The set itself is mutable—you can add or remove elements at any time. However, all elements placed inside must be hashable. Numbers, strings, and tuples of immutable elements are fine. Lists and dictionaries, being mutable, are not allowed:

```
s = {1, 2}
```

```
s.add(3)
```

```
print(s) # {1, 2, 3}

# s.add([4, 5]) # TypeError: unhashable type: 'list'
```

This restriction exists because mutability could change an element's hash, making it “disappear” or become misplaced inside the hash table.

4. Backed by a Hash Table

The hallmark of sets is their speed. Hash tables allow average $O(1)$ time complexity for insertion, deletion, and membership testing. Even with millions of elements, checking whether a value exists in a set is nearly instantaneous.

```
big_set = set(range(1000000))

print(999999 in big_set) # True, and extremely fast
```

In rare cases of hash collisions, performance may degrade, but Python's hashing strategy makes collisions uncommon.

5. A Gotcha: Boolean Values and Integers

A subtle but important detail is that booleans in Python (True and False) are actually subclasses of integers. Specifically, `True == 1` and `False == 0`, and they even share the same hash values.

```
s = {True, 1, 2}

print(len(s)) # 2

print(s) # {1, 2}
```

Logic: When 1 is inserted after True, Python finds that `True == 1` and their hashes match. Hence, only one of them is stored. The printed representation may prefer 1 over True. This quirk can cause confusion, so be careful when mixing booleans and integers in sets.

Why Sets Matter

Understanding these characteristics makes it clear why sets are such a powerful tool in Python:

- They guarantee uniqueness automatically.
- They offer blazing-fast membership checks.
- They eliminate the need for manual deduplication logic.

- They provide the foundation for advanced set operations (union, intersection, difference) used in filtering and grouping tasks.

Compared to lists and tuples, sets sacrifice order and indexing in exchange for performance and uniqueness. This trade-off is deliberate, and once you internalise it, you will know exactly when sets are the right choice.

2.2 Creating and Initialising Sets

You can create sets in several ways. The choice depends on whether you want to list elements explicitly, convert from another iterable, start empty, or build a set by comprehension.

1. Set literal with braces {}

```
s = {1, 2, 3, "a", "b"}
```

Logic: Python directly allocates a hash table and inserts the provided elements, collapsing duplicates on the fly.

2. From an iterable via set() constructor

```
items = [1, 2, 2, 3, 3, 3]
```

```
s = set(items)
```

```
print(s) # {1, 2, 3}
```

Logic: Python iterates the list and inserts each item into the set, skipping duplicates through hash/equality checks.

3. Empty set: set() (not {})

```
empty_set = set()
```

```
empty_dict = {} # This is a dict, not a set
```

Logic: {} is reserved for dictionaries. An empty set must be created with set().

4. Set comprehension

```
squares = {n*n for n in range(10)}
```

```
evens = {n for n in range(20) if n % 2 == 0}
```

Logic: Comprehensions evaluate the expression for each item in the iterable, and the set auto-deduplicates results.

5. From strings and other iterables

```
unique_chars = set("Mississippi")  
  
print(unique_chars) # {'p', 'M', 'i', 's'}
```

Logic: A string is iterable by character. Duplicates collapse.

6. frozenset for immutable sets

```
frozen = frozenset([1, 2, 3])  
  
# frozen.add(4) # AttributeError: 'frozenset' object has no attribute 'add'
```

Logic: frozenset is hashable itself, so you can store it inside another set or use it as a dict key.

Common initialisation patterns:

- Deduplicate a list while losing order:
- `deduped = list(set([5, 5, 3, 2, 2, 1]))`
- Deduplicate but preserve original order (prefer list + seen set):

```
data = [5, 5, 3, 2, 2, 1]  
  
seen = set()  
  
ordered_deduped = []  
  
for x in data:  
    if x not in seen:  
        seen.add(x)  
        ordered_deduped.append(x)  
  
# ordered_deduped: [5, 3, 2, 1]
```

Logic: Use a set only for fast membership; keep a list to preserve the order of first appearance.

2.3 Properties of Sets (Unordered, Unique Elements, Mutable)

Unordered

You cannot rely on position. Access by index is invalid:

```
s = {10, 20, 30}  
  
# s[0] # TypeError
```

If you need a deterministic order for presentation or further processing, convert or view through

sorted():

```
for x in sorted(s):
```

```
    print(x)
```

Logic: sorted(s) returns a new list sorted by the natural order of elements, leaving s unchanged.

Unique Elements

Sets automatically eliminate duplicates:

```
tags = {"ai", "ml", "ai", "python"}
```

```
print(tags) # {'ai', 'ml', 'python'}
```

Logic: On each insertion, Python checks for an existing equal element and keeps one.

Hash and equality rule:

An element is considered a duplicate if it compares equal (`__eq__`) to an existing element and has a matching hash (`__hash__`). This is why elements must be hashable. Lists and dicts are mutable and unhashable, so they cannot be set elements:

```
try:
```

```
    s = { [1, 2], 3 } # TypeError
```

```
except TypeError as e:
```

```
    print(e)
```

Tuples are fine if all items inside are hashable:

```
pairs = {(1, 2), (1, 2), (2, 3)}
```

```
print(pairs) # {(1, 2), (2, 3)}
```

Mutable Container

You can add and remove elements:

```
s = {1, 2}
```

```
s.add(3)      # add single element
```

```
s.update([3, 4, 5]) # add multiple from iterable; duplicates ignored  
print(s)          # {1, 2, 3, 4, 5}
```

```
s.remove(5)        # remove exact element; raises KeyError if absent  
s.discard(10)      # remove if present; no error if absent  
x = s.pop()        # remove and return an arbitrary element  
s.clear()          # remove all elements
```

Logic:

- `add(x)` computes `hash(x)`, inserts if not equal to any existing element.
- `update(iterable)` repeats `add` for each element.
- `remove(x)` searches for `x` via hash, raises if missing; `discard(x)` silently does nothing if missing.
- `pop()` chooses an arbitrary element based on internal table state (not “first” or “last”).
- `clear()` empties the hash table.

Copying correctly:

```
a = {1, 2, 3}  
b = a    # same object (alias)  
c = a.copy() # shallow copy (new set with same elements)  
a.add(99)  
print(b) # {1, 2, 3, 99}  
print(c) # {1, 2, 3}
```

Logic: Assignment passes a reference. `copy()` builds a new set with the same elements.

2.4 Accessing Elements in a Set (Iteration, Membership Test)

Because sets are unordered and non-indexable, you access contents by iterating or by testing membership.

Iteration

```
students = {"Asha", "Ravi", "Meera"}
```

```
for name in students:
```

```
    print(name)
```

Logic: Python iterates the internal hash table, yielding each unique element once. The order is arbitrary and may vary across runs.

If you need a stable order during iteration, sort first:

```
for name in sorted(students):
```

```
    print(name)
```

You can also combine enumeration with sorting if you need indices for printing:

```
for i, name in enumerate(sorted(students), start=1):
```

```
    print(i, name)
```

Membership Test

Membership testing is a classic strength of sets:

```
blacklist = {"tmp", "cache", "node_modules"}
```

```
folder = "cache"
```

```
if folder in blacklist:
```

```
    print("Skip scanning this folder.")
```

Logic: in computes the hash of folder and checks the appropriate bucket. Average $O(1)$.

Practical filtering example:

```
allowed_domains = {"learnbay.in", "innokidz.org", "example.com"}
```

```
emails = ["a@learnbay.in", "b@spam.co", "c@innokidz.org"]
```

```
def domain_of(email):
```

```
    return email.split("@")[-1]
```

```
filtered = [e for e in emails if domain_of(e) in allowed_domains]
```

```
print(filtered) # ['a@learnbay.in', 'c@innokidz.org']
```

Logic: Each membership check against the set is fast. This pattern scales far better than checking against a list for large data.

Case-insensitive membership:

```
keywords = {"python", "sql", "linux"}
```

```
query = "Python"
```

```
if query.lower() in keywords:
```

```
    print("Match (case-insensitive)")
```

Logic: Normalise both sides to the same case before membership testing.

2.5 Difference Between Sets, Lists, and Tuples

Python provides a variety of container types, each designed to store and manage collections of objects. Among these, sets, lists, and tuples are commonly used. While they may seem similar at first glance—since all three can hold multiple elements—their underlying behavior, use-cases, and performance characteristics differ greatly. Choosing the correct container type can lead to more efficient programs, better readability, and fewer logical errors. This section compares sets, lists, and tuples based on key conceptual differences.

Order and Indexing

Lists are ordered collections, meaning the order in which you insert elements is preserved. This allows you to access elements by position (indexing). For example:

```
lst = [10, 20, 30]
```

```
print(lst[0]) # 10
```

Here, `lst[0]` refers to the first element.

Tuples are also ordered and support indexing just like lists. The difference lies in their immutability, which will be discussed later. Example:

```
tup = (10, 20, 30)
```

```
print(tup[1]) # 20
```

The indexing works exactly like lists.

Sets, however, are unordered. They do not maintain insertion order conceptually (even though modern Python versions preserve insertion order for implementation reasons, programmers must not rely on it). Most importantly, sets do not support indexing. Attempting to access `s[0]` will raise a `TypeError`. If you want a predictable order from a set, you must explicitly sort it:

```
s = {10, 20, 30}
```

```
print(sorted(s)) # [10, 20, 30]
```

Thus, when you need positional access, lists and tuples are appropriate; when order does not matter, sets are preferable.

Duplicates

Another significant difference lies in how each container treats duplicate values.

Lists allow duplicates freely:

```
lst = [1, 1, 2, 2, 3]
```

```
print(lst) # [1, 1, 2, 2, 3]
```

This is useful when you want to keep track of repeated occurrences, such as logging multiple identical events.

Tuples also allow duplicates. Since they are immutable, the duplicates remain exactly as they are defined:

```
tup = (1, 1, 2, 2, 3)
```

```
print(tup) # (1, 1, 2, 2, 3)
```

Sets, on the other hand, automatically eliminate duplicates. Only unique elements are stored:

```
s = {1, 1, 2, 2, 3}
```

```
print(s) # {1, 2, 3}
```

This property makes sets excellent for deduplication tasks, such as cleaning raw data or extracting unique values.

Mutability

Mutability refers to whether the container's contents can be changed after creation.

Lists are mutable. You can append, insert, update, or remove elements at any time:

```
lst = [1, 2, 3]
lst.append(4)
lst[0] = 10
print(lst) # [10, 2, 3, 4]
```

Tuples are immutable. Once created, you cannot modify, add, or delete elements. Attempting to reassign `tup[0] = 5` will raise a `TypeError`. This immutability makes tuples reliable for representing fixed collections, such as coordinates or database keys.

Sets are mutable as containers: you can add or remove elements with methods like `add()` or `remove()`. However, the elements themselves must be hashable—which typically means immutable. For instance, integers, strings, and tuples of immutable objects are allowed, but lists and dictionaries are not:

```
s = {1, 2}
s.add(3)
# s.add([4, 5]) # TypeError: unhashable type: 'list'
```

Thus, sets strike a balance: the container can change, but elements must be stable.

Membership Test Performance

Membership testing (`x in container`) is one of the most common operations across all data structures.

In lists and tuples, membership testing is $O(n)$ on average. Python checks each element one by one until a match is found or the container ends. For small collections this is fine, but for large datasets, this can be inefficient:

```
lst = list(range(100000))
print(99999 in lst) # True, but slow for large lists
```


Sets, by contrast, use a hash table internally. When you check if an element is present, Python computes its hash and directly checks the appropriate bucket. This operation is $O(1)$ on average, making sets far superior for membership checks:

```
s = set(range(100000))
```

```
print(99999 in s) # True, very fast
```

Therefore, when frequent membership tests are required, sets are the best choice.

Typical Use Cases

Each container type has scenarios where it excels:

- **Lists:**

Ideal for ordered sequences, stacks, queues, or situations requiring indexing. Common use cases include storing tasks, maintaining logs, and processing data where duplicates matter.

- **Tuples:**

Best for fixed collections of data. They are often used for returning multiple values from functions, as dictionary keys (since they are hashable), or representing structured records (e.g., coordinates (x,y)). Their immutability provides safety where accidental modification is undesirable.

- **Sets:**

Perfect for operations requiring uniqueness and fast lookups. They shine in deduplication, filtering, and mathematical set operations such as union, intersection, and difference. For example, finding common students in two courses or removing invalid entries from a dataset is most efficiently handled with sets.

Code comparisons

Indexing vs no indexing

```
lst = [10, 20, 30]
```

```
print(lst[1]) # 20
```

```
s = {10, 20, 30}
```

```
# print(s[1]) # TypeError
```

Deduplication

```
lst = [1, 1, 2, 3, 3]
```

```
deduped = list(set(lst))
```

deduped is [1, 2, 3] but order is not guaranteed.

When order matters, prefer list with a membership set

```
data = ["ai", "ml", "ai", "python", "ml"]
```

```
seen = set()
```

```
result = []
```

```
for item in data:
```

```
    if item not in seen:
```

```
        seen.add(item)
```

```
        result.append(item)
```

```
print(result) # ['ai', 'ml', 'python']
```

Logic: Use the set for fast “seen?” checks; keep the list to preserve first occurrence order.

Immutability of tuple

```
t = (1, 2, 3)
```

```
# t[0] = 9 # TypeError: 'tuple' object does not support item assignment
```

Set element hashability requirement

```
# s = {[1, 2], 3} # TypeError: unhashable type: 'list'
```

```
s = {(1, 2), 3} # OK, tuple is hashable if its elements are hashable
```

Performance intuition:

- Use a set when you frequently ask “is x present?” or when you must ensure uniqueness.
- Use a list when the order and positional operations matter.

- Use a tuple when you need an immutable, ordered, fixed-size record or a hashable compound key.

2.6 Worked Examples

Example 1: Deduplicating while preserving order

Problem: Given a list of course codes where duplicates may appear, keep only the first occurrence of each code and preserve the original order.

```
codes = ["DSAI101", "GENAI201", "DSAI101", "SQL150", "GENAI201", "LINUX101"]
seen = set()
unique_codes = []
for code in codes:
    # Membership is O(1) average
    if code not in seen:
        seen.add(code)      # Remember we've seen it
        unique_codes.append(code) # Keep first appearance
print(unique_codes)
# ['DSAI101', 'GENAI201', 'SQL150', 'LINUX101']
```

Logic:

We use a set (`seen`) for fast membership checks to avoid an $O(n^2)$ approach. Each code is checked against `seen`; only the first occurrence passes the test. We append to `unique_codes` (list) to preserve order. This pattern is a staple when order matters but you still need uniqueness.

Example 2: Fast filtering with a set

Problem: From a stream of emails, keep only those whose domain belongs to an approved list.

```
approved = {"learnbay.in", "innokidz.org"}
emails = [
```

```
"a@learnbay.in", "x@spam.com", "c@innokidz.org", "z@fake.net", "b@learnbay.in"
]

def domain(email):

    return email.rsplit("@", 1)[-1].lower()

filtered = [e for e in emails if domain(e) in approved]

print(filtered)

# ['a@learnbay.in', 'c@innokidz.org', 'b@learnbay.in']
```

Logic:

We normalise domains to lowercase for consistent matching. The set `approved` makes each membership test $O(1)$ average, which is crucial when filtering thousands of emails.

Example 3: Preventing unhashable elements and using tuples

Problem: You want to collect unique coordinates (x, y) . Lists cannot be used because they are mutable and unhashable; tuples solve this.

```
coords = set()

coords.add((10, 20))

coords.add((10, 20)) # duplicate ignored

coords.add((15, 25))

print(coords)      # {(10, 20), (15, 25)}

# coords.add([5, 6]) # TypeError: unhashable type: 'list'
```

Logic:

Each coordinate is a tuple of integers, which are hashable. Tuples ensure hash stability, satisfying the set's requirement.

Example 4: Choosing set vs list for membership heavy workloads

Scenario: You need to repeatedly check whether a product ID appears in a reference collection. With a list, each in test is $O(n)$; with a set, it's $O(1)$ average.

```
import random

reference_list = list(range(100000))

reference_set = set(reference_list) # one-time cost to build

queries = [random.randint(0, 120000) for _ in range(1000)]

# List membership ( $O(n)$  per query)
list_hits = sum(1 for q in queries if q in reference_list)

# Set membership ( $O(1)$  average per query)
set_hits = sum(1 for q in queries if q in reference_set)

print(list_hits, set_hits)
```

Logic:

Even with only 1,000 queries, the set often runs much faster because it avoids scanning the whole list for every test. This performance advantage scales with data size.

Example 5: Safe removals with discard

Problem: You want to clean a set by removing possible outliers if they exist, without risking errors.

```
scores = {10, 20, 30}

scores.discard(999) # No error even if 999 not present

scores.remove(10)  # OK; 10 is present

# scores.remove(999) # Would raise KeyError

print(scores)      # {20, 30}
```

Logic:

Use `discard` when you are not sure an element is present. Use `remove` when you expect it to be there and want an error if it isn't.

Example 6: Building sets with comprehensions

Goal: Unique lowercase words from a sentence, ignoring punctuation.

```
text = "AI, ai, and Python! Python; AI?"

# Normalize and split on non-letters quickly:

clean = "".join(ch.lower() if ch.isalpha() else " " for ch in text)

unique_words = {w for w in clean.split() if w}

print(unique_words) # {'ai', 'python', 'and'}
```

Logic:

We normalize to lowercase and replace punctuation with spaces, then split. The set comprehension collects distinct words automatically.

SELF-ASSESSMENT QUESTIONS – 1

Multiple Choose Questions	
1	Which statement about Python sets is true? A. Sets preserve insertion order and support indexing B. Sets allow duplicate elements C. Sets are unordered and contain unique, hashable elements D. Sets are immutable containers
2	Which of the following correctly creates an empty set? A. {} B. [] C. set() D. empty()
3	What is the average-case time complexity of the membership test $x \in s$ for a Python set s ?

	A. $O(n)$ B. $O(\log n)$ C. $O(1)$ D. $O(n \log n)$
4	What happens with <code>len({True, 1, 2})</code> in Python and why? A. 3, because True and 1 are different types B. 2, because True equals 1 and they share the same hash C. 3, because sets never collapse numeric duplicates D. 1, because only the largest value remains
5	Which operation will raise an error if the element is absent? A. <code>s.discard(x)</code> B. <code>s.remove(x)</code> C. <code>x in s</code> D. <code>s.update([x])</code>

3. SET OPERATIONS IN PYTHON

Set operations in Python provide a systematic and efficient way to perform mathematical and logical manipulations on collections of unique elements. A set is an unordered collection that automatically eliminates duplicates and supports fast membership testing due to its hash-based implementation. These properties make sets particularly suitable for data cleaning, comparison, and analytical tasks.

Python supports a wide range of operations that correspond directly to classical set theory. The union operation (`|` or `union()`) combines elements from multiple sets, ensuring uniqueness across all members. The intersection operation (`&` or `intersection()`) identifies elements that are common between sets, while the difference operation (`-` or `difference()`) isolates elements present in one set but not in another. The symmetric difference operation (`^` or `symmetric_difference()`) returns elements that belong exclusively to one of the sets but not both. Each of these operations can also be performed in-place using assignment operators such as `|=`, `&=`, `-=`, and `^=`, which optimise memory usage.

Modification of set contents is supported through various methods. The `add()` method inserts a single element, whereas `update()` adds multiple elements from an iterable. Deletion can be performed using `remove()` (which raises an error if the element is absent), `discard()` (which safely ignores missing items), `pop()` (which removes an arbitrary element), and `clear()` (which empties the entire set). These mutation operations provide flexibility in dynamic data management.

Python also defines relational set operations through methods like `issubset()`, `issuperset()`, and `isdisjoint()`, which enable comparison between sets. These methods are valuable for determining hierarchical or exclusive relationships between datasets, such as verifying permissions or dependencies.

In scenarios requiring immutability, Python offers the `frozenset`, an immutable and hashable variant of the standard set. `Frozensets` can be used as keys in dictionaries or as elements within other sets, making them useful in applications involving fixed configurations or graph structures.

Additional built-in functions enhance set usability. Functions like `len()` provide the number of elements, while `min()`, `max()`, and `sum()` allow direct aggregation and evaluation of comparable data within sets. Combined with set comprehensions, these capabilities facilitate concise expressions for filtering, transformation, and aggregation of data.

3.1 Adding and Removing Elements

A set is mutable, which means you can dynamically add and remove elements after creation. Python provides several methods for this purpose. Each method has slightly different behavior, so it is important to understand them clearly.

3.1.1 add()

The `add()` method inserts a single new element into the set. If the element is already present, the set remains unchanged because sets automatically maintain uniqueness.

```
fruits = {"apple", "banana"}  
fruits.add("orange")  
fruits.add("banana") # already exists  
print(fruits) # {'orange', 'apple', 'banana'}
```

Logic:

When `add("orange")` is called, Python computes its hash and inserts it into the internal hash table if it does not already exist. Adding "banana" again has no effect because it matches an existing entry.

3.1.2 update()

The `update()` method adds multiple elements at once from an iterable such as a list, tuple, or another set.

```
colors = {"red", "blue"}  
colors.update(["green", "yellow"])  
colors.update({"blue", "black"}) # duplicates ignored  
print(colors) # {'red', 'yellow', 'blue', 'green', 'black'}
```

Logic:

For each item in the iterable, Python performs the same check as `add()`. Only unique items are inserted, duplicates are ignored silently.

3.1.3 remove()

The `remove()` method deletes a specified element. If the element is not present, it raises a `KeyError`.

```
numbers = {1, 2, 3}
numbers.remove(2)
print(numbers) # {1, 3}
# numbers.remove(5) # Raises KeyError
```

Logic:

Python checks for the element using hashing. If it is not found, the method raises an error. Use `remove()` only when you are sure the element exists.

3.1.4 discard()

Similar to `remove()`, but `discard()` does not raise an error if the element is absent.

```
items = {"pen", "pencil", "eraser"}
items.discard("pencil")
items.discard("marker") # ignored
print(items) # {'pen', 'eraser'}
```

Logic:

This is safer when you are not certain about an element's existence. `discard()` silently does nothing if the element is not found.

3.1.5 pop()

Removes and returns an arbitrary element. Since sets are unordered, you cannot predict which element will be removed.

```
letters = {"a", "b", "c"}
removed = letters.pop()
```

```
print(removed) # could be 'a', 'b', or 'c'
```

```
print(letters) # remaining elements
```

Logic:

pop() removes whichever element is found first in the internal hash table structure. The unpredictability enforces the idea that sets have no order.

3.1.6 clear()

Removes all elements from the set, leaving it empty.

```
data = {1, 2, 3}
```

```
data.clear()
```

```
print(data) # set()
```

Logic:

Python simply empties the internal hash table. This is faster than removing elements one by one.

3.2 Mathematical Set Operations

Sets in Python support mathematical operations directly through both methods and operators. This makes the syntax concise and expressive.

3.2.1 Union (|, union())

The union of two sets combines all unique elements from both.

```
a = {1, 2, 3}
```

```
b = {3, 4, 5}
```

```
print(a | b)      # {1, 2, 3, 4, 5}
```

```
print(a.union(b)) # {1, 2, 3, 4, 5}
```

Logic:

Union traverses both sets and inserts elements into a new set, automatically skipping duplicates.

3.2.2 Intersection (&, intersection())

The intersection contains only elements common to both sets.

```
x = {1, 2, 3}
```

```
y = {2, 3, 4}
```

```
print(x & y)      # {2, 3}
```

```
print(x.intersection(y)) # {2, 3}
```

Logic:

Python compares hash values from one set against the other, collecting only the matches.

3.2.3 Difference (-, difference())

The difference contains elements present in the first set but not in the second.

```
m = {1, 2, 3, 4}
```

```
n = {3, 4, 5}
```

```
print(m - n)      # {1, 2}
```

```
print(m.difference(n)) # {1, 2}
```

Logic:

Each element of m is checked against n. If not found, it stays in the result.

3.2.4 Symmetric Difference (^, symmetric_difference())

The symmetric difference contains elements present in either set but not both.

```
p = {1, 2, 3}
```

```
q = {3, 4, 5}
```

```
print(p ^ q)      # {1, 2, 4, 5}
```

```
print(p.symmetric_difference(q)) # {1, 2, 4, 5}
```

Logic:

Equivalent to $(p - q) \cup (q - p)$. Useful for finding mismatched values between datasets.

3.3 Subset and Superset Operations

Sets also support relationships between groups of elements.

3.3.1 issubset()

Checks if all elements of one set are present in another.

```
small = {1, 2}
```

```
large = {1, 2, 3, 4}
```

```
print(small.issubset(large)) # True
```

Logic:

For each element in small, Python tests membership in large. If all succeed, it returns True.

3.3.2 issuperset()

Checks if a set contains all elements of another set.

```
a = {1, 2, 3, 4}
```

```
b = {2, 3}
```

```
print(a.issuperset(b)) # True
```

Logic:

The opposite of subset. It verifies whether all elements in b exist inside a.

3.4 Frozen Sets – Immutable Sets

A frozenset is an immutable version of a set. Once created, you cannot add or remove elements. This immutability makes frozensets hashable, meaning they can be used as dictionary keys or elements inside another set.

```
fs = frozenset([1, 2, 3])  
  
# fs.add(4) # AttributeError  
  
normal_set = {fs, frozenset([4, 5])}  
  
print(normal_set)
```

Logic:

Since the contents cannot change, the frozenset's hash value is stable. This is why it qualifies as a key in mappings or as an element of another set.

3.5 Built-in Functions with Sets

Python's built-in functions often work directly on sets.

1. `len()` – Counts elements.
2. `s = {10, 20, 30}`
3. `print(len(s)) # 3`
4. `max()` and `min()` – Return the maximum or minimum element (according to natural ordering).
5. `numbers = {5, 9, 1}`
6. `print(max(numbers)) # 9`
7. `print(min(numbers)) # 1`
8. `sum()` – Returns the total of numeric elements.
9. `values = {2, 4, 6}`
10. `print(sum(values)) # 12`

Logic:

These functions iterate over the set once, applying their specific aggregation. Since sets are iterable, the process works like with lists or tuples.

3.6 Practical Applications of Sets

Sets shine in scenarios where uniqueness and fast membership are critical.

3.6.1 Duplicate Removal

```
data = [1, 2, 2, 3, 3, 4]
unique = set(data)
print(unique) # {1, 2, 3, 4}
```

Logic:

The set automatically discards duplicate entries.

3.6.2 Membership Testing

```
stop_words = {"a", "an", "the"}
word = "the"
if word in stop_words:
    print("Skip word")
```

Logic:

Checking membership in a set is much faster than in a list, especially for large collections.

3.6.3 Data Cleaning

Sets help filter inconsistent or noisy data.

```
allowed = {"yes", "no"}
responses = ["yes", "no", "maybe", "yes", "no", "unknown"]
cleaned = [r for r in responses if r in allowed]
print(cleaned) # ['yes', 'no', 'yes', 'no']
```

Logic:

We retain only valid responses, discarding invalid ones.

3.6.4 Detecting Commonality Across Datasets

```
students_math = {"Asha", "Ravi", "Kiran"}  
students_science = {"Ravi", "Kiran", "Meera"}  
common = students_math & students_science  
print(common) # {'Ravi', 'Kiran'}
```

Logic:

Intersection reveals students enrolled in both courses.

3.7 Worked Examples

Example 1 — Deduplicate While Preserving First-Seen Order

Problem: You receive enrollment IDs with repeats. Keep only the first time each ID appears, preserving the original order.

```
enrollments = ["LB101", "LB102", "LB101", "LB103", "LB102", "LB104"]  
seen = set()  
unique_ordered = []  
for eid in enrollments:  
    if eid not in seen:    # O(1) average  
        seen.add(eid)  
        unique_ordered.append(eid)  
print(unique_ordered) # ['LB101', 'LB102', 'LB103', 'LB104']
```

Logic:

- Use a set for fast membership checks and a list to maintain order.
- `set.add()` inserts only when new, so duplicates silently collapse.
- Complexity: $O(n)$ average, because each `in` seen is $O(1)$ on average.

Why not `list(set(enrollments))`? That loses the original order. The pattern above is the standard “order-preserving dedup.”

Example 2 — Domain Whitelist Filtering (Membership at Scale)

Problem: Clean a mailing list to include only emails from approved domains.

```
approved = {"learnbay.in", "innokidz.org"}

raw = ["a@learnbay.in", "b@spam.com", "c@innokidz.org", "d@fake.net"]

def domain(email):

    return email.rsplit("@", 1)[-1].lower()

clean = [e for e in raw if domain(e) in approved]

print(clean) # ['a@learnbay.in', 'c@innokidz.org']
```

Logic:

- Membership against a set is $O(1)$ average, which scales to large lists.
- Normalizing to lowercase avoids case mismatches.
- The approach remains fast even for hundreds of thousands of emails.

Example 3 — Inventory Difference: What's Out of Stock?

Problem: Show items requested by students that are not currently in stock.

```
requested = {"HDMI cable", "USB-C charger", "Notebook", "Whiteboard markers"}

in_stock = {"Notebook", "HDMI cable", "Pen"}

out_of_stock = requested - in_stock

print(out_of_stock) # {'USB-C charger', 'Whiteboard markers'}
```

Logic:

- $A - B$ keeps only elements in A not in B.
- Internally, Python checks each element of requested for membership in in_stock.

When to use: Pre-checks, validation, and control flows (“only proceed if nothing is missing”).

Example 4 — Intersection for Common Registrants

Problem: You need the list of learners enrolled in both “Python Basics” and “SQL Basics.”

```
python = {"Asha", "Ravi", "Meera", "Kiran"}

sql = {"Ravi", "Kiran", "Zara"}
```

both = python & sql

```
print(both) # {'Ravi', 'Kiran'}
```

Logic:

- A & B returns only those present in both sets.
- Real uses: “common customers,” “overlapping tags,” “shared constraints.”

Example 5 — Symmetric Difference to Spot Mismatches

Problem: Two systems export IDs. Find IDs that exist in exactly one system (potential sync errors).

```
sys_a = {"S001", "S002", "S003", "S006"}
```

```
sys_b = {"S002", "S003", "S004", "S005"}
```

```
mismatched = sys_a ^ sys_b
```

```
print(mismatched) # {'S001', 'S004', 'S005', 'S006'}
```

Logic:

- $A \oplus B$ = elements in A or B but not both.
- Great for reconciliation and auditing.

Example 6 — Subset/Superset for Permission Checks

Problem: A role requires a minimal capability set. Check whether a user’s permissions meet the requirement.

```
required = {"read_reports", "download_csv"}
```

```
user_perms = {"login", "read_reports", "download_csv", "share"}
```

```
print(required.issubset(user_perms)) # True
```

```
print(user_perms.issuperset(required)) # True (equivalent check)
```

Logic:

- `issubset` confirms all required privileges exist.
- The inverse view is `issuperset`.
- Combine with difference to see missing permissions:
- `missing = required - user_perms`

- if missing:
- `print("Missing:", missing)`

Example 7 — Using update, discard, remove, pop, clear Safely

Problem: Maintain a live set of “active tickets.”

```
active = {"T101", "T102"}
active.add("T103")          # add single
active.update({"T104", "T101"}) # add many (duplicates ignored)

# Safe removal when unsure:
active.discard("T999")      # no error

# Strict removal when sure:
active.remove("T104")       # KeyError if absent

# Pop an arbitrary element:
any_one = active.pop()      # unpredictable which one
print(any_one)

# Clear all:
active.clear()
print(active) # set()
```

Logic (key differences):

- add vs update (single vs many).
- discard won't crash if absent; remove will.
- pop underlines that sets have no positional order.
- clear is an $O(1)$ clearance of internal structure.

Example 8 — Frozenset for Unordered Pairs (Graph Edges)

Problem: Represent undirected edges between nodes without caring about order (A—B is the same as B—A). Use frozenset so an edge is hashable and can live inside a set.

edges = set()

```
edges.add(frozenset({"A", "B"}))
edges.add(frozenset({"B", "A"})) # same edge, ignored
edges.add(frozenset({"A", "C"}))
print(edges)
# {frozenset({'A', 'B'}), frozenset({'A', 'C'})}
```

Logic:

- frozenset is immutable and hashable; {A, B} equals {B, A} as a frozenset.
- This is the standard pattern for modeling undirected edges, de-duplicating automatically.

Example 9 — Unique Normalized Words (Data Cleaning + Set Comprehension)

Problem: Extract unique words from text, ignoring punctuation and case.

```
text = "AI, ai, and Python! Python; AI?"
clean = "".join(ch.lower() if ch.isalpha() else " " for ch in text)
unique_words = {w for w in clean.split() if w}
print(unique_words) # {'and', 'python', 'ai'}
```

Logic:

- Normalize: keep letters, demote to lowercase.
- Split into words; the set comprehension deduplicates.
- Useful for building vocabularies, tag clouds, or indexing.

Example 10 — Jaccard Similarity for Tag Overlap

Problem: Compare two learners' interests as tags and compute a similarity score:

```
learner_a = {"python", "sql", "linux", "ml"}
learner_b = {"sql", "ml", "docker", "git"}

inter = learner_a & learner_b

union = learner_a | learner_b

jaccard = len(inter) / len(union) if union else 0.0

print(inter)  # {'sql', 'ml'}
print(union)  # {'python', 'sql', 'linux', 'ml', 'docker', 'git'}
print(jaccard) # 0.333...
```

Logic:

- Intersection and union map exactly to the formula.
- Use cases: recommendation systems, clustering by interest, near-duplicate detection.

SELF-ASSESSMENT QUESTIONS – 2

Fill in the blanks:	
6	The ____() method adds a single new element to a set.
7	The operator corresponds to the set operation called ____.
8	The discard() method differs from remove() because it does not raise a(n) ____.
9	A frozenset is an immutable and therefore ____.
10	The function len(s) returns the total number of ____ in set s.

4. SUMMARY

- A set is an unordered collection of unique, hashable elements.
- Sets can be created using literals, the `set()` constructor, comprehensions, or as immutable frozensets.
- Sets are unordered, contain unique elements, and are mutable as containers.
- Elements are accessed through iteration or membership testing.
- Lists are ordered and allow duplicates, tuples are ordered and immutable, while sets are unordered and unique.
- Elements can be added using `add()` or `update()`.
- Elements can be removed using `remove()`, `discard()`, `pop()`, or `clear()`.
- Sets support union, intersection, difference, and symmetric difference operations.
- Subset and superset relationships can be checked using `issubset()` and `issuperset()`.
- Sets are applied in duplicate removal, membership testing, data cleaning, and relationship analysis between datasets.

5. GLOSSARY

Set	- An unordered, mutable collection of unique and hashable elements in Python, used for fast membership testing and eliminating duplicates.
Hashable	- A property of an object that guarantees a fixed hash value for its lifetime, allowing it to be used in sets or as dictionary keys (e.g., numbers, strings, tuples of hashable items).
add()	- A method that inserts a single element into a set. If the element already exists, the set remains unchanged.
update()	- A method that adds multiple elements from any iterable (list, tuple, set, etc.) into the set, ignoring duplicates.
remove() vs discard()	- Both delete elements from a set. <code>remove()</code> raises a <code>KeyError</code> if the element is missing, while <code>discard()</code> silently ignores absent elements.
pop()	- A method that removes and returns an arbitrary element from a set, emphasizing that sets are unordered and non-indexable.
Union	- A set operation (using <code> </code> or <code>union()</code>) that combines all unique elements from two or more sets.
Intersection	- A set operation (using <code>&</code> or <code>intersection()</code>) that returns only elements common to both sets.
Difference	- A set operation (using <code>-</code> or <code>difference()</code>) that returns elements present in one set but not the other.
Symmetric Difference	- A set operation (using <code>^</code> or <code>symmetric_difference()</code>) that returns elements present in either set, but not both.
Subset and Superset	- Relationships between sets. A set A is a subset of B if all elements of A exist in B (<code>issubset()</code>), and B is a superset if it contains all elements of A (<code>issuperset()</code>).
Frozenset	- An immutable (unchangeable) version of a set. Since its contents cannot be altered, a frozenset is hashable and can itself be stored inside other sets or used as a dictionary key.

6. TERMINAL QUESTIONS

1. Why can't Python sets contain lists or dictionaries, but can contain tuples? Give an example.
2. Write a Python program to remove duplicates from a list of integers using a set while preserving order.
3. Differentiate between the `remove()` and `discard()` methods of sets.
4. Given two sets $A = \{1,2,3,4,5\}$ and $B = \{4,5,6,7\}$, find:
 - (a) Elements in either A or B but not both
 - (b) Elements common to both sets
5. What is the average time complexity of membership testing (`x in set`) and why is it faster than in a list?
6. Use a set to find unique words in the string: "Python is great and Python is powerful". Display them in sorted order.
7. What is a frozenset in Python, and in what situations would you use it?
8. Given a registered student set `{"Asha", "Ravi", "Meera"}` and attendee list `["Ravi", "Kiran", "Zara"]`, write code to:
 - (a) Find new students not registered
 - (b) Find students who were both registered and attended
9. Explain with an example the difference between `issubset()` and `issuperset()` methods.
10. You have two sets:

```
skills_required = {"python", "sql", "linux"}
```

```
skills_candidate = {"python", "sql", "git"}
```

Write code to check if the candidate meets requirements, and list missing skills if any.

7. ANSWERS

7.1. Self-Assessment Answers:

1. C – Sets are unordered and contain unique, hashable elements.
2. C – `set()` creates an empty set; `{}` creates an empty dict.
3. C – Average-case membership in a set is $O(1)$ due to hashing.
4. B – `True == 1`, both hash to the same value, so the set collapses them.
5. B – `remove` raises `KeyError` if `x` is not found; `discard` does nothing.
6. `add()`
7. Union
8. `KeyError`
9. Hashable
10. Elements

7.2. Terminal Question Answers

Answer 1: Python sets require elements to be hashable. Lists and dictionaries are mutable, so their hash can change, making them unfit for sets. Tuples are immutable (if they contain only hashable items), so they are allowed. Example:

```
s = {(1,2), (3,4)}
```

works, but

```
s = {[1,2], [3,4]}
```

raises `TypeError`. Tuples ensure stability in hashing, whereas lists/dicts may change, breaking set integrity.

Answer 2: To remove duplicates while preserving order, combine a set with a loop:

```
nums = [1,2,2,3,4,3,5]
```

```
seen, unique = set(), []
```

```
for n in nums:
```

if n not in seen:

```
seen.add(n)
```

```
unique.append(n)
```

```
print(unique) # [1,2,3,4,5]
```

Here, the set tracks “seen” values for fast lookups, and the list keeps insertion order. This ensures deduplication while maintaining sequence.

Answer 3: The `remove()` method deletes a specified element but raises a `KeyError` if it’s absent. The `discard()` method also deletes, but does nothing if the element is missing. For example:

```
s = {1,2,3}
```

```
s.remove(2) # works
```

```
s.discard(5) # safe, no error
```

Use `remove()` when you are sure the element exists, and `discard()` when you want safety without exceptions.

Answer 4: For sets $A = \{1,2,3,4,5\}$ and $B = \{4,5,6,7\}$:

(a) Elements in either but not both:

```
print(A ^ B) # {1,2,6,7}
```

(b) Elements common to both:

```
print(A & B) # {4,5}
```

The symmetric difference ($\wedge$) gives unique-only elements, while intersection ($\&$) shows shared elements. Both operations are efficient due to hash-based membership.

Answer 5: Membership testing in sets (x in s) averages $O(1)$ because sets use a hash table internally. When you check x in s , Python computes the hash of x and looks directly in the appropriate bucket. In contrast, lists must scan each element sequentially, resulting in $O(n)$ time. Thus, sets are far faster for repeated membership queries, especially with large data collections.

Answer 6: To find unique words in a sentence:

```
text = "Python is great and Python is powerful"
```

```
unique = set(text.lower().split())
```

```
print(sorted(unique))
```

Output: ['and', 'great', 'is', 'powerful', 'python'].

The `split()` separates words, the set removes duplicates, and `sorted()` organises them alphabetically. Sets make this task simple and efficient by handling uniqueness automatically without additional logic.

Answer 7: A frozenset is an immutable version of a set. Unlike normal sets, you cannot add or remove elements after creation. Example:

```
fs = frozenset([1,2,3])
```

It is hashable, so it can be used as a key in dictionaries or as an element of another set. Useful in cases where a fixed group of values must remain unchanged, such as representing graph edges or configuration constants.

Answer 8: Registered students: {"Asha", "Ravi", "Meera"}. Attendees: ["Ravi", "Kiran", "Zara"].

(a) New students:

```
new = set(attendees) - registered
```

```
print(new) # {'Kiran','Zara'}
```

(b) Already registered and attended:

```
present = registered & set(attendees)
```

```
print(present) # {'Ravi'}
```

Here, difference highlights outsiders, and intersection finds overlaps. Both operations are highly efficient for such membership analysis.

Answer 9: `issubset()` checks if all elements of one set are inside another, while `issuperset()` checks the opposite. Example:

```
a = {1,2}
```

```
b = {1,2,3,4}
```

```
print(a.issubset(b)) # True
```

```
print(b.issuperset(a)) # True
```

This shows that a is fully contained in b, and b fully covers a. These methods are critical for permissions, prerequisites, and dataset containment checks.

Answer 10: Given:

```
skills_required = {"python","sql","linux"}
```

```
skills_candidate = {"python","sql","git"}
```

Check requirements:

```
print(skills_required.issubset(skills_candidate)) # False
```

```
print(skills_required - skills_candidate)      # {'linux'}
```

The candidate misses linux. Subset checking confirms readiness, while the difference operation explicitly lists gaps. This approach is widely applied in recruitment systems, certification checks, and project team validations.

8. REFERENCES

- Python Software Foundation. The Python Standard Library: set — Built-in Types. Available at: <https://docs.python.org/3/library/stdtypes.html#set>
- Van Rossum, G., & Drake, F. L. (2009). The Python Language Reference Manual. Network Theory Ltd.
- Lutz, M. (2013). Learning Python (5th Edition). O'Reilly Media. – Comprehensive coverage of Python data structures, including sets.
- Downey, A. (2015). Think Python: How to Think Like a Computer Scientist (2nd Edition). Green Tea Press. – Free online resource with clear explanations of sets and operations.
- Beazley, D. (2009). Python Essential Reference (4th Edition). Addison-Wesley Professional. – Concise details of set operations and their internal implementation.
- Hetland, M. L. (2017). Beginning Python: From Novice to Professional (3rd Edition). Apress. – Beginner-friendly introduction to sets and other collections.
- Summerfield, M. (2010). Programming in Python 3: A Complete Introduction to the Python Language (2nd Edition). Addison-Wesley. – Practical guide with examples of using sets in real-world tasks.
- Python Enhancement Proposals (PEP 218). Adding a Built-In Set Object Type. Available at: <https://peps.python.org/pep-0218/> – Historical context on why sets were added to Python.
- Real Python. Python's Built-in Set Types. Available at: <https://realpython.com/python-sets/> – Tutorial with practical coding use cases.

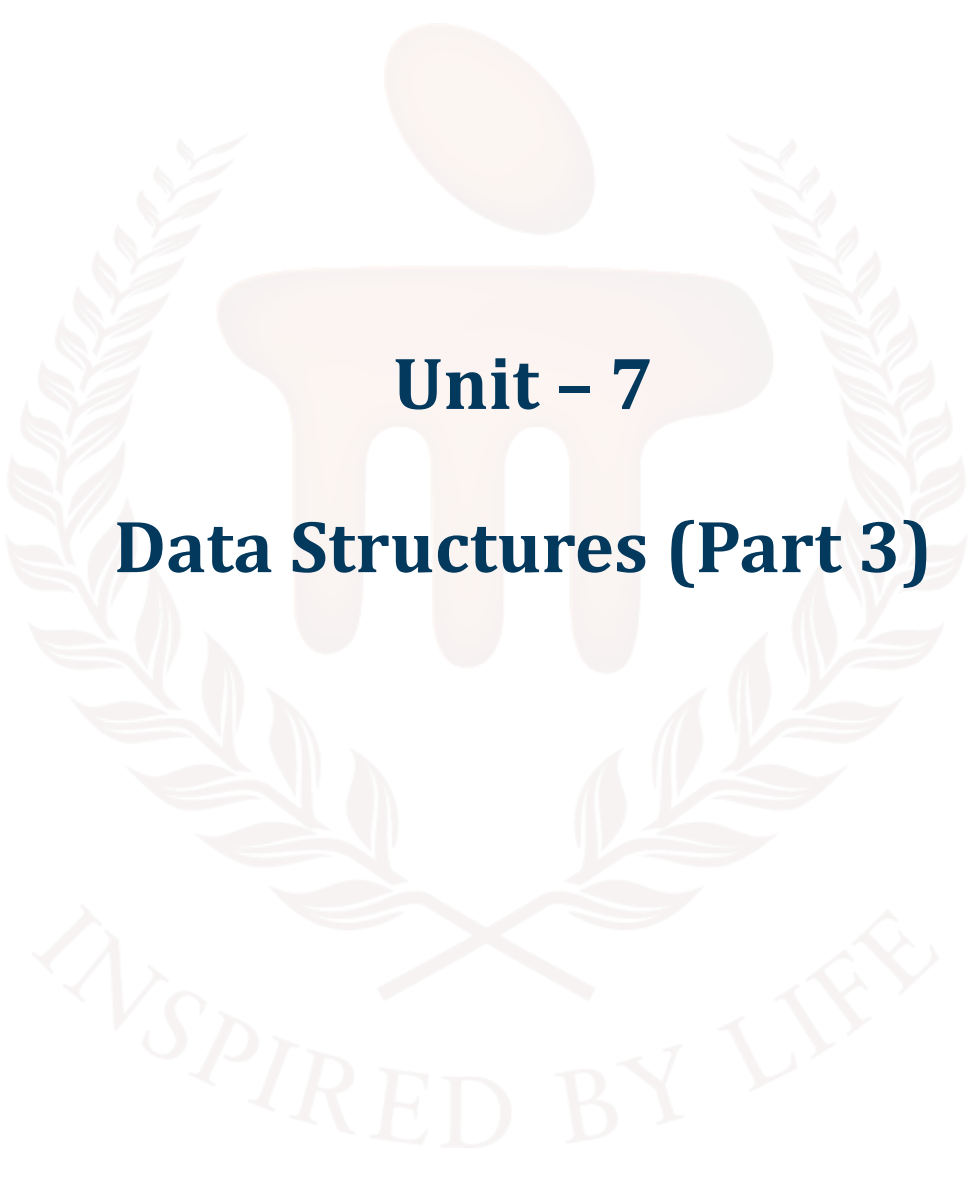
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 7

Data Structures (Part 3)

TABLE OF CONTENTS

Sl No	Topic	Fig No / Table / Graph	Saq / Activity	Page No
1	Introduction	-	-	5-6
1.1	Learning Objectives	-	-	
2	Basics Of Tuples In Python	-	1	7-24
2.1	Definition And Characteristics Of Tuples	-	-	
2.2	Creating And Initialising Tuples	-	-	
2.3	Indexing And Accessing Tuple Elements	-	-	
2.4	Immutable Nature Of Tuples	-	-	
2.5	Difference Between Tuples, Lists, And Sets	-	-	
3	Tuple Operations	-	2	25-50
3.1	Concatenation And Repetition	-	-	
3.2	Membership Testing ('In', 'Not In')	-	-	
3.3	Iterating Over Tuples	-	-	
3.4	Nested Tuples	-	-	
3.5	Built-In Functions With Tuples ('Len()', 'Max()', 'Min()', 'Sum()')	-	-	
3.6	Tuple Packing And Unpacking	-	-	
3.7	Practical Applications Of Tuples	-	-	
4	Summary	-	-	51
5	Glossary	-	-	52
6	Terminal Questions	-	-	53
7	Answers	-	-	54-56
7.1	Self-Assessment Answers	-	-	
7.2	Terminal Questions Answers	-	-	
8	References	-	-	57

1. INTRODUCTION

In this unit, learners will explore the fundamentals of Tuples in Python, an essential data structure widely used for handling fixed collections of items. Tuples are a key part of Python's data handling capabilities, offering both simplicity and efficiency. By studying this unit, learners will understand how tuples differ from other sequence types such as lists and sets, and why their immutability makes them a reliable choice for certain programming scenarios.

The unit begins with the characteristics of tuples, explaining how they represent ordered collections of elements that cannot be modified once created. Learners will understand how this immutability ensures data safety and performance stability, especially when working with fixed datasets such as coordinates, configuration values, or database records. Next, they will learn how to create and initialise tuples using different syntaxes, including parentheses and tuple constructors, as well as how single-element tuples are defined carefully using commas.

Learners will then explore indexing and accessing tuple elements, which follows the same zero-based indexing concept as lists. Through examples, they will see how positive and negative indexing, as well as slicing, can be used to retrieve specific values efficiently. The immutable nature of tuples will be highlighted next, explaining that once a tuple is created, its elements cannot be altered or removed. This section will clarify the benefits of immutability, such as making tuples hashable and thus suitable for use as dictionary keys or elements in sets.

The topic also includes the comparison between tuples, lists, and sets, emphasising their unique features. While lists are mutable and flexible, tuples are fixed and faster in processing. Sets, on the other hand, are unordered and exclude duplicates. Understanding these differences helps learners choose the most appropriate data structure for different use cases.

Moving forward, the unit discusses tuple operations. Learners will practise concatenation and repetition to combine or multiply tuples, and explore membership testing using the `in` and `not in` operators. They will also learn iteration techniques, using for loops to traverse tuple elements effectively. The concept of nested tuples—tuples within tuples—will be introduced to show how complex data can be structured hierarchically.

The unit then explains how to use built-in functions such as `len()`, `max()`, `min()`, and `sum()` to perform computations or retrieve properties from tuples. These operations make tuples a powerful tool in data analysis and programming logic. Learners will also gain practical experience with tuple packing

and unpacking, a feature that allows storing multiple values in a single variable and extracting them easily.

Finally, learners will explore the practical applications of tuples in real-world programming—such as returning multiple values from functions, storing constant data, and using tuples in databases and configurations.

1.1. Learning Objectives

By the end of this Unit, you will be able to:

- Define the Tuples and its syntax.
- Compare lists and tuples.
- List the tuple operations like indexing, slicing, concatenation, repetition, iteration, and membership testing.
- Apply built-in functions on tuples such as `len()`, `min()`, `max()`, and `sum()`.
- Differentiate between mutable and immutable structures
- Use tuples in practical scenarios, including nested tuples, returning multiple values from functions, and employing tuples as dictionary keys.



2. BASICS OF TUPLES IN PYTHON

2.1 Definition and Characteristics of Tuples

In Python, a **tuple** is a built-in data structure used to store an ordered collection of items. Unlike lists, tuples are **immutable**, meaning that once a tuple is created, its contents cannot be changed. This immutability provides stability, faster access in some contexts, and the ability to use tuples as dictionary keys or elements of a set.

A tuple is represented using **parentheses ()**, where elements are separated by commas. Example:

```
numbers = (10, 20, 30)
```

A tuple is essentially a **sequence data type**, just like lists and strings. The items in a tuple are:

1. **Ordered** – The position of elements is preserved. The first element will always remain first unless a new tuple is created.
2. **Immutable** – Elements cannot be changed, added, or deleted after creation. This makes tuples “read-only” lists.
3. **Heterogeneous** – Tuples can store different types of data (integers, strings, floats, even other tuples).
4. **Hashable** – Tuples can be used as dictionary keys or stored in sets if they contain only immutable items.
5. **Indexed** – Elements are accessed using integer indices starting from 0.

Tuples are lightweight and often used when data should remain constant, such as representing geographic coordinates (latitude, longitude) or storing configuration values (host, port).

Code Example 1: Basic Tuple Creation

```
# Creating a tuple of integers
```

```
numbers = (1, 2, 3, 4, 5)
```

```
# Creating a tuple of mixed data types
```

```
person = ("Alice", 25, 5.6)
```


Printing both tuples

```
print("Numbers Tuple:", numbers)
```

```
print("Person Tuple:", person)
```

Output:

Numbers Tuple: (1, 2, 3, 4, 5)

Person Tuple: ('Alice', 25, 5.6)

Logic:

- In the first case, all elements are integers, stored in an ordered and immutable sequence.
- In the second case, elements include a string, an integer, and a float—showing tuples can hold mixed types.

Code Example 2: Nested Tuples

A tuple containing another tuple

```
student = ("John", ("Math", "Science", "History"))
```

```
print("Student Name:", student[0])
```

```
print("Subjects:", student[1])
```

Output:

Student Name: John

Subjects: ('Math', 'Science', 'History')

Logic:

Here, the outer tuple holds two elements:

1. A string "John"
2. Another tuple ("Math", "Science", "History").

This shows that tuples can be **nested** and are useful for representing structured data.

Code Example 3: Empty and Single-Element Tuples

Empty tuple

```
empty = ()
```

Single-element tuple (comma required)

```
single = (5,)
```

```
print("Empty Tuple:", empty)
```

```
print("Single-element Tuple:", single)
```

Output:

Empty Tuple: ()

Single-element Tuple: (5,)

Logic:

- An empty tuple is written as `()`.
- For a single element, a **comma is mandatory** `(5,)`; otherwise, Python interprets `(5)` as an integer wrapped in parentheses.

2.2 Creating and Initialising Tuples

Creating a tuple in Python is straightforward: you place **comma-separated values** inside **parentheses ()**. However, the parentheses are optional in many cases—simply separating values by commas also creates a tuple.

Example:

```
my_tuple = (1, 2, 3) # with parentheses
```

```
another_tuple = 4, 5, 6 # without parentheses
```

Both are valid tuples.

There are multiple ways to create and Initialise tuples in Python:

1. **Using parentheses ()** – The most common way.

```
numbers = (10, 20, 30)
```

2. **Without parentheses (comma-separated values)** – Called *tuple packing*.

```
coordinates = 12.5, 45.8
```

3. **Empty tuple** – Created using empty parentheses.

```
empty = ()
```

4. **Single-element tuple** – Requires a **trailing comma**.

```
single = (5,) # Correct
```

```
not_tuple = (5) # Just an integer
```

5. **Using the tuple() constructor** – Converts another iterable (like a list, string, or set) into a tuple.

```
list_data = [1, 2, 3]
```

```
tuple_data = tuple(list_data)
```

6. **Nested tuples** – Tuples containing other tuples.

```
student = ("Alice", (90, 85, 92))
```

7. **Tuple unpacking** – Assigning values from a tuple to variables.

```
person = ("Bob", 28, "Engineer")
```

```
name, age, profession = person
```

Code Example 1: Different Ways of Creating Tuples

1. Using parentheses

```
t1 = (1, 2, 3)
```

2. Without parentheses

```
t2 = 4, 5, 6
```

3. Empty tuple

```
t3 = ()
```

4. Single-element tuple

```
t4 = (7,)
```

5. Using tuple() constructor

```
t5 = tuple([8, 9, 10])
```

```
print("t1:", t1)
```

```
print("t2:", t2)
```

```
print("t3:", t3)
```

```
print("t4:", t4)
```

```
print("t5:", t5)
```

Output:

```
t1: (1, 2, 3)
```

```
t2: (4, 5, 6)
```

```
t3: ()
```

```
t4: (7,)
```

```
t5: (8, 9, 10)
```

Logic:

- Tuples can be created in multiple syntactic ways.
- A trailing comma is necessary for single-element tuples.
- The tuple() function is useful for converting lists, strings, or sets into tuples.

Code Example 2: Tuple from a String and Range

```
# Tuple from string
```

```
chars = tuple("Python")
```

```
# Tuple from range
```

```
numbers = tuple(range(5))
```

```
print("Tuple from string:", chars)
```

```
print("Tuple from range:", numbers)
```

Output:

```
Tuple from string: ('P', 'y', 't', 'h', 'o', 'n')
```

```
Tuple from range: (0, 1, 2, 3, 4)
```

Logic:

- The string "Python" is broken into individual characters inside a tuple.
- range(5) generates numbers from 0 to 4, which are converted into a tuple.

Code Example 3: Tuple Packing and Unpacking

Packing: multiple values into one tuple

```
person = "John", 30, "Teacher"
```

Unpacking: extracting values

```
name, age, profession = person
```

```
print("Tuple Packed:", person)
```

```
print("Unpacked Values:", name, age, profession)
```

Output:

Tuple Packed: ('John', 30, 'Teacher')

Unpacked Values: John 30 Teacher

Logic:

- Packing automatically groups values into a tuple.
- Unpacking assigns each tuple element to a variable in the same order.
- This feature is widely used in functions that return multiple values.

Code Example 4: Nested Tuples

A tuple with nested elements

```
student = ("Alice", (95, 88, 92))
```

```
print("Student Name:", student[0])
```

```
print("Marks Tuple:", student[1])
```

```
print("First Subject Marks:", student[1][0])
```

Output:

Student Name: Alice

Marks Tuple: (95, 88, 92)

First Subject Marks: 95

Logic:

- Nested tuples are useful for storing structured data.

- Here, `student[1]` returns another tuple (95, 88, 92).
- Accessing `student[1][0]` digs deeper into the nested tuple.

2.3 Indexing and Accessing Tuple Elements

Indexing refers to the process of accessing individual elements of a tuple using their position (index). Since tuples are **ordered**, every element is assigned a fixed position starting from 0. Python allows both **positive indexing** (from the start) and **negative indexing** (from the end).

- **Positive Indexing:** Starts from 0 for the first element, 1 for the second, and so on.
- **Negative Indexing:** Starts from -1 for the last element, -2 for the second last, and so on.

Accessing an element is done using **square brackets []** with the index number.

1. Positive Indexing

```
t = (10, 20, 30, 40, 50)
print(t[0]) # First element
print(t[3]) # Fourth element

t[0] → 10
t[3] → 40
```

2. Negative Indexing

```
t = (10, 20, 30, 40, 50)
print(t[-1]) # Last element
print(t[-2]) # Second last element

t[-1] → 50
t[-2] → 40
```

3. Nested Indexing – Accessing elements from tuples within tuples.

```
student = ("Alice", (95, 88, 92))
print(student[1][0]) # First mark inside the nested tuple

student[1] → (95, 88, 92)
student[1][0] → 95
```

Index Errors – If you try to access an index that does not exist, Python raises an `IndexError`.

```
t = (1, 2, 3)
```

```
print(t[5]) # Error
```

Code Example 1: Positive Indexing

```
fruits = ("apple", "banana", "cherry", "mango")
```

```
print("First fruit:", fruits[0])
```

```
print("Second fruit:", fruits[1])
```

```
print("Fourth fruit:", fruits[3])
```

Output:

First fruit: apple

Second fruit: banana

Fourth fruit: mango

Logic:

- Index starts at 0.
- Accessing `fruits[3]` retrieves "mango".

Code Example 2: Negative Indexing

```
fruits = ("apple", "banana", "cherry", "mango")
```

```
print("Last fruit:", fruits[-1])
```

```
print("Second last fruit:", fruits[-2])
```

Output:

Last fruit: mango

Second last fruit: cherry

Logic:

- Negative indices allow access from the end.
- -1 refers to the last element, -2 to the second last.

Code Example 3: Nested Tuples with Indexing

```
student = ("John", ("Math", "Science", "History"))  
  
print("Student Name:", student[0])  
  
print("Subjects Tuple:", student[1])  
  
print("Second Subject:", student[1][1])
```

Output:

Student Name: John

Subjects Tuple: ('Math', 'Science', 'History')

Second Subject: Science

Logic:

- The outer tuple has two elements.
- student[1] returns another tuple.
- student[1][1] digs into the nested tuple to return "Science".

Code Example 4: Index Out of Range

```
numbers = (1, 2, 3)  
  
try:  
    print(numbers[5])  
except IndexError as e:  
    print("Error:", e)
```

Output:

Error: tuple index out of range

Logic:

- Tuples have fixed size.
- Accessing beyond valid indices raises an IndexError.

Code Example 5: Iterating with Index

```
colors = ("red", "green", "blue")
```

```
for i in range(len(colors)):

    print(f"Index {i} -> {colors[i]}")
```

Output:

Index 0 -> red

Index 1 -> green

Index 2 -> blue

Logic:

- `range(len(colors))` generates indices.
- Each index retrieves an element via `colors[i]`.

2.4 Immutable Nature of Tuples

A tuple in Python is **immutable**, meaning that once a tuple is created, its elements cannot be changed, added, or deleted. Unlike lists, which are mutable and allow modification, tuples are “read-only” collections. This immutability provides stability and makes tuples hashable, allowing them to be used as dictionary keys and elements in sets.

1. No Modification of Elements

- You cannot reassign an element at a given index.
- For example, `t[0] = 99` will raise a `TypeError`.

2. No Addition of Elements

- Methods like `append()` or `extend()` are not available for tuples.

3. No Removal of Elements

- Methods like `remove()` or `pop()` also do not exist for tuples.

4. Workaround by Creating a New Tuple

- You can always create a new tuple by combining existing tuples or by converting a tuple into a list, modifying it, and then converting back into a tuple.

5. Performance and Security

- Since tuples cannot be changed, they are faster to process in certain contexts and provide data integrity.

Code Example 1: Attempting Modification

```
t = (10, 20, 30)
```

```
try:
```

```
    t[0] = 99
```

```
except TypeError as e:
```

```
    print("Error:", e)
```

Output:

Error: 'tuple' object does not support item assignment

Logic:

- Python prevents reassigning elements in a tuple.
- The immutability ensures that the tuple's content is constant once created.

Code Example 2: No Append or Remove Methods

```
t = (1, 2, 3)
```

```
# Check available methods
```

```
print("Tuple Methods:", dir(t))
```

Output (trimmed):

Tuple Methods: ['count', 'index', ...]

Logic:

- Unlike lists, tuples do not have `append()`, `extend()`, or `remove()`.
- They only support `count()` and `index()` as built-in methods.

Code Example 3: Creating a New Tuple

```
t = (1, 2, 3)
```

```
# Create a new tuple by concatenation
```

```
new_t = t + (4, 5)
```

```
print("Original Tuple:", t)

print("New Tuple:", new_t)
```

Output:

Original Tuple: (1, 2, 3)

New Tuple: (1, 2, 3, 4, 5)

Logic:

- Concatenation creates a new tuple without altering the old one.
- Immutability ensures the original tuple remains intact.

Code Example 4: Converting Tuple to List and Back

```
t = (10, 20, 30)

# Convert to list for modification

temp_list = list(t)

temp_list.append(40)

# Convert back to tuple

t = tuple(temp_list)

print("Modified Tuple:", t)
```

Output:

Modified Tuple: (10, 20, 30, 40)

Logic:

- Tuples themselves can't be modified, but you can bypass this by temporarily converting them into a list, modifying it, and converting back into a tuple.

Code Example 5: Tuples as Dictionary Keys

```
# Using tuples as dictionary keys

coordinates = {

    (10, 20): "Point A",

    (30, 40): "Point B"
```

```
}  
  
print(coordinates[(10, 20)])
```

Output:

Point A

Logic:

- Tuples are hashable (if elements inside are immutable).
- This property allows them to be used as dictionary keys or stored in sets, unlike lists.

Why Immutability Matters

1. **Data Integrity:** Prevents accidental changes.
2. **Performance:** Tuples are faster to access than lists.
3. **Hashability:** Enables use as keys in dictionaries.
4. **Functional Programming Style:** Immutable data structures are common in functional programming, and tuples align with this principle.

2.5 Difference Between Tuples, Lists, and Sets

Python provides multiple collection data types—**tuples, lists, and sets**—each with distinct properties and use cases. While they may look similar at first glance, their **mutability, ordering, and uniqueness rules** set them apart.

- **Tuple (()):** An ordered, immutable collection of items.
- **List ([]):** An ordered, mutable collection of items.
- **Set ({}):** An unordered, mutable collection of unique items.

1. Tuples

- **Ordered:** Elements retain their sequence.
- **Immutable:** Cannot be changed after creation.
- **Duplicates allowed:** Can store repeated values.
- **Use case:** Fixed records (e.g., coordinates, database rows).

2. Lists

- **Ordered:** Maintain the sequence of elements.
- **Mutable:** Items can be modified, added, or removed.
- **Duplicates allowed.**
- **Use case:** Dynamic data where frequent modifications are needed.

3. Sets

- **Unordered:** No defined index; elements may appear in random order.
- **Mutable:** Can add or remove elements.
- **No duplicates allowed:** Each element must be unique.
- **Use case:** Mathematical operations (union, intersection), storing unique items.

Code Example 1: Basic Creation

```
# Tuple
t = (1, 2, 3, 3)

# List
l = [1, 2, 3, 3]

# Set
s = {1, 2, 3, 3}

print("Tuple:", t)
print("List:", l)
print("Set:", s)
```

Output:

Tuple: (1, 2, 3, 3)

List: [1, 2, 3, 3]

Set: {1, 2, 3}

Logic:

- Tuple and list preserve duplicates.
- Set automatically removes duplicates.

Code Example 2: Mutability Difference

```
# List can be modified

l = [10, 20, 30]

l[0] = 99

print("Modified List:", l)

# Tuple cannot be modified

t = (10, 20, 30)

try:

    t[0] = 99

except TypeError as e:

    print("Error with Tuple:", e)

# Set allows adding/removing

s = {10, 20, 30}

s.add(40)

s.remove(20)

print("Modified Set:", s)
```

Output:

Modified List: [99, 20, 30]

Error with Tuple: 'tuple' object does not support item assignment

Modified Set: {40, 10, 30}

Logic:

- Lists are mutable.
- Tuples are immutable.
- Sets are mutable but unindexed.

Code Example 3: Ordering and Indexing

Indexing works for list and tuple

```
t = (5, 10, 15)
```

```
l = [5, 10, 15]
```

```
print("Tuple element:", t[1])
```

```
print("List element:", l[1])
```

Sets do not support indexing

```
s = {5, 10, 15}
```

```
try:
```

```
    print(s[1])
```

```
except TypeError as e:
```

```
    print("Error with Set:", e)
```

Output:

Tuple element: 10

List element: 10

Error with Set: 'set' object is not subscriptable

Logic:

- Lists and tuples maintain order and support indexing.
- Sets are unordered and cannot be accessed via index.

Code Example 4: Use in Dictionary Keys

Tuples can be dictionary keys

```
coords = {(10, 20): "A", (30, 40): "B"}
```

```
print(coords[(10, 20)])
```

Lists cannot be dictionary keys

```
try:
```

```
    d = {[1, 2]: "value"}
```

```
except TypeError as e:
```

```
    print("Error with List as Key:", e)
```

```
# Sets cannot be dictionary keys
```

```
try:
```

```
    d = {{1, 2}: "value"}
```

```
except TypeError as e:
```

```
    print("Error with Set as Key:", e)
```

Output:

A

Error with List as Key: unhashable type: 'list'

Error with Set as Key: unhashable type: 'set'

Logic:

- Tuples are hashable (if elements are immutable).
- Lists and sets are mutable → unhashable → cannot be used as dictionary keys.

Tabular Comparison

Feature / Property	Tuple ()	List ()	Set ()
Ordering	Maintains order (sequential, fixed positions)	Maintains order (insertion order preserved)	Unordered, no guarantee of element order
Mutability	Immutable (cannot be changed after creation)	Mutable (can add, remove, or modify elements)	Mutable (can add/remove elements but not modify by index)
Duplicates	Allowed (e.g., (1, 2, 2, 3))	Allowed (e.g., [1, 2, 2, 3])	Not allowed (e.g., {1, 2, 2} → {1, 2})
Indexing	Supported (positive & negative indexing)	Supported (positive & negative indexing)	Not supported (cannot access by index)

Slicing	Supported (e.g., t[1:3])	Supported (e.g., l[0:2])	Not supported
Hashable	Yes (if elements are immutable)	No (lists are mutable, hence unhashable)	No (sets are mutable, hence unhashable)
Nesting	Allowed (can contain other tuples, lists, or sets)	Allowed (can contain tuples, lists, or sets)	Allowed but sets cannot contain mutable elements like lists or other sets
Methods Available	Very few (count(), index())	Many (append(), extend(), remove(), pop())	Some (add(), remove(), discard(), union())
Performance	Faster due to immutability and lightweight structure	Slower than tuples in read-only operations	Faster for membership checks due to hashing
Memory Usage	Consumes less memory	Consumes more memory than tuples	Consumes more due to hashing overhead
Use in Dictionaries	Can be used as keys if fully immutable	Cannot be used as keys	Cannot be used as keys
Iteration	Possible (sequentially iterated)	Possible (sequentially iterated)	Possible (unordered iteration)
Applications	Good for fixed data, records, coordinates, configs	Best for dynamic data, lists of items, sequences	Useful for uniqueness checks, set theory operations (union, intersection)

SELF-ASSESSMENT QUESTIONS – 1

Multiple Choose Questions	
1	Which of the following is the correct way to create a tuple with a single element? a) (5) b) (5,) c) tuple(5) d) {5}
2	What happens if you try to change an element in a tuple?

	a) It changes successfully b) It raises a TypeError c) It raises a ValueError d) It ignores the assignment
3	Which of the following can be used as a dictionary key? a) List b) Set c) Tuple d) Dictionary
4	What will be the output of: <pre>t = (1, 2, 3, 4) print(t[1:3])</pre> a) (2, 3, 4) b) (1, 2, 3) c) (2, 3) d) Error
5	Which of the following statements about tuples is false? a) Tuples are ordered b) Tuples allow duplicates c) Tuples are mutable d) Tuples can store multiple data types

3. TUPLE OPERATIONS

3.1 Concatenation and Repetition

Tuples, like other sequence data types in Python, support **concatenation** and **repetition**. These operations allow us to combine multiple tuples into one or repeat the contents of a tuple multiple times. Since tuples are immutable, these operations do not modify the original tuple but instead produce a new tuple as output.

- **Concatenation (+)**: Joining two or more tuples end-to-end.
- **Repetition (*)**: Repeating the elements of a tuple a specified number of times.

1. Concatenation (+)

- When you use the + operator with two tuples, Python creates a **new tuple** that contains elements of the first tuple followed by elements of the second.
- Original tuples remain unchanged, which highlights the immutable nature of tuples.

Example:

`(1, 2) + (3, 4)` results in `(1, 2, 3, 4)`.

This is useful when you want to combine data sources, such as merging two sets of coordinates (x, y) and (z, w) into one.

2. Repetition (*)

- The * operator repeats the contents of a tuple n times.
- Example: `(1, 2) * 3` produces `(1, 2, 1, 2, 1, 2)`.

This is handy in scenarios like creating placeholder data structures, default configuration values, or testing repeated sequences.

3. Why They Matter

- Both concatenation and repetition **return new tuples**—they never modify the originals.
- They allow efficient manipulation of read-only data structures without compromising immutability.

- Since tuples can store mixed data, concatenation and repetition can be used in advanced data modeling, such as building hierarchical records or repeating constants in simulations.

Coding Example 1 (Concatenation + Repetition)

Concatenation

```
t1 = ("A", "B")
```

```
t2 = ("C", "D")
```

```
combined = t1 + t2
```

Repetition

```
repeated = t1 * 3
```

```
print("Concatenated Tuple:", combined)
```

```
print("Repeated Tuple:", repeated)
```

Output:

Concatenated Tuple: ('A', 'B', 'C', 'D')

Repeated Tuple: ('A', 'B', 'A', 'B', 'A', 'B')

Logic:

- `t1 + t2` creates a new tuple with all elements from both.
- `t1 * 3` repeats ("A", "B") three times.

3.2 Membership Testing (in, not in)

Membership testing in Python allows you to check whether a given element exists in a sequence (like a tuple, list, or string). For tuples, this is done using the operators:

- **in** → Returns True if the element exists in the tuple.
- **not in** → Returns True if the element does **not** exist in the tuple.

This operation is essential for quick lookups, decision-making, and validation in programs.

1. How It Works Internally

- When you use `in` or `not in`, Python sequentially checks each element of the tuple from start to end until it finds a match or reaches the end.

- Since tuples are immutable, the search process does not involve modification—it only reads values.
- For small tuples, this is fast enough, but for very large tuples, membership testing can become slower compared to using a set or dictionary (since those use hashing).

2. Practical Scenarios

- **Validation:** Checking if a student's grade is in the tuple of valid grades `(("A", "B", "C"))`.
- **Filtering:** Testing whether a keyword is in a tuple of restricted words.
- **Decision Making:** Verifying if a product ID exists in a tuple before processing an order.

3. Comparison with Lists and Sets

- Membership testing works the same way for tuples and lists, but both are **linear operations** ($O(n)$ time complexity).
- Sets, on the other hand, are optimised for membership checks using hashing ($O(1)$ on average).
- However, tuples are often chosen when data integrity is important (because they cannot be changed once created).

4. Important Notes

- Membership testing is **case-sensitive** when dealing with strings inside tuples. Example: `"apple" in ("Apple", "Banana")` → False.
- For nested tuples, membership testing only checks the **top level** unless you explicitly search deeper.

Real-World Analogy

Imagine you're at an airport security check with a list of **banned items** stored as a tuple. When the officer inspects your bag, they check item by item:

- If your bag has "knife" and the banned tuple contains "knife", the officer says "True – banned item found".
- If your bag has "toothbrush" and it isn't in the banned tuple, the officer says "True – not in the list".

This is exactly how in and not in work—confirming whether an item belongs to a fixed collection.

Coding Example 2 (Membership Testing)

```
fruits = ("apple", "banana", "cherry", "mango")
```

```
print("apple" in fruits)    # True
```

```
print("grape" in fruits)    # False
```

```
print("orange" not in fruits) # True
```

Output:

True

False

True

Logic:

- "apple" in fruits → True because it exists in the tuple.
- "grape" in fruits → False because it isn't present.
- "orange" not in fruits → True because "orange" is absent.

Advanced Example (Nested Tuples Membership)

If we test membership with nested tuples:

```
data = ("Alice", ("Math", "Science"), "Bob")
```

```
print("Alice" in data)      # True
```

```
print("Math" in data)       # False
```

```
print(("Math", "Science") in data) # True
```

Output:

True

False

True

Logic:

- "Alice" exists at the top level → True.
- "Math" is inside a nested tuple, not at the top level → False.

- The entire nested tuple ("Math", "Science") matches → True.

3.3 Iterating Over Tuples

Iteration refers to the process of traversing each element of a collection one by one. In Python, iteration over a tuple means systematically accessing every element of the tuple in sequence. Because tuples are ordered collections, iteration always respects the order in which elements were defined.

Python offers multiple ways to iterate through a tuple:

1. Using a **for loop** (most common).
2. Using a **while loop** with indices.
3. Using **enumerate()** to get both index and value.
4. Using **unpacking in loops** (when tuples store structured data).

Why Iteration Matters

Iteration is a fundamental programming concept because it allows you to:

- **Access all items** in a collection without manually writing repeated code.
- **Perform computations** on each element (e.g., summing numbers, converting to uppercase).
- **Filter or validate data** (e.g., checking which students passed based on marks stored in tuples).
- **Work with nested structures** (e.g., looping over a tuple that contains multiple smaller tuples).

For tuples specifically:

- Iteration is always **safe** because tuples are immutable, meaning no accidental changes will occur during traversal.
- Iteration guarantees **deterministic order** because tuples preserve sequence.

The Nature of Iteration in Tuples

1. Immutability Advantage:

Since tuples cannot be changed, iteration does not have to worry about the tuple being modified during the process. This ensures stability and predictability.

2. Read-Only Access:

Iteration over tuples always results in read-only operations. If you need to alter elements, you would typically copy the data into another structure like a list.

3. Sequential Traversal:

Tuples are traversed linearly, from index 0 to the last element, unless you explicitly apply conditions (e.g., `continue`/`break`) or reverse logic.

Iteration Approaches in Detail

1. Basic for Loop

The most common way to iterate:

```
colors = ("red", "green", "blue")
```

```
for color in colors:
```

```
    print(color)
```

This prints each color sequentially.

- **Simplicity:** No need to handle indices.
- **Readability:** Directly works with the elements.

2. while Loop with Indices

Sometimes, explicit indexing is required.

- Use a counter starting at 0.
- Increment until it reaches the tuple length.

This is less Pythonic but still relevant when you need index-based control.

3. Using `enumerate()`

When both **index** and **value** are required, `enumerate()` is ideal. It returns pairs (index, element) that can be unpacked directly in the loop.

Example:

```
for i, value in enumerate(("apple", "banana", "cherry")):
```

```
    print(i, "->", value)
```

This shows both index and element.

4. Iterating Over Nested Tuples

Tuples may contain other tuples. Iterating in this case requires either:

- A nested loop (outer for main tuple, inner for inner tuple).
- Or recursive logic if deeply nested.

Example:

```
data = (("Alice", 25), ("Bob", 30))
```

```
for name, age in data:
```

```
    print(name, "is", age, "years old")
```

This technique is widely used in real-world applications like database records.

Real-World Applications of Tuple Iteration

1. Processing Database Records

Many database query results are returned as a tuple of tuples. Iterating over them allows extracting and processing each record.

Example:

```
students = (
```

```
    ("Alice", "A"),
```

```
    ("Bob", "B"),
```

```
    ("Charlie", "C")
```

```
)
```

```
for name, grade in students:
```

```
    print(f'{name} scored {grade}')
```

2. Working with Coordinates

Tuples often represent coordinates. Iteration helps in tasks like distance calculation.

```
coordinates = ((0,0), (3,4), (6,8))
```

```
for x, y in coordinates:
```

```
    print(f'Point ({x},{y})')
```

3. Configuration Values

System settings (e.g., server address, port numbers) stored as tuples can be iterated to validate or display.

Advantages of Iterating Tuples

- **Safety:** The tuple cannot be modified accidentally during traversal.
- **Efficiency:** Tuples are lightweight, making iteration fast.
- **Consistency:** The order is always guaranteed.
- **Simplicity:** Fewer methods compared to lists mean less confusion when iterating.

Comparison with Iterating Lists and Sets

Feature	Tuple Iteration	List Iteration	Set Iteration
Order	Preserved	Preserved	Not preserved
Safety	Cannot change	Elements can change during iteration	Elements can change but unordered
Use Case	Fixed records	Dynamic collections	Unique, unordered membership checks

This shows why tuples are often preferred in scenarios where data integrity is crucial.

Coding Example 3 (Iteration with Enumerate)

```
fruits = ("apple", "banana", "cherry", "mango")
```

```
for index, fruit in enumerate(fruits):
```

```
    print(f"Index {index}: {fruit}")
```

Output:

Index 0: apple

Index 1: banana

Index 2: cherry

Index 3: mango

Logic:

- `enumerate()` provides both index and value.
- Useful for debugging, logging, or cases where element position matters.

Common Mistakes in Tuple Iteration**1. Trying to Modify Elements**

```
for i in (1,2,3):
```

```
    i = i * 2
```

This doesn't change the tuple—it only reassigns the loop variable.

2. Assuming Sets/Lists Work the Same

With sets, iteration order is not preserved. Tuples always respect original order.

3. Shallow vs Deep Iteration

When a tuple contains nested tuples, iterating once only goes one level deep. You need explicit nested loops or recursion for deeper levels.

Advanced Uses of Tuple Iteration**1. Unpacking Multiple Values at Once**

```
data = (("Alice", 85, "A"), ("Bob", 78, "B"))
```

```
for name, score, grade in data:
```

```
    print(f"{name} got {grade}")
```

This is powerful for structured datasets.

2. Combining with `zip()`

Tuples often combine naturally with `zip()` to pair elements from multiple collections.

```
names = ("Alice", "Bob")
```

```
scores = (85, 90)
```

```
for name, score in zip(names, scores):
```

```
    print(f"{name}: {score}")
```

3. Iterating in Reverse

You can reverse tuples using slicing `[::-1]` and then iterate.

```
for value in (1,2,3)[::-1]:
```

```
    print(value)
```

Iteration in Large Datasets

In real-world projects (like log processing, stock data, IoT readings):

- Iteration over tuples ensures stable, read-only access.
- Tuples take up less memory than lists, making them efficient for large but fixed datasets.
- Combined with generators, iteration over tuples can process streams of data efficiently without memory overload.

3.4 Nested Tuples

A **nested tuple** is a tuple that contains one or more tuples as its elements. In other words, it's a tuple inside another tuple. This structure allows you to represent **hierarchical** or **multi-dimensional data** in a compact, immutable format.

Example:

```
student = ("Alice", (85, 90, 92))
```

Here, the outer tuple has two elements: the string "Alice" and another tuple (85, 90, 92) representing marks.

1. Why Nest Tuples?

- **Data Organisation:** When you want to group related information together (e.g., name and scores of a student).
- **Multi-Dimensional Data:** Tuples can represent matrices, grids, or coordinate systems.
- **Immutability in Layers:** Since tuples cannot be modified, nesting ensures even complex structures remain constant.

2. Accessing Nested Tuples

- Elements are accessed using **multiple levels of indexing**.
- Example: `student[1][0] → 85`.

- The first index `student[1]` gets the inner tuple `(85, 90, 92)`. The second index `[0]` fetches its first element.

3. Iteration with Nested Tuples

- You can loop through the outer tuple and then loop again through inner tuples.
- Alternatively, you can use **tuple unpacking** if each inner tuple has the same structure.

Real-World Applications of Nested Tuples

1. Representing Student Records

Each student can be represented as a tuple, and marks stored in another tuple:

```
students = (  
    ("Alice", (85, 90, 92)),  
    ("Bob", (78, 88, 80))  
)
```

This is very similar to rows in a database table.

2. Geographic Coordinates

A nested tuple can store multiple (latitude, longitude) pairs:

```
locations = (  
    ("New York", (40.7128, -74.0060)),  
    ("London", (51.5074, -0.1278))  
)
```

This format is widely used in GPS and mapping applications.

3. Storing Matrix Data

Nested tuples naturally represent a matrix because each row can be another tuple:

```
matrix = (  
    (1, 2, 3),  
    (4, 5, 6),  
    (7, 8, 9)
```

)

Access: `matrix[1][2] → 6`.

4. Configurations and Settings

Nested tuples are often used in configurations, where different settings are grouped but must remain unchanged.

Example: Storing database connection parameters as `("localhost", (3306, "admin", "password"))`.

Advantages of Nested Tuples

1. Compact Representation

- Stores hierarchical data in a structured way without extra classes or objects.

2. Read-Only Integrity

- Ideal for representing fixed relationships like database results, GPS coordinates, or immutable constants.

3. Predictable Access

- Index-based access ensures deterministic behavior.

4. Hashability in Layers

- As long as the inner tuples contain immutable items, the entire nested tuple is hashable.

Comparison: Nested Tuples vs Other Structures

Feature	Nested Tuples	Nested Lists	Dictionaries
Mutability	Immutable	Mutable	Mutable
Order	Preserved	Preserved	Order (since Python 3.7)
Duplicates	Allowed	Allowed	Keys must be unique
Access	By index	By index	By key
Use Case	Fixed hierarchical records	Editable hierarchical records	Key-value mappings

Advanced Considerations

1. Performance

- Nested tuples are slightly more memory-efficient than nested lists because of immutability.

- Ideal for large but fixed datasets.

2. Limitations

- Cannot modify inner tuples directly.
- Deeply nested tuples can become difficult to manage, leading to readability issues.

3. Best Practices

- Use nested tuples for **read-only structured data**.
- If frequent updates are needed, prefer lists or dictionaries.
- Document the structure clearly (e.g., comments or named tuples) to avoid confusion.

Reusing Coding Example (from earlier section)

```
student = ("John", ("Math", "Science", "History"))  
print("Student Name:", student[0])    # John  
print("Subjects Tuple:", student[1])  # ('Math', 'Science', 'History')  
print("Second Subject:", student[1][1]) # Science
```

Output:

Student Name: John

Subjects Tuple: ('Math', 'Science', 'History')

Second Subject: Science

Logic:

- The outer tuple has "John" and another tuple.
- Nested indexing allows precise access.

3.5 Built-in Functions with Tuples (len(), max(), min(), sum())

One of the strengths of Python is its rich set of **built-in functions** that can operate directly on standard data structures like tuples. These functions simplify common operations such as finding the size of a tuple, identifying the largest or smallest element, and even calculating totals for numeric data.

Since tuples are immutable, these functions **do not modify** the tuple; instead, they provide useful information or computed results. This makes them perfectly aligned with the “read-only” philosophy of tuples.

1. len() – Length of a Tuple

The len() function returns the **number of elements** in a tuple.

Explanation

- It counts all elements, regardless of type (strings, numbers, nested tuples, etc.).
- Nested tuples are treated as a single element. For example:
`len((1, (2, 3))) → 2.`

Real-World Example

- Counting the number of student records in a database query represented as tuples.
- Quickly validating whether a tuple has the expected number of configuration values (e.g., (host, port)).

2. max() – Maximum Element

The max() function returns the **largest element** in a tuple.

Explanation

- Works on numeric values (returns highest number).
- Works on strings (returns lexicographically largest string).
- Does not work if the tuple has mixed incompatible types (e.g., integers + strings).

Real-World Example

- Determining the highest test score in a tuple of student marks.
- Finding the last alphabetical word in a tuple of keywords.

3. min() – Minimum Element

The min() function returns the **smallest element** in a tuple.

Explanation

- Works on numbers (lowest value).

- Works on strings (lexicographically smallest).
- Similar limitations to max() with mixed types.

Real-World Example

- Determining the lowest price in a tuple of stock values.
- Identifying the earliest alphabetical word in a tuple of names.

4. sum() – Sum of Elements

The sum() function returns the **total of numeric elements** in a tuple.

Explanation

- Only works on numeric data (integers or floats).
- Raises an error if the tuple contains non-numeric types.
- An optional second argument can be provided to start the sum from a given value.

Real-World Example

- Summing sales values stored as a tuple of numbers.
- Adding time durations represented as numbers inside a tuple.

Coding Example 4 (Built-in Functions)

```
numbers = (10, 20, 30, 40, 50)
print("Length:", len(numbers)) # 5
print("Maximum:", max(numbers)) # 50
print("Minimum:", min(numbers)) # 10
print("Sum:", sum(numbers)) # 150
```

Output:

Length: 5

Maximum: 50

Minimum: 10

Sum: 150

Logic

- **len(numbers)**: Counts all 5 elements.
- **max(numbers)**: Highest value → 50.
- **min(numbers)**: Smallest value → 10.
- **sum(numbers)**: Adds all → 150.

Advanced Considerations

1. With Strings

```
words = ("apple", "banana", "cherry")
```

```
print("Max:", max(words)) # cherry
```

```
print("Min:", min(words)) # apple
```

- "cherry" is max because "c" comes later in alphabetical order than "a" or "b".
- "apple" is min because "a" comes first.

2. With Nested Tuples

len() counts nested tuples as single elements. Example:

```
nested = (1, (2, 3), (4, 5, 6))
```

```
print(len(nested)) # 3
```

3. Performance

- len() runs in constant time ($O(1)$) since Python stores the length internally.
- max(), min(), and sum() take linear time ($O(n)$), scanning all elements.

3.6 Tuple Packing and Unpacking

Python provides two powerful concepts—**packing** and **unpacking**—that make tuples extremely flexible. These features are not only syntactic conveniences but also represent one of the most expressive and elegant aspects of Python programming.

- **Packing**: Putting multiple values into a single tuple.
- **Unpacking**: Extracting tuple elements into separate variables.

Together, they allow for **clean, readable, and concise code**, especially in functions that return multiple values or in loops working with structured data.

1. Tuple Packing

- When multiple values are assigned to a single variable, Python automatically groups them into a tuple.
- Example:
- `packed = 10, 20, 30`
- # Equivalent to `(10, 20, 30)`

2. Tuple Unpacking

- When a tuple is assigned to multiple variables, its elements are distributed (unpacked) into those variables.
- Example:
- `a, b, c = (10, 20, 30)`
- # `a = 10, b = 20, c = 30`

1. Tuple Packing

- Packing happens automatically whenever values are separated by commas.
- Parentheses `()` are optional during packing.
- Useful for storing structured data quickly without explicitly calling the `tuple()` constructor.
- Example:
- `coordinates = 5, 10`
- `print(coordinates) # (5, 10)`

Key Idea:

Packing is like **wrapping gifts into a box**. Each gift (value) is placed in the box (tuple), which can be passed around as a single unit.

2. Tuple Unpacking

- Unpacking assigns tuple elements into variables in order.

- Number of variables must generally match the number of tuple elements.
- Example:
- `person = ("Alice", 25, "Engineer")`
- `name, age, profession = person`
- After unpacking:
 - `name → "Alice"`
 - `age → 25`
 - `profession → "Engineer"`

Key Idea:

Unpacking is like **taking gifts out of the box**. Each item is assigned to its own place.

3. Extended Unpacking (* Operator)

Python allows one variable to “catch” multiple remaining values during unpacking using `*`.

Example:

```
data = (1, 2, 3, 4, 5)
```

```
a, *b, c = data
```

```
# a = 1, b = [2, 3, 4], c = 5
```

- `a` gets the first element.
- `c` gets the last.
- `b` gets everything in between.

This feature is extremely useful when handling **variable-length tuples**.

4. Multiple Assignments

Tuple unpacking allows assigning multiple values in a single line:

```
x, y = 10, 20
```

This is more concise and elegant than writing:

```
x = 10
```

```
y = 20
```

Python internally packs (10, 20) into a tuple, then unpacks it into x and y.

5. Swapping Variables

One of the most famous uses of tuple unpacking is **variable swapping** without a temporary variable:

```
a, b = b, a
```

- Traditional swap requires a temporary storage:
- `temp = a`
- `a = b`
- `b = temp`
- Tuple unpacking makes it a one-liner.

This is not just syntactic sugar; it also improves code readability and reduces errors.

6. Unpacking in Loops

When iterating over a tuple of tuples, unpacking allows directly splitting elements:

```
students = [("Alice", 85), ("Bob", 90)]
```

```
for name, score in students:
```

```
    print(name, "scored", score)
```

Without unpacking, you'd have to access elements using indices, which is less readable.

7. Functions Returning Tuples

Python functions can return multiple values as tuples, and unpacking them is convenient:

```
def min_max(values):
```

```
    return min(values), max(values)
```

```
low, high = min_max([3, 7, 1, 9])
```

Here, the function returns (1, 9) which is unpacked into low and high.

This makes Python functions more powerful and expressive compared to languages that allow only a single return value.

Real-World Applications

1. Database Operations

- Database rows are often tuples. Packing and unpacking allows developers to extract specific fields directly into variables.

2. row = ("Alice", "Physics", 90)

3. name, subject, marks = row

4. **Coordinate Systems**

- In games or simulations, coordinates (x, y, z) are stored as tuples.
- Packing/unpacking helps assign or extract them easily.

5. **Data Transmission**

- Network packets often contain multiple fields (source, destination, payload). Representing them as tuples makes unpacking logical and clear.

6. **Mathematical Computations**

- Returning multiple values (like slope and intercept in a regression function) is clean with tuples.

7. **Swapping Without Temporary Variables**

- Widely used in algorithms (sorting routines, optimisation).

Coding Example 5 (Packing and Unpacking)

Packing

```
person = "John", 28, "Developer"
```

```
print("Packed Tuple:", person)
```

Unpacking

```
name, age, profession = person
```

```
print("Name:", name)
```

```
print("Age:", age)
```

```
print("Profession:", profession)
```

Extended Unpacking

```
data = (1, 2, 3, 4, 5)
```

```
first, *middle, last = data
```

```
print("First:", first)
```

```
print("Middle:", middle)
```

```
print("Last:", last)
```

Output:

Packed Tuple: ('John', 28, 'Developer')

Name: John

Age: 28

Profession: Developer

First: 1

Middle: [2, 3, 4]

Last: 5

Logic:

- Values are automatically packed into a tuple.
- Unpacking distributes tuple elements into distinct variables.
- The * operator collects variable-length elements into a list.

Advanced Insights**1. Error Handling in Unpacking**

- If the number of variables does not match the tuple length, Python raises a ValueError.
- Extended unpacking (*) helps avoid this problem in flexible situations.

2. Memory Efficiency

- Packing allows storing structured values in a compact immutable object.
- Unpacking prevents repetitive indexing operations, making code faster to read and maintain.

3. Parallel Assignment

- Packing/unpacking enables **parallel assignment**, where multiple variables can be updated at once.

3.7 Practical Applications of Tuples

Tuples may look simple compared to lists and dictionaries, but they are **heavily used in real-world Python programs** because of their immutability, readability, and performance benefits. Their key strengths—being ordered, immutable, and hashable—make them suitable for situations where **data integrity and consistency** are more important than frequent modifications.

1. Database Records

Most Python database libraries (such as SQLite, MySQL connector, or psycopg2 for PostgreSQL) return rows of query results as **tuples**.

- Example: A query like `SELECT name, age, grade FROM students` might return rows such as:

`("Alice", 20, "A")`

`("Bob", 21, "B")`

- Why tuples?
 - Records are immutable once retrieved.
 - The database row should not be accidentally changed in memory.
 - Predictable, ordered structure.

This is one of the most common and practical uses of tuples in enterprise applications.

2. Dictionary Keys

Since tuples are **hashable** (if they contain only immutable elements), they can be used as **keys in dictionaries**.

Practical scenarios:

- Storing **coordinates as keys**:

```
grid = {  
    (0,0): "Start",  
    (1,2): "Tree",  
    (3,4): "Treasure"
```

```
}
```

- Mapping **student-course pairs to grades**.
- Representing **graph edges** in algorithms.

Unlike lists, tuples can safely serve as keys because they cannot change after creation.

3. Returning Multiple Values from Functions

In Python, a function can only return one object. Tuples make it possible to **return multiple values elegantly**.

- Example: A function that computes both minimum and maximum values from a dataset can return (min_value, max_value).
- The caller can unpack these values directly:
- `low, high = get_min_max(data)`

This is widely used in mathematical computations, statistics, and machine learning workflows.

4. Coordinate and Geometric Data

In games, simulations, or geographic applications, tuples are perfect for storing fixed coordinate values:

- A **2D point** as (x, y).
- A **3D point** as (x, y, z).
- A **GPS location** as (latitude, longitude).

Since coordinates represent fixed points, immutability ensures they remain constant throughout the program, avoiding accidental changes.

5. Structured but Read-Only Data

When working with data that should never change after , tuples provide security. Examples:

- **Months of a year:**

```
months = ("January", "February", ..., "December")
```

- **Days of the week:**

```
days = ("Monday", "Tuesday", ..., "Sunday")
```

Using a tuple prevents accidental modification, unlike lists.

6. Performance Optimisation

Tuples are slightly **faster and more memory-efficient** than lists. This matters in:

- Large-scale computations where millions of records are being iterated.
- Situations requiring lightweight, read-only structures.

Benchmarking often shows tuples have **faster creation and iteration times** compared to lists because of immutability.

7. Nested and Hierarchical Data

Tuples allow building structured, hierarchical datasets without creating complex objects.

- Example: Representing a **student record** as:

("Alice", (85, 90, 92), "Computer Science")

- Example: Representing a **matrix**:

```
(  
    (1, 2, 3),  
    (4, 5, 6),  
    (7, 8, 9)  
)
```

This is a compact way of grouping data, especially in math-heavy applications.

8. Configuration and Settings

Tuples are often used to store **system configuration values** that should not be modified during runtime:

- Example: Server configuration → (host, port)
- Example: User privileges → ("read", "write")

Since these values represent constants, tuples are the best fit.

9. Using Tuples in Sets

Because tuples are hashable, they can be elements of a **set**.

- Example: Collecting **unique pairs** of (username, role).
- Example: Storing **unique coordinate points** in a set for a mapping algorithm.

This is impossible with lists because lists are mutable.

10. Data Serialization and Transfer

In networking and APIs, tuples are lightweight and compact, making them useful for packaging fixed sets of data before serialisation.

- Example: Representing (status_code, message) pairs in HTTP responses.
- Example: Transferring (x, y, z) sensor values from IoT devices.

11. Tuples in Python Internals

Many Python built-in functions and libraries internally use tuples:

- Functions like enumerate() return (index, element) pairs.
- Iteration with zip() produces tuples.
- Exception handling uses tuples when capturing error details.

This makes tuples a **foundational part of the Python ecosystem**.

Advantages of Tuples in Practical Use

1. **Safety:** No accidental modifications.
2. **Performance:** Faster and lighter than lists.
3. **Hashability:** Can serve as dictionary keys and set elements.
4. **Readability:** Clearly signals to developers that the data is fixed.
5. **Versatility:** Works across databases, networks, mathematical models, and configuration systems.

Limitations

- Tuples are not ideal if frequent modifications are required.
- Deeply nested tuples can reduce readability.
- Lack of methods (compared to lists) means fewer manipulation tools.

SELF-ASSESSMENT QUESTIONS - 2

Fill in the blanks:	
6	A tuple in Python is an _____ collection of ordered elements that cannot be modified once created.
7	The operator used to check if an element exists in a tuple is _____, while the operator used to check if it does not exist is _____.
8	When multiple values are assigned to a single variable without parentheses, Python automatically performs _____, creating a tuple.
9	The built-in function _____ returns the total number of elements in a tuple, while _____ returns the highest element.
10	Tuples can be used as _____ in dictionaries because they are immutable and hashable, unlike lists or sets.

4. SUMMARY

- Tuples are immutable, ordered collections in Python.
- They can hold heterogeneous data types, including other tuples.
- Tuples are created using parentheses () or the tuple() constructor.
- Indexing (positive/negative) and slicing allow element access.
- Their immutability ensures data integrity and faster performance.
- Tuples differ from lists (mutable) and sets (unordered, unique elements).
- Concatenation and repetition create new tuples.
- Membership testing (in, not in) checks for element existence.
- Built-in functions like len(), max(), min(), and sum() enhance usability.
- Tuples are widely used in databases, coordinates, function returns,

5. GLOSSARY

Tuple	- An immutable, ordered sequence of elements in Python.
Immutability	- Property of an object that prevents modification after creation.
Indexing	- Accessing tuple elements using position numbers.
Slicing	- Extracting sub-parts of a tuple without altering it.
Concatenation	- Combining two or more tuples into one.
Repetition	- Repeating elements of a tuple a specified number of times.
Membership Testing	- Checking whether an element exists in a tuple using in or not in.
Packing	- Grouping multiple values into one tuple.
Unpacking	- Assigning tuple elements to multiple variables.
Hashability	- Ability of an object to serve as a dictionary key or set element.
Nested Tuple	- A tuple containing other tuples as its elements.
Built-in Functions	- Predefined functions like len(), max(), min(), and sum() that operate directly on tuples.

6. TERMINAL QUESTIONS

1. What are the key characteristics that differentiate tuples from lists and sets?
2. Explain how tuples are created and Initialised in Python.
3. Describe how indexing and slicing are used to access tuple elements.
4. Why are tuples considered immutable, and what benefits does this provide?
5. How do concatenation and repetition operations work on tuples?
6. Demonstrate how membership testing (in, not in) is applied to tuples.
7. Explain how iteration works with tuples, including nested structures.
8. What is tuple packing and unpacking? Give a practical example.
9. List three real-world applications where tuples are more suitable than lists.
10. Why are tuples hashable, and how does this property benefit Python programs?

7. ANSWERS

7.1. Self-Assessment Answers:

1. b) (5,)
2. b) It raises a TypeError
3. c) Tuple
4. c) (2, 3)
5. c) Tuples are mutable
6. immutable
7. in, not in
8. packing
9. len(), max()
10. keys

7.2. Terminal Question Answers

Answer 1: Tuples are ordered, immutable collections, unlike lists, which are mutable, and sets, which are unordered and disallow duplicates. Tuples retain element order, allow duplicates, and can be used as dictionary keys due to immutability. Their immutability ensures stability, making them ideal for fixed datasets, while lists are suited for flexible, editable data. Sets excel when uniqueness and mathematical operations like union or intersection are required, but lack indexing.

Answer 2: Tuples can be created using parentheses (), by separating elements with commas, or by using the tuple() constructor. For example, (1, 2, 3), 1, 2, 3, and tuple([1, 2, 3]) are all valid. Special cases include empty tuples (), and single-element tuples (5,), where the comma is mandatory. Tuples can also be nested, such as ("Alice", (85, 90)), allowing structured and hierarchical data storage.

Answer 3: Indexing allows accessing tuple elements by position: positive indices start from 0, while negative indices start from -1 for the last element. For example, t[0] retrieves the first element, and t[-1] retrieves the last. Slicing retrieves sub-sequences, using t[start:end]. For instance, t[1:3] retrieves elements from index 1 to 2. Slicing is useful for extracting ranges without altering the original tuple, which remains immutable.

Answer 4: Tuples are immutable, meaning once created, their elements cannot be changed, added, or removed. This immutability provides safety against accidental modifications, enhances

performance since memory is optimized, and allows tuples to be used as keys in dictionaries. For example, `t = (1, 2, 3)` cannot be altered. If modifications are necessary, the tuple must be converted into a list, edited, and then converted back into a tuple.

Answer 5: Concatenation (+) joins two tuples into a new one, such as `(1, 2) + (3, 4)` producing `(1, 2, 3, 4)`. Repetition (*) repeats tuple elements a given number of times, e.g., `(1, 2) * 2` producing `(1, 2, 1, 2)`. Both operations leave the original tuples unchanged due to immutability. These operations are useful for merging fixed datasets or generating repeated patterns in simulations or placeholders.

Answer 6: Membership testing uses `in` and `not in` operators to check if elements exist in a tuple. For instance, `"apple" in ("apple", "banana")` returns `True`, while `"grape" in ("apple", "banana")` returns `False`. Similarly, `"grape" not in ("apple", "banana")` returns `True`. Membership testing is case-sensitive, so `"Apple" in ("apple",)` returns `False`. It is commonly used for validation, filtering, and ensuring whether certain fixed values exist within constant datasets.

Answer 7: Iteration over tuples can be done using `for` loops, while loops with indices, or `enumerate()` to retrieve both index and element. Tuples preserve order, so iteration is predictable. Nested tuples require nested loops or unpacking to access inner elements. For example, iterating over `(("Alice", 85), ("Bob", 90))` allows direct unpacking into `name, score` variables. Iteration is read-only, ensuring no accidental modifications occur during traversal, aligning with tuple immutability.

Answer 8: Packing is the process of grouping multiple values into one tuple, e.g., `person = "Alice", 25, "Engineer"`. Unpacking is the reverse—assigning tuple elements to variables, e.g., `name, age, job = person`. Extended unpacking with * handles variable-length tuples, e.g., `a, *b, c = (1, 2, 3, 4, 5)` results in `a=1, b=[2,3,4], c=5`. This feature simplifies multiple assignments, function returns, and structured data processing, making code more concise and readable.

Answer 9: Tuples are particularly suitable in scenarios requiring immutability and stability. Examples include:

1. **Database records** – query results returned as tuples prevent accidental modifications.
2. **Coordinates in geometry or GPS** – fixed values like (latitude, longitude) are naturally immutable.
3. **Dictionary keys** – tuples can be hashable, unlike lists, making them ideal for mapping composite keys, such as `(student_id, subject)` in academic systems. These real-world contexts benefit from tuples' safety and structural clarity.

Answer 10: Tuples are hashable because of their immutability, meaning their contents cannot change after creation. This property allows them to serve as dictionary keys or be stored in sets, unlike mutable structures like lists. Hashability is valuable when representing fixed pairs or combinations, such as grid coordinates (x, y) or composite identifiers (name, id). Python relies on hash values for fast lookups, making tuples ideal in performance-critical dictionary and set operations.



8. REFERENCES

- Python Software Foundation. Python 3 Documentation – Tuples.
<https://docs.python.org/3/tutorial/datastructures.html>
- Lutz, M. Learning Python, 5th Edition. O'Reilly Media, 2013.
- Sweigart, A. Automate the Boring Stuff with Python. No Starch Press, 2015.
- Beazley, D. Python Essential Reference, 4th Edition. Addison-Wesley, 2009.
- Van Rossum, G., & Drake, F. An Introduction to Python. Network Theory Ltd, 2003.
- Hetland, M. L. Beginning Python: From Novice to Professional. Apress, 2017.
- Pilgrim, M. Dive Into Python 3. Apress, 2009.
- McKinney, W. Python for Data Analysis, 2nd Edition. O'Reilly Media, 2017.
- Summerfield, M. Programming in Python 3: A Complete Introduction to the Python Language. Addison-Wesley, 2010.
- Langtangen, H. P. A Primer on Scientific Programming with Python. Springer, 2016.
- Python Software Foundation. Built-in Functions.
<https://docs.python.org/3/library/functions.html>
- Downey, A. Think Python: How to Think Like a Computer Scientist. Green Tea Press, 2015.

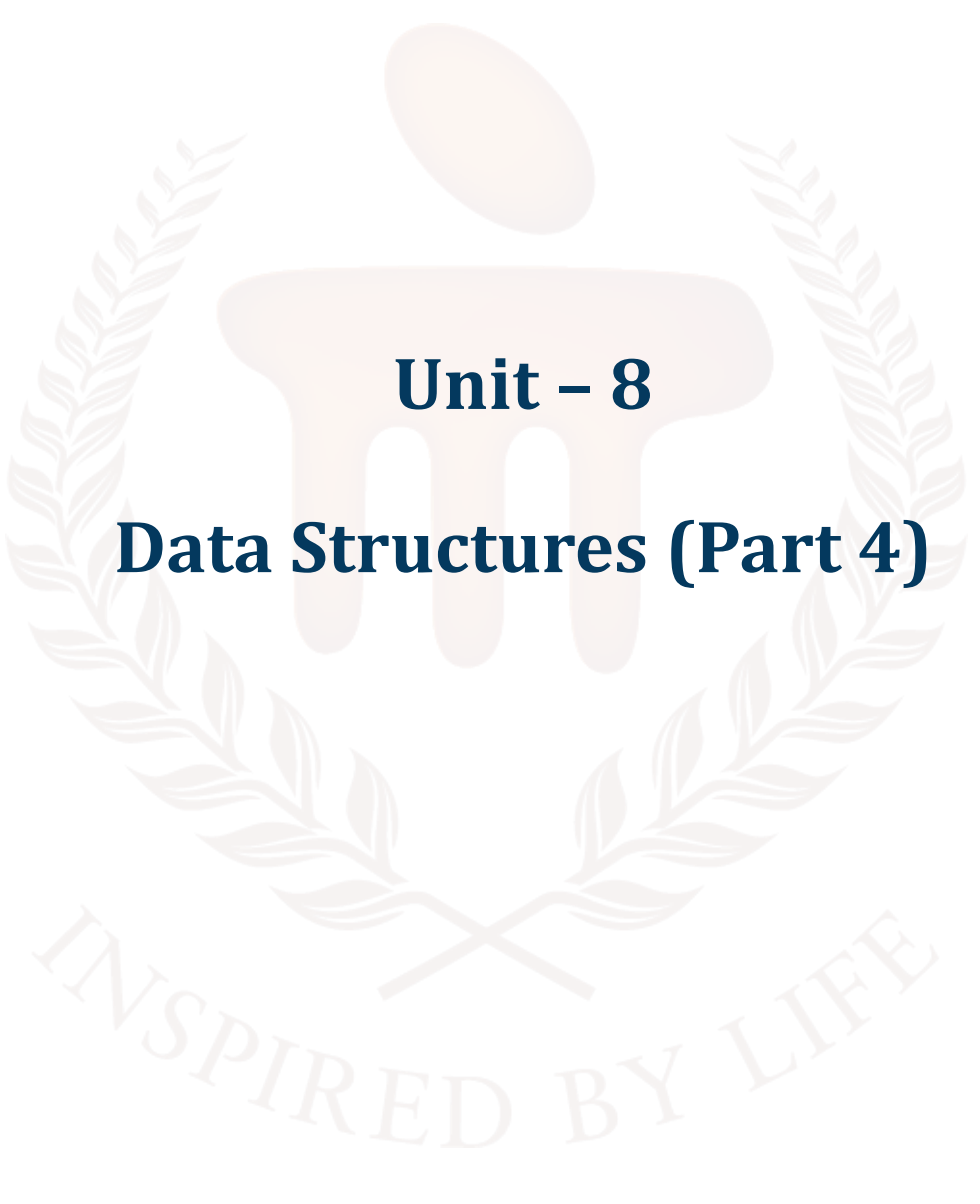
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 8

Data Structures (Part 4)

TABLE OF CONTENTS

Sl No	Topic	Fig No / Table / Graph	Saq / Activity	Page No
1	Introduction	-	-	5-6
1.1	Learning Objectives	-	-	
2	Basics Of Dictionaries In Python	-	1	7-17
2.1	Definition And Characteristics Of Dictionaries	-	-	
2.2	Creating And Initialising Dictionaries	-	-	
2.3	Accessing Dictionary Elements (Keys, Values, Items)	-	-	
2.4	Difference Between Dictionaries And Other Data Structures	-	-	
3	Dictionary Operations	-	2	18-36
3.1	Adding And Updating Elements	-	-	
3.2	Removing Elements ('Pop()', 'Popitem()', 'Clear()', 'Del')	-	-	
3.3	Iterating Over Dictionaries (Keys, Values, Items)	-	-	
3.4	Dictionary Methods ('Get()', 'Update()', 'Setdefault()')	-	-	
3.5	Built-In Functions With Dictionaries ('Len()', 'Sorted()')	-	-	
3.6	Nested Dictionaries	-	-	
3.7	Practical Applications Of Dictionaries	-	-	
4	Summary	-	-	37
5	Glossary	-	-	38
6	Terminal Questions	-	-	39

7	Answers	-	-	40-42
7.1	Self-Assessment Answers	-	-	
7.2	Terminal Questions Answers	-	-	
8	References	-	-	43



1. INTRODUCTION

In this unit, learners will explore one of Python's most powerful and flexible data structures — the Dictionary. Dictionaries allow users to store data in key-value pairs, making data retrieval and manipulation both fast and intuitive. Unlike lists or tuples that rely on positional indexes, dictionaries use keys that uniquely identify each value. This makes them ideal for handling structured information such as student records, employee databases, or configuration settings in programs.

Learners will first understand the characteristics of dictionaries. A dictionary is an unordered, mutable, and indexed collection of data. It allows quick access to values using unique keys, supports heterogeneous data types, and is efficient for searching and updating. Learners will then explore how to create and initialise dictionaries using literals `{}` or the `dict()` constructor, along with examples showing empty, nested, and pre-filled dictionaries. The section will further explain accessing dictionary elements using keys, retrieving all keys, values, and items collectively, and safely accessing data using methods like `get()`.

Next, learners will examine the differences between dictionaries and other data structures such as lists, tuples, and sets. While lists and tuples rely on numeric indices, dictionaries work with keys, offering faster lookups and more flexible data mapping. This comparison helps learners choose the right data structure for the right problem.

Learners will explore dictionary operations. They study how to add and update elements dynamically by assigning values to new or existing keys. The section also covers removing elements using built-in functions like `pop()`, `popitem()`, `clear()`, and the `del` statement — each serving a distinct purpose for managing dictionary contents.

The next topic explains iteration over dictionaries — an essential skill for traversing through keys, values, or key-value pairs efficiently in loops. Learners will then explore several dictionary methods such as `get()`, `update()`, and `setdefault()`, which simplify common operations and make code cleaner and safer. The use of built-in functions like `len()` for measuring size and `sorted()` for ordering keys will also be demonstrated with real-time coding examples.

Finally, learners will learn about nested dictionaries, which allow storing dictionaries within dictionaries to represent complex hierarchical data. Examples such as storing student details (name, age, marks) under a single key will help in understanding how Python supports multi-level data representation.

1.1. Learning Objectives

By the end of this Unit, you will be able to:

- Define Dictionaries and its syntax
- Explain the characteristics of Python dictionaries.
- Initialise dictionaries using different methods.
- Perform basic operations: insertion, deletion, updating, and retrieval of values.
- Apply dictionary methods such as `.keys()`, `.values()`, and `.items()`.
- Differentiate between dictionaries and other data structures (lists, tuples, sets).
- Use dictionaries in practical scenarios like data modeling, configuration storage, and JSON manipulation.



2. BASICS OF DICTIONARIES IN PYTHON

2.1 Definition and Characteristics of Tuples

Definition

A **dictionary** in Python is a built-in data structure designed to store data in the form of **key-value pairs**. It acts like a real-world dictionary where a word (the key) maps to its meaning (the value). Unlike lists or tuples that rely on numerical indexing, dictionaries use keys to retrieve values directly. This key-based lookup makes dictionaries one of the most powerful and efficient data structures in Python.

Dictionaries are enclosed in **curly braces {}**, and each key is separated from its corresponding value by a **colon :**. Multiple pairs are separated by commas. For example:

```
student = {"name": "Alice", "age": 20, "grade": "A"}
```

Here, "name", "age", and "grade" are keys, while "Alice", 20, and "A" are their corresponding values.

The **keys** in a dictionary must be of an **immutable data type**, such as strings, integers, or tuples. This is because keys need to be **hashable**, meaning their hash value remains constant for fast lookups. The **values**, however, can be of any type, including lists, sets, or even other dictionaries. This flexibility allows dictionaries to model complex real-world data effectively.

One defining feature of dictionaries is that they were historically **unordered collections** (up to Python 3.6). However, starting from Python 3.7, dictionaries preserve the **insertion order of keys** as an implementation detail, and from Python 3.8 onwards, this became part of the official language specification. This means that the order in which items are added to the dictionary is preserved when iterating over it, giving developers predictable behavior.

Dictionaries are highly optimised for **fast retrieval**. When you provide a key, Python uses a **hashing algorithm** to quickly locate the value in memory, making dictionary lookups on average an **O(1) operation**. This efficiency is why dictionaries are often used in scenarios requiring fast searches, indexing, or data mapping.

Some important characteristics of dictionaries include:

1. **Mutability:** Dictionaries are mutable, meaning you can add, update, or delete key–value pairs after creation.
2. **Uniqueness of Keys:** A dictionary cannot contain duplicate keys. If you try to insert a duplicate key, the new value overwrites the old one.
3. **Dynamic Size:** Dictionaries can grow or shrink as elements are added or removed.
4. **Heterogeneous Values:** Keys must be immutable, but values can be of any type, including complex objects or nested structures.
5. **Hashability:** Only immutable objects (like strings, numbers, tuples) can be keys. Lists, sets, or dictionaries themselves cannot be used as keys.
6. **Efficient Lookups:** Thanks to hashing, retrieving values by keys is much faster than searching through lists.

In real-world applications, dictionaries are widely used in:

- **Databases** (storing records as key–value mappings).
- **JSON data handling** (JSON maps directly to Python dictionaries).
- **Caching and indexing** (fast retrieval of data).
- **Configuration storage** (system or application settings).

Dictionaries represent a **mapping data type** where keys act as unique identifiers, and values represent the data associated with those identifiers. Their combination of flexibility, speed, and ease of use makes them a cornerstone of Python programming.

Detailed Explanation

Dictionaries are fundamentally different from sequence types (lists, tuples) because they provide **associative access** instead of positional access. Instead of remembering where a value is stored (index), you use a descriptive label (key).

For example, if you store student information in a list, you might access the age using `student[1]`. With a dictionary, you simply use `student["age"]`. This approach improves readability and reduces errors since the code is self-explanatory.

Dictionaries also support **nesting**, meaning you can store another dictionary inside as a value. This is extremely powerful for modeling hierarchical or structured data. For instance:

```
student = {  
    "name": "Alice",  
    "marks": {"math": 90, "science": 88, "history": 92}  
}
```

Dictionaries are **mutable**, allowing updates and deletions. For instance, `student["age"] = 21` updates the age. If a new key is assigned, it is automatically added.

A key design principle of dictionaries is their use of **hash tables** internally. Each key is hashed, and the hash guides Python to the location of its value. This provides constant-time complexity for searches, insertions, and deletions under most conditions.

In summary, dictionaries trade off some memory efficiency for extreme speed and readability, making them indispensable in real-world applications like web development, data analysis, and AI/ML pipelines.

Code Example

```
# Creating a dictionary  
student = {"name": "Alice", "age": 20, "grade": "A"}  
  
# Printing dictionary  
print("Student Dictionary:", student)  
  
# Accessing elements  
print("Name:", student["name"])  
print("Age:", student["age"])  
print("Grade:", student["grade"])
```

Output

```
Student Dictionary: {'name': 'Alice', 'age': 20, 'grade': 'A'}  
  
Name: Alice  
  
Age: 20  
  
Grade: A
```

Logic

In the code, we create a dictionary `student` with three key-value pairs. Each key is a string (`"name"`, `"age"`, `"grade"`) and each value is either a string or an integer. When we print the dictionary, Python displays it using curly braces `{}` with each key-value pair separated by a colon. Accessing values is done by referencing the key inside square brackets—for example, `student["name"]`. This retrieves the value associated with that key. Unlike lists, where `student[0]` might represent the first element, dictionaries make the code more intuitive and readable by associating meaningful labels with data. The immutability of keys ensures stability during lookups, while the mutability of the dictionary allows us to later update `"age"` or add new keys like `"subject"`. This demonstrates how dictionaries combine readability with efficiency.

2.2 Creating and Initialising Dictionaries

A **dictionary** in Python can be created and Initialised in multiple ways, offering flexibility to programmers depending on the situation. Since dictionaries are central to Python programming, understanding the different ways to create them is critical for efficient coding.

The most common method of creating a dictionary is by using **curly braces `{}`** with key-value pairs separated by colons. Example:

```
student = {"name": "Alice", "age": 20}
```

Another approach is using the built-in **`dict()` constructor**, which can accept various forms of data (like sequences of pairs or keyword arguments) and convert them into dictionaries. Example:

```
student = dict(name="Alice", age=20)
```

Dictionaries can also be created from **lists or tuples of pairs**, where each pair contains a key and its corresponding value. This is particularly useful when you have structured data that needs to be mapped. Example:

```
pairs = [("name", "Alice"), ("age", 20)]
```

```
student = dict(pairs)
```

In addition, **dictionary comprehensions** provide a Pythonic way to generate dictionaries dynamically. For example, to create a dictionary of squares for numbers 1 to 5:

```
squares = {x: x*x for x in range(1, 6)}
```

Special cases include creating an **empty dictionary** with `{}` or `dict()`, which can later be populated with key-value pairs. Example:

```
empty = {}  
empty["key"] = "value"
```

Python also allows dictionaries to be created using the **fromkeys() method**, where a collection of keys is Initialised with the same default value. Example:

```
keys = ("a", "b", "c")  
my_dict = dict.fromkeys(keys, 0)
```

Key points about dictionary creation:

1. **Keys must be unique** – duplicate keys overwrite existing values.
2. **Keys must be immutable** – strings, numbers, or tuples are common choices.
3. **Values can be any type** – integers, strings, lists, sets, or even another dictionary.
4. **Flexibility** – Initialisation can be manual (explicit declaration), derived (from existing structures), or generated (using comprehensions).

In practice, the choice of Initialisation method depends on the problem. For static data, curly braces are quick and readable. For dynamic data (like generating mappings programmatically), comprehensions or `dict()` conversions are more efficient.

Detailed Explanation

Dictionaries are versatile because they support multiple Initialisation techniques. Let's explore the **main approaches**:

1. **Curly Braces** – The most common and readable way:

```
student = {"name": "Alice", "age": 20}
```

2. **dict() Constructor** – Useful when passing keyword arguments or converting iterables:

```
student = dict(name="Alice", age=20)
```

3. **From List or Tuple of Pairs** – Often used when transforming external data into mappings:

```
data = [("name", "Alice"), ("age", 20)]  
student = dict(data)
```


4. **Comprehensions** – Provide concise dynamic dictionary creation:

```
squares = {x: x*x for x in range(5)}
```

5. **fromkeys() Method** – Initialises multiple keys with a shared value:

```
my_dict = dict.fromkeys(["a", "b", "c"], 0)
```

These methods allow Python programmers to handle both static datasets (configuration files, constants) and dynamic data (user input, computed values) efficiently. Unlike lists, where elements are positional, dictionaries map meaningful keys to values, improving readability and reducing logical errors.

Code Example

```
# 1. Using curly braces
student1 = {"name": "Alice", "age": 20}

# 2. Using dict() constructor
student2 = dict(name="Bob", age=22)

# 3. From list of pairs
student3 = dict([("name", "Charlie"), ("age", 23)])

# 4. Using comprehension
squares = {x: x*x for x in range(1, 6)}

# 5. Using fromkeys()
keys = ("id", "status", "role")
default_dict = dict.fromkeys(keys, "N/A")

print("Curly braces:", student1)
print("dict() constructor:", student2)
print("List of pairs:", student3)
print("Comprehension:", squares)
print("fromkeys():", default_dict)
```

Output

```
Curly braces: {'name': 'Alice', 'age': 20}
dict() constructor: {'name': 'Bob', 'age': 22}
```



```
List of pairs: {'name': 'Charlie', 'age': 23}
Comprehension: {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
fromkeys(): {'id': 'N/A', 'status': 'N/A', 'role': 'N/A'}
```

Logic

This code demonstrates five ways to create dictionaries. First, curly braces `{}` directly declare keys and values. The `dict()` constructor provides a function-based method, particularly handy when arguments are provided as keywords. Converting from lists of pairs `[("key", value), ...]` is useful when external data needs transformation. Dictionary comprehensions `{key: expression for item in iterable}` allow dynamic generation, here used to build squares of numbers. Finally, `fromkeys()` Initialises multiple keys with a shared default value—helpful when setting default configurations. Each approach highlights Python’s flexibility. Importantly, duplicate keys overwrite previous ones, keys must be immutable, and values can be any type. Together, these methods make dictionaries adaptable for various scenarios like handling JSON, modeling configurations, or transforming tabular data into key–value mappings.

2.3 Accessing Dictionary Elements (Keys, Values, Items)

Definition

Dictionaries in Python are designed to map **keys** to **values**, and the most fundamental operation after creating a dictionary is accessing its elements. Unlike lists and tuples where data is accessed through numeric indices, dictionaries use **keys** for retrieval. This key-based access is what makes dictionaries both powerful and intuitive.

There are multiple ways to access dictionary data:

1. Access by Key

- The most direct method is to reference a value using its key inside square brackets.
- Example:

```
student = {"name": "Alice", "age": 20}
print(student["name"])  # Alice
```
- If the key does not exist, Python raises a **KeyError**.

2. Using the `get()` Method

- This method allows safer access to values. If a key doesn’t exist, it returns `None` or a specified default value instead of throwing an error.

- Example:
- `print(student.get("grade", "Not Available"))`

3. Accessing Keys, Values, and Items

- Dictionaries provide three built-in views to access different aspects:
 - **keys()** → Returns all dictionary keys.
 - **values()** → Returns all dictionary values.
 - **items()** → Returns all key-value pairs as tuples.
- These views are dynamic, meaning changes to the dictionary are reflected immediately in them.

4. Iterating Over Elements

- You can loop directly over keys:
- `for k in student:`
- `print(k, student[k])`
- Or iterate over values using `.values()`, or both keys and values using `.items()`.

5. Membership Testing

- The `in` operator checks for the presence of keys (not values) in a dictionary.
- Example:
- `"name" in student # True`

The **keys** of a dictionary act as the unique identifiers, while the **values** store the associated data. The `.items()` method is particularly powerful since it returns pairs of keys and values, which can be unpacked directly in loops, providing structured access.

In real-world applications, accessing dictionary elements is crucial for tasks like:

- Looking up configuration values by name.
- Retrieving user details in a web application.
- Extracting structured data from JSON objects.

Detailed Explanation

Accessing dictionary elements revolves around the principle of **key-based retrieval**. This design offers two major advantages:

1. Readability and Expressiveness

Instead of retrieving data by position (`list[0]`), you use a descriptive label (`student["name"]`). This makes code easier to understand and maintain.

2. Efficiency

Dictionaries use **hash tables** internally. Keys are hashed, and the hash is used to find the value's location in memory. This ensures average-case constant-time complexity ($O(1)$) for lookups.

2.4 Different methods of access provide flexibility:

- Square bracket access (`dict[key]`) is direct but risky if the key doesn't exist.
- The `get()` method is safer because it prevents runtime errors by returning a fallback value.
- The `keys()`, `values()`, and `items()` methods provide high-level views of dictionary contents. These views can be cast to lists or iterated directly.

For example, if you want only the student names from a class record dictionary, `.values()` gives you direct access without needing to loop manually over the entire structure. Similarly, `.items()` is invaluable when both the key and value need to be processed together, such as when exporting to a file or displaying in a formatted table.

Code Example

```
student = {"name": "Alice", "age": 20, "grade": "A"}
# Direct access by key
print("Name:", student["name"])

# Safe access with get()
print("Grade:", student.get("grade", "Not Available"))
print("Subject:", student.get("subject", "Not Assigned"))
# Accessing keys, values, and items
print("Keys:", list(student.keys()))
print("Values:", list(student.values()))
print("Items:", list(student.items()))
```

Output

```
Name: Alice
```

Grade: A

Subject: Not Assigned

Keys: ['name', 'age', 'grade']

Values: ['Alice', 20, 'A']

Items: [('name', 'Alice'), ('age', 20), ('grade', 'A')]

Logic

This code demonstrates multiple ways of accessing dictionary elements. The first approach, `student["name"]`, retrieves the value "Alice", but would raise an error if the key didn't exist. The second approach uses `get()`, which safely handles missing keys by returning "Not Assigned" instead of an error. This is particularly useful in real-world applications where input data may be incomplete.

The `.keys()`, `.values()`, and `.items()` methods return views of the dictionary. Converting them to lists makes the output easier to read, but they can also be directly iterated in loops. `.items()` is powerful because it returns pairs of keys and values, making it easy to unpack in a single loop construct.

Together, these methods highlight the balance of safety, readability, and efficiency that dictionaries provide, ensuring they remain one of the most versatile data structures in Python.

SELF-ASSESSMENT QUESTIONS – 1

Multiple Choose Questions	
1	Which of the following statements about Python dictionaries is true? a) Keys in a dictionary can be lists or other dictionaries. b) Dictionaries allow duplicate keys. c) Dictionary keys must be immutable. d) Dictionary values must be strings.
2	What will happen if you try to access a non-existent key using square brackets, e.g., <code>student["grade"]</code> ? a) Returns None b) Returns an empty string c) Raises a <code>KeyError</code> d) Creates a new key with value None

3	Which method returns a list-like view of key-value pairs in a dictionary? a) <code>.pairs()</code> b) <code>.items()</code> c) <code>.keys()</code> d) <code>.values()</code>
4	What is the average time complexity for dictionary lookups in Python? a) $O(1)$ b) $O(n)$ c) $O(\log n)$ d) $O(n^2)$
5	Which of the following is the correct way to create a dictionary with default values for multiple keys? a) <code>dict(["a", "b", "c"], 0)</code> b) <code>dict.fromkeys(["a", "b", "c"], 0)</code> c) <code>dict.keys(["a", "b", "c"], 0)</code> d) <code>dict.default(["a", "b", "c"], 0)</code>

3. DICTIONARY OPERATIONS

3.1 Adding and Updating Elements

One of the fundamental features of Python dictionaries is their **mutability**. Unlike tuples, which are immutable, or sets, which focus solely on uniqueness, dictionaries allow you to **add new elements** and **update existing ones** freely. This flexibility makes dictionaries indispensable when working with dynamic datasets such as user profiles, configurations, or real-time analytics.

Adding elements to a dictionary involves introducing a new **key-value pair**. If the key does not already exist in the dictionary, Python creates it and assigns the given value. Updating elements involves modifying the value associated with an **existing key**. In both cases, the syntax is straightforward:

```
dict_name[key] = value
```

If `key` exists, the operation updates its value; if not, a new key-value pair is added. This dual behavior makes dictionary assignment a powerful and concise tool.

For example:

```
student = {"name": "Alice", "age": 20}
student["grade"] = "A"           # Adding new key
student["age"] = 21              # Updating existing key
```

Here, "grade" is newly added, while "age" is updated.

Dictionaries also allow **bulk updates** using the `.update()` method, where multiple key-value pairs can be added or modified at once. This is particularly useful for merging datasets or refreshing configuration files.

Key points about adding/updating elements:

1. **New Key Creation:** Adding with `dict[key] = value` introduces a new pair.
2. **Overwrite on Duplicate Key:** If the key exists, its value is overwritten.
3. **Dynamic Growth:** Dictionaries expand automatically as elements are added.
4. **Bulk Update:** Using `.update()` allows adding/updating multiple elements at once.
5. **Data Integrity:** Updating does not affect dictionary order (since Python 3.7+ maintains insertion order).

This mutability feature makes dictionaries highly adaptable. For instance, in a web application, you can add new user details as they become available or update existing information without reconstructing the entire structure.

Detailed Explanation

Dictionaries differ from lists and tuples in that they do not require predefined size or sequential indexing. The ability to **add or update elements at will** aligns with real-world use cases where information is constantly changing.

- **Adding:** When you assign a value to a non-existent key, Python dynamically creates it. This eliminates the need for explicit “append” methods like those found in lists.
- **Updating:** When you assign a value to an existing key, Python simply replaces the old value with the new one. This is efficient and does not require reallocation of the entire dictionary.

For example, in a shopping cart application:

- **Adding:** `cart["apple"] = 3` introduces "apple" with quantity 3.
- **Updating:** `cart["apple"] = 5` modifies the quantity to 5.

The `.update()` method allows dictionary modification with another dictionary or iterable of key-value pairs. Example:

```
student.update({"grade": "A", "age": 22})
```

This updates "age" and adds "grade".

Such operations ensure dictionaries remain highly efficient for managing dynamic, associative data.

Code Example

```
# Initial dictionary
student = {"name": "Alice", "age": 20}

# Adding new key-value pair
student["grade"] = "A"

# Updating existing key
student["age"] = 21

# Bulk update using update()
student.update({"course": "Data Science", "age": 22})
```



```
print("Final Dictionary:", student)
```

Output

```
Final Dictionary: {'name': 'Alice', 'age': 22, 'grade': 'A', 'course': 'Data Science'}
```

Logic

The dictionary `student` is first created with two keys: `"name"` and `"age"`. When `"grade"` is assigned, it did not previously exist, so Python adds it. Next, `"age"` is reassigned from 20 to 21, updating the existing entry. Finally, the `.update()` method is used with another dictionary: it changes `"age"` again (to 22) and adds `"course"`.

This illustrates the core behavior of dictionary mutability:

- Keys act as **unique identifiers**.
- Assignments either create or update entries based on key existence.
- `.update()` provides a bulk mechanism for merging or modifying entries.

This flexibility is critical in real applications where data evolves over time—whether tracking user profiles, updating financial records, or refreshing configuration settings. Dictionaries ensure these changes happen quickly and efficiently without reconstructing the entire data structure.

3.2 Removing Elements (`pop()`, `popitem()`, `clear()`, `del`)

While dictionaries in Python are mutable and flexible, there are many scenarios where data needs to be **removed**—either selectively or completely. Removing elements ensures that a dictionary reflects the most up-to-date and relevant information. Python provides several methods and keywords for removal operations, each suited to specific use cases.

1. `pop(key[, default])`

- Removes a specific key-value pair using the provided key.
- If the key exists, the corresponding value is returned, and the key is removed.
- If the key does not exist and no default value is provided, it raises a **KeyError**.
- If a default is given, that value is returned instead of raising an error.
- Example:

2. `student.pop("age")`

3. `popitem()`

- Removes and returns the **last inserted key-value pair** as a tuple (**key**, **value**) .
- Useful for iterative removal in cases where dictionary order is not crucial.
- Raises a **KeyError** if the dictionary is empty.

4. `clear()`

- Removes **all key-value pairs** from the dictionary, leaving it empty (`{}`) .
- Often used when resetting a data structure but keeping the dictionary object intact.

5. **del keyword**

- A Python keyword (not a dictionary method) that can be used to delete a specific key-value pair.
- Example:
- `del student["name"]`
- Can also delete the dictionary object entirely with `del dict_name`.
- Raises a **KeyError** if the key doesn't exist.

Key differences:

- `pop()` is for removing by key (with optional default).
- `popitem()` removes the last entry.
- `clear()` removes everything.
- `del` is versatile: it deletes keys or the entire dictionary.

These removal operations are essential in workflows like **cleaning datasets, updating user information, managing caches, or resetting configurations**.

Detailed Explanation

The variety of dictionary removal methods reflects real-world programming needs.

- `pop()` is ideal when you know the exact key to remove. It is also safe if combined with a default value, making it resilient in cases where the key may or may not exist. For example, in user management, you might remove a session entry safely without risking an exception.

- `popitem()` leverages Python's insertion-ordered nature (3.7+). It removes the last added key-value pair, mimicking a "stack-like" behavior. This can be useful when dictionaries are being consumed progressively.
- `clear()` provides a quick way to reset. In caching systems or temporary storage, `clear()` is useful to wipe all contents without deleting the dictionary object itself.
- **del keyword** offers flexibility. It can target specific keys or remove the entire dictionary object from memory. For instance, `del config["timeout"]` deletes just that entry, while `del config` removes the whole dictionary.

Each method has its strengths:

- Use `pop()` for selective removal.
- Use `popitem()` for progressive cleanup.
- Use `clear()` for a full reset.
- Use `del` when precision or complete deletion is needed.

Code Example

```
# Initial dictionary
student = {"name": "Alice", "age": 20, "grade": "A"}

# Using pop()
removed_age = student.pop("age")

# Using popitem()
last_removed = student.popitem()

# Adding back a key for demonstration
student["course"] = "Data Science"

# Using del keyword
del student["course"]

# Using clear()
student.clear()

print("Removed age:", removed_age)
print("Removed last item:", last_removed)
print("Dictionary after clear:", student)
```

Output

```
Removed age: 20
Removed last item: ('grade', 'A')
Dictionary after clear: {}
```

Logic

This code demonstrates all major dictionary removal operations. First, `pop("age")` removes the "age" key and returns its value (20). Next, `popitem()` removes and returns the last inserted key-value pair (("grade", "A")). Then, "course" is added and deleted using the `del` keyword, showing how individual keys can be removed explicitly. Finally, `clear()` removes all elements, leaving the dictionary empty ({}).

Each method showcases different removal strategies:

- `pop()` is targeted and safe with defaults.
- `popitem()` is sequential and efficient when consuming dictionaries.
- `del` gives explicit control over keys or entire objects.
- `clear()` resets the dictionary for reuse.

Together, they make dictionaries flexible for real-world applications like cleaning user data, managing sessions, and handling configuration resets.

3.3 Iterating Over Dictionaries (Keys, Values, Items)

Iteration is the process of traversing each element of a data structure in sequence. In Python, dictionaries support multiple forms of iteration, allowing programmers to loop through **keys**, **values**, or **key-value pairs (items)**. This flexibility makes dictionaries particularly useful in scenarios like data analysis, configuration handling, and dynamic reporting.

Unlike lists and tuples where iteration always follows a sequential order of indices, dictionaries rely on **insertion order of keys** (since Python 3.7, officially guaranteed in Python 3.8+). This means that when you iterate over a dictionary, elements are retrieved in the order they were added.

There are three major approaches:

1. Iterating Over Keys (Default Behavior)

- By default, iterating over a dictionary returns its keys.
- Example:

- `for key in student:`
- `print(key)`

2. Iterating Using `.keys()`

- Explicitly retrieves all keys.
- This is useful when clarity matters in collaborative code.
- Example:
- `for key in student.keys():`
- `print(key)`

3. Iterating Over Values

- The `.values()` method allows iteration through values without concern for keys.
- Example:
- `for value in student.values():`
- `print(value)`

4. Iterating Over Items

- The `.items()` method returns key–value pairs as tuples.
- These tuples can be unpacked directly within the loop.
- Example:
- `for key, value in student.items():`
- `print(key, ":", value)`

Characteristics of dictionary iteration:

- **Order Preservation:** Iteration order follows insertion order.
- **Dynamic Reflection:** Any changes to the dictionary (like adding a new key) during iteration may affect traversal results.
- **Efficiency:** Iteration is fast because dictionaries are optimised with hash tables, though traversal still takes linear time ($O(n)$).

This versatility makes iteration over dictionaries a critical tool in real-world programming, especially when processing structured datasets or exporting dictionary contents.

Detailed Explanation

Iterating over dictionaries is highly practical because it allows direct access to structured data.

- **Iterating keys** is the default behavior. This is useful when you only need identifiers, such as retrieving all usernames from a login dictionary.
- **Iterating values** is helpful when data labels are not important. For instance, extracting all scores from a student record dictionary.
- **Iterating items** is the most powerful because it combines both key and value in a tuple. Using unpacking, you can process both simultaneously, which is common in reporting systems and database handling.

Example in real life: imagine a dictionary storing product IDs as keys and stock counts as values. Iterating over items allows you to generate an inventory report directly.

The `.keys()`, `.values()`, and `.items()` methods return **dynamic views**, meaning they reflect dictionary changes in real-time. If you add or remove entries after retrieving the view, those changes appear immediately in the iteration process.

Thus, dictionary iteration is not only flexible but also synchronised with updates, making it ideal for real-time data applications.

Code Example

```
student = {"name": "Alice", "age": 20, "grade": "A"}
# Iterating over keys
print("Keys:")
for key in student:
    print(key)
# Iterating over values
print("\nValues:")
for value in student.values():
    print(value)
# Iterating over items
print("\nItems:")
for key, value in student.items():
    print(f"{key} : {value}")
```

Output

```
Keys:
name
age
grade
Values:
```

```
Alice
20
A
Items:
name : Alice
age : 20
grade : A
```

Logic

This code demonstrates all three primary forms of dictionary iteration. The first loop iterates directly over the dictionary, printing only the keys ("name", "age", "grade"). The second loop uses `.values()` to print all values ("Alice", 20, "A"), showing that values can be accessed independently of keys. The third loop employs `.items()` to iterate over key-value pairs as tuples. Using unpacking, both the key and value are accessed at once, allowing formatted output like `name : Alice`.

This example highlights how dictionary iteration can be tailored to different needs: key-based operations (like validation), value-only processing (like calculations), or key-value pair handling (like reporting). The ability to iterate flexibly over dictionaries, while preserving insertion order, makes them more powerful and intuitive than most other Python data structures.

3.4 Dictionary Methods `(get(), update(), setdefault())`

Python dictionaries come with several built-in methods that enhance their usability and make them more adaptable to real-world scenarios. Among these, three of the most widely used are **`get()`**, **`update()`**, and **`setdefault()`**. These methods simplify common operations such as safe retrieval of values, bulk updates, and conditional insertion of default values.

1. `get(key[, default])`

The `get()` method retrieves the value for a given key without raising an error if the key does not exist. Instead, it returns `None` by default or a user-defined fallback value.

- **Syntax:**
- `dict.get(key, default_value)`
- If `key` exists → returns its value.
- If `key` doesn't exist → returns `default_value` or `None`.

This prevents runtime crashes that occur when accessing missing keys directly with `dict[key]`. It is especially useful when handling user input or incomplete datasets.


```
2. update([other])
```

The `update()` method adds or modifies multiple key-value pairs at once. It accepts another dictionary, iterable of pairs, or keyword arguments.

- **Syntax:**

- `dict.update(other_dict)`
- If a key exists → its value is updated.
- If a key doesn't exist → it is added.

This is ideal for merging dictionaries, refreshing configuration settings, or updating user profiles dynamically.

```
3. setdefault(key[, default])
```

The `setdefault()` method retrieves the value of a key. If the key doesn't exist, it adds the key with a specified default value and returns that value.

- **Syntax:**

- `dict.setdefault(key, default_value)`
- If `key` exists → returns its value.
- If `key` doesn't exist → inserts it with `default_value` and returns `default_value`.

This is particularly useful for Initialising dictionaries where default values are required, such as grouping elements or counting frequencies.

Detailed Explanation

These three methods serve distinct but complementary purposes:

- `get()` ensures safety during value retrieval. For example, when accessing student grades, if the record is missing, the program won't crash but can provide "Not Available" instead.
- `update()` is powerful for merging or refreshing data. Imagine synchronising two user profiles—`update()` can add new preferences while overwriting outdated ones.
- `setdefault()` combines retrieval and insertion. It is widely used in frequency counting problems. For example, counting word occurrences in text:

```
freq = {}  
for word in ["apple", "banana", "apple"]:  
    freq.setdefault(word, 0)  
    freq[word] += 1
```

This way, `setdefault()` Initialises missing keys automatically.

Together, these methods make dictionary handling safer, more efficient, and more expressive.

Code Example

```
student = {"name": "Alice", "age": 20}
# Using get()
print("Grade:", student.get("grade", "Not Assigned"))
# Using update()
student.update({"grade": "A", "age": 21})
# Using setdefault()
course = student.setdefault("course", "Data Science")
print("Updated Student Dictionary:", student)
print("Course:", course)
```

Output

```
Grade: Not Assigned
Updated Student Dictionary: {'name': 'Alice', 'age': 21, 'grade': 'A', 'course': 'Data Science'}
Course: Data Science
```

Logic

This code demonstrates the three methods in action. First, `get("grade", "Not Assigned")` safely attempts to retrieve "grade". Since "grade" didn't exist, it returns the fallback "Not Assigned" instead of causing a `KeyError`.

Next, `update()` merges the dictionary with another: it adds a new key "grade" with "A", and updates "age" from 20 to 21. This shows how existing values are modified while new ones are added seamlessly.

Finally, `setdefault("course", "Data Science")` checks for the key "course". Since it wasn't present, the method inserts it with the default "Data Science" and returns that value. If "course" already existed, it would simply return its value without overwriting.

Together, these methods highlight Python's emphasis on **safety, efficiency, and readability** in dictionary operations.

3.5 Built-in Functions with Dictionaries (`len()`, `sorted()`)

Python provides several **built-in functions** that work directly with dictionaries. Among them, two of the most commonly used are `len()` and `sorted()`. These functions enhance dictionary usability by

enabling quick measurement of size and easy ordering of elements, tasks that are crucial in data processing, reporting, and algorithm design.

1. `len()` Function

The `len()` function returns the **number of key-value pairs** in a dictionary.

- **Syntax:**
- `len(dictionary)`
- Returns an integer representing the count of items.
- Works in constant time ($O(1)$) because Python stores the size internally.

Example:

```
student = {"name": "Alice", "age": 20, "grade": "A"}  
print(len(student)) # Output: 3
```

Characteristics of `len()` for dictionaries:

1. Counts **pairs**, not individual keys or values separately.
2. Works even on empty dictionaries, returning 0.
3. Provides a quick way to validate the size of dynamic data structures.

2. `sorted()` Function

The `sorted()` function returns a **new sorted list** from the iterable passed to it. When used with dictionaries, it sorts based on keys by default.

- **Syntax:**
- `sorted(dictionary)`
- Returns a list of keys in ascending order.
- Can be customised with the `key` parameter to sort based on values or even custom rules.

Example:

```
student = {"name": "Alice", "age": 20, "grade": "A"}  
print(sorted(student)) # Output: ['age', 'grade', 'name']
```

Sorting by values:

```
print(sorted(student.items(), key=lambda x: x[1]))
```

Characteristics of `sorted()` for dictionaries:

1. Always returns a list, not a dictionary.
2. Default sort is based on keys (ascending order).
3. Custom sorting functions can be applied for flexible arrangements.

Detailed Explanation

The `len()` function is extremely straightforward but powerful. For example, in a student management system, checking `len(records)` quickly tells you how many students are stored. In caching or memory-limited systems, `len()` helps enforce size constraints by preventing dictionaries from growing beyond a threshold.

The `sorted()` function is more advanced. By default, it only returns keys in sorted order, but combining it with `.items()` provides full key-value sorting. With the `key` argument, sorting can be tailored:

- `key=lambda x: x[0]` sorts by keys.
- `key=lambda x: x[1]` sorts by values.
- `reverse=True` allows descending order.

For example, sorting students by grade or age in a dictionary becomes straightforward. The fact that `sorted()` always returns a **list** means the original dictionary remains unmodified, preserving immutability where needed.

Together, `len()` and `sorted()` provide fundamental insights into dictionary data—**size and order**—which are critical in analysis, validation, and reporting.

Code Example

```
student = {"name": "Alice", "age": 20, "grade": "A"}
# Using len()
print("Number of items:", len(student))
# Using sorted() - keys
print("Sorted keys:", sorted(student))
# Using sorted() - items by value
print("Sorted by values:", sorted(student.items(), key=lambda x: x[1]))
```

Output

```
Number of items: 3
Sorted keys: ['age', 'grade', 'name']
Sorted by values: [('grade', 'A'), ('name', 'Alice'), ('age', 20)]
```

Logic

The first operation uses `len(student)` to count the number of key-value pairs. Since there are three pairs (`"name"`, `"age"`, `"grade"`), the result is 3. The next operation, `sorted(student)`, sorts the dictionary keys alphabetically, producing a list `['age', 'grade', 'name']`. Finally, `sorted(student.items(), key=lambda x: x[1])` sorts based on values rather than keys. It produces a list of tuples ordered by the values `(("A", "Alice", 20))`.

This demonstrates how `len()` is useful for size validation, while `sorted()` adds analytical power by enabling order-based processing. Importantly, neither function alters the dictionary itself—`len()` only reports size, and `sorted()` always returns a new list. This ensures data integrity while still providing flexible operations.

3.6 Nested Dictionaries

A **nested dictionary** is a dictionary inside another dictionary. Just as nested lists or nested tuples allow hierarchical data representation, nested dictionaries enable developers to model **multi-level, structured, or relational data** in a single container. Each key in the outer dictionary maps either to a simple value (string, integer, etc.) or to another dictionary, which can itself contain keys and values.

For example:

```
students = {  
    "Alice": {"age": 20, "grade": "A"},  
    "Bob": {"age": 21, "grade": "B"}  
}
```

Here, `"Alice"` and `"Bob"` are outer dictionary keys, each pointing to an inner dictionary that holds their individual attributes.

Key characteristics of nested dictionaries:

1. Hierarchical Structure

- They allow you to represent entities with attributes and sub-attributes.
- For instance, a company's database can be stored as `{"department": {"employee": {"details": ...}}}`.

2. Scalability

- Nested dictionaries can represent large, complex datasets without requiring external databases.

3. Access through Multiple Keys

- To retrieve deeply nested values, you must chain multiple key lookups. Example:
- `students["Alice"]["grade"]` # 'A'

4. Mutability

- Since dictionaries are mutable, nested dictionaries allow adding, updating, or deleting keys at any depth.

5. Use in Real-World Applications

- Nested dictionaries are often used in **JSON-like structures**, which are common in APIs, configuration files, and databases.

Nested dictionaries transform a flat mapping into a structured, hierarchical model, making them indispensable for advanced Python applications.

Detailed Explanation

Nested dictionaries are especially useful when data has **multiple levels of association**. For example, consider storing employee details:

```
employees = {  
    "E101": {"name": "Alice", "role": "Engineer"},  
    "E102": {"name": "Bob", "role": "Manager"}  
}
```

Here, the outer dictionary maps employee IDs ("E101", "E102") to their personal information. Each inner dictionary provides attributes like "name" and "role".

To access nested elements, multiple keys are chained:

- `employees["E101"]["name"]` → "Alice"
- `employees["E102"]["role"]` → "Manager"

Updates are also straightforward:

```
employees["E101"]["role"] = "Senior Engineer"
```

New attributes can be added dynamically to inner dictionaries:

```
employees["E101"]["salary"] = 70000
```

Nested dictionaries closely resemble **JSON objects**. When interacting with APIs, JSON responses (like weather data, product catalogs, or user profiles) are often converted into Python nested dictionaries for easy access. This structural similarity makes them vital in web development, data science, and configuration management.

Code Example

```
students = {  
    "Alice": {"age": 20, "grade": "A"},  
    "Bob": {"age": 21, "grade": "B"}  
}  
  
# Accessing nested values  
print("Alice's grade:", students["Alice"]["grade"])  
  
# Updating nested values  
students["Bob"]["grade"] = "A-"  
  
# Adding a new nested key  
students["Alice"]["course"] = "Data Science"  
  
print("Updated Students Dictionary:", students)
```

Output

```
Alice's grade: A  
Updated Students Dictionary: {  
    'Alice': {'age': 20, 'grade': 'A', 'course': 'Data Science'},  
    'Bob': {'age': 21, 'grade': 'A-'}  
}
```

Logic

In this code, the dictionary `students` is defined with two outer keys (`"Alice"`, `"Bob"`), each mapping to an inner dictionary. Accessing `students["Alice"]["grade"]` retrieves Alice's grade from the inner dictionary. Updating `students["Bob"]["grade"]` changes Bob's grade from `"B"` to `"A-"`. A new key `"course"` is added to Alice's dictionary dynamically, showing that nested structures can evolve at any depth.

This demonstrates the power of nested dictionaries: they allow structured storage of related attributes within a single entity. Instead of managing multiple parallel dictionaries (e.g., `names`, `ages`, `grades`), nested dictionaries combine everything into one, simplifying management and improving readability. Such structures are common when parsing JSON data from APIs, where information naturally comes in hierarchical form.

3.7 Practical Applications of Dictionaries

Dictionaries in Python are not just academic constructs; they are among the most **practically useful data structures** for real-world problem solving. Because they combine **fast lookups, mutability, and key-value mappings**, dictionaries appear in nearly every domain of Python programming—from web development to data science and artificial intelligence.

A **practical application** means leveraging dictionaries in scenarios that mirror real-world systems, such as databases, caching systems, or structured information models. Unlike lists (which depend on numeric indices), dictionaries allow data retrieval through **meaningful keys**, making programs more expressive and less error-prone.

Some of the most significant practical applications include:

1. Databases and Record Storage

- Dictionaries can mimic database rows, where column names act as keys and cell values as dictionary values.
- Example: `{"id": 101, "name": "Alice", "marks": 92}`

2. JSON and API Responses

- Many APIs return data in JSON format, which maps directly to Python dictionaries. Developers can parse and manipulate API responses easily.

3. Configuration Files

- Application settings such as file paths, timeouts, and permissions can be stored in dictionaries.
- Example: `{"theme": "dark", "timeout": 300, "version": "1.0"}`

4. Caching and Memoization

- Dictionaries provide a way to store computed results, avoiding redundant calculations.
- Example: caching Fibonacci numbers or expensive database queries.

5. Data Analysis

- Dictionaries are used for frequency counting, mapping items to values (like word counts), or grouping records by categories.

6. Graph and Network Representation

- Graphs can be modeled as dictionaries of adjacency lists:

- `graph = {"A": ["B", "C"], "B": ["C"], "C": []}`

7. AI and Machine Learning

- Dictionaries store hyperparameters, mappings of class labels, or feature metadata.

The **real power of dictionaries** lies in their ability to mirror real-world data structures. Consider a **student management system**: instead of storing names, ages, and grades in parallel lists, a dictionary allows storing all information together under one key (`"student_id"`). This drastically reduces complexity and improves clarity.

In **web development**, configuration dictionaries hold routing information, authentication details, or API keys. For example, a Flask application often uses dictionaries for settings like `{"DEBUG": True, "SECRET_KEY": "xyz"}`.

In **data science**, dictionaries are critical for counting frequencies, grouping data, or even representing hierarchical data structures. The classic word count problem in natural language processing relies on dictionaries: each word becomes a key, and its frequency becomes the value.

In **graph theory**, dictionaries naturally represent nodes and edges. Each node (key) maps to its adjacency list (value), allowing flexible graph traversal algorithms.

Finally, dictionaries are widely used for **caching**. Suppose a function is computationally expensive—its results can be stored in a dictionary with inputs as keys and outputs as values. This avoids repeated computation and saves time, particularly in AI/ML tasks where models may process millions of records.

Code Example

```
# Practical Example: Word Frequency Counter
text = "apple banana apple orange banana apple"
words = text.split()
```

```
freq = {}
for word in words:
    freq[word] = freq.get(word, 0) + 1
```

```
print("Word Frequencies:", freq)
```

Output

```
Word Frequencies: {'apple': 3, 'banana': 2, 'orange': 1}
```

Logic

In this example, the problem is to count the frequency of words in a text. The string is split into words using `.split()`, producing a list `["apple", "banana", "apple", "orange", "banana", "apple"]`. An empty dictionary `freq` is created to store the counts.

For each word in the list, the program uses `dict.get(word, 0)` to safely retrieve its current count (defaulting to 0 if the word isn't already present). Then, it increments the count by 1. The result is a dictionary mapping each unique word to its frequency: `{"apple": 3, "banana": 2, "orange": 1}`.

This example showcases how dictionaries provide a simple yet powerful way to solve frequency-counting problems. Such applications extend naturally to larger datasets, such as analysing social media hashtags, counting product sales, or processing logs in real-time systems.

SELF-ASSESSMENT QUESTIONS – 2

Fill in the blanks:	
6	A dictionary in Python stores data in the form of _____ pairs.
7	The method _____ removes a specific key and returns its value, while the method _____ removes the last inserted item.
8	Iterating directly over a dictionary by default gives its _____.
9	The dictionary method _____ safely retrieves a value without raising a <code>KeyError</code> , while the method _____ adds or updates multiple key-value pairs at once.
10	A _____ dictionary is a dictionary that contains another dictionary as its value.

4. SUMMARY

- Python dictionaries store data as **key-value pairs**, where keys are unique and immutable, and values can be of any type.
- Dictionaries can be created using `{}`, the `dict()` constructor, from sequences of pairs, comprehensions, or `fromkeys()`.
- Values are accessed via keys (`dict[key]`), safely with `.get()`, or through dictionary views (`.keys()`, `.values()`, `.items()`).
- Keys are **immutable** (string, number, tuple), while values can be mutable objects like lists or even other dictionaries.
- Dictionaries differ from lists, tuples, and sets by offering **associative access** (by keys) instead of positional or uniqueness-based access.
- Elements can be added or updated with `dict[key] = value`, and multiple updates can be applied at once using `.update()`.
- Removal operations include `pop()` (specific key), `popitem()` (last item), `del` (specific/all), and `.clear()` (empties dictionary).
- Iteration can be done by **keys**, **values**, or **items**, with `.items()` allowing simultaneous unpacking of key-value pairs.
- **Nested dictionaries** allow hierarchical data structures, similar to JSON, useful for APIs, configurations, and complex data models.
- Dictionaries are widely applied in **databases, caching, JSON parsing, configurations, frequency counting, graph modeling, and machine learning pipelines**.

5. GLOSSARY

Dictionary	- A Python data structure that stores data in key–value pairs.
Key	- An immutable identifier (string, number, or tuple) used to access values in a dictionary.
Value	- The data associated with a key in a dictionary; can be any type.
KeyError	- An error raised when attempting to access a non-existent dictionary key directly.
Mutable	- A property that allows data structures like dictionaries to be updated after creation.
Hashing	- The process of mapping keys to memory addresses for fast dictionary lookups.
Dictionary View	- Objects returned by .keys(), .values(), and .items(), reflecting dictionary contents..
Nested Dictionary	- A dictionary containing another dictionary as a value, allowing hierarchical data storage.
Update Method	- The .update() method used to modify or add multiple key–value pairs simultaneously.
Setdefault Method	- A method that retrieves a value or inserts a default value if the key doesn't exist.
Dictionary	- A Python data structure that stores data in key–value pairs.
Key	- An immutable identifier (string, number, or tuple) used to access values in a dictionary.

6. TERMINAL QUESTIONS

1. What are the key characteristics of Python dictionaries that make them unique from other data structures?
2. Explain different ways of creating and Initialising dictionaries in Python.
3. How can you safely access dictionary values without risking a `KeyError`?
4. Compare dictionaries with lists, tuples, and sets, highlighting their differences.
5. How are new elements added and existing ones updated in dictionaries?
6. Describe different methods of removing elements from a dictionary with examples.
7. What are the main ways to iterate over a dictionary and their practical uses?
8. Explain the purpose and usage of dictionary methods `get()`, `update()`, and `setdefault()`.
9. What is a nested dictionary? Give an example of how it can be used.
10. List at least three practical applications of dictionaries in real-world programming.

7. ANSWERS

7.1. Self-Assessment Answers:

1. c) Dictionary keys must be immutable.
2. c) Raises a KeyError.
3. b) .items().
4. a) O(1).
5. b) dict.fromkeys(["a","b","c"], 0).
6. key-value
7. pop(), popitem()
8. keys
9. get(), update()
10. nested

7.2. Terminal Question Answers

Answer 1: Python dictionaries are mutable collections storing data in key-value pairs. Keys must be unique and immutable, while values can be of any type. They provide associative access instead of positional indexing, making retrieval faster and more intuitive. Dictionaries also preserve insertion order (Python 3.7+). These features distinguish them from lists, tuples, and sets, making dictionaries ideal for structured, dynamic, and fast-access data handling in real-world applications like databases, caching, and configuration storage.

Answer 2: Dictionaries can be created using curly braces {}, the `dict()` constructor, dictionary comprehensions, or the `fromkeys()` method. For example: `{"name": "Alice", "age": 20}` creates a dictionary directly. Using `dict(name="Alice", age=20)` leverages the constructor. From a list of tuples, `dict([("name", "Bob"), ("age", 22)])` works. Comprehensions allow dynamic generation, such as `{x: x*x for x in range(5)}`. `dict.fromkeys(["a","b"], 0)` Initialises keys with default values. These flexible options suit different programming needs.

Answer 3: To safely access dictionary values, the `.get()` method is used instead of direct indexing. Accessing with `dict[key]` raises a `KeyError` if the key is absent, while `dict.get(key, default)` returns `None` or a user-specified default. Example: `student.get("grade", "Not Assigned")`. This prevents crashes and allows graceful handling of incomplete datasets. It is particularly useful when working with uncertain inputs, user data, or APIs where missing values are common, ensuring program stability and reliability.

Answer 4: Dictionaries differ from lists, tuples, and sets in access and storage. Lists and tuples use positional indexing, while sets ensure uniqueness without key-value mappings. Dictionaries provide associative access using descriptive keys, enabling fast lookups. Tuples are immutable, lists are mutable sequences, sets handle unique items, but dictionaries map meaningful keys to values. This allows them to represent structured data efficiently, making them suitable for records, configurations, and databases. Each structure has unique strengths depending on use cases.

Answer 5: Elements are added or updated using `dict[key] = value`. If the key doesn't exist, a new entry is created; if it exists, the value is updated. For example: `student["grade"]="A"` adds a grade, while `student["age"]=21` updates age. The `.update()` method allows bulk updates, such as `student.update({"course":"Data Science", "age":22})`. This dual functionality—creation and modification—ensures dictionaries remain dynamic and flexible for real-time data changes in applications like user profiles or system configurations.

Answer 6: Dictionaries provide multiple removal methods. `pop(key)` removes a specific key and returns its value; `popitem()` removes and returns the last inserted pair. The `del` keyword deletes a key or the entire dictionary object, while `.clear()` removes all elements, leaving an empty dictionary. Example: `student.pop("age")`. Each method serves different needs: selective removal, progressive cleanup, or complete reset. These tools make dictionaries highly adaptable for cleaning datasets, managing caches, or resetting dynamic structures efficiently.

Answer 7: Iteration over dictionaries can be done by keys, values, or items. Iterating directly or via `.keys()` returns keys. `.values()` iterates over all values. `.items()` returns key-value pairs as tuples, which can be unpacked for simultaneous processing. Example:

```
for k,v in student.items():  
    print(k,v)
```

This flexibility makes iteration useful in tasks like generating reports, validating data, or exporting information. Iteration is insertion-ordered (Python 3.7+), predictable, and essential for structured data processing workflows.

Answer 8: The `.get()` method retrieves values safely, returning `None` or a default if the key is missing. `.update()` allows adding or updating multiple key-value pairs at once, often used for merging configurations. `.setdefault()` retrieves a value if the key exists or inserts it with a default value otherwise. Example:

```
d.setdefault("count", 0)
```

This ensures stability in frequency counting or grouping tasks. Together, these methods enhance safety, efficiency, and flexibility when managing dictionary data in real-world scenarios.

Answer 9: A nested dictionary is a dictionary within another dictionary, enabling hierarchical data representation. Example:

```
students = {"Alice":{"age":20,"grade":"A"},  
            "Bob":{"age":21,"grade":"B"}}
```

Here, outer keys map to student names, while inner dictionaries hold their details. Nested dictionaries mirror JSON objects and are widely used in APIs, configuration files, and databases. They allow structured storage of complex data, making retrieval and updates manageable in multi-level systems like employee databases or product catalogs.

Answer 10: Dictionaries have numerous practical applications. They represent database records with fields as keys, store configurations, and handle JSON data from APIs. They are vital in caching, where results of computations or queries are stored for fast reuse. In data analysis, they are used for frequency counts and grouping. Graphs can also be modeled using adjacency lists in dictionaries. Their speed, flexibility, and expressiveness make them core to web development, AI, machine learning, and enterprise systems.

8. REFERENCES

- Python Software Foundation – *Python 3.11 Documentation: Data Structures*.
- Lutz, Mark – *Learning Python, 5th Edition*. O'Reilly Media, 2013.
- Beazley, David – *Python Essential Reference, 4th Edition*. Addison-Wesley, 2009.
- Sweigart, Al – *Automate the Boring Stuff with Python*. No Starch Press, 2015.
- Van Rossum, Guido – *An Introduction to Python*. Network Theory Ltd, 2003.
- Hetland, Magnus Lie – *Beginning Python: From Novice to Professional*. Apress, 2017.
- McKinney, Wes – *Python for Data Analysis, 2nd Edition*. O'Reilly Media, 2017.
- Pilgrim, Mark – *Dive Into Python 3*. Apress, 2009.
- Summerfield, Mark – *Programming in Python 3: A Complete Introduction to the Python Language*. Addison-Wesley, 2010.
- Downey, Allen – *Think Python: How to Think Like a Computer Scientist*. Green Tea Press, 2015.

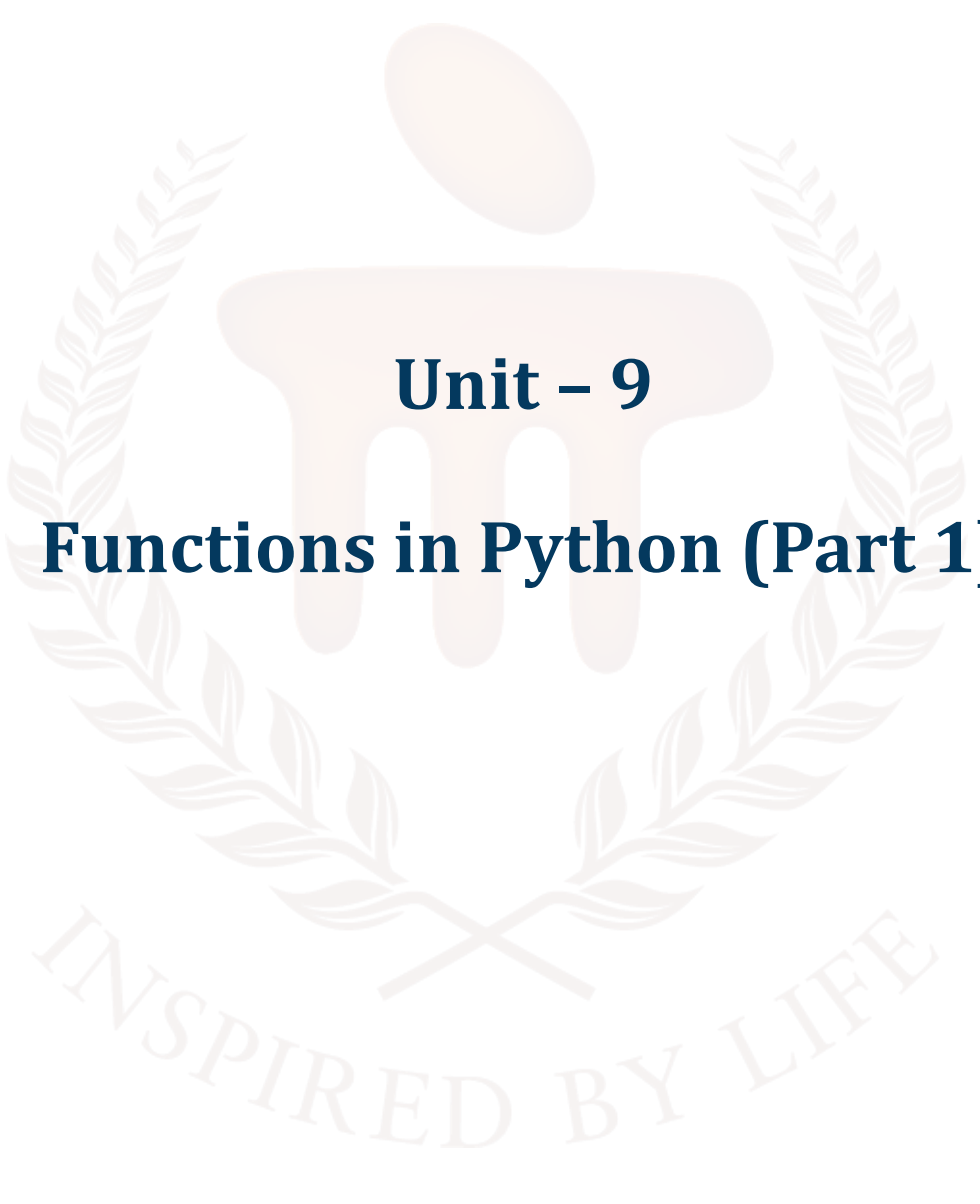
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 9

Functions in Python (Part 1)

TABLE OF CONTENTS

Sl No	Topic	Fig No / Table / Graph	Saq / Activity	Page No
1	Introduction	-	-	5-6
1.1	Learning Objectives	-	-	
2	Basics of Functions	-	1	7-14
2.1	Definition and Purpose of a Function	-	-	
2.2	Built-in Functions Versus User-defined Functions	1	-	
2.3	Anatomy of a Function: Name, Signature, Body	2	-	
2.4	Advantages of Organising Code with Functions	3	-	
3	Function Definitions		2	15-24
3.1	The Def Statement and Correct Syntax	-	-	
3.2	Indentation, Function Body and Single-line Functions	4	-	
3.3	Docstrings and Simple Type Annotations	5	-	
3.4	The Return Statement and Returning Multiple Values	-	-	
3.5	Naming Conventions and Readability Tips	-	-	
4	Calling Functions	-	3	25-35
4.1	Simple Calls and Expression Contexts	-	-	
4.2	Positional and Keyword Arguments	6	-	
4.3	Default Arguments and Common Pitfalls	-	-	
4.4	Variable-length Argument Lists: *args and **kwargs	-	-	
4.5	Argument Evaluation Order and Side Effects	-	-	
5	Summary	-	-	36
6	Glossary	-	-	37
7	Terminal Questions	-	-	38
8	Answers	-	-	39-41
8.1	Self-Assessment Answers	-	-	
8.2	Terminal Questions Answers	-	-	
9	References	-	-	42

1. INTRODUCTION

In this unit, we begin exploring one of the most important building blocks of Python programming functions. Functions allow programmers to structure code into smaller, reusable sections that perform specific tasks. This approach not only improves the readability of a programme but also helps in maintaining and testing it efficiently. The unit introduces the concept of functions, explaining their purpose, structure, and how they contribute to modular programming.

The unit also covers how to define and call functions in Python. Learners will study the syntax of the `def` statement, the use of parameters and arguments, and how the `return` statement is used to send results back to the calling code. It distinguishes between built-in and user-defined functions, explores various argument-passing methods such as positional, keyword, and default arguments, and explains how Python executes function calls and manages scope.

To study this unit effectively, start by revising the basic syntax rules of Python, including indentation and variable naming conventions. Then, practise writing simple functions to perform small, well-defined tasks. Focus on calling these functions correctly and observing how arguments and return values are handled. Working through small examples and experimenting with different argument types will strengthen your grasp of function-based programming.

1.1. Learning Objectives

By the end of this Unit, you will be able to:

- Define and create functions in Python using correct syntax.
- Identify the components of a function, including parameters and return values.
- Apply different methods of passing arguments to functions.
- Use built-in and user-defined functions effectively in programmes.
- Analyse function calls and manage variable scope within functions.



2. BASICS OF FUNCTIONS

In Python programming, one of the most important aspects of writing efficient, organised, and maintainable code is understanding and using functions. Before this stage, a programmer might have written all instructions sequentially in a single block of code. While this approach works for small programmes, it quickly becomes difficult to manage when the code grows in size and complexity. To overcome this limitation, Python provides a mechanism known as a *function* that allows programmers to structure programmes into smaller and more manageable parts.

Functions serve as building blocks of a programme, allowing specific tasks to be grouped together and executed when required. They make it easier to reuse code, avoid repetition, and isolate individual operations for better understanding and debugging. In larger projects, functions contribute to modular programming, where different sections of a programme are developed independently and then integrated seamlessly.

This section introduces the basic idea of functions in Python. It explains what a function is, why it is used, and how it differs between built-in and user-defined types. It also describes the internal structure or anatomy of a function and highlights the benefits of organising code using functions. By the end of this section, the learner should be able to understand the logic behind function creation, identify built-in functions, write user-defined ones, and appreciate how functions contribute to the overall structure of a Python programme.

2.1 Definition and Purpose of a Function

A function in Python is a reusable block of organised and related statements designed to perform a specific task. Functions help in structuring code into manageable sections, making it more readable and easier to maintain. In programming, repetitive tasks often occur, such as calculating the area of a circle, finding the average of numbers, or validating input. Instead of rewriting the same code multiple times, these operations can be written once inside a function and called whenever needed.

From a logical perspective, a function acts like a miniature programme within a larger programme. It takes some input (called arguments or parameters), performs certain operations, and returns an output or carries out an action. Python supports both built-in functions, such as `len()`, `print()`, and `range()`, and user-defined functions created by programmers for specific needs.

In Python, functions are defined using the keyword `def`, followed by the function name and a pair of parentheses that may include parameters. A colon marks the start of the function body, and the body itself is indented, as per Python's indentation rules.

Example:

```
def greet():  
    print("Hello, welcome to Python programming!")
```

In the above example, the function `greet()` prints a message when called. The keyword `def` indicates that a new function is being defined, while `print()` is a statement inside the function body. Once defined, the function can be called anywhere in the programme using its name followed by parentheses:

```
greet()
```

When executed, this will output:

Hello, welcome to Python programming!

The primary purpose of using functions in programming includes:

- **Code Reusability:** A function can be used multiple times throughout a programme without rewriting the same block of code.
- **Code Organisation:** Functions help in breaking down complex problems into smaller, manageable tasks.
- **Abstraction:** Functions hide the internal details of how a task is performed, exposing only the required interface.
- **Ease of Maintenance:** Changes to a function's internal code automatically reflect everywhere it is called, reducing redundancy and errors.

2.2 Built-in Functions Versus User-defined Functions

Python provides two main types of functions: built-in functions and user-defined functions. Both play a key role in simplifying programming tasks, but their origin and usage differ.

Built-in Functions: These are pre-defined functions that come with Python's standard library. They are available for immediate use and require no separate definition. Built-in functions perform

commonly needed operations such as mathematical computation, string manipulation, type conversion, and input/output handling.

Some examples of Python's built-in functions include:

- `print()` – displays output on the screen.
- `len()` – returns the length of a list, string, or tuple.
- `type()` – determines the type of a given variable.
- `range()` – generates a sequence of numbers.
- `sum()` – computes the sum of a list or iterable.
- `max()` and `min()` – find the largest and smallest items in a sequence.

For example:

```
numbers = [4, 9, 2, 7, 5]
print("Total numbers:", len(numbers))
print("Maximum number:", max(numbers))
```

These functions simplify tasks that would otherwise require multiple lines of code. Python's standard library extends functionality even further through modules such as `math`, `random`, and `datetime`, each containing additional pre-built functions.

User-defined Functions: User-defined functions are those created by the programmer to perform specific operations that are not already available through Python's built-in library. Such functions reflect the programmer's design for solving a particular problem within an application. They are defined using the `def` keyword followed by a chosen function name, parameters (if any), and an indented block of code.

Example:

```
def area_of_circle(radius):
    area = 3.1416 * radius * radius
    return area
print(area_of_circle(5))
```

In this example, the programmer defines a new function called `area_of_circle()` that takes the radius as an argument, computes the area, and returns it.

Table 1: Comparison between Built-in and User-Defined Functions

Aspect	Built-in Functions	User-defined Functions
Definition	Predefined in Python and ready for use	Created by the programmer
Availability	Part of the standard library	Must be explicitly defined
Examples	<code>print()</code> , <code>len()</code> , <code>sum()</code>	<code>area_of_circle()</code> , <code>greet_user()</code>
Flexibility	Limited to existing functionality	Fully customisable
Purpose	To provide common operations efficiently	To solve specific problems within a programme

2.3 Anatomy of a Function: Name, Signature, Body

Every function in Python has a particular structure that defines how it operates. This structure consists of several key parts, each serving a specific role. Understanding this anatomy is important for writing correct and effective functions.

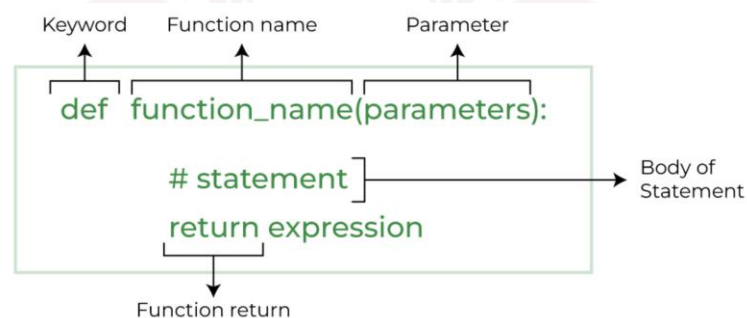


Figure 1: Function Syntax

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20220721172423/51.png>

This figure delineates the fundamental syntax for defining a function in the Python programming language. The process begins with the required keyword `def`, signalling the start of a function definition, immediately followed by the chosen function name. Any necessary input variables are listed within the parentheses as parameters, concluding with a colon to mark the start of the function

block. The body of statement contains the indented code that executes when the function is called, often including one or more statements to perform a task. Finally, the optional return keyword facilitates the function return, allowing a value or expression to be passed back to the calling section of the programme. This structure is essential for creating reusable and modular code components.

1. Function Name - A function name uniquely identifies a function in a programme. The name is used when calling the function. Like variable names, function names must follow Python's naming rules:

- They must start with a letter or underscore.
- They can contain letters, digits, and underscores.
- They should not contain spaces or special characters.
- Names are case-sensitive, meaning `calculate()` and `Calculate()` would be considered different.

Choosing descriptive function names makes code easier to read. For example, `get_average()` is more meaningful than `func1()`.

2. Function Signature (Header) - The function header defines the function name and the parameters it accepts. It begins with the keyword `def`, followed by the name and a pair of parentheses. Parameters are listed inside the parentheses and represent the input data required by the function.

Syntax:

```
def function_name(parameter1, parameter2, ...):
```

The signature also defines the scope of the function, meaning the variables inside it are not accessible outside unless returned.

Example:

```
def add_numbers(a, b):  
    return a + b
```

In this case, `add_numbers` is the function name, and `(a, b)` are the parameters that the function accepts.

- 3. Function Body** - The function body is the indented block of code that performs the desired operation. The first line after the colon is typically a documentation string (docstring), which explains what the function does. This is optional but recommended for clarity.

Example:

```
def multiply(x, y):  
    """This function multiplies two numbers and returns the  
    result."""  
    result = x * y  
    return result
```

Here, the body contains a docstring, an operation (`result = x * y`), and a return statement that outputs the result.

- 4. Return Statement** - The return statement ends the function's execution and sends the result back to the caller. If no return statement is given, the function returns `None` by default.

Example:

```
def square(num):  
    return num * num
```

When called with `square(4)`, it returns 16.

If a function performs an action but does not need to send data back, it may omit the return statement entirely.

- 5. Parameters and Arguments** - Parameters are variables listed in the function definition, while arguments are actual values passed to the function during a call. For instance:

```
def greet_user(name):  
    print("Hello, ", name)
```

```
greet_user("Alice")
```

Here, `name` is a parameter, and `"Alice"` is an argument.

Python supports various types of parameters, such as positional parameters, keyword parameters, default parameters, and variable-length parameters, which allow flexible handling of inputs.

2.4 Advantages of Organising Code with Functions

Functions form one of the fundamental building blocks of any structured programme. Organising code using functions provides several advantages that improve clarity, reusability, and reliability of software projects.

- **Improved Code Reusability** - Functions allow code that performs a particular operation to be reused throughout the programme without rewriting it. For instance, if a calculation or data validation is needed in multiple places, it can be written once inside a function and called wherever required. This reduces repetition and shortens development time.
- **Easier Code Maintenance** - When a programme is organised using functions, updates and modifications can be made easily. If an error or logic change occurs within a function, only that function needs to be updated, rather than every part of the programme that uses the same code pattern.
- **Enhanced Readability and Structure** - Functions make long programmes easier to read by dividing them into smaller, logical segments. Each function performs a distinct task, and descriptive function names convey the purpose of that segment clearly. This modular design improves readability and makes it easier for others to understand the code.
- **Simplified Debugging and Testing** - Functions isolate specific tasks, which simplifies debugging. When a problem occurs, it can be traced to the specific function responsible, rather than searching through an entire programme. Each function can also be tested individually to ensure correctness before integration with the rest of the system.
- **Reduced Code Duplication** - Programmes often include repetitive logic, such as validating input or computing totals. Functions help centralise such logic, ensuring that the same behaviour is consistent across different parts of the code. This reduces redundancy and ensures consistency.
- **Promotes Modular Programming** - Functions support modular programming, where the programme is divided into independent modules that can be developed, tested, and maintained separately. Each module can contain several functions related to a specific task or operation, which promotes a cleaner architecture.
- **Better Collaboration and Integration** - In large development teams, different programmers can work on different functions or modules without interfering with each other's work. Once

tested, these functions can be combined to form the complete application. This encourages parallel development and faster delivery.

- **Abstraction and Encapsulation** - Functions allow hiding of complex details. The user of a function does not need to know how it works internally, only how to use it. For instance, when calling `len()`, the user does not need to know the internal algorithm used to count elements. This abstraction makes code easier to understand and maintain.
- **Enhanced Scalability** - Well-structured functions make it easier to expand a programme's features. New functions can be added without rewriting existing ones, supporting scalability and growth of applications over time.
- **Reproducibility in Data Processing** - In data analysis or scientific computation, functions enable reproducibility. Once a function performs a data cleaning operation or analysis method, it can be applied to new datasets without rewriting the logic, ensuring consistent results.

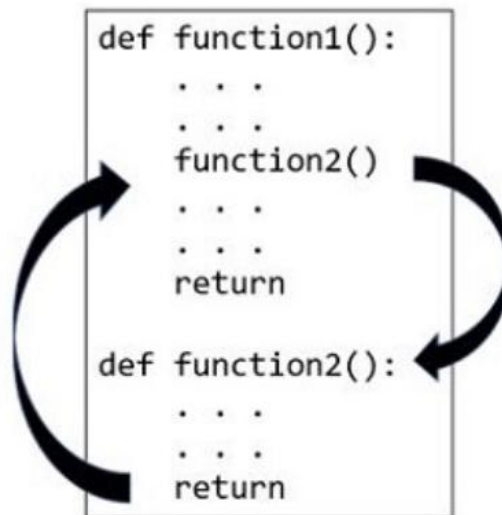


Figure 2: Function Workflow

Source: https://www.tutorialspoint.com/python/images/python_functions.jpg

The figure illustrates a fundamental principle of modular programming through the interaction between separate functions in a code base, exemplified here in Python. Function1 represents the main block or a higher-level process, which executes a call to function2 during its operation. The curved arrows signify the flow of control: the programme temporarily leaves function1 to execute the statements within function2, after which it returns to the exact point of the call in function1 to continue execution. This clear division of labour ensures reusability of code, allows for easier testing

of individual components, and helps manage complexity by breaking the overall task into smaller, manageable subroutines.

Functions are the foundation of organised programming in Python. They allow a programmer to divide large, complex tasks into smaller parts that are easier to manage, understand, and reuse. Through functions, the same code can be executed multiple times across different parts of a programme, saving time and ensuring consistency.

Understanding the basics of functions—what they are, how they are defined, and how they differ between built-in and user-defined types—builds the groundwork for more advanced programming concepts. The structure of a function, including its name, signature, and body, defines how it interacts with data and the rest of the programme.

By organising code into functions, programmers achieve modularity, clarity, and efficiency, which are essential qualities for any scalable software application. This approach not only simplifies development but also ensures long-term maintainability of projects. As we progress to the next section, these fundamental concepts will serve as the base for understanding how to define and call functions effectively in Python.

SELF-ASSESSMENT QUESTIONS – 1

Multiple Choose Questions	
1	Which statement best defines a function in Python? a) A statement that repeats code automatically. b) A type of loop used to iterate a sequence. c) A reusable block of organised and related statements designed to perform a specific task. d) A module imported from the standard library.
2	Which keyword is used to define a new function in Python? a) func b) def c) define d) function
3	In the anatomy of a function, which terms correctly pair the variables in the function definition with the actual values passed when calling the function? a) Parameters are in the definition and arguments are the actual values passed.

	b) Arguments are in the definition and parameters are the actual values passed. c) Signatures are the actual values and bodies are parameters. d) Returns are the parameters and parameters are the returns.
4	What is the purpose of the return statement in a Python function? a) To start the function definition. b) To declare local variables. c) To document what the function does. d) To end function execution and send a value back to the caller.
5	Which of the following is an example of a user-defined function from the notes? a) len() b) area_of_circle(radius) c) print() d) range()



3. FUNCTION DEFINITIONS

Functions are a fundamental component of any programming language, and in Python, they are used extensively to improve structure, readability, and reusability of code. The definition of a function forms the foundation upon which the entire concept of modular programming rests. A function definition specifies how the function behaves—what it takes as input, what operations it performs, and what it produces as output.

In Python, defining a function involves more than merely writing code blocks; it is about expressing logic clearly, using correct syntax, maintaining consistent structure, and adhering to conventions that make the code understandable to others. Each function definition must follow a specific format and respect Python's indentation rules, since indentation in Python is not just stylistic—it dictates the structure of the programme itself.

This section explains how to properly define functions using the `def` statement, discusses the role of indentation and structure in defining function bodies, explores the use of documentation strings (docstrings), and covers the concept of type annotations for clarity. It also explains how to use return statements effectively, including returning multiple values, and concludes with practical tips on naming conventions and readability that help maintain clean, professional, and maintainable code.

3.1 The Def Statement and Correct Syntax

The process of defining a function in Python begins with the `def` statement. This keyword stands for “define” and signals to the Python interpreter that a new function is being declared. The function definition specifies the function's name, its parameters, and the block of code that will execute when the function is called.

The general syntax of defining a function is as follows:

```
def function_name(parameters):  
    """Optional documentation string."""  
    # function body  
    statement(s)  
    return expression
```

Each part of this structure plays a specific role:

1. **The `def` Keyword:** This tells Python that a function definition is starting.

2. **Function Name:** The name must follow Python's identifier rules—letters, digits, and underscores are allowed, but it cannot start with a digit. The name should describe the task of the function clearly.
3. **Parentheses and Parameters:** Parameters (if any) are enclosed within parentheses. A function can accept zero, one, or more parameters. If no parameters are required, the parentheses remain empty.
4. **Colon :** The colon marks the start of the function body. The indented lines following it form the body of the function.
5. **Indented Block (Function Body):** The body of the function contains statements that perform the intended task. All lines must be consistently indented.
6. **Return Statement (Optional):** A function can return a value using the return keyword. If omitted, the function returns None by default.

Example 1:

```
def greet_user():  
    print("Hello! Welcome to Python programming.")
```

Here, the function `greet_user()` does not take any parameters and simply prints a message. To execute it, we call the function by writing:

```
greet_user()
```

Example 2 (Function with Parameters):

```
def add(a, b):  
    sum = a + b  
    return sum
```

When called as `add(5, 3)`, it returns 8.

A function must always be defined before it is called in a programme. If a function call appears before its definition, Python raises a `NameError` since the interpreter reads code from top to bottom.

3.2 Indentation, Function Body and Single-line Functions

Indentation in Python Functions: In Python, indentation determines the structure of code. Unlike many other programming languages that use braces `{ }`, Python uses indentation to indicate code

blocks. This applies to functions, loops, and conditional statements. The function body must be indented under the definition line to show that it belongs to that function.

Example:

```
def display_message():  
    print("This is part of the function.") # indented line  
    print("Python uses indentation instead of braces.")
```

If indentation is missing or inconsistent, Python raises an `IndentationError`. Proper indentation is essential not only for correct execution but also for readability and professional coding style.

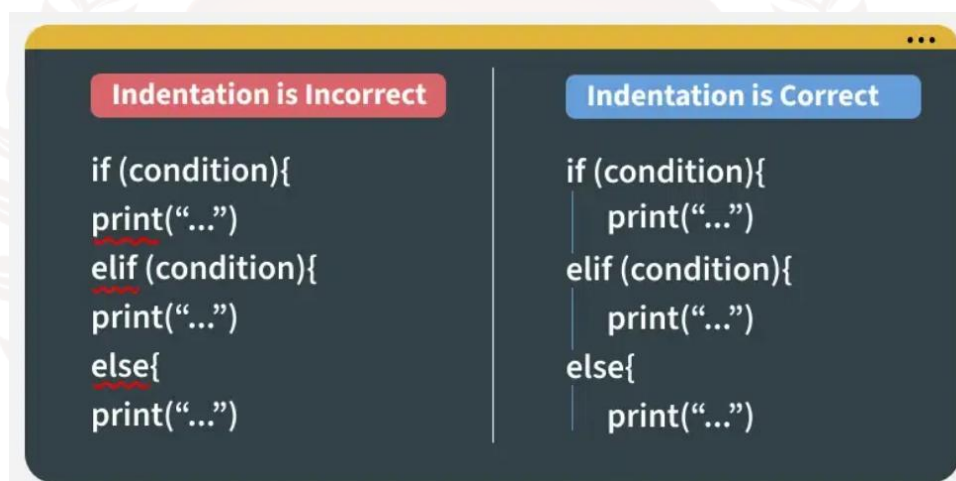


Figure 3: Indentation in Python

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20241203113516014762/Indentation-in-python.webp>

The figure clearly contrasts the correct and incorrect use of indentation, a crucial aspect of Python syntax that defines code blocks. The left panel shows that statements directly following control flow keywords, such as `if`, `elif`, and `else`, are incorrectly aligned with their parent statement, indicated by the wavy underlines which would represent a syntax error. Conversely, the right panel illustrates the correct structure, where the statements within each block, specifically the `print` function calls, are uniformly indented from the preceding control statement. This mandatory consistent indentation is not merely stylistic but is fundamental to the language's parser for determining the logical grouping and execution flow of the programme.

Structure of the Function Body: The body of a function can contain:

- Variable declarations or assignments.

- Conditional statements.
- Loops.
- Function calls.
- Return statements.
- Docstrings (optional).

Example:

```
def check_number(num):  
    if num % 2 == 0:  
        return "Even"  
    else:  
        return "Odd"
```

Here, the function body uses a conditional structure to check the nature of the input number.

Single-line Functions: In some cases, a function performs a very short operation that fits comfortably in one line. Python allows defining such functions in a compact form.

Example:

```
def square(x): return x * x
```

This is syntactically valid, but excessive use of single-line functions can reduce readability in longer programmes. They are best used for simple mathematical operations or short return expressions.

Lambda Functions (Anonymous Functions): Although this section focuses on normal functions, it is worth noting that Python also supports small, unnamed functions using the lambda keyword.

Example:

```
square = lambda x: x * x
```

Both the normal and lambda versions behave similarly, though lambda functions are often used in cases where a short, one-time-use function is sufficient.

3.3 Docstrings and Simple Type Annotations

Docstrings (Documentation Strings): Good programming practice requires every function to describe what it does. Python provides an elegant way to document functions through *docstrings*. A docstring is a string literal placed immediately after the function definition line, enclosed in triple quotes `"""`. It serves as internal documentation that can be viewed by users through the built-in `help()` function or the `__doc__` attribute.

Example:

```
def calculate_area(radius):  
    """This function calculates the area of a circle given its radius."""  
    return 3.1416 * radius * radius
```

Docstrings improve code readability and are valuable when developing large projects where multiple programmers need to understand the purpose of functions written by others.

Multi-line docstrings are used when more detailed documentation is required:

```
def calculate_area(radius):  
    """  
    Calculate the area of a circle.  
    Parameters:  
        radius (float): The radius of the circle.  
    Returns:  
        float: The computed area.  
    """  
    return 3.1416 * radius * radius
```

When someone types `help(calculate_area)`, Python displays the docstring contents, making it easy to understand the function's purpose and usage.

Example:

```
def addtownumber(a, b):  
    """Python program to add two numbers."""  
    print('Sum of two numbers: ',a+b)  
    print(addtownumber.__doc__)
```

Output:

```
Python program to add two numbers.
```

Figure 4: Docstrings in Python

Source: <https://www.stechies.com/python-docstrings/>

The figure demonstrates the implementation and retrieval of a docstring within a Python function, a key feature for documentation. The function `addtownumber` is defined with two parameters, and immediately following the header, a triple-quoted string is included, which serves as the docstring, providing a concise description of the function's purpose. Crucially, the docstring is not executed as code but is stored as an attribute of the function object. As shown in the output, this internal documentation can be accessed programmatically at runtime using the special attribute `__doc__`.

which in this case returns the descriptive phrase 'Python program to add two numbers,' assisting both the original programmer and future developers.

Simple Type Annotations: Python allows programmers to specify the expected types of parameters and return values using *type annotations*. These annotations do not affect execution but serve as hints for readability and external tools such as linters or IDEs.

Syntax:

```
def function_name(parameter: type) -> return_type:
```

Example:

```
def add_numbers(a: int, b: int) -> int:
    return a + b
```

Here, *a* and *b* are expected to be integers, and the function returns an integer. Python itself does not enforce these types, but static type checkers such as *mypy* can verify type correctness before execution.

More complex examples:

```
def greet_user(name: str) -> None:
    print("Hello,", name)
```

Type annotations improve clarity, particularly in larger projects, where function definitions serve as documentation of intended usage.

3.4 The Return Statement and Returning Multiple Values

The return statement plays a vital role in Python functions. It ends the function's execution and sends a value back to the point where the function was called. If no return statement is provided, the function returns *None* by default.

Basic Usage:

```
def multiply(a, b):
    return a * b
```

When called as `multiply(3, 4)`, it returns 12.

Once a return statement is executed, the function terminates immediately. Any code written after return within the same block is ignored.

Example:

```
def test_return():
    print("Before return")
    return "Done"
```

```
print("After return") # This line will never execute
```

Returning Expressions and Variables: The return statement can return computed expressions or variables, depending on the requirement.

```
def square(num):  
    result = num * num  
    return result
```

Returning Multiple Values: Python allows a function to return more than one value at a time by separating them with commas. Internally, Python groups them into a tuple.

Example:

```
def get_details():  
    name = "Alice"  
    age = 25  
    city = "London"  
    return name, age, city
```

```
info = get_details()  
print(info)
```

Output:

```
('Alice', 25, 'London')
```

The returned tuple can also be unpacked into separate variables:

```
name, age, city = get_details()  
print(name)  
print(age)  
print(city)
```

This feature makes Python flexible and expressive, allowing complex results to be returned cleanly from a single function.

Returning Lists or Dictionaries: Sometimes, structured data is better returned as a list or dictionary rather than multiple values.

Example (returning list):

```
def squares(numbers):  
    result = []  
    for n in numbers:  
        result.append(n * n)  
    return result
```

Example (returning dictionary):

```
def student_info():  
    return {"name": "John", "age": 20, "grade": "A"}
```

3.5 Naming Conventions and Readability Tips

The clarity of a programme depends heavily on how functions are named and written. Well-chosen names and clear formatting make code self-explanatory, while poor naming leads to confusion even if the logic is correct.

General Naming Rules:

- Function names should be descriptive and indicate the purpose clearly. Example: `calculate_total()` is better than `ct()`.
- Use lowercase letters with underscores to separate words (following *snake_case* convention).
- Avoid using names that conflict with Python's built-in functions (e.g., `list`, `max`, `sum`).
- Function names must start with a letter or underscore, never a digit.
- Keep names concise but meaningful.

Examples of Good Function Names:

- `get_average_score()`
- `convert_temperature()`
- `display_menu()`

Examples of Poor Function Names:

- `func1()`, `doit()`, `process()`, or `x_y_z()`

Readability Tips:

- **Keep Functions Short:** A function should ideally perform one logical task. If a function grows too large, it should be divided into smaller helper functions.
- **Use Docstrings for Documentation:** Every function should include a short explanation of what it does and how to use it.
- **Maintain Consistent Indentation:** The standard in Python is four spaces per indentation level. Mixing tabs and spaces should be avoided.
- **Avoid Side Effects:** A function should not change global variables or external states unless necessary. It should ideally depend only on its inputs and return results.

- **Include Type Hints Where Possible:** Type hints improve readability and help others understand what type of arguments are expected.
- **Keep Return Behaviour Clear:** Ensure every function either always returns a value or always performs an action; mixing the two may confuse users.
- **Follow PEP 8 Style Guidelines:** PEP 8 is the official Python style guide. It encourages using meaningful names, consistent spacing, and clear documentation.

Defining functions correctly is one of the most critical skills in Python programming. The `def` statement provides a simple yet powerful mechanism to structure code into logical units that are easy to test, maintain, and reuse. A well-defined function includes a meaningful name, appropriate parameters, proper indentation, and a clear return statement.

Documentation strings and type annotations make functions more readable and self-explanatory, while consistent naming conventions and formatting ensure that code remains professional and understandable across teams. Returning multiple values or structured data further enhances Python's flexibility in handling complex results.

Ultimately, the art of defining functions lies not just in writing correct syntax but in expressing logic clearly, following conventions, and maintaining clarity. These practices form the basis for writing clean, reliable, and scalable Python programmes, and they prepare the foundation for exploring more advanced concepts such as recursive functions, higher-order functions, and parameter handling in future sections.

SELF-ASSESSMENT QUESTIONS – 2

Multiple Choose Questions	
6	If a function call appears before the function is defined in the same module, what is the usual outcome? a) The function executes using a default definition. b) Python raises a <code>NameError</code>. c) The interpreter converts the call into a lambda. d) The call is silently ignored and returns <code>None</code>.
7	Which of the following is the correct way to add simple type annotations to a function that accepts a string and returns nothing? a) <code>def greet(name) -> None:</code>

	b) <code>def greet(name: string) -> void:</code> c) <code>def greet(name: str) -> None:</code> d) <code>def greet(str name) -> None:</code>
8	Which statement accurately describes the role of indentation within a Python function definition? a) Indentation groups the indented lines into the function body and is required for correct parsing. b) Indentation is optional and used only for readability. c) Indentation is replaced by braces <code>{}</code> in modern Python. d) Indentation must always use tabs and never spaces.
9	Which phrase best characterises a lambda function in Python? a) A named function with multiple statements. b) An anonymous, single-expression function typically used for short operations. c) A special function used only for recursion. d) A decorator applied to classes.
10	How does a Python function return multiple values to its caller? a) By using multiple return statements executed in sequence. b) By returning a generator object that yields values automatically. c) By returning values separated with semicolons. d) By returning comma-separated values which are packaged as a tuple.

4. CALLING FUNCTIONS

Once functions are defined, the next essential step is understanding how to call them correctly. Function calls form the execution point where defined logic is triggered. While definitions describe what a function does, calls determine when and how it is executed within a programme. Every call transfers control to the function, executes its body, and returns a result to the point from where it was called.

Calling functions properly involves more than just writing the function name. It requires understanding of argument types, their order, and how values are passed during the call. Python provides flexibility in function invocation, allowing both positional and keyword-based arguments, default parameter values, and even variable-length arguments. This flexibility, while powerful, can also introduce subtle errors if not used carefully.

This section explains how functions are called, the distinction between positional and keyword arguments, how default arguments behave, how to handle variable numbers of parameters using `*args` and `**kwargs`, and how Python evaluates arguments during a call. It also discusses potential side effects that may occur when functions modify mutable objects or depend on global variables.

4.1 Simple Calls and Expression Contexts

A function call activates the function's body using its name followed by parentheses. Inside the parentheses, values called arguments are supplied, corresponding to the parameters defined in the function header. Once called, Python temporarily transfers control from the main programme to the function. After execution, control returns to the point of the call, usually carrying a return value.

Basic Function Call Syntax:

```
def greet():  
    print("Hello, welcome to Python!")
```

```
# Calling the function  
greet()
```

When Python encounters `greet()`, it jumps to the function definition, executes its statements, and returns to continue with the rest of the code.

Function Calls with Arguments:


```
def add(a, b):  
    return a + b  
result = add(10, 5)  
print(result)
```

Here, the call `add(10, 5)` sends the values 10 and 5 to the parameters `a` and `b`, respectively. The function computes their sum and returns the value 15.

Function Calls as Expressions:

Function calls can appear in expressions, not just as standalone statements. This allows their return values to be used directly.

Example:

```
def square(x):  
    return x * x  
  
value = square(4) + square(2)  
print(value)
```

Here, `square(4)` and `square(2)` are evaluated as expressions, and their results are used in further computation.

Nested Function Calls:

Functions can also be called inside other functions or even within their own expressions.

```
def double(x):  
    return x * 2  
  
def add_and_double(a, b):  
    return double(a + b)  
  
result = add_and_double(3, 4)  
print(result)
```

The inner call `a + b` computes 7, which is then passed to `double()`, returning 14.

Flow of Execution during a Function Call:

1. Python evaluates the arguments in the call.

2. It creates a new local namespace for the function.
3. Arguments are assigned to the corresponding parameters.
4. The function body executes.
5. Control returns to the caller with the result (if any).

4.2 Positional and Keyword Arguments

In Python, arguments can be passed in two main ways—positional and keyword. Understanding the distinction between them is crucial for writing clear and predictable code.

- **Positional Arguments:** These are the most common type of arguments. The order of values in the function call must match the order of parameters in the definition.

Example:

```
def describe_person(name, age):  
    print(f"{name} is {age} years old.")
```

```
describe_person("Alice", 25)
```

Here, "Alice" corresponds to name, and 25 corresponds to age. The position of each argument determines which parameter it fills.

If the number or order of arguments does not match, Python raises a `TypeError`.

Example of Error:

```
describe_person(25, "Alice") # Incorrect order
```

Output:

```
TypeError: describe_person() takes 2 positional arguments
```

- **Keyword Arguments:** Keyword arguments are passed by explicitly naming the parameter along with its value. This method enhances readability and allows arguments to be specified in any order.

Example:

```
describe_person(age=30, name="John")
```

This call produces the same output as the earlier example, even though the order is reversed, because each argument is associated by name rather than position.

Mixing Positional and Keyword Arguments: Python allows a mix of positional and keyword arguments, but all positional arguments must come before any keyword arguments.

Example:

```
def display_info(name, age, city):
    print(f"Name: {name}, Age: {age}, City: {city}")
```

```
display_info("Sam", age=22, city="London")
```

However, the following is invalid:

```
display_info(name="Sam", "London", 22) # Causes SyntaxError
```

Advantages of Keyword Arguments:

- They make the function call self-explanatory.
- The order of arguments becomes flexible.
- Default values (discussed in the next subsection) are easier to manage.

	Positional Arguments	Keyword Arguments
Definition	Arguments passed based on their position in the function call.	Arguments passed by explicitly specifying the parameter name.
Order	Must be in the correct order to match the parameters in the function definition.	Can be in any order, as each value is assigned by name.
Syntax	Passed without specifying parameter names. e.g., <code>func(1, 2)</code> .	Passed with parameter names. e.g., <code>func(x=1, y=2)</code> .
Optional/Default Values	Typically used for required parameters without default values.	Commonly used for parameters with default values; overrides the default if provided.
Readability	May be less readable if there are many arguments, as positions matter.	Often clearer and self-documenting, as each argument is labeled.
Flexibility	Limited flexibility; requires the exact order.	Offers more flexibility; order does not matter, and defaults can be easily overridden.
Examples	<pre>def person(name, age): person("PJC", 34)</pre>	<pre>def person(name="User", age=25): person(age=30, name="Patrick")</pre>

Figure 5: Positional and Keyword Arguments

Source: https://miro.medium.com/v2/resize:fit:1100/format:webp/1*T1A-A3CxuCFiykb0EEqJwg.png

The figure provides a clear comparative analysis of two methods for passing data to functions: positional arguments and keyword arguments. Positional arguments rely entirely on the order of values in the function call to match the parameter sequence in the function definition, mandating a strict order but using a simple syntax. In contrast, keyword arguments offer greater flexibility and readability because values are explicitly assigned to parameter names, meaning their order in the call is irrelevant, and they are commonly used to override default parameter values. The table highlights that while positional arguments are straightforward for required parameters, keyword arguments significantly enhance code clarity and modularity, particularly when dealing with functions that have numerous optional parameters.

4.3 Default Arguments and Common Pitfalls

Sometimes, functions are designed to accept optional parameters. Python supports *default arguments*, which assign predefined values to parameters when the caller does not supply them.

Syntax:

```
def greet(name, message="Hello"):  
    print(message, name)
```

```
greet("Alice")  
greet("Bob", "Good morning")
```

Output:

```
Hello Alice  
Good morning Bob
```

Here, message has a default value of "Hello". If no value is supplied during the call, Python uses this default.

Rules for Default Arguments:

1. Default parameters must come **after** non-default ones in the definition.

Example (incorrect):

2. `def func(a=1, b):` # Invalid
3. `pass`

This raises a `SyntaxError`.

4. Default values are evaluated **only once** when the function is defined, not every time it is called.

Common Pitfall: Mutable Default Arguments

One of the most common mistakes occurs when mutable objects such as lists or dictionaries are used as default values.

Example of Incorrect Behaviour:

```
def add_item(item, item_list=[]):  
    item_list.append(item)  
    return item_list
```

```
print(add_item("apple"))  
print(add_item("banana"))
```

Output:

```
['apple']  
['apple', 'banana']
```

This happens because the same list object is reused every time the function is called, as default values are created only once during definition.

Correct Approach:

```
def add_item(item, item_list=None):  
    if item_list is None:  
        item_list = []  
    item_list.append(item)  
    return item_list
```

Now each call creates a new list when no argument is provided.

Advantages of Default Arguments:

- Simplifies function calls by reducing repetitive parameters.
- Useful when parameters have common default behaviours.
- Enhances readability by clarifying which arguments are optional.

4.4 Variable-length Argument Lists: *args and **kwargs

In real-world applications, it is often impossible to predict how many arguments will be passed to a function. Python addresses this with *variable-length arguments*, allowing flexibility when defining functions that can accept an arbitrary number of inputs.

Using *args

The asterisk * before a parameter name collects extra positional arguments into a tuple.

Example:

```
def sum_all(*numbers):  
    total = 0  
    for num in numbers:  
        total += num  
    return total  
  
print(sum_all(1, 2, 3))  
print(sum_all(5, 10, 15, 20))
```

Output:

```
6  
50
```

Here, *numbers gathers all extra arguments into a tuple, enabling the function to handle any number of inputs.

*Rules for args:

- The *args parameter must appear after regular positional parameters.
- Only one *args is allowed per function definition.

Example:

```
def display(first, *others):  
    print("First:", first)  
    print("Others:", others)  
  
display(1, 2, 3, 4)
```

Output:

First: 1

Others: (2, 3, 4)

Using **kwargs

Similarly, double asterisks ** allow capturing variable numbers of keyword arguments. These arguments are stored in a dictionary, where keys represent parameter names and values represent arguments passed.

Example:

```
def print_details(**info):  
    for key, value in info.items():  
        print(f"{key}: {value}")  
  
print_details(name="Alice", age=25, city="London")
```

Output:

```
name: Alice  
age: 25  
city: London
```

****Rules for kwargs:**

- **kwargs must appear after *args in the parameter list.
- It can accept zero or more keyword arguments.
- The resulting dictionary is mutable and can be processed like any other dictionary.

Combining All Types:

```
def example(a, b=2, *args, **kwargs):  
    print(a, b, args, kwargs)  
example(1, 3, 5, 7, x=9, y=10)
```

Output:

```
1 3 (5, 7) {'x': 9, 'y': 10}
```

4.5 Argument Evaluation Order and Side Effects

When a function is called, Python follows a specific order for evaluating arguments. Understanding this order is crucial to prevent unexpected results or side effects.

Order of Evaluation:

1. Evaluate all argument expressions from left to right.
2. Assign evaluated values to parameters.
3. Execute the function body.

Example:

```
def show(a, b):  
    print(a, b)  
def f1():  
    print("f1 called")  
    return 10  
def f2():  
    print("f2 called")  
    return 20  
show(f1(), f2())
```

Output:

```
f1 called  
f2 called  
10 20
```

This confirms that both argument expressions are evaluated before the function call executes.

Side Effects in Function Calls: A side effect occurs when a function modifies an external state or mutable object passed as an argument. This can lead to unintended consequences if not handled carefully.

Example:

```
def append_item(lst, item):  
    lst.append(item)  
    return lst  
my_list = [1, 2, 3]  
append_item(my_list, 4)  
print(my_list)
```

Output:

```
[1, 2, 3, 4]
```

The original list is modified because lists are mutable objects. To avoid this, functions should work with copies unless modification is intended.

Avoiding Side Effects:

- Use immutable data types (tuples, strings) when passing arguments that should remain unchanged.
- Document clearly when a function modifies input data.
- Avoid reliance on global variables inside functions.

Example of Global Variable Side Effect:

```
count = 0

def increment():
    global count
    count += 1
```

```
increment()
print(count)
```

Here, the function modifies a global variable directly, which can make debugging difficult in complex programmes. Prefer returning values instead:

```
def increment(value):
    return value + 1
```

Calling functions correctly is as important as defining them. The call mechanism in Python follows a structured pattern that determines how values are passed and results are returned. Simple function calls form the basis, while positional and keyword arguments offer flexibility and clarity. Default arguments simplify calls by allowing optional parameters, though care must be taken with mutable defaults.

Variable-length arguments using `*args` and `**kwargs` enable dynamic handling of inputs, making functions more adaptable. Understanding the evaluation order of arguments ensures predictable outcomes, and awareness of side effects promotes safe programming practices.

Together, these concepts form the foundation of function execution in Python. Mastery of calling conventions, argument handling, and evaluation behaviour allows programmers to write robust,

maintainable, and modular code, setting the stage for advanced topics such as recursion, variable scopes, and parameter passing mechanisms discussed in later sections.

SELF-ASSESSMENT QUESTIONS – 3

Multiple Choose Questions	
11	What happens when a function is called in Python? a) The function is copied into memory but not executed. b) Control is temporarily transferred to the function, which executes and may return a result. c) The function definition is deleted after execution. d) The function executes only if no arguments are provided.
12	In Python, what determines which parameter receives which value in a positional argument call? a) The alphabetical order of parameter names. b) The order of parameters in the function definition. c) The data types of the arguments. d) The values assigned in the global scope.
13	Which of the following function definitions would cause a SyntaxError? a) <code>def greet(name, message="Hi"): pass</code> b) <code>def func(a=1, b): pass</code> c) <code>def sum_all(*numbers): pass</code> d) <code>def show(a, *args, **kwargs): pass</code>
14	What is the key difference between <code>*args</code> and <code>**kwargs</code> in Python function definitions? a) <code>*args</code> collects variable-length keyword arguments; <code>**kwargs</code> collects positional ones. b) <code>*args</code> collects extra positional arguments as a tuple; <code>**kwargs</code> collects extra keyword arguments as a dictionary. c) <code>*args</code> is used for default parameters; <code>**kwargs</code> for required ones. d) <code>*args</code> and <code>**kwargs</code> are identical but differ in syntax.
15	Why might a mutable default argument cause unexpected behaviour in Python? a) Because mutable defaults are re-created for every function call. b) Because mutable defaults are shared across all calls, persisting changes between executions. c) Because Python does not support mutable objects as function parameters. d) Because mutable defaults automatically convert into immutable types during execution.

5. SUMMARY

- Functions in Python are reusable blocks of code designed to perform specific tasks, promoting modularity and reducing redundancy in programmes.
- Built-in functions such as `print()`, `len()`, and `range()` are pre-defined by Python, while user-defined functions are created by programmers using the `def` keyword.
- A function's anatomy consists of its name, parameters, body, and an optional return statement, forming a structured way to execute code segments.
- Organising code with functions improves readability, debugging, and maintainability by breaking large programmes into manageable parts.
- The `def` statement defines a function's structure, requiring correct syntax and indentation to form a valid block of code.
- The return statement ends a function's execution and optionally passes a value back to the point of the call, while multiple values can be returned as tuples.
- Functions are called by writing their name followed by parentheses, optionally containing arguments that match the parameters defined.
- Python supports various types of arguments: positional, keyword, default, and variable-length (`*args` and `**kwargs`), offering flexibility in how functions are invoked.
- Default arguments simplify function calls but must be handled carefully, especially when using mutable data types such as lists or dictionaries.
- Understanding argument evaluation order and avoiding side effects ensures predictable programme behaviour and prevents unintended data modification.

6. GLOSSARY

Function	-	A reusable block of organised code designed to perform a particular operation or computation.
Parameter	-	A variable listed inside a function's parentheses during definition, acting as a placeholder for input values.
Argument	-	The actual value supplied to a function when it is called, corresponding to its parameters.
Return Statement	-	A command that ends a function and optionally sends back a result to the caller.
Positional Argument	-	An argument passed to a function based on its order relative to the parameter list.
Keyword Argument	-	An argument passed by explicitly naming the parameter and assigning its value in the call.
Default Argument	-	A parameter with a pre-assigned value used when no argument is provided during the function call.
Variable-length Arguments	-	Special parameters (*args and **kwargs) that allow a function to accept an arbitrary number of arguments.
Docstring	-	A string written at the start of a function that documents its purpose and usage, enclosed in triple quotes.
Side Effect	-	An unintended change in a programme's external state caused by a function, such as modifying a global variable or mutable object.
Function	-	A reusable block of organised code designed to perform a particular operation or computation.
Parameter	-	A variable listed inside a function's parentheses during definition, acting as a placeholder for input values.

7. TERMINAL QUESTIONS

1. Explain, in your own words, why functions improve the structure and maintainability of a Python programme.
2. Distinguish between a parameter and an argument, and give an example showing both.
3. Describe the role of the def statement and list the parts that must appear in a correct function header.
4. Explain how and why you would use a docstring in a function; show a short example.
5. Give three examples of built-in functions and explain a practical situation where each would be useful.
6. Describe the problem that mutable default arguments can cause and show the recommended pattern to avoid it.
7. Compare positional and keyword arguments: when would you prefer one over the other and why?
8. Explain how *args and **kwargs work, and write a small function that demonstrates both.
9. Describe what happens to program flow and local state when a function is called and then returns a value.
10. Discuss two naming and readability practices you should follow when writing functions and explain how they help team development.

8. ANSWERS

8.1. Self-Assessment Answers:

1. c) A reusable block of organised and related statements designed to perform a specific task.
2. b) `def`
3. a) Parameters are in the definition and arguments are the actual values passed.
4. d) To end function execution and send a value back to the caller.
5. b) `area_of_circle(radius)`
6. b) Python raises a `NameError`.
7. c) `def greet(name: str) -> None:`
8. a) Indentation groups the indented lines into the function body and is required for correct parsing.
9. b) An anonymous, single-expression function typically used for short operations.
10. d) By returning comma-separated values which are packaged as a tuple.
11. b) Control is temporarily transferred to the function, which executes and may return a result.
12. b) The order of parameters in the function definition.
13. b) `def func(a=1, b): pass`
14. b) `*args` collects extra positional arguments as a tuple; `**kwargs` collects extra keyword arguments as a dictionary.
15. b) Because mutable defaults are shared across all calls, persisting changes between executions.

8.2. Terminal Question Answers

Answer 1: Functions improve structure and maintainability by breaking a programme into small, named units that each do one task; this reduces duplication and makes it easier to update or test parts without changing the whole programme. Using functions also clarifies programme flow and lets teams work on separate components independently. Reference: Section 2.4.

Answer 2: A parameter is the named variable in a function definition, while an argument is the actual value supplied when the function is called; for example, in `def greet(name):` `name` is a parameter and in `greet("Alice")` the string `"Alice"` is the argument. Parameters act as placeholders inside the function; arguments fill those placeholders at call time. Reference: Section 2.3.

Answer 3: The `def` statement starts a function definition and must include the keyword `def`, the function name, parentheses (which may list parameters), a colon, and an indented body; the header

therefore contains the function name and its signature. Correct syntax and indentation are required so the interpreter recognises the function block and its scope. Reference: Section 3.1.

Answer 4: A docstring is a triple-quoted string placed immediately after the function header that documents the function's purpose, parameters and return value; it is accessible at runtime via `help()` or the `__doc__` attribute and helps other developers understand how to use the function. Example:

```
def calculate_area(radius):  
    """Return the area of a circle given radius (float)."""  
    return 3.1416 * radius * radius
```

Reference: Section 3.3.

Answer 5: Examples of built-in functions are `print()` for displaying output (useful for simple logging or user prompts), `len()` for obtaining the size of a sequence (useful when iterating or validating input), and `max()` for finding the largest item in a collection (useful in summaries or selection tasks). These save effort by providing common operations without extra code. Reference: Section 2.2.

Answer 6: Mutable default arguments (for example a list used as a default) are created once at definition time and then shared across calls, which can cause accumulated state between invocations; the usual remedy is to use `None` as the default and create a fresh object inside the function, for example:

```
def add_item(item, item_list=None):  
    if item_list is None:  
        item_list = []  
    item_list.append(item)  
    return item_list
```

Reference: Section 4.3.

Answer 7: Positional arguments are assigned by order and are concise for required inputs, while keyword arguments are passed by name, improving readability and allowing any order; prefer positional for short, fixed-parameter calls and choose keyword when clarity is important or when many optional parameters exist. Mixing both is allowed provided positional arguments come first. Reference: Section 4.2.

Answer 8: `*args` collects extra positional arguments into a tuple and `**kwargs` collects extra keyword arguments into a dictionary, allowing a function to accept a variable number of inputs; for example:

```
def demo(a, *args, **kwargs):  
    print(a)           # required positional  
    print(args)        # tuple of extra positional values
```

```
print(kwargs)      # dict of extra keyword values
```

```
demo(1, 2, 3, x=4, y=5)
```

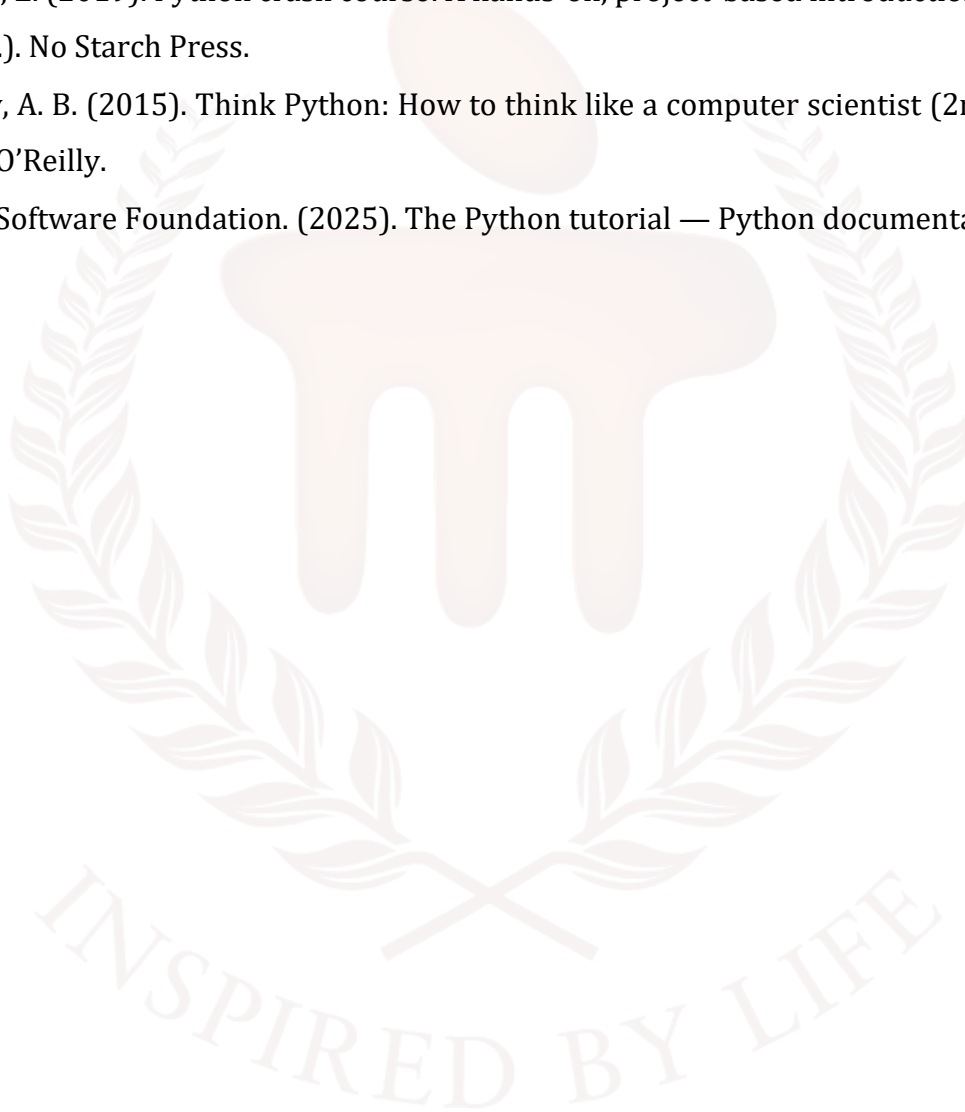
Reference: Section 4.4.

Answer 9: When a function is called, Python evaluates the argument expressions, creates a new local namespace, binds arguments to parameters within that namespace, executes the function body, and then returns control (and any return value) to the caller; local variables are discarded after return unless returned or referenced externally. This creates isolation between caller and callee and keeps state local to the function. Reference: Section 4.1.

Answer 10: Two naming and readability practices are: (a) use descriptive names in snake_case (for example calculate_total) so the function's purpose is clear at a glance; and (b) keep functions short and single-purpose so each unit is easy to read, test and reuse. Both practices make code easier for team members to understand and reduce risks when integrating changes. Reference: Section 3.5.

9. REFERENCES

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- Ramalho, L. (2022). Fluent Python (2nd ed.). O'Reilly Media.
- Matthes, E. (2019). Python crash course: A hands-on, project-based introduction to programming (2nd ed.). No Starch Press.
- Downey, A. B. (2015). Think Python: How to think like a computer scientist (2nd ed.). Green Tea Press / O'Reilly.
- Python Software Foundation. (2025). The Python tutorial — Python documentation.



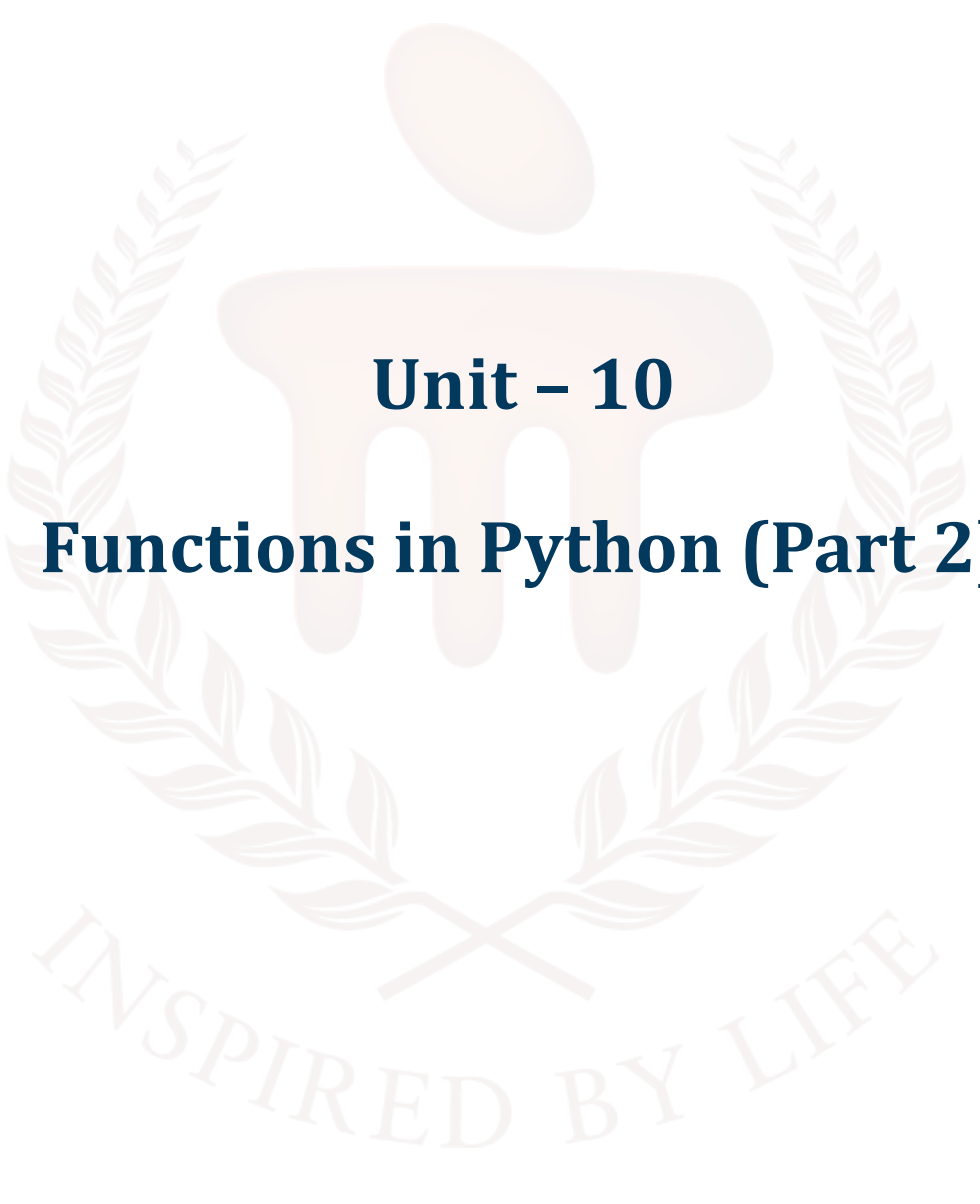
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 10

Functions in Python (Part 2)

TABLE OF CONTENTS

Sl No	Topic	Fig No / Table / Graph	Saq / Activity	Page No
1	Introduction	-	-	4
1.1	Learning Objectives	-	-	
2	Concept of Recursion	-	1	5-14
2.1	Definition and Basic Idea of Recursion	-	-	
2.2	Need for Recursion in Programming	1	-	
2.3	Structure and Working of Recursive Functions	2	-	
2.4	Base Case, Recursive Case and Flow of Control	3	-	
2.5	Stack Frames and Function Call Stack	-	-	
3	Types of Recursion	-	2	15-24
3.1	Direct and Indirect Recursion	4	-	
3.2	Tail and Non-tail Recursion	5	-	
3.3	Linear and Tree Recursion	6	-	
3.4	Common Errors and Infinite Recursion	-	-	
4	Applications of Recursion	-	3	25-34
4.1	Mathematical Problems (Factorial, Fibonacci Series)	-	-	
4.2	Divide and Conquer Algorithms (Binary Search, Merge Sort)	7	-	
4.3	Recursive Operations on Data Structures (Linked Lists, Trees)	-	-	
4.4	Practical Problem-Solving Using Recursion	-	-	
5	Summary	-	-	35
6	Glossary	-	-	36
7	Terminal Questions	-	-	37
8	Answers	-	-	38-40
8.1	Self-Assessment Answers	-	-	
8.2	Terminal Questions Answers	-	-	
9	References	-	-	41

1. INTRODUCTION

In the previous chapter, we studied the fundamental aspects of functions in Python. It covered the basics of defining and calling functions, their structure, the purpose of using functions for code organisation, and the use of arguments, return statements, and scope. The chapter also explained the difference between built-in and user-defined functions and demonstrated how functions make a programme modular, efficient, and easier to maintain.

This chapter continues the exploration of Python functions by introducing the concept of recursion. Recursion is a powerful programming technique where a function calls itself to solve smaller instances of a problem. It is particularly useful for tasks that can be broken down into repetitive sub-problems, such as mathematical computations, searching, sorting, and traversing hierarchical data structures.

The chapter explains how recursion works in Python by detailing the base case and recursive case, and by showing how the function call stack operates during recursive calls. It also introduces different types of recursion such as direct, indirect, tail, and tree recursion, supported by practical examples. The goal is to understand both the strengths and limitations of recursion as a problem-solving method. To study this chapter effectively, begin by reviewing the structure of a function from the previous unit, then move on to understand the logic behind recursive calls. Practise tracing the flow of recursive functions step-by-step and try writing small examples like factorial and Fibonacci series. Visualising the call stack and identifying the base case in every recursive function will help in mastering the concept.

1.1. Learning Objectives

By the end of this Unit, you will be able to:

- Explain the concept and working of recursion in Python.
- Identify and implement base and recursive cases in a function.
- Distinguish between different types of recursion such as direct, indirect, and tail recursion.
- Apply recursion to solve mathematical and algorithmic problems.
- Evaluate when recursion is appropriate compared to iterative solutions.



2. CONCEPT OF RECURSION

In programming, problem-solving often involves breaking down complex tasks into smaller, simpler subproblems. One of the most elegant techniques that captures this idea is recursion. Recursion allows a function to call itself, either directly or indirectly, to solve a problem by reducing it into smaller instances of the same problem.

Python supports recursion naturally, and it is widely used in tasks involving repetitive decomposition such as factorial computation, tree traversal, file system scanning, and mathematical series evaluation. Recursion provides an alternative to loops, especially when a task can be defined in terms of itself.

2.1 Definition and Basic Idea of Recursion

Recursion is a programming technique in which a function calls itself, either directly or through another function, to solve a problem by breaking it down into smaller subproblems of the same type. The recursive approach continues until a simple, easily solvable condition—known as the base case—is reached.

In Python, recursion follows the same structure as normal function calls, except that the function's name appears within its own body. When invoked, each recursive call creates a new instance of the function with its own set of parameters and local variables.

Simple Example:

```
def countdown(n):  
    if n == 0:  
        print("Time's up!")  
    else:  
        print(n)  
        countdown(n - 1)
```

When `countdown(5)` is called, the function prints numbers from 5 to 1 and finally prints “Time’s up!”. Each call to `countdown()` decreases the value of `n`, moving closer to the base case `n == 0`.

A recursive function approaches a problem by calling itself with a smaller or simpler version of the original input, breaking the task into manageable steps. Each call pauses until the subsequent call finishes its work, which creates a chain of suspended executions. The process continues until a base case is reached, which provides a direct answer without the need for further calls. This method reflects the way people often reason through problems. For instance, the value of 5 factorial can be

thought of as 5 multiplied by 4 factorial, and 4 factorial can be reduced in the same manner until reaching 1 factorial, which completes the chain.

Mathematical Representation:

If a function $f(n)$ is defined recursively, it can take the form:

$$f(n) = f(n - 1) + \text{some operation}$$

with a condition such as $f(0)$ or $f(1)$ providing the base case.

2.2 Need for Recursion in Programming

Recursion plays an important role in programming because it allows certain problems to be expressed in a clear and elegant manner that would otherwise require complex looping structures. Some problems are naturally recursive in nature, meaning they are defined in terms of smaller instances of themselves.

Reasons for Using Recursion:

- **Natural Fit for Divide-and-Conquer Problems:** Problems such as sorting (QuickSort, MergeSort) or searching (Binary Search) can be divided into smaller subproblems that follow the same pattern as the original one. Recursion fits perfectly in these cases.
- **Simplified Code:** Recursive solutions are often shorter and easier to read compared to iterative ones. For example, traversing nested structures like directories or trees can be implemented neatly with recursion rather than multiple nested loops.
- **Mathematical and Logical Clarity:** Many mathematical definitions such as factorial, Fibonacci series, and power functions are inherently recursive. Writing them recursively mirrors their mathematical form, improving conceptual clarity.
- **Reduced Use of Loops:** Recursion can replace repetitive loops, particularly when the number of repetitions depends on changing conditions or variable input depth (for instance, processing nested JSON objects).
- **Problem Decomposition:** Recursion encourages breaking a complex task into smaller sub-tasks that are easier to understand, implement, and maintain.
- **Ease in Handling Dynamic Data Structures:** Structures like trees, graphs, and linked lists are often best handled using recursion since they can contain self-referential links.

Recursion is particularly suitable for problems that involve naturally branching or self-similar structures. Tree traversal methods such as preorder, inorder and postorder fit well with this approach because each node leads to smaller subtrees that can be processed in the same manner. Directory traversal in file systems also mirrors this pattern as each folder may contain subfolders that require repeated exploration. Many mathematical problems, including factorials and the Fibonacci sequence, are often more intuitive to express recursively. Well-known algorithms such as QuickSort, MergeSort and Binary Search are built around recursive decomposition of data. Backtracking tasks, such as navigating a maze or solving the N-Queens problem, also rely on recursion to explore possible paths and return when a route fails.

Table 1: Iterative vs Recursive Approach

Aspect	Iterative	Recursive
Method	Uses loops	Function calls itself
Memory Use	Lower	Higher (due to call stack)
Readability	Sometimes complex	Often clearer for hierarchical problems
Termination	Based on loop condition	Based on base case
Overhead	Minimal	Higher (function call overhead)

2.3 Structure and Working of Recursive Functions

A recursive function in Python generally follows a consistent pattern comprising three essential parts:

1. **Base Case** – The stopping condition that ends the recursion.
2. **Recursive Case** – The part where the function calls itself to reduce the problem.
3. **Return Statement** – The mechanism to return the computed value back through each recursive level.

General Form of a Recursive Function:

```
def recursive_function(parameters):
    if base_condition:
        return result
    else:
        return recursive_function(modified_parameters)
```

Step-by-Step Working:

```
def factorial(n):  
    if n == 1:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

When `factorial(5)` is invoked, the function does not return straight away; instead it calls `factorial(4)`, which calls `factorial(3)`, and this chain continues until the base case `factorial(1)` is reached. At that point `factorial(1)` returns 1, and the pending calls resume in reverse order, each multiplying its own `n` by the value returned from the next level down. The final result is produced as these multiplications are carried back up the call chain.

The sequence of calls can be represented as:

```
factorial(5)  
= 5 * factorial(4)  
= 5 * 4 * factorial(3)  
= 5 * 4 * 3 * factorial(2)  
= 5 * 4 * 3 * 2 * factorial(1)  
= 5 * 4 * 3 * 2 * 1  
= 120
```

Each call creates a separate stack frame containing the current value of `n` and the return address.

A recursive function creates a fresh instance of itself every time it is called, and each instance maintains its own local variables and state. Control keeps moving from one call to the next until the base condition is satisfied, at which point the chain of calls begins to unwind. As the unwinding progresses, the individual results produced by each call are combined or passed upwards, eventually forming the final outcome.

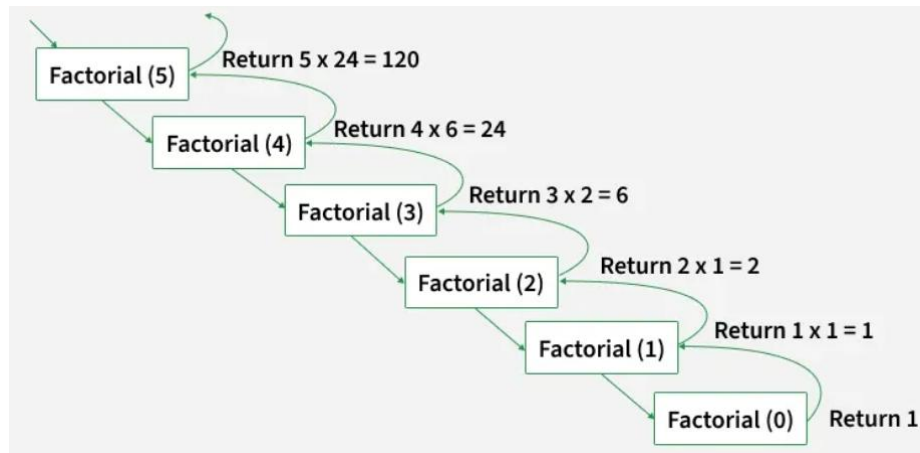


Figure 1: Function Call for Factorial of 5

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20250702172044227332/dsa.webp>

This figure illustrates how a recursive function processes the calculation of the factorial of five by breaking the problem into smaller calls until it reaches the base case. Each step shows the function calling itself with a reduced value, moving from five down to zero. Once the base case is reached, the returns begin to move back up the chain, with each level multiplying its number by the value returned from the next call. This creates a clear view of the recursive flow, showing how intermediate results build up to produce the final answer of one hundred and twenty.

2.4 Base Case, Recursive Case and Flow of Control

For recursion to work correctly and terminate safely, it must have both a base case and a recursive case. Understanding these two is crucial for avoiding infinite recursion and potential runtime errors such as `RecursionError` in Python.

Base Case - The base case is the simplest form of the problem that can be solved directly without further recursion. It serves as the stopping condition to prevent infinite loops. Without a base case, recursion would continue indefinitely, exhausting system memory.

Example:

```
def countdown(n):  
    if n == 0:  
        print("Stop")  
    else:  
        print(n)  
        countdown(n - 1)
```

Here, the base case if $n == 0$: stops the recursion when n reaches zero.

Recursive Case - The recursive case defines how the problem is reduced into smaller subproblems that move toward the base case. It involves the function calling itself with modified parameters.

Example (Fibonacci sequence):

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)
```

Each recursive case breaks the problem into two smaller Fibonacci computations, eventually reaching the base case.

Flow of Control - The control flow of a recursive function involves two key phases:

1. **Recursive Descent (Going Down):** Each recursive call progresses deeper into smaller subproblems, moving towards the base case.
2. **Recursive Ascent (Coming Up):** Once the base case is reached, the results are returned back through each previous call.

Example:

For factorial(3), the flow is:

factorial(3)

→ factorial(2)

→ factorial(1)

At factorial(1), the base case returns 1, and the recursion unwinds:

factorial(2) returns $2 * 1 = 2$

factorial(3) returns $3 * 2 = 6$

Every recursive process must move closer to its base case, otherwise the function will continue calling itself without end and cause a RecursionError. Each call operates with its own local variables, which keeps the individual steps independent from one another. Throughout this process, the call stack records the sequence of calls and keeps track of where each function should return once its work is complete.

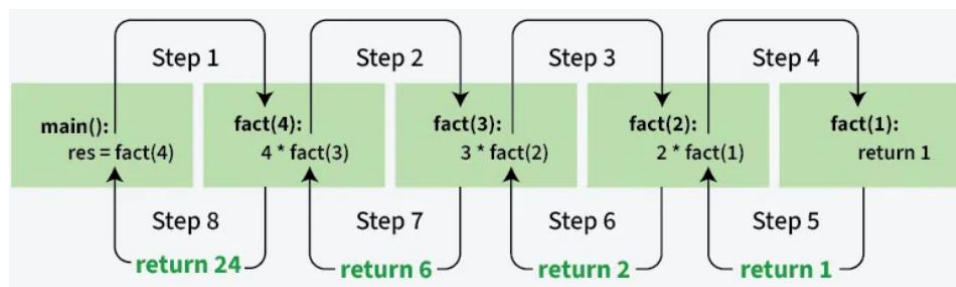


Figure 2: Illustration of Recursion

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20240924132748/factorial.webp>

The figure shows how the factorial function progresses through its recursive calls, beginning with `fact(4)` in the main program and then moving step by step to `fact(3)`, `fact(2)` and `fact(1)`. Each call is placed on the call stack during this expansion phase, where the function keeps breaking the problem into a smaller version of itself based on the rule $n! = n \times (n - 1)!$. The process reaches the base case at `fact(1)`, which returns 1 without making any further calls. Once this value is returned, the contraction phase begins. Each suspended call resumes in reverse order, multiplies its own number by the value returned from the next level and passes the result upwards. This continues until `fact(4)` returns 24 to the main program. The diagram captures these two movements clearly: the downward path towards the base case and the upward path as each result is completed and returned.

2.5 Stack Frames and Function Call Stack

When recursion occurs, Python manages function calls using an internal structure known as the call stack. Each time a function is called, a stack frame is created to store information such as parameter values, local variables, and the return address.

The Function Call Stack - The call stack operates on the Last In, First Out (LIFO) principle. This means the most recent function call must finish before the previous one can continue. This structure allows Python to remember the order of calls and return correctly after each function finishes execution.

Illustration with Factorial Example:

For the recursive call `factorial(3)`:

1. Call `factorial(3)` → stack stores this call.
2. Calls `factorial(2)` → pushed on top of the stack.
3. Calls `factorial(1)` → pushed again.

4. Base case returns → factorial(1) is popped from the stack.
5. factorial(2) resumes, computes, and is popped.
6. Finally, factorial(3) completes and is popped, leaving the stack empty.

Each level of recursion adds a new frame to the stack, which holds:

- Function name
- Parameters and local variables
- Return address (where to continue after returning)

Stack Frames in Python - Each recursive call's local environment is independent. Even if the same variable name is used, each frame maintains its own copy.

Example:

```
def recursive_count(n):  
    print("Entering level:", n)  
    if n > 0:  
        recursive_count(n - 1)  
    print("Exiting level:", n)
```

For recursive_count(3), the execution order will show how each level opens and closes in stack order.

Output:

Entering level: 3

Entering level: 2

Entering level: 1

Entering level: 0

Exiting level: 0

Exiting level: 1

Exiting level: 2

Exiting level: 3

This pattern shows how recursive calls are nested and unwound in LIFO order.

Stack Overflow and Recursion Limit - Every recursive call consumes stack memory. If the recursion is too deep or lacks a proper base case, Python will eventually raise a `RecursionError`.

Example:

```
def infinite_recursion():  
    infinite_recursion()
```

```
infinite_recursion()
```

Error message:

`RecursionError: maximum recursion depth exceeded`

By default, Python limits recursion depth to prevent infinite loops. The current recursion limit can be viewed or modified using the `sys` module:

```
import sys  
print(sys.getrecursionlimit())  
sys.setrecursionlimit(2000)
```

However, increasing this limit must be done cautiously as excessive recursion can cause the interpreter to crash due to memory exhaustion.

Managing recursive depth requires careful planning to ensure that a function does not call itself more times than the system can handle. A clear and reachable base case is essential, as it prevents the function from continuing indefinitely. When the depth of recursion is likely to grow very large, an iterative approach is often safer because it avoids the overhead of maintaining many stack frames. In some situations techniques such as tail recursion or memoisation can help reduce repeated work or simplify the structure of the function, though Python does not apply tail call optimisation automatically, so deep recursion can still lead to a stack overflow if not controlled properly.

Recursion is a powerful and conceptually elegant programming technique that enables complex problems to be expressed through self-referential definitions. By breaking a problem into smaller instances of itself, recursion aligns closely with the principle of divide and conquer. Understanding recursion requires grasping its structure—each recursive function must include a base case to stop infinite calls and a recursive case that reduces the problem. The function call stack plays a crucial role in managing these recursive layers, ensuring that each call completes before the programme resumes previous states.

Although recursion can produce concise and intuitive code, it should be used with awareness of its memory implications. Improperly designed recursion may result in stack overflow or inefficiency.

However, when applied correctly, recursion simplifies the logic for operations involving hierarchical data, repetitive decomposition, and natural mathematical formulations. By mastering the concept of recursion, programmers gain a deeper understanding of both problem-solving methodology and how Python manages execution internally through its call stack mechanism.

Self-Assessment Questions – 1

Multiple Choose Questions	
1	What is the best description of recursion in programming? a) A function that calls itself to solve smaller instances of the same problem b) A loop that repeats until a condition is false c) A variable that refers to itself in memory d) A library for handling repetitive tasks
2	Which pair are the two essential components that control a recursive function's flow? a) Parameters and return statement b) Base case and recursive case c) Global variables and local variables d) Loop condition and increment
3	Which problem is most naturally solved using recursion? a) Summing elements of a flat list with a for loop b) Sorting a small fixed array using insertion sort c) Traversing a binary tree to visit all nodes d) Formatting a single string for output
4	Why does Python raise a RecursionError during deep recursion? a) Because Python disallows more than one function call at a time b) Because recursive functions cannot use return values c) Because global variables run out of space d) Because each recursive call adds a stack frame and the call stack can exceed the recursion limit
5	Which statement about tail recursion and Python is correct? a) Python does not perform tail call optimisation automatically and so deep tail recursion can still exhaust the stack b) Python always converts tail recursion into iteration to save memory c) Tail recursion and memoisation are the same technique d) Tail recursion uses less memory than any other recursion in Python

3. TYPES OF RECURSION

Recursion, being a fundamental programming concept, takes different forms depending on the structure of function calls and the way results are accumulated or propagated. Understanding the types of recursion is essential to applying it effectively in problem-solving and ensuring that functions perform efficiently without unnecessary memory use or execution overhead. In Python and other languages, recursion can be used in multiple ways—direct, indirect, tail, non-tail, linear, or tree—each suited to different computational problems. This section explores these variations, how they differ in logic and performance, and what common mistakes programmers encounter when applying recursive methods.

3.1 Direct and Indirect Recursion

Recursion occurs whenever a function calls itself directly or indirectly through another function. The distinction between direct and indirect recursion lies in how the control flow of function calls is organised.

Direct Recursion - Direct recursion takes place when a function calls itself within its own body. This is the simplest and most common type of recursion, often used in problems such as computing factorials, Fibonacci numbers, or traversing nested data structures.

Example of direct recursion:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

In the above example, the function factorial calls itself directly to compute the factorial of a number. Each recursive call reduces the size of the problem (n decreases by 1) until the base case ($n == 0$) is reached, at which point the recursion stops.

Characteristics of Direct Recursion:

- The function contains a direct call to itself.
- The recursion depth depends on the size of the input.

- Base and recursive cases are within the same function.
- Easier to implement and understand for simple problems.

Direct recursion is often appreciated for its clarity, especially when expressing mathematical ideas or problems that naturally describe themselves in terms of smaller versions of the same task. It can make compact functions easier to read and follow, and for simple cases it is straightforward to test and reason about each step. However, direct recursion also has certain drawbacks. If the base case is missing or cannot be reached, the chain of calls will continue until the program encounters a stack overflow. It may also be less efficient for problems that generate deep or repeated calls, since each recursive step creates a new stack frame and may repeat work already done in earlier calls.

Indirect Recursion - Indirect recursion happens when two or more functions call each other in a cycle. This form of recursion is less common but useful in some logical or state-dependent problems, such as alternating operations or simulating mutual dependencies.

Example of indirect recursion:

```
def functionA(n):  
    if n > 0:  
        print(n)  
        functionB(n - 1)  
  
def functionB(n):  
    if n > 1:  
        print(n)  
        functionA(n // 2)  
  
functionA(10)
```

In this example, functionA calls functionB, and functionB calls functionA. The sequence continues until the base conditions are met.

Characteristics of Indirect Recursion:

- Involves two or more functions calling each other.
- Control alternates between multiple functions.
- Base conditions must be properly defined in each function to stop the recursion.

Indirect recursion involves two or more functions calling one another in a cycle, and it can be helpful when a task is better organised across several smaller parts. By distributing the logic between different functions, it supports a modular style of programming in which each function handles a specific role. Despite these benefits, indirect recursion can be more difficult to follow because the flow of control moves between several places, making it harder to trace and debug. It also carries a greater risk of inconsistent or missing base cases, which can cause the calls to continue in a loop and result in infinite recursion.

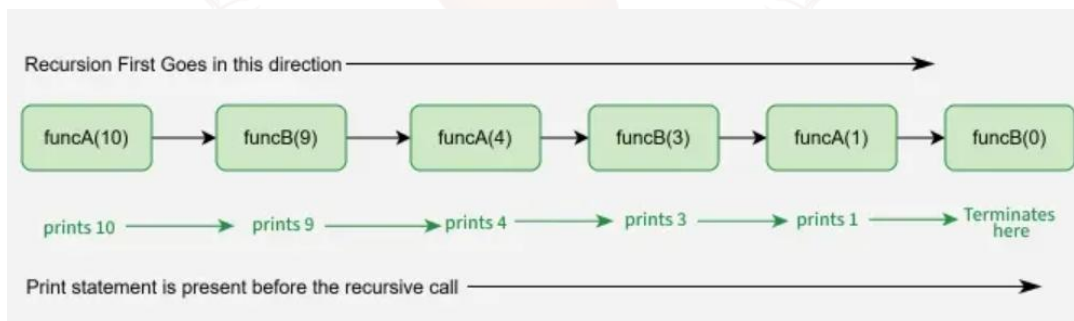


Figure 3: Indirect Recursion

Source: https://media.geeksforgeeks.org/wpcontent/uploads/20250611105645906495/indirect_recursion_in_c.webp

The figure clearly depicts the execution sequence of indirect recursion, a programming technique where two distinct functions, funcA and funcB, call one another sequentially. The recursive descent begins with funcA(10), which then calls funcB(9), initiating an alternating chain of calls (funcA(4), funcB(3), funcA(1)) that proceeds in the direction of the flow arrow. Critically, since the print statement is positioned *before* the recursive call in both functions, the argument values (10, 9, 4, 3, 1) are output immediately upon function entry, before the subsequent call is made. The entire recursive process ceases when funcB(0) is executed, meeting the base case and terminating the function chain.

3.2 Tail and Non-tail Recursion

The distinction between tail and non-tail recursion concerns where the recursive call appears within the function and how the results are processed.

Tail Recursion - Tail recursion occurs when the recursive call is the last statement executed in the function. No further computation is performed after the recursive call returns, which allows compilers (in some languages) to optimise memory usage. Python, however, does not perform tail call optimisation, but understanding this concept helps in writing efficient recursive logic.

Example of tail recursion:

```
def tail_sum(n, accumulator=0):  
    if n == 0:  
        return accumulator  
    else:  
        return tail_sum(n - 1, accumulator + n)
```

Here, the recursive call to `tail_sum` is the last operation in the function, and the intermediate result is carried in the accumulator.

Characteristics of Tail Recursion:

- The recursive call appears at the end of the function.
- The result of the recursive call is directly returned without modification.
- Can be converted to an iterative solution easily.

Tail recursion places the recursive call at the end of the function, which makes the structure resemble an iterative process and simplifies the work required after each call. This arrangement can reduce the amount of computation performed during the unwinding phase, since most of the work is handled before the final call is made. However, Python does not apply tail call optimisation, so each call still uses a separate stack frame. In some cases tail recursion also needs an extra parameter to keep track of intermediate results, which can make the function slightly less natural to write.

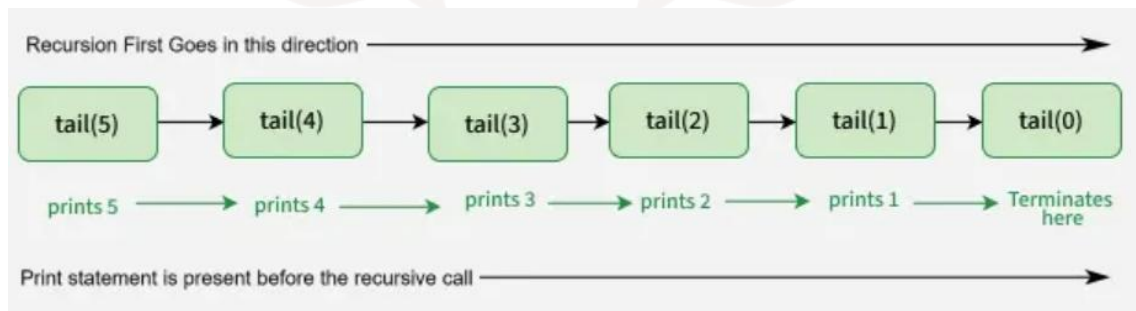


Figure 4: Tail Recursion

Source: https://media.geeksforgeeks.org/wpcontent/uploads/20250611105623429620/tail_recursion_in_c.webp

The figure clearly illustrates the execution flow of tail recursion, a specific type of linear recursion where the recursive call is the very last operation in the function. The sequence begins with `tail(5)`, which immediately calls `tail(4)`, and this process continues in a descending, linear fashion through

tail(3), tail(2), and tail(1), following the direction indicated by the arrow. Because the print statement is positioned *before* the recursive call in the function, the argument values (5, 4, 3, 2, 1) are printed immediately upon entry into each function call, before the next function in the chain is invoked. The entire recursive process ceases when tail(0) is executed, which serves as the base case and terminates the sequence.

Non-tail Recursion - In non-tail recursion, the recursive call is not the final action in the function. After returning from the recursive call, further operations are performed before producing the final result.

Example of non-tail recursion:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

In this case, after the recursive call factorial(n - 1) returns, the multiplication $n * \text{factorial}(n - 1)$ is carried out, making it non-tail recursive.

Characteristics of Non-tail Recursion:

- Recursive call is followed by another operation.
- Often represents mathematical recursion where results depend on multiple recursive outcomes.
- Typically uses more stack space as intermediate results must be retained.

Non-tail recursion is often easier to express for many mathematical problems because the structure follows the natural progression of breaking a task into smaller parts and then combining the results. It appears frequently in functional and academic examples, where clarity of expression is valued. The drawback is that each call must retain its intermediate state while waiting for the deeper calls to return, which increases memory usage. If the recursion becomes too deep, this build-up of stack frames can lead to a stack overflow and cause the program to stop unexpectedly.

3.3 Linear and Tree Recursion

Recursion can also be classified based on the number of recursive calls made during each function invocation.

Linear Recursion - Linear recursion occurs when a function makes only one recursive call at each level. Each call produces one subsequent call until the base condition is met, creating a linear chain of calls.

Example of linear recursion:

```
def countdown(n):  
    if n == 0:  
        print("Blast off!")  
    else:  
        print(n)  
        countdown(n - 1)
```

In the example, each call reduces the problem by one and makes exactly one further call, forming a straight line of recursive calls.

Characteristics of Linear Recursion:

- One recursive call per function invocation.
- Forms a single chain of calls and returns.
- Often used in simple sequences or accumulative operations.

Linear recursion is straightforward to understand because each call leads to only one further call, which keeps the flow of control simple and predictable. This makes it suitable for basic tasks such as traversing a list, computing a sum or carrying out a simple reduction, and it tends to be easier to debug since the call chain forms a single, clear path. Its simplicity also brings certain limits. Linear recursion cannot express more complex branching behaviour because it handles only one sub-problem at a time, and when the input size becomes large the depth of recursion can grow quickly, increasing the risk of running out of stack space.

Tree Recursion - Tree recursion occurs when a function makes more than one recursive call within its body. This type of recursion is used in problems where each stage branches into multiple subproblems, such as Fibonacci series or divide-and-conquer algorithms.

Example of tree recursion:

```
def fibonacci(n):  
    if n <= 1:  
        return n
```

```
else:  
    return fibonacci(n - 1) + fibonacci(n - 2)
```

In this example, each call to `fibonacci` generates two further recursive calls. The recursion thus forms a branching structure resembling a tree.

Characteristics of Tree Recursion:

- Each function invocation results in multiple recursive calls.
- Generates an exponential number of total calls for larger inputs.
- Suitable for problems that naturally split into smaller subproblems.

Tree recursion is useful for problems that branch into several smaller sub-problems, as it can explore many possibilities in a structured way and fits naturally with approaches such as divide and conquer or backtracking. This makes it effective for tasks like generating combinations, exploring decision pathways or processing hierarchical structures. Its main drawback is the significant computational and memory cost, since the number of calls can grow rapidly as each call may produce several more. Without memoisation or caching, the same sub-problems may be solved repeatedly, which increases the overall time and resource usage.

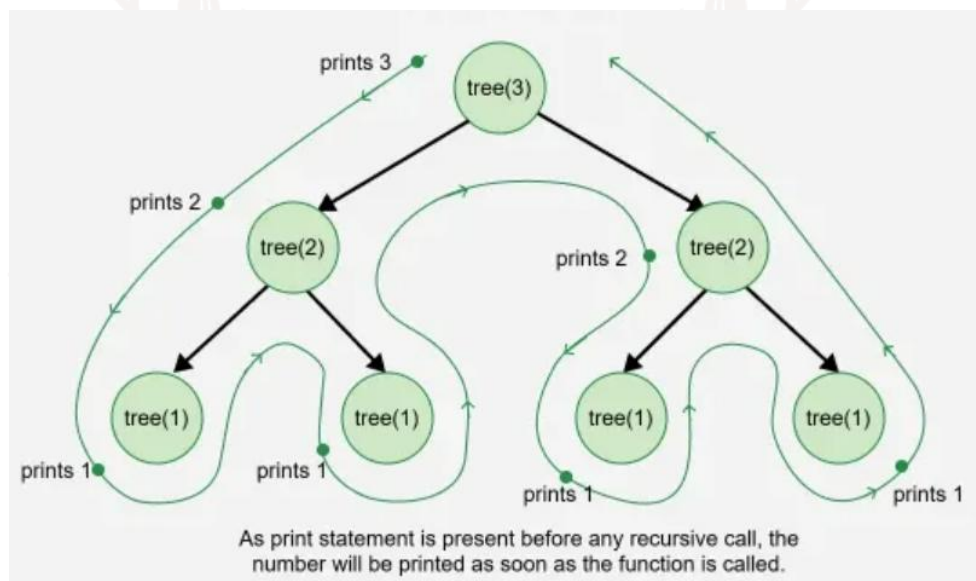


Figure 5: Tree Recursion

Source: [https://media.geeksforgeeks.org/wp-content/uploads/20250611105645773438/tree recursion in c.webp](https://media.geeksforgeeks.org/wp-content/uploads/20250611105645773438/tree%20recursion%20in%20c.webp)

The figure clearly depicts the process of tree recursion, a form of multiple recursions where a function calls itself more than once within its own body. Starting with, the function makes two subsequent

calls, leading to a branching structure that resembles an inverted tree. The entire sequence, from the initial call down to the leaf nodes of, is followed by tracing the green curved line, illustrating the execution path. Importantly, as the print statement is positioned *before* any recursive call, the output (3, 2, 1, 1, 2, 1, 1) is generated immediately upon function entry, following the depth-first, left-to-right traversal of the recursive calls.

3.4 Common Errors and Infinite Recursion

Recursion can be an elegant way to solve problems, but it is prone to certain mistakes that can lead to runtime errors, performance issues, or non-terminating behaviour.

Common Errors in Recursive Functions

- **Missing Base Case:** A recursive function must always have a base condition that stops further recursive calls. Without it, recursion continues indefinitely until a runtime error occurs.

Example:

```
def countdown(n):  
    print(n)  
    countdown(n - 1)  # No base case - infinite recursion
```

- **Incorrect Base Case:** Even when present, an incorrect base case can cause the function to skip termination conditions or return wrong results.
- **Improper Recursive Step:** The recursive step should reduce the problem size or move it towards the base condition. If it fails to do so, the recursion may never converge.
- **State Mutation Across Calls:** Using global or mutable variables within recursive calls can lead to unpredictable results, as each call may modify shared state.
- **Stack Overflow:** Deep recursion can consume large amounts of memory, leading to a stack overflow error. Python's recursion depth limit (by default around 1000) prevents uncontrolled growth.

Infinite Recursion - Infinite recursion occurs when the base case is missing or never reached due to faulty logic. This type of error causes the program to run until the call stack overflows, raising a `RecursionError` in Python.

Example:

```
def infinite_call(x):
```

```
print(x)
infinite_call(x + 1)  # Base condition missing
```

Here, the function keeps calling itself indefinitely because there is no condition to stop recursion.

Prevention Techniques:

- Always verify the base case and ensure it is reachable.
- Include logging or print statements for debugging recursion depth.
- Use iteration where recursion becomes too deep or inefficient.
- Apply memoisation or iterative conversion when appropriate.

Understanding the types of recursion is crucial for designing efficient and reliable recursive functions. Direct recursion provides simplicity, while indirect recursion allows logical separation across functions. Tail and non-tail recursion distinguish between efficient termination and post-processing operations. Linear recursion handles sequential problems effectively, whereas tree recursion models branching computations like Fibonacci or combinatorial problems. Awareness of common mistakes such as missing base cases or faulty recursion steps helps prevent infinite loops and runtime errors. By mastering these concepts, programmers can write recursive functions that are both correct and optimised for their specific problem domains.

Self-Assessment Questions – 2

Multiple Choose Questions	
6	Which best describes indirect recursion? a) A function that calls itself directly within its body b) Two or more functions calling one another in a cycle c) A recursion where the call is the final statement d) A recursion that uses an accumulator to carry results
7	What characterises tail recursion? a) The recursive call is the last operation and its result is returned directly b) The recursive call is always followed by further computation c) It is automatically optimised by Python to use constant stack space d) It always requires multiple recursive calls per invocation
8	Which statement correctly defines tree recursion? a) A recursion that uses an accumulator to hold intermediate results

	b) A function that calls itself only once per invocation c) Mutual calls between two functions that alternate control d) A function that makes more than one recursive call within its body
9	Which of the following is a common cause of a RecursionError? a) Using too many local variables in a function b) Calling <code>sys.setrecursionlimit</code> with a low value c) Missing or unreachable base case leading to infinite recursion d) Returning values from every recursive call
10	Which statement about stack frames and Python's recursion limit is true? a) Python stores all recursion state in global variables rather than stack frames b) Each recursive call adds a new stack frame and the interpreter enforces a recursion depth limit c) Recursive functions never consume additional memory for call state d) <code>sys.getrecursionlimit()</code> cannot be used to view the current recursion limit

4. APPLICATIONS OF RECURSION

Recursion forms a cornerstone of computer science and programming because it allows complex problems to be broken down into smaller, more manageable components. Instead of using repetitive loops or manual iteration, recursive functions solve a problem by repeatedly calling themselves on smaller instances until a base condition is met. In Python, recursion provides an elegant and logical way to express repetitive problem-solving strategies. This section explores the wide-ranging applications of recursion, beginning with classical mathematical problems, moving through divide and conquer algorithms, examining recursive manipulation of data structures, and finally exploring how recursion helps in real-world problem-solving.

4.1 Mathematical Problems (Factorial, Fibonacci Series)

Many mathematical problems have natural recursive definitions. Recursion provides a straightforward and intuitive way to express these problems in Python, making code concise and closely aligned with mathematical formulae.

Factorial - The factorial of a non-negative integer n (denoted as $n!$) is defined as the product of all positive integers less than or equal to n . It can be defined recursively as:

- Base case: $0! = 1$
- Recursive case: $n! = n \times (n - 1)!$

Recursive implementation:

```
def factorial(n):  
    if n == 0:  
        return 1  
    else:  
        return n * factorial(n - 1)
```

Each call to factorial reduces the problem by one until it reaches 0, at which point it returns 1 and begins unwinding.

Explanation of flow:

- factorial(4) calls factorial(3)
- factorial(3) calls factorial(2)
- factorial(2) calls factorial(1)

- `factorial(1)` calls `factorial(0)` which returns 1
- Then, the results multiply back up: $1 \times 1 \times 2 \times 3 \times 4 = 24$

Using recursion for calculating factorial offers a direct link to its mathematical definition, which makes the logic easy to understand and express. The function is often shorter and clearer than an equivalent iterative version, and for small inputs it is straightforward to test and confirm that the output is correct. However, recursion becomes less practical as the numbers grow larger because each call consumes a stack frame, increasing the chance of reaching Python's recursion limit. Iterative solutions usually run faster for large values, as they avoid the overhead of repeated function calls.

Fibonacci Series - The Fibonacci sequence is another classical example where recursion fits naturally. The sequence is defined as:

- Base cases: $F(0) = 0$, $F(1) = 1$
- Recursive case: $F(n) = F(n - 1) + F(n - 2)$

Recursive implementation:

```
def fibonacci(n):  
    if n <= 1:  
        return n  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)
```

This function generates Fibonacci numbers by calling itself twice for each term.

Example for $n = 5$:

- $\text{fibonacci}(5) = \text{fibonacci}(4) + \text{fibonacci}(3)$
- $\text{fibonacci}(4) = \text{fibonacci}(3) + \text{fibonacci}(2)$
- and so on, until the base cases $\text{fibonacci}(1)$ and $\text{fibonacci}(0)$ are reached.

This direct approach leads to redundant computations. For instance, $\text{fibonacci}(3)$ is computed multiple times. To optimise, memoisation or iterative methods are often used.

Improved version using memoisation:

```
memo = {0: 0, 1: 1}  
def fibonacci_memo(n):  
    if n not in memo:  
        memo[n] = fibonacci_memo(n - 1) + fibonacci_memo(n - 2)  
    return memo[n]
```

Recursive Fibonacci is mainly helpful in teaching because it shows clearly how a function can branch into several paths as it breaks a problem into smaller parts. It serves as a good starting point for understanding recursion trees, where each call produces two more, and it helps explain why techniques such as dynamic programming or memoisation are needed to avoid repeated calculations in more advanced work.

4.2 Divide and Conquer Algorithms (Binary Search, Merge Sort)

Recursion plays a key role in divide and conquer algorithms, where a problem is divided into smaller parts, each solved recursively, and their results combined to form the final output. This method is efficient for large datasets and forms the basis for many classic algorithms.

Binary Search - Binary search is a recursive algorithm for finding an element in a sorted list by repeatedly dividing the search interval in half.

Algorithm logic:

1. Compare the middle element of the list with the target value.
2. If the middle element matches the target, the search is successful.
3. If the target is smaller, search recursively in the left half.
4. If the target is larger, search recursively in the right half.

Recursive implementation:

```
def binary_search(arr, low, high, target):  
    if low > high:  
        return -1  
    mid = (low + high) // 2  
    if arr[mid] == target:  
        return mid  
    elif arr[mid] > target:  
        return binary_search(arr, low, mid - 1, target)  
    else:  
        return binary_search(arr, mid + 1, high, target)
```

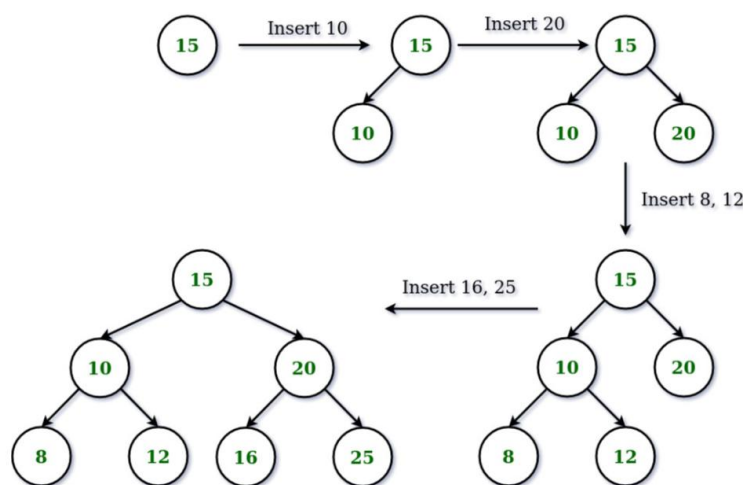


Figure 6: Binary Tree using Recursion

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20200206162306/Untitled-Diagram311.jpg>

The figure illustrates the step-by-step construction and property of a Binary Search Tree (BST), where the process begins with the root node, 15. Subsequent elements are inserted based on the BST property: values less than the current node are placed in the left sub-tree, and values greater than the current node are placed in the right sub-tree. Specifically, 10 is inserted to the left of 15, and 20 to the right. Following this, the insertion of 8 and 12 places them to the left and right of 10, respectively, while 16 and 25 are correctly positioned to the left and right of 20, maintaining the ordered structure. This recursive partitioning ensures efficient search, insertion, and deletion operations within the data structure.

Binary search recursion is efficient because it reduces the problem size by half with each step, giving it a time complexity of $O(\log n)$, which makes it suitable for large datasets. Since it compares only a few key positions rather than scanning the entire list, it performs fewer checks than a linear search. Its limitations arise from the need for the data to be sorted before the search can work correctly, and the function must handle indices with care, as small mistakes in calculating midpoints or boundaries can lead to off-by-one errors or incorrect results.

Merge Sort - Merge sort is another divide and conquer algorithm that uses recursion to sort a list efficiently. It works by dividing the list into two halves, recursively sorting them, and then merging the sorted halves.

Algorithm steps:

1. Divide the list into two halves until single-element lists remain.
2. Merge the smaller lists in sorted order.

3. Continue merging upwards until the entire list is sorted.

Recursive implementation:

```
def merge_sort(arr):
    if len(arr) > 1:
        mid = len(arr) // 2
        left = arr[:mid]
        right = arr[mid:]
        merge_sort(left)
        merge_sort(right)
        i = j = k = 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
```

Merge sort offers predictable performance because it always runs in $O(n \log n)$ time, which makes it reliable even when working with very large datasets. It is also a stable sorting method, meaning that elements with the same value keep their original order, which is important in many data-handling tasks. The main drawback is the extra memory needed to hold the temporary lists during the merge step, and the recursive calls add a certain amount of overhead. These factors can make it less suitable in environments where memory is limited or where very small lists need to be sorted frequently.

4.3 Recursive Operations on Data Structures (Linked Lists, Trees)

Recursion is particularly effective for working with recursive data structures—structures defined in terms of smaller versions of themselves. Examples include linked lists and trees.

Linked Lists - A linked list is a collection of nodes, where each node contains data and a reference to the next node. Recursive operations on linked lists include traversal, searching, and reversing.

Example: Recursive traversal of a linked list

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

def traverse(node):
    if node is None:
        return
    print(node.data)
    traverse(node.next)
```

The function begins by printing the data stored in the current node, and once this step is complete it calls itself again using the next node in the sequence. This pattern continues until it reaches a node that has no successor, at which point the recursion stops because there are no further nodes to process.

Example: Recursive reversal of a linked list

```
def reverse_list(node):
    if node is None or node.next is None:
        return node
    new_head = reverse_list(node.next)
    node.next.next = node
    node.next = None
    return new_head
```

This example reverses a linked list recursively by adjusting node pointers during the unwinding phase.

Trees - Trees are naturally recursive structures where each node may have child nodes that themselves are trees. Recursion is ideal for tree traversal operations such as preorder, inorder, and postorder traversal.

Example: Recursive inorder traversal

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def inorder(node):
    if node:
```

```
inorder(node.left)
print(node.value)
inorder(node.right)
```

The traversal works by visiting the left subtree first, then processing the root node, and finally moving to the right subtree, which reflects the natural structure of a binary tree. Using recursion suits this pattern well because each call handles its own subtree in the same manner, making the hierarchical movement through the tree easy to express and follow.

Recursion is well suited to handling data structures that have a natural hierarchy, as it allows each level of the structure to be processed in a clear and direct way without the need to manage a stack manually. This makes operations such as tree traversal easier to express because the function's structure matches the organisation of the data itself. Even so, recursion brings certain risks. If the tree is heavily unbalanced, the depth of the recursive calls can become quite large, increasing the chance of reaching the recursion limit. In some situations an iterative method can run more efficiently, particularly when it avoids the overhead of repeated function calls.

4.4 Practical Problem-Solving Using Recursion

Beyond mathematical and structural examples, recursion finds practical use in numerous real-world computational problems. These problems often involve repetitive exploration, hierarchical processing, or decomposition of large problems.

Example 1: Directory Traversal

Recursion can be used to explore file systems where folders contain subfolders and files.

```
import os

def list_files(path):
    for entry in os.listdir(path):
        full_path = os.path.join(path, entry)
        if os.path.isdir(full_path):
            list_files(full_path)
        else:
            print(full_path)
```

The function goes through each file and folder in the current directory, and whenever it comes across another directory, it calls itself to inspect everything inside it. This creates a natural descending process where the function continues exploring each level of the folder structure until no further subdirectories remain, after which it unwinds back through the earlier calls.

Example 2: Solving Puzzles (Towers of Hanoi)

The Towers of Hanoi puzzle is a classic demonstration of recursion. The problem involves moving a set of disks from one peg to another using an auxiliary peg, with the constraint that only one disk can be moved at a time and no larger disk may be placed on a smaller one.

Recursive solution:

```
def hanoi(n, source, target, auxiliary):
    if n == 1:
        print(f"Move disk 1 from {source} to {target}")
        return
    hanoi(n - 1, source, auxiliary, target)
    print(f"Move disk {n} from {source} to {target}")
    hanoi(n - 1, auxiliary, target, source)
```

Example 3: Fractals and Graphics

Fractal patterns, such as the Sierpinski triangle or Koch snowflake, are naturally recursive, where the same shape appears repeatedly at different scales.

Simple example (Sierpinski triangle logic):

- Draw a triangle.
- Recursively draw smaller triangles in each corner.
- Continue until a minimal triangle size is reached.

This use demonstrates recursion in visual computation and geometry.

Example 4: Backtracking Problems

Recursive backtracking explores all possibilities to find solutions, such as:

- Solving mazes.
- Generating permutations and combinations.
- Solving Sudoku puzzles.

Example: generating all permutations of a list:

```
def permute(data, start, end):
    if start == end:
        print(''.join(data))
    else:
        for i in range(start, end + 1):
```



```
data[start], data[i] = data[i], data[start]
permute(data, start + 1, end)
data[start], data[i] = data[i], data[start]
```

Here, recursion systematically explores all arrangements.

Recursion serves as a fundamental concept that simplifies complex problems by expressing them in terms of smaller, self-similar subproblems. From solving mathematical equations like factorial and Fibonacci to handling divide and conquer algorithms such as binary search and merge sort, recursion provides clarity and structure. It is indispensable when working with recursive data structures like linked lists and trees, where it simplifies traversal and manipulation. Beyond theory, recursion powers practical applications, including directory traversal, problem-solving in games and puzzles, and graphical fractal generation. Understanding these applications strengthens a programmer's ability to approach problems logically and design clean, modular, and efficient solutions.

Self-Assessment Questions – 3

Multiple Choose Questions	
11	Which algorithm is a classic example of divide and conquer implemented with recursion? a) Bubble sort b) Linear search c) Merge sort d) Selection sort
12	Why is the naive recursive implementation of the Fibonacci sequence inefficient for large n? a) It uses too much extra memory for temporary arrays b) It recalculates the same subproblems many times c) It requires the list to be sorted beforehand d) It cannot be expressed in terms of smaller subproblems
13	Which recursive operation is most appropriate for a binary tree? a) Inorder traversal that visits left subtree, node, then right subtree b) Iteratively reversing the tree using a stack c) Sorting the tree nodes using bubble sort d) Converting every node value to a string without visiting children
14	When writing a recursive directory traversal function in Python, which statement is true? a) It always uses constant extra memory regardless of depth b) It does not need a base case because os.listdir handles termination

	c) It can only list files, not directories d) The function calls itself on subdirectories and unwinds when no further subdirectories remain
15	What is the correct high-level recursive strategy for solving the Towers of Hanoi for n disks? a) Move all n disks directly from source to target in one step b) Move $n - 1$ disks to the auxiliary peg, move the largest disk to the target, then move $n - 1$ from auxiliary to target c) Swap the source and target and repeat without moving disks d) Use an iterative loop that never reduces the problem size



5. SUMMARY

- Recursion is a method where a function solves a problem by calling itself with a smaller version of the same task until a stopping point is reached.
- The basic structure of a recursive function contains two essential parts: a base case that ends the process and a recursive case that continues it.
- Recursion is useful in programming because it offers a clear way to express solutions that are naturally repetitive or hierarchical in structure.
- The flow of control in recursion moves downward through repeated calls until the base case is met and then unwinds as each call returns its result.
- When a recursive function runs, Python stores information about each call in the form of stack frames on the function call stack.
- Direct recursion occurs when a function calls itself, while indirect recursion takes place when two or more functions call each other in a cycle.
- Tail recursion places the recursive call at the end of the function, while non-tail recursion performs extra work after the recursive call returns.
- Linear recursion creates one recursive call at each stage, while tree recursion produces multiple recursive calls, such as in the Fibonacci sequence.
- Common errors in recursion include missing base cases, unreachable base cases, and incorrect conditions that cause indefinite repetition.
- Recursion has a wide range of applications, including mathematical functions, divide and conquer algorithms, data structure operations, directory exploration, puzzles, and graphical patterns.

6. GLOSSARY

Recursion	-	A technique where a function repeatedly calls itself to solve smaller parts of a problem.
Base Case	-	The condition that stops further recursive calls and returns a direct answer.
Recursive Case	-	The part of the function where it calls itself with reduced input.
Stack Frame	-	A block of memory that stores details about a single function call, including variables and return addresses.
Call Stack	-	The structure that holds stack frames in a last-in, first-out order as functions run and return.
Direct Recursion	-	When a function calls itself without involving any other function.
Indirect Recursion	-	When a function calls another function which then calls the original function.
Tail Recursion	-	A form of recursion where the recursive call is the final action in the function.
Tree Recursion	-	A recursive pattern where each call leads to two or more recursive calls, forming a branching structure.
Divide and Conquer	-	A strategy that splits a problem into smaller parts, solves them recursively, and then combines the results.

7. TERMINAL QUESTIONS

1. Explain, in your own words, why a clear base case is essential for a recursive function and describe what can happen if it is missing or unreachable.
2. Compare and contrast tail recursion and non-tail recursion, and give an example scenario where converting to an iterative approach would be preferable.
3. Describe how the function call stack and stack frames behave during a recursive call sequence, using the factorial example to illustrate entry and exit points.
4. Outline the differences between direct and indirect recursion, then provide a short pseudocode example of indirect recursion that would be appropriate for a state machine.
5. For a binary tree, explain how inorder, preorder and postorder traversals are implemented recursively and state one use case for each traversal order.
6. Discuss the performance implications of tree recursion using the naive Fibonacci implementation as an example, and explain how memoisation changes the time complexity.
7. Present a recursive algorithm for reversing a singly linked list, explain the role of the return value during unwinding, and compare it to an iterative solution in terms of space usage.
8. Describe how divide and conquer algorithms use recursion, then explain why merge sort guarantees $O(n \log n)$ time while quicksort may perform worse in some cases.
9. Provide a recursive approach to directory traversal that safely handles symbolic links and permission errors, and outline the safeguards you would include to avoid infinite loops.
10. List common mistakes programmers make with recursion (at least four), and for each give a practical debugging or design technique to prevent or detect the problem.

8. ANSWERS

8.1. Self-Assessment Answers:

1. A function that calls itself to solve smaller instances of the same problem
2. Base case and recursive case
3. Traversing a binary tree to visit all nodes
4. Because each recursive call adds a stack frame and the call stack can exceed the recursion limit
5. Python does not perform tail call optimisation automatically and so deep tail recursion can still exhaust the stack
6. Two or more functions calling one another in a cycle
7. The recursive call is the last operation and its result is returned directly
8. A function that makes more than one recursive call within its body
9. Missing or unreachable base case leading to infinite recursion
10. Each recursive call adds a new stack frame and the interpreter enforces a recursion depth limit
11. Merge sort
12. b) It recalculates the same subproblems many times
13. Inorder traversal that visits left subtree, node, then right subtree
14. The function calls itself on subdirectories and unwinds when no further subdirectories remain
15. Move $n - 1$ disks to the auxiliary peg, move the largest disk to the target, then move $n - 1$ from auxiliary to target

8.2. Terminal Question Answers

Answer 1: A clear base case is essential because it provides the stopping condition that prevents endless self-calls; without it the recursion never terminates and eventually raises a `RecursionError` or exhausts memory. A reachable and correctly defined base case ensures the recursive descent finishes and the unwinding phase can return final results. Refer 2.4.

Answer 2: Tail recursion places the recursive call as the final action so no work remains after the call, whereas non-tail recursion performs further computation after the recursive call returns. Because Python does not perform tail-call optimisation, converting deep tail-recursive routines (for example, a large summation carried by an accumulator) to an iterative loop is often preferable to avoid heavy stack use. Refer 3.2.

Answer 3: Each recursive invocation pushes a stack frame containing parameters, local variables and the return address; using factorial as an example, factorial(3) pushes frames for 3, 2 and 1, and when the base case returns those frames pop in LIFO order as results propagate back up. Observing entry and exit prints or tracing the stack makes the descent and unwinding clear and shows how intermediate state is preserved per frame. Refer 2.5.

Answer 4: Direct recursion has a function calling itself within its own body, while indirect recursion involves two or more functions calling each other in a cycle; indirect forms suit problems split across distinct behaviours or states. A simple state-machine pseudocode: funcA(state): if stop: return; funcB(next_state); funcB(state): if stop: return; funcA(next_state2). Refer 3.1.

Answer 5: Inorder traversal recursively visits left subtree, then node, then right subtree and is used to produce sorted output from a binary search tree; preorder visits node before its subtrees and is useful for serialising or copying a tree; postorder visits children before the node and is suitable for deletion or for evaluating expression trees. Each is implemented by recursive calls that mirror the tree's structure. Refer 4.3.

Answer 6: Tree recursion such as the naive Fibonacci implementation branches into many overlapping calls and therefore has exponential time complexity, causing severe performance problems for large n. Applying memoisation caches results for each subproblem so repeated work is avoided and the time complexity falls to linear in n. Refer 4.1.

Answer 7: A standard recursive reverse for a singly linked list returns the new head when unwinding: if node is None or node.next is None return node; new_head = reverse_list(node.next); node.next.next = node; node.next = None; return new_head. The return value is vital during unwinding because it supplies the updated head to higher frames; compared with the iterative three-pointer method, recursion uses $O(n)$ call-stack space while the iterative approach uses $O(1)$ extra space. Refer 4.3.

Answer 8: Divide and conquer splits a problem into smaller parts solved recursively and then combines their results; merge sort always splits roughly in half and merges in linear time at each level, yielding $O(n \log n)$ across all levels. Quicksort can also be recursive and is average $O(n \log n)$, but poor pivot choices produce unbalanced partitions and can degrade to $O(n^2)$ in the worst case. Refer 4.2.

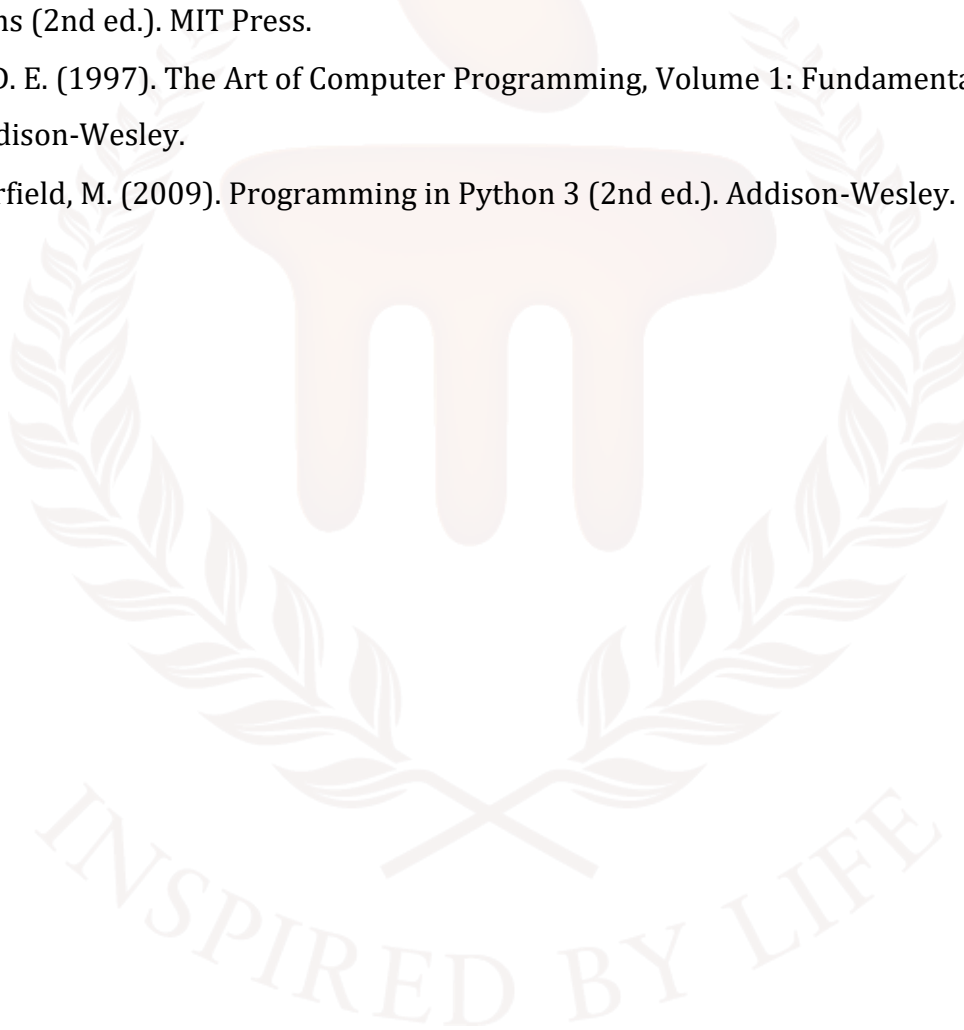
Answer 9: A safe recursive directory traversal uses a try/except around directory reads, checks and optionally skips symbolic links, and records visited inodes or absolute paths to avoid revisiting them; on encountering PermissionError or symlink loops the function should skip that entry and continue. Additional safeguards include limiting recursion depth, using os.scandir for efficient iteration, and normalising paths before adding them to a visited set to prevent infinite loops. Refer 4.4.

Answer 10: Common mistakes include missing or incorrect base cases, failure to reduce the problem size, unintended mutation of shared state, and ignoring recursion depth limits; practical techniques are to add explicit base-case assertions and unit tests, include logging or depth counters to trace progress, avoid global mutable variables or pass state explicitly, use memoisation where subproblems repeat, and convert to iteration or limit recursion depth when necessary. Refer 3.4.



9. REFERENCES

- Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data Structures and Algorithms in Python. Wiley.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.
- Abelson, H., Sussman, G. J., & Sussman, J. (1996). Structure and Interpretation of Computer Programs (2nd ed.). MIT Press.
- Knuth, D. E. (1997). The Art of Computer Programming, Volume 1: Fundamental Algorithms (3rd ed.). Addison-Wesley.
- Summerfield, M. (2009). Programming in Python 3 (2nd ed.). Addison-Wesley.



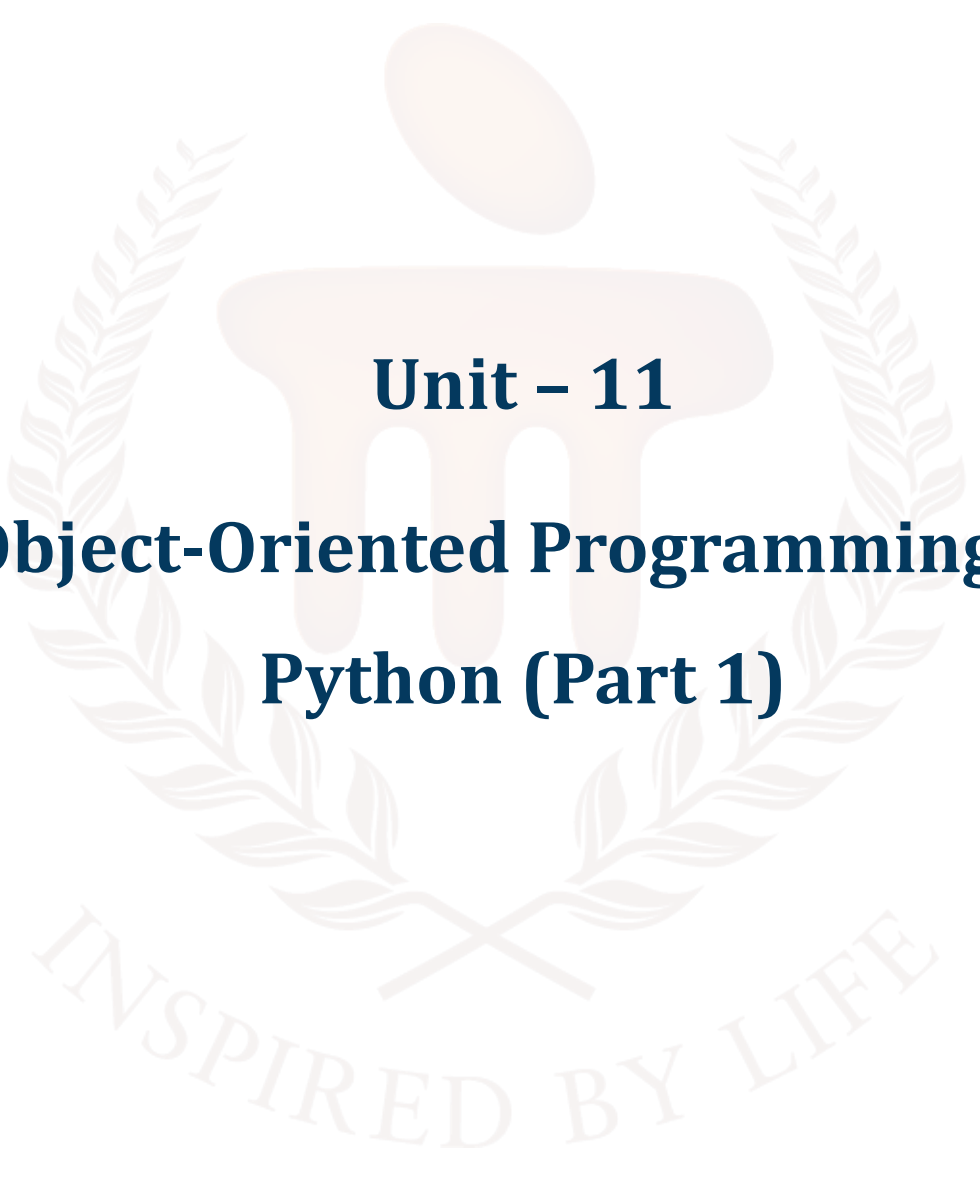
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 11

Object-Oriented Programming in Python (Part 1)

TABLE OF CONTENTS

Sl No	Topic	Fig No / Table / Graph	Saq / Activity	Page No
1	Introduction	-	-	4
	1.1 Learning Objectives	-	-	
2	Classes and Objects	-	1	5-11
	2.1 Concept of Object-Oriented Programming	-	-	
	2.2 Defining Classes in Python	-	-	
	2.3 Creating and Using Objects	1	-	
	2.4 Attributes and Methods	2	-	
3	Class Variables	-	2	12-17
	3.1 Definition and Purpose	-	-	
	3.2 Declaration of Class Variables	3	-	
	3.3 Accessing Class Variables	-	-	
	3.4 Use Cases of Shared Data	-	-	
4	Instance Variables	-	3	18-23
	4.1 Meaning of Instance Variables	4	-	
	4.2 Initialising Instance Variables	-	-	
	4.3 Object-Specific Data Storage	-	-	
	4.4 Difference Between Class and Instance Variables	-	-	
5	Using Classes in Python Programs	-	4	24-31
	5.1 Creating Multiple Objects	-	-	
	5.2 Data Encapsulation Basics	5	-	
	5.3 Real-World Modelling with Classes	-	-	
	5.4 Best Practices in Class Design	-	-	
6	Summary	-	-	32
7	Glossary	-	-	33
8	Terminal Questions	-	-	34
9	Answers	-	-	35-37
	9.1 Self-Assessment Answers	-	-	
	9.2 Terminal Questions Answers	-	-	
10	References	-	-	38

1. INTRODUCTION

The previous chapter focused on recursion in Python, where problems were solved by calling a function within itself. You studied how recursive solutions use a base case and a recursive case to break complex problems into smaller, manageable parts. This approach helped in improving problem-solving skills and understanding function execution flow, especially in tasks such as mathematical computations and data processing.

This chapter introduces Object Oriented Programming in Python, starting with the basic building blocks such as classes and objects. It explains how classes act as blueprints for creating objects and how objects represent real-world entities within a program. These concepts help in organising code in a structured and logical manner.

The chapter also covers class variables and instance variables, highlighting their differences and usage. You will learn how data can be shared across all objects using class variables, while instance variables store object-specific information. This distinction is important when designing scalable and reusable Python programs.

To study this chapter effectively, practise writing small programs that define classes and create multiple objects. Experiment with class and instance variables to observe their behaviour during execution. Regular coding practice combined with real-life examples will help in building a strong foundation in object-oriented programming.

1.1. Learning Objectives

By the end of this Unit, you will be able to:

- Apply object-oriented concepts to create and use classes and objects in Python.
- Differentiate between class variables and instance variables through practical examples.
- Construct Python programs using object-oriented design principles.
- Organise program data using appropriate class structures.
- Evaluate the impact of variable scope on object behaviour in Python programs.



2. CLASSES AND OBJECTS

Object-Oriented Programming (OOP) is a paradigm in computer programming that uses “objects” to represent data and methods. Unlike procedural programming, which focuses on functions and sequential execution, OOP models real-world entities by encapsulating both data and behaviour. Python, being a versatile and high-level programming language, provides extensive support for object-oriented programming, allowing developers to write code that is modular, reusable, and easier to maintain.

The main building blocks of OOP in Python are classes and objects. A class serves as a blueprint for creating objects, while objects are instances of classes that hold data and perform actions. This section will explore the concepts of classes, objects, attributes, and methods, which form the foundation of object-oriented programming in Python.

2.1 Concept of Object-Oriented Programming

Object-oriented programming is centred around the concept of objects. An object is a self-contained unit that contains both data, in the form of attributes (also called properties or fields), and functionality, in the form of methods (functions defined within the class). This approach allows for modelling real-world entities more intuitively in software.

The four fundamental principles of OOP are:

- **Encapsulation:** Encapsulation is the mechanism of restricting access to some components of an object and only exposing necessary parts. It is achieved by defining attributes and methods within a class and controlling their accessibility. This helps protect the internal state of the object and ensures that changes to data occur in a controlled manner.
- **Abstraction:** Abstraction involves hiding the internal details of an object and exposing only the necessary features. Users interact with objects through a simplified interface, without needing to understand the complex inner workings.
- **Inheritance:** Inheritance allows one class to derive properties and methods from another class, promoting code reuse and hierarchical relationships. A child class inherits attributes and behaviour from a parent class but can also have additional features or override existing ones.

- **Polymorphism:** Polymorphism refers to the ability of different objects to respond to the same method in different ways. This allows for flexibility in code and enables the use of generic interfaces that work with multiple object types.

In Python, everything is an object, and classes are used to define new object types. The ability to combine data and behaviour into a single unit simplifies programming and mirrors real-world concepts, making OOP a powerful tool for software development.

2.2 Defining Classes in Python

A class is essentially a blueprint for creating objects. It defines the attributes (data) and methods (functions) that the objects created from it will have. In Python, classes are defined using the class keyword followed by the class name and a colon.

Syntax:

```
class ClassName:
    # class body
    # attributes and methods
```

For example:

```
class Car:
    # class attributes
    wheels = 4

    # constructor method
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year

    # method to display car details
    def display_info(self):
        print(f"{self.year} {self.make} {self.model} with {self.wheels} wheels")
```

Car is the class name, and by convention, class names in Python are written using PascalCase to clearly distinguish them from variables and functions. The `__init__` method acts as a constructor and is automatically invoked when a new object of the class is created; its purpose is to initialise the attributes of the object. The keyword `self` refers to the current instance of the class and is used within class methods to access instance attributes and other methods. In this context, `wheels` is a class attribute that is shared by all objects created from the class, whereas `make`, `model`, and `year` are

instance attributes that hold data unique to each object. The `display_info` method is a class method designed to output the details of a car by printing the values of its attributes.

Classes in Python act as templates used to create objects, defining both the data they can store and the behaviour they can perform. They make it possible to group related data and functionality within a single structured unit, which improves organisation and readability in programs. Since Python follows dynamic typing, attributes are not fixed at compile time, meaning new attributes can be added to objects even after they have been created, offering flexibility during program execution.

2.3 Creating and Using Objects

Once a class has been defined, we can create objects from it. An object is an instance of a class that contains its own set of attributes and can perform actions through its methods.

Creating an object:

```
my_car = Car("Toyota", "Corolla", 2023)
```

Accessing attributes and methods:

```
print(my_car.make) # Output: Toyota
```

```
my_car.display_info() # Output: 2023 Toyota Corolla with 4 wheels
```

Each object created from a class maintains its own separate copy of instance attributes, so modifying an attribute for one object does not cause any change in the attributes of other objects. Methods that are defined within a class can be invoked by using the object name followed by the method name and parentheses, which allows the method to operate on the data specific to that object.

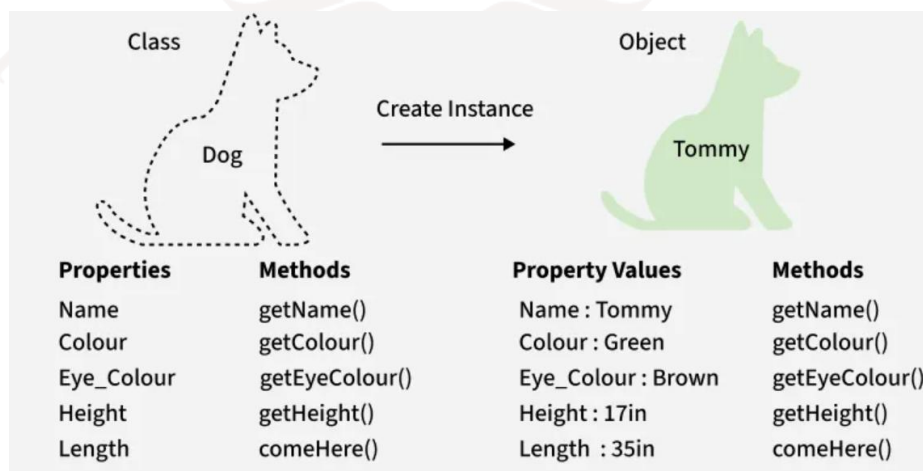


Figure 1: Classes and Objects

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20250906151500826906/420046617.webp>

The figure illustrates the fundamental concept of Object-Oriented Programming (OOP), specifically the relationship between a class and an object in a language like Python. This figure displays the class, named Dog, as a conceptual blueprint or template which defines the general properties (attributes) such as Name and Colour, and methods (behaviours) like getName() and comeHere(). The process of 'Create Instance' then yields an object, named Tommy, which is a tangible instance of the class. This specific object possesses the same structure of properties and methods but is instantiated with actual values for the properties, such as Name: Tommy and Colour: Green. Therefore, the class provides the structure, and the object provides the specific data that populates that structure.

Example of multiple objects:

```
car1 = Car("Honda", "Civic", 2022)
car2 = Car("Ford", "Mustang", 2021)
```

```
car1.display_info() # Output: 2022 Honda Civic with 4 wheels
car2.display_info() # Output: 2021 Ford Mustang with 4 wheels
```

This demonstrates how a single class can create multiple objects, each with its own distinct data.

2.4 Attributes and Methods

Attributes and methods are the two main components of a class.

Attributes are variables that belong to an object or class. They store the data associated with objects.

There are two types:

1. **Instance Attributes:** Unique to each object. Defined inside the constructor (`__init__`) using `self`.
2. **Class Attributes:** Shared among all objects of the class. Defined directly inside the class but outside any method.

Methods are functions that belong to a class. They define the behaviour of an object. There are several types:

1. **Instance Methods:** Operate on instance attributes. The first parameter is always `self`.
2. **Class Methods:** Operate on class attributes. Defined using the `@classmethod` decorator and take `cls` as the first parameter.
3. **Static Methods:** Do not operate on instance or class attributes. Defined using the `@staticmethod` decorator. They behave like regular functions but belong to the class's namespace.

Example of class, instance attributes, and methods:

```
class Student:
    # class attribute
    school = "Greenwood High"

    def __init__(self, name, age):
        self.name = name # instance attribute
        self.age = age   # instance attribute

    # instance method
    def display(self):
        print(f"{self.name}, Age: {self.age}, School: {Student.school}")

    # class method
    @classmethod
    def school_name(cls):
        print(f"The school is {cls.school}")

    # static method
    @staticmethod
    def greet():
        print("Welcome to the school!")
```

Usage:

```
s1 = Student("Alice", 14)
s1.display()           # Output: Alice, Age: 14, School: Greenwood High
Student.school_name()  # Output: The school is Greenwood High
Student.greet()        # Output: Welcome to the school!
```

Instance methods must be called using an object because they operate on the data stored within that specific instance. In contrast, class methods and static methods can be invoked using the class name, as they are not dependent on a particular object's data. Methods play an important role in providing controlled access to attributes and in encapsulating behaviour, ensuring that data and the operations performed on it are logically grouped within the class.

Attributes		Methods
>>> Door		>>>headLightOn
>>> Seats		>>>Start
>>> Licence plate		>>>Brake
>>> Wheel		>>>Horn
>>> Side mirror		>>>turnOnAC
>>> Handle		>>>startWiper
>>> Make		>>>muffler
>>> Color		>>>exhaustFumes
Attributes and Methods of a Car		

Figure 2: Attributes and Methods

Source: <https://raphealenike.medium.com/object-oriented-programming-oop-series-attributes-and-methods-5d1e51b11e3e>

The figure and the previously discussed figure detailing the Dog class serve to illustrate the core principles of Object-Oriented Programming (OOP), specifically the concepts of attributes and methods within a class structure. In this context, the class acts as a template for a real-world entity, such as a car, defining its inherent attributes—the data or properties it possesses, like Colour, Make, and Seats. Furthermore, the class defines methods, which are the behaviours or actions the entity can perform, such as Start or Brake. When an object (an instance of the class) is created, it inherits these attributes and methods, allowing specific data to be assigned to the attributes and the methods to be executed, modelling the object's specific state and behaviour in a program.

Object-oriented programming in Python allows for better organisation and structure in programming by modelling real-world entities. Classes act as blueprints, while objects are instances that encapsulate data and behaviour. Attributes store data, while methods define actions. Understanding the principles of OOP and the relationship between classes, objects, attributes, and methods is fundamental to writing modular, maintainable Python code.

SELF-ASSESSMENT QUESTIONS – 1

Multiple Choose Questions	
1	What is the purpose of the init method in a Python class? a) To define class-level constants b) To register the class with the interpreter c) To initialise instance attributes when an object is created d) To convert an object to a string
2	Which OOP principle involves hiding internal details and exposing only the necessary features? a) Encapsulation b) Polymorphism c) Inheritance d) Abstraction

3	Which statement about class attributes in Python is correct? a) They are unique to each object instance b) They are shared across all instances of the class c) They must be declared inside the init method d) They cannot be accessed using the class name
4	How does a @classmethod differ from a @staticmethod? a) A classmethod receives cls and can access class attributes; a staticmethod receives no implicit first argument and cannot access class or instance data b) A classmethod can only be called on instances; a staticmethod can only be called on the class c) A classmethod always acts as a constructor; a staticmethod always acts as a destructor d) They are interchangeable and behave the same
5	Which example best illustrates polymorphism in object-oriented programming? a) A subclass inheriting attributes from a parent class b) Using self to access instance attributes inside a method c) Two different classes implementing a method with the same name but different behaviour d) Marking an attribute as private by prefixing it with an underscore



3. CLASS VARIABLES

In object-oriented programming, data associated with a class can be divided into two main types: instance variables and class variables. While instance variables are unique to each object, class variables are shared across all instances of a class. They provide a mechanism to store information that is common to every object created from that class. Understanding class variables is essential for scenarios where shared data management is required, such as maintaining counters, configuration values, or constants.

Class variables contribute to better memory management because a single copy is shared by all objects, unlike instance variables which are duplicated for each object. This section explores the definition, declaration, access, and use cases of class variables in Python.

3.1 Definition and Purpose

A class variable is a variable that is defined within a class but outside any method, including the constructor (`__init__`). Unlike instance variables, class variables are shared by all instances of the class. Changes to a class variable affect all objects that reference it.

Purpose of class variables:

- To store data that should be consistent across all instances.
- To maintain global counters or shared settings within a class.
- To reduce memory usage by avoiding duplication of common data.
- To provide a mechanism for communication between objects of the same class.

Consider a class that represents a bank account to understand the distinction between different types of variables. Each bank account object would store individual details such as the account holder's name and the current balance as instance variables, since these values differ from one account to another. At the same time, information such as the bank name or branch code would remain the same for all accounts and can therefore be defined as class variables shared by every object of the class.

3.2 Declaration of Class Variables

Class variables are declared directly inside the class but outside any instance methods or constructors. The syntax is straightforward:


```
class BankAccount:
    bank_name = "Global Bank" # class variable

    def __init__(self, account_holder, balance):
        self.account_holder = account_holder # instance variable
        self.balance = balance # instance variable
```

In this example, `bank_name` is defined as a class variable, meaning it is shared across all instances of the class and holds the same value for every account. On the other hand, `account_holder` and `balance` are instance variables, so each object maintains its own separate values for these attributes based on the specific account it represents.

Key points about declaration:

- Class variables are usually declared at the beginning of the class, before the constructor, for readability and maintainability.
- They can be accessed by both the class name and object references.
- Modifying a class variable using the class name affects all instances, whereas modifying it using an object reference may create a new instance variable for that object, leaving the class variable unchanged.

Table 1: Comparison Between Class Variables and Instance Variables

Aspect	Class Variables	Instance Variables
Scope	Shared across all objects of the class	Limited to the specific object
Storage	Only one copy exists in memory for the entire class	Separate copy exists for each object
Definition	Declared inside the class but outside any method	Usually declared inside the constructor using <code>self</code>
Access	Can be accessed using the class name or an object	Accessed using the object name
Behaviour when modified	A change affects all objects of the class	A change affects only the specific object
Purpose	Used for shared data common to all objects	Used for storing object-specific data

3.3 Accessing Class Variables

Class variables can be accessed in multiple ways, either through the class name or an object of the class.

Example:

```
class Employee:
    company_name = "Tech Solutions" # class variable

    def __init__(self, name, position):
        self.name = name # instance variable
        self.position = position # instance variable

# Access using class name
print(Employee.company_name) # Output: Tech Solutions

# Access using object
emp1 = Employee("John", "Manager")
emp2 = Employee("Alice", "Developer")

print(emp1.company_name) # Output: Tech Solutions
print(emp2.company_name) # Output: Tech Solutions
```

Modifying class variables:

```
# Changing class variable using class name
Employee.company_name = "Global Tech"

print(emp1.company_name) # Output: Global Tech
print(emp2.company_name) # Output: Global Tech
```

An important point to remember is that when a class variable is modified using an object reference, Python does not change the original class variable. Instead, it creates a new instance variable with the same name only for that particular object. As a result, the modification applies only to that object and has no impact on the class variable or on other objects created from the same class.

```
emp1.company_name = "Local Tech"
print(emp1.company_name) # Output: Local Tech (instance variable)
print(emp2.company_name) # Output: Global Tech (class variable unchanged)
```

This behaviour demonstrates the distinction between class-level data and object-level data and highlights the importance of using class variables carefully when shared data is intended.

3.4 Use Cases of Shared Data

Class variables are particularly useful in scenarios where data needs to be consistent across all objects of a class. Common use cases include:

1. **Counting Instances:** Keeping track of the number of objects created from a class.

```
class Student:
    count = 0 # class variable

    def __init__(self, name):
        self.name = name
        Student.count += 1

s1 = Student("Alice")
s2 = Student("Bob")
print(Student.count) # Output: 2
```

2. **Shared Configuration or Constants:** Storing configuration values or constants that should remain consistent.

```
class AppSettings:
    default_language = "English"
    default_theme = "Light"
```

3. **Managing Shared Resources:** Keeping track of shared resources such as available seats, inventory items, or network connections.

```
class Theatre:
    total_seats = 100

    def __init__(self, booked_seats):
        self.booked_seats = booked_seats
        Theatre.total_seats -= booked_seats
```

4. **Data Common to All Objects:** Any attribute that is the same for all objects, such as company name, school name, or product category, can be defined as a class variable.

Using class variables offers several advantages in object-oriented programming. Since only a single copy of the variable is shared among all objects of the class, memory usage becomes more efficient, especially when dealing with large numbers of objects. Class variables also help simplify program structure by storing common data at the class level, making it easily accessible to all instances. This shared nature allows objects to communicate or refer to common information without the need for separate global variables or external storage mechanisms.

Despite these benefits, class variables come with certain limitations that require careful handling. If a class variable is unintentionally modified, the change is reflected across all objects of the class, which can lead to unexpected behaviour in the program. For this reason, class variables should be used only for data that is truly common to all objects, while object-specific or frequently changing data should be stored in instance variables to ensure independent object states.

Class variables are an integral part of object-oriented programming in Python. They provide a mechanism to store data that is common to all instances of a class. Proper use of class variables can improve memory efficiency and enable shared data management across objects. It is essential to distinguish between class variables and instance variables to avoid unexpected behaviour. Understanding how to declare, access, and apply class variables is crucial for creating well-structured and maintainable Python programs.

SELF-ASSESSMENT QUESTIONS – 2

Multiple Choose Questions	
6	Which statement best describes a class variable? a) It is shared across all instances of the class b) It is unique to each object instance c) It must be declared inside the init method d) It cannot be accessed using the class name
7	What happens if you assign a value to a class variable through an object reference? a) The class variable is updated for every instance b) A new instance variable with the same name is created for that object, leaving the class variable unchanged c) Python raises an error d) The class variable is deleted
8	What happens if you assign a value to a class variable through an object reference? a) The class variable is updated for every instance b) A new instance variable with the same name is created for that object, leaving the class variable unchanged c) Python raises an error d) The class variable is deleted
9	How can you access a class variable in Python?

	a) Only from within the class body b) Only through class methods c) Only using an instance of the class d) Using either the class name or an instance of the class
10	Why should class variables be used with care? a) Because they are private by default and inaccessible b) Because they always increase memory usage compared with instance variables c) Because they cannot be changed after class creation d) Because an unintended modification can affect every instance of the class

4. INSTANCE VARIABLES

In object-oriented programming, data associated with a class can be categorised as either class variables or instance variables. While class variables are shared among all objects of a class, instance variables are unique to each object. They provide a mechanism for storing object-specific data, allowing each object to maintain its own state. In Python, instance variables are crucial for creating objects with distinct characteristics and behaviour.

Understanding instance variables is essential for designing classes that model real-world entities, as each object often needs to hold its own information independent of other objects. This section explores the meaning, initialisation, usage, and distinctions of instance variables.

4.1 Meaning of Instance Variables

An instance variable is a variable that is defined within a class and belongs specifically to an object. It represents the attributes or properties of an individual object and is used to store information that varies from one object to another.

Key characteristics of instance variables:

- **Object-specific:** Each object has its own copy of the instance variables, so changes in one object do not affect others.
- **Defined inside methods:** Instance variables are typically defined within the constructor method `__init__` using the `self` keyword.
- **Encapsulation:** Instance variables help encapsulate the state of an object, ensuring that each object can maintain its own data independently.

Example:

```
class Book:
    def __init__(self, title, author, pages):
        self.title = title # instance variable
        self.author = author # instance variable
        self.pages = pages # instance variable
```

Here, `title`, `author`, and `pages` are instance variables. Each `Book` object created from this class will have its own values for these variables, allowing objects to store individual data.

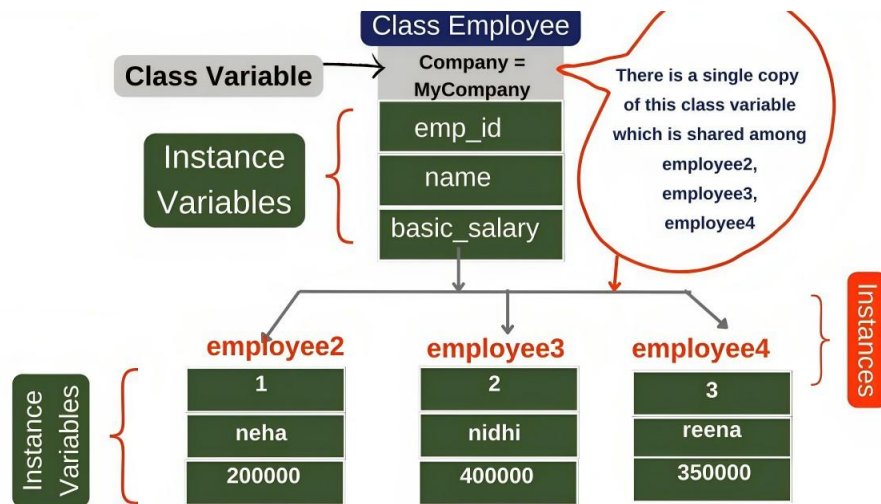


Figure 3: Instance Variables and Class Variables

Source: https://media.licdn.com/dms/image/v2/D5612AQHV66zfYxtSIw/article-cover_image-shrink 720 1280/article-cover image-shrink 720 1280/0/1697370358247?e=2147483647&v=beta&t= etJM7vJsQv0g94lh2VQC6Jt4EzO DlsydrNLU12L05o

The figure illustrates the key difference between a class variable and instance variables in Object-Oriented Programming (OOP), as often implemented in Python. The Class Employee blueprint defines the structure, including the class variable `$Company = 'MyCompany'`. Crucially, only a single copy of this class variable exists, and it is shared across all instances (objects) created from the class, such as *employee2*, *employee3*, and *employee4*. Conversely, instance variables, such as *emp_id*, *name*, and *basic_salary*, are specific to each instance or object. Each employee object (e.g., *employee2*) holds its own unique copy of these variables with distinct values (e.g., name: 'neha', salary: 200000), meaning the data is not shared and can be independently modified without affecting other instances.

4.2 Initialising Instance Variables

Instance variables are usually initialised within the constructor method `__init__`. The constructor is a special method that is automatically called when an object is created. Initialisation ensures that each object starts with a defined state.

Syntax:

```
class ClassName:
    def __init__(self, attribute1, attribute2):
        self.attribute1 = attribute1
        self.attribute2 = attribute2
```


Example:

```
class Student:
    def __init__(self, name, age, grade):
        self.name = name # instance variable
        self.age = age    # instance variable
        self.grade = grade # instance variable
```

```
# Creating objects
student1 = Student("Alice", 14, "8th Grade")
student2 = Student("Bob", 15, "9th Grade")
```

```
print(student1.name) # Output: Alice
print(student2.grade) # Output: 9th Grade
```

In this example, `self.name`, `self.age`, and `self.grade` are instance variables that store data specific to each object created from the class. The constructor parameters `name`, `age`, and `grade` make it possible to pass values while creating new objects, ensuring that each object is initialised with its own data. Since every object maintains an independent copy of its instance variables, any changes made to `student1` do not affect the data stored in `student2`.

Dynamic creation of instance variables: In Python, it is also possible to create instance variables outside the constructor.

```
student1.section = "A" # dynamically adding an instance variable
print(student1.section) # Output: A
```

Although this is allowed, it is recommended to initialise all instance variables inside the constructor to maintain clarity and consistency.

4.3 Object-Specific Data Storage

The primary purpose of instance variables is to store object-specific data. This allows each object to have its own unique attributes while still belonging to the same class. Object-specific storage is essential when modelling real-world entities where each instance has distinct properties.

Example: Car objects

```
class Car:
    def __init__(self, make, model, year):
        self.make = make
        self.model = model
        self.year = year
```

```
car1 = Car("Toyota", "Corolla", 2023)
car2 = Car("Honda", "Civic", 2022)
```

Here, car1 and car2 are two objects of the Car class. Each has its own values for make, model, and year. This allows the program to track each car separately, demonstrating the importance of object-specific data.

Accessing and modifying instance variables:

```
print(car1.model) # Output: Corolla
car2.year = 2023
print(car2.year) # Output: 2023
```

Changes to instance variables affect only the object whose data is modified. This encapsulation ensures that objects operate independently, a fundamental principle of object-oriented programming.

Common use cases of instance variables:

- **Storing personal data:** Names, ages, or contact details of individuals.
- **Tracking product information:** Product name, price, and quantity for each product object.
- **Maintaining object state:** Current progress in a game, balance in a bank account, or location of a vehicle.
- **Dynamic attributes:** Instance variables can be updated as the object interacts with the program.

4.4 Difference Between Class and Instance Variables

While class variables and instance variables are both part of a class, they serve different purposes and behave differently. Understanding the differences is crucial for proper object-oriented design. **Example demonstrating the difference:**

```
class Employee:
    company = "Tech Solutions" # class variable

    def __init__(self, name, salary):
        self.name = name # instance variable
        self.salary = salary # instance variable

emp1 = Employee("John", 50000)
emp2 = Employee("Alice", 60000)

# Access class variable
print(emp1.company) # Tech Solutions
```

```
print(emp2.company)    # Tech Solutions

# Modify instance variable
emp1.salary = 55000
print(emp1.salary)    # 55000
print(emp2.salary)    # 60000

# Modify class variable
Employee.company = "Global Tech"
print(emp1.company)    # Global Tech
print(emp2.company)    # Global Tech
```

This example clearly shows that class variables are shared, whereas instance variables remain unique to each object.

When designing classes in Python, it is important to follow certain best practices to ensure clarity and maintainability. Class variables should be used only for data that is truly common to all objects, such as shared configuration values or constants. Any data that is specific to an individual object should be stored in instance variables to maintain independent object states. Modifying class variables through objects should be avoided, as this can unintentionally create instance variables with the same name and lead to confusing behaviour. To improve readability and consistency, all instance variables should be initialised within the constructor, making it clear what data each object is expected to hold.

Instance variables are fundamental in object-oriented programming for storing object-specific data. They are defined within the constructor using the `self` keyword and allow each object to maintain its own state independently of other objects. Proper use of instance variables ensures encapsulation, clarity, and flexibility in program design. The distinction between instance and class variables must be understood to prevent errors and to structure Python programs effectively.

SELF-ASSESSMENT QUESTIONS – 3

Multiple Choose Questions	
11	What best describes an instance variable in Python? a) A variable shared across all instances of a class b) A variable declared outside the class body c) A variable unique to each object, usually defined in the <code>init</code> method using <code>self</code> d) A variable that cannot be changed after the object is created
12	Where are instance variables usually initialised?

	a) Inside the constructor (init) using self b) At the top level of the module c) Using the @staticmethod decorator d) Directly on the class body, outside any method
13	If you change an instance variable for one object, what happens? a) The change is reflected in every instance of the class b) Only that particular object is affected c) Python converts the instance variable into a class variable d) An exception is raised
14	Which statement about dynamically adding instance variables outside the constructor is correct? a) It is not permitted in Python b) It is preferable to always add them dynamically rather than in init c) Doing so creates or modifies a class variable instead d) It is allowed but it is clearer and more maintainable to initialise instance variables in init
15	Which scenario is a typical use case for instance variables? a) Storing settings that should be identical for every object b) Keeping a global counter of created instances c) Holding object-specific data such as a product's name or price d) Sharing large immutable data to save memory

INSPIRED BY LIFE

5. USING CLASSES IN PYTHON PROGRAMS

Object-oriented programming in Python provides a framework for creating structured and modular programs. Classes serve as the blueprint for objects, encapsulating both data and behaviour. This section focuses on practical usage of classes in Python programs, including creating multiple objects, understanding data encapsulation, modelling real-world scenarios, and following best practices in class design.

Using classes effectively allows programmers to manage complexity, improve code readability, and ensure reusability of components. In Python, classes are not just theoretical constructs—they are actively used to model real-world entities, manage shared and unique data, and organise program logic.

5.1 Creating Multiple Objects

A single class can be used to create multiple objects, each with its own set of attributes and behaviour. This is one of the core benefits of object-oriented programming: the ability to generate as many instances of a class as needed without duplicating code.

Creating multiple objects involves calling the constructor method (`__init__`) of a class repeatedly. Each call produces a distinct object with its own memory allocation for instance variables.

Example:

```
class Laptop:
    def __init__(self, brand, model, price):
        self.brand = brand
        self.model = model
        self.price = price

# Creating multiple objects
laptop1 = Laptop("Dell", "Inspiron", 600)
laptop2 = Laptop("HP", "Pavilion", 750)
laptop3 = Laptop("Apple", "MacBook Air", 1200)

# Accessing object data
print(laptop1.brand)  # Output: Dell
print(laptop2.price)  # Output: 750
print(laptop3.model)  # Output: MacBook Air
```

In this example, `Laptop` serves as a template. Objects `laptop1`, `laptop2`, and `laptop3` each have unique attributes while sharing the same methods defined in the class.

When multiple objects are created from the same class, each object remains independent and maintains its own set of instance variables. Any method called on an object works with the data specific to that object unless the method explicitly accesses a class variable shared across all instances. At the same time, memory usage remains efficient because the class definition and its methods are stored only once, while each object stores only its individual data values.

Practical applications:

- Creating multiple user accounts in a software system.
- Managing inventory items in an e-commerce application.
- Tracking multiple vehicles in a transportation system.

5.2 Data Encapsulation Basics

Encapsulation is a core concept in object-oriented programming. It involves restricting access to certain parts of an object's data and only exposing necessary functionality. In Python, encapsulation is achieved by defining private and public attributes and methods.

1. **Public attributes and methods:** Accessible from outside the class. By default, all attributes and methods in Python are public.

```
class Employee:
    def __init__(self, name, salary):
        self.name = name # public attribute
        self.salary = salary # public attribute

    def display_info(self):
        print(f"Name: {self.name}, Salary: {self.salary}")
```

2. **Private attributes and methods:** Not accessible directly from outside the class. In Python, a leading double underscore (`__`) is used to indicate a private variable or method.

```
class BankAccount:
    def __init__(self, account_number, balance):
        self.__account_number = account_number # private attribute
        self.__balance = balance # private attribute

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance
```

In this example, `__balance` and `__account_number` are private attributes. They cannot be accessed directly from outside the class:

```
account = BankAccount("12345", 1000)
print(account.__balance)  # Error: AttributeError
print(account.get_balance())  # Output: 1000
```

Encapsulation offers several important benefits in object-oriented programming. It helps protect object data from unintended or unauthorised changes by restricting direct access to internal attributes. By providing controlled ways to access or modify data through methods, encapsulation ensures that objects behave in a predictable and consistent manner. It also supports the principle of data hiding, which is essential for building secure, reliable, and well-structured software systems.

Accessor and mutator methods: Also called getters and setters, these methods allow controlled access to private attributes.

```
class Employee:
    def __init__(self, name, salary):
        self.__name = name
        self.__salary = salary

    # Getter
    def get_name(self):
        return self.__name

    # Setter
    def set_salary(self, salary):
        if salary > 0:
            self.__salary = salary
        else:
            print("Invalid salary")
```

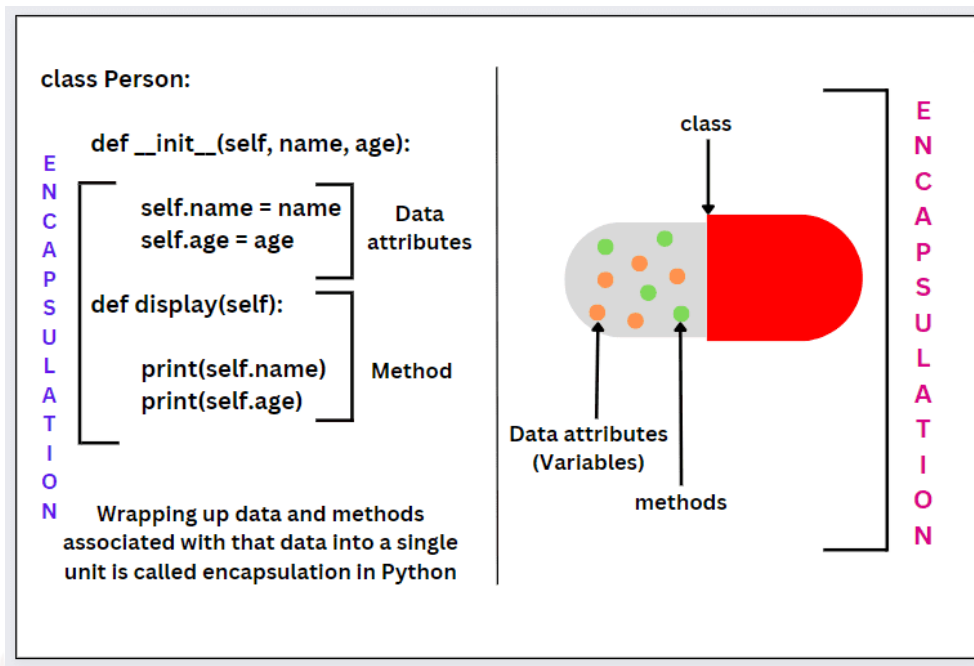



Figure 4: Encapsulation in Python

Source: <https://www.scientecheasy.com/wp-content/uploads/2023/10/python-encapsulation-example.png>

The figure illustrates encapsulation, a core principle of Object-Oriented Programming (OOP), typically demonstrated using the structure of a Python class. This figure visually represents the concept as a capsule, where the data attributes (variables) and the methods (functions) that operate on that data are wrapped together into a single unit—the class, such as class Person. The key mechanism is that the data and its associated behaviours are bundled, with the methods providing the defined interface for accessing and manipulating the data. This process effectively hides the internal implementation details of the object and prevents direct, uncontrolled modification of the data from outside the class, thereby promoting data integrity and modularity in the code.

5.3 Real-World Modelling with Classes

One of the most powerful applications of classes in Python is modelling real-world entities. By combining attributes (data) and methods (behaviour) in a single class, programmers can simulate real-life objects and interactions within software.

Example: Modelling a Library System

```

class Book:
    def __init__(self, title, author, copies):
        self.title = title
  
```

```
        self.author = author
        self.copies = copies

    def borrow(self):
        if self.copies > 0:
            self.copies -= 1
            print(f"{self.title} borrowed successfully")
        else:
            print(f"{self.title} is not available")

    def return_book(self):
        self.copies += 1
        print(f"{self.title} returned successfully")

# Creating objects
book1 = Book("1984", "George Orwell", 3)
book2 = Book("To Kill a Mockingbird", "Harper Lee", 2)

# Borrowing and returning
book1.borrow() # Output: 1984 borrowed successfully
book2.borrow() # Output: To Kill a Mockingbird borrowed successfully
book1.return_book() # Output: 1984 returned successfully
```

In this scenario, each Book object stores its own set of data such as the title, author, and number of available copies, ensuring that information is maintained separately for every book. The borrow and return_book methods operate on object-specific data, updating the values associated with the particular book instance on which they are called. By creating multiple objects from the Book class, the library system is able to track the status and availability of each book independently.

Other examples of real-world modelling:

- **Banking systems:** Modelling accounts, transactions, and balances.
- **E-commerce platforms:** Modelling products, customers, and orders.
- **School management systems:** Modelling students, teachers, and classes.
- **Transport management:** Modelling vehicles, routes, and drivers.

Using classes for real-world modelling promotes code clarity, modularity, and scalability. It allows complex systems to be broken down into smaller, manageable components.

5.4 Best Practices in Class Design

Effective class design is critical for writing maintainable, readable, and reusable code. Poorly designed classes can lead to code duplication, errors, and difficulties in extending functionality. The following best practices should be observed when creating classes in Python:

- **Follow naming conventions:**
 1. Class names should use PascalCase (Book, EmployeeDetails).
 2. Attributes and methods should use snake_case (borrow_book, account_balance).
- **Keep classes focused:**
 1. A class should represent a single concept or entity. Avoid creating classes that try to do too much.
 2. For example, a Book class should not manage library members; a separate Member class should be created.
- **Use constructors effectively:**
 1. Initialise all necessary instance variables in the `__init__` method to ensure consistent object state.
 2. Avoid dynamically creating instance variables outside the constructor unless necessary.
- **Encapsulation and data hiding:**
 1. Keep sensitive attributes private using double underscores (`__`).
 2. Provide accessor and mutator methods for controlled access.
- **Avoid excessive class variables:**
 1. Use class variables only for truly shared data, such as constants or counters.
 2. Object-specific data should always be stored in instance variables.
- **Document classes and methods:**
 1. Use docstrings to explain the purpose of the class and its methods.
 2. This improves code readability and maintainability.

```
class Student:  
    """
```

Represents a student with name, age, and grade.
Methods allow updating and retrieving student details.

```
"""
```

```
def __init__(self, name, age, grade):
```

```
    self.name = name
```

```
    self.age = age
```

```
    self.grade = grade
```

```
def display_info(self):
```

```
    """Displays the student's information"""
```

```
    print(f"Name:      {self.name},      Age:      {self.age},      Grade: {self.grade}")
```

- **Reusability through modular design:**

- Break complex systems into multiple classes that interact with each other.
- Promote code reuse by defining methods that can be called by multiple objects.

- **Avoid deep inheritance hierarchies:**

- Prefer composition over inheritance where possible to simplify relationships.
- Only use inheritance when a clear “is-a” relationship exists between classes.

- **Use meaningful attribute and method names:**

- Names should clearly indicate their purpose. For instance, `calculate_salary` is preferable to `calc1`.

Following best practices in object-oriented programming makes code easier to debug and maintain, as the structure and intent of each class are clear and well organised. It helps reduce the chances of errors and unintended side effects by encouraging disciplined use of variables and methods. Well-designed code is also easier for other developers to read and understand, especially when they work on the project in the future. Over time, this structured approach supports scalability, allowing larger applications to grow and evolve without becoming difficult to manage.

Using classes in Python allows programmers to create structured, modular, and scalable programs. Multiple objects can be created from a single class, each with its own data, while methods encapsulate behaviour. Data encapsulation ensures that object data is protected and accessed in a controlled manner. Classes enable real-world modelling, from banking systems to library management, and allow complex systems to be broken down into manageable components. Following best practices in class design, including proper naming, constructor usage, encapsulation, and modularity, ensures

code maintainability, readability, and reusability. By adhering to these principles, Python programmers can create efficient, reliable, and flexible software systems.

SELF-ASSESSMENT QUESTIONS – 4

Multiple Choose Questions	
16	What is a primary advantage of creating multiple objects from a single class? a) It forces each object to share the same state permanently b) It reduces code duplication by reusing the class blueprint for different instances c) It prevents the use of instance methods d) It makes the programme run without a constructor
17	What is the main goal of encapsulation in class design? a) To allow unrestricted direct access to all attributes b) To hide internal details and provide controlled access through methods c) To ensure every attribute is a class variable d) To increase coupling between classes
18	Which practice is recommended when initialising instance variables? a) Initialise them inside the constructor (init) to keep object state consistent b) Always create instance variables dynamically outside the constructor for flexibility c) Use class variables for data that differs between instances d) Avoid using the self keyword to reduce verbosity
19	Which example best illustrates modelling a real-world system with classes? a) Using a Book class with title, author and copies, plus borrow and return methods b) Storing all data in global variables and using standalone functions to manipulate them c) Creating only static methods for everything and never instantiating objects d) Writing long procedural scripts without classes or objects
20	Which of the following is a recommended guideline for class design? a) Use deep inheritance hierarchies wherever possible to reuse code b) Keep classes focused on a single responsibility and prefer composition over complex inheritance c) Avoid docstrings because they make the code verbose d) Store frequently changing, object-specific data as class variables

6. SUMMARY

- Classes in Python act as blueprints that define both data and behaviour for the objects created from them.
- A single class can be used to create many objects, each with its own independent set of attributes.
- The constructor method is responsible for initialising instance variables each time a new object is created.
- Using multiple objects helps organise program logic by keeping related data and actions grouped together.
- Data encapsulation protects internal attributes by restricting direct access and allowing controlled interaction through methods.
- Python marks private attributes using two leading underscores, preventing accidental modification from outside the class.
- Real-world modelling with classes allows systems such as libraries, banking platforms, and e-commerce applications to be represented in code.
- Methods inside a class operate on the data stored within each object, keeping behaviour consistent and predictable.
- Good class design involves clear naming, focusing on one responsibility, proper use of constructors, and keeping sensitive data private.
- Following structured class design principles supports scalability, readability, and ease of maintenance in larger software systems.

7. GLOSSARY

Class	-	A template used to define the structure and behaviour of objects in Python.
Object	-	An instance created from a class, holding its own data and methods
Constructor	-	The init method that sets up instance variables when an object is created.
Instance variable	-	Data that belongs to a specific object rather than the class itself.
Class variable	-	A variable shared across all objects created from a class.
Encapsulation	-	The practice of hiding internal data and controlling access through methods.
Private attribute	-	An attribute marked with two leading underscores to prevent direct access.
Getter	-	A method that returns the value of a private attribute.
Setter	-	A method that updates the value of a private attribute in a controlled way.
Composition	-	Building complex systems by combining simple classes rather than relying on deep inheritance.

8. TERMINAL QUESTIONS

1. Explain the difference between a class and an object in Python, using an example.
2. Describe the role of the init method and explain why the self parameter is necessary.
3. Compare and contrast class variables and instance variables, and give an example where each would be appropriate.
4. How do @classmethod and @staticmethod differ in purpose and usage? Provide code examples to illustrate.
5. What is encapsulation and how does Python support data hiding for attributes? Discuss pros and cons of Python's name-mangling approach.
6. Define polymorphism in object-oriented programming and show a Python example where two different classes implement the same method name with different behaviour.
7. When should you prefer composition over inheritance? Give a real-world scenario that favours composition.
8. Explain how you would implement and use accessor (getter) and mutator (setter) methods in Python, and discuss modern alternatives (for example, property).
9. Outline best practices for designing a class in Python, including naming, constructor usage and documentation.
10. Give a short design for a simple library system (classes and key methods) and explain how objects would interact to borrow and return books.

9. ANSWERS

9.1. Self-Assessment Answers:

1. c) To initialise instance attributes when an object is created
2. Abstraction
3. b) They are shared across all instances of the class
4. a) A classmethod receives cls and can access class attributes; a staticmethod receives no implicit first argument and cannot access class or instance data
5. c) Two different classes implementing a method with the same name but different behaviour
6. a) It is shared across all instances of the class
7. b) A new instance variable with the same name is created for that object, leaving the class variable unchanged
8. c) Keeping a count of how many instances of the class have been created
9. d) Using either the class name or an instance of the class
10. d) Because an unintended modification can affect every instance of the class
11. c) A variable unique to each object, usually defined in the init method using self
12. a) Inside the constructor (init) using self
13. b) Only that particular object is affected
14. d) It is allowed but it is clearer and more maintainable to initialise instance variables in init
15. c) Holding object-specific data such as a product's name or price
16. b) It reduces code duplication by reusing the class blueprint for different instances
17. b) To hide internal details and provide controlled access through methods
18. a) Initialise them inside the constructor (init) to keep object state consistent
19. a) Using a Book class with title, author and copies, plus borrow and return methods
20. b) Keep classes focused on a single responsibility and prefer composition over complex inheritance

9.2. Terminal Question Answers

Answer 1: A class is a blueprint that defines attributes and methods, while an object is a concrete instance created from that blueprint holding its own data; for example, a Car class defines make and model, and `my_car = Car("Toyota", "Corolla", 2023)` is an object with those values. Refer: Sections 2.2 and 2.3.

Answer 2: The `__init__` method is the constructor that initialises an object's instance attributes when the object is created; the `self` parameter refers to that particular instance so methods can read and modify the object's own data. Refer: Section 2.2.

Answer 3: Class variables are declared in the class body and are shared by all instances, while instance variables are created per object (usually in `__init__`) and hold object-specific data; use a class variable for a bank name and an instance variable for an account balance. Refer: Sections 3.1–3.2 and 4.4.

Answer 4: A `@classmethod` receives `cls` and can access or modify class-level data, whereas a `@staticmethod` receives no implicit first argument and behaves like a function placed in the class namespace; for example, `@classmethod` `def set_default(cls, v): cls.default = v` and `@staticmethod` `def util(x): return x*2`. Refer: Section 2.4.

Answer 5: Encapsulation means restricting direct access to an object's internals and exposing a controlled interface; Python supports this with naming conventions and name-mangling (double underscores) which helps prevent accidental access but does not enforce strict privacy, since mangled names remain reachable if required. Refer: Section 5.2.

Answer 6: Polymorphism is the ability for different object types to respond to the same method name in their own way; for example, two classes can each implement `speak()`, and a loop over `[Dog(), Cat()]` calling `speak()` invokes the appropriate behaviour for each. Refer: Sections 2.1 and 2.4.

Answer 7: Prefer composition when you want a “has-a” relationship or to avoid complex inheritance hierarchies—for instance, a `Car` should contain an `Engine` object rather than inherit from `Engine`—because composition keeps responsibilities separate and simplifies maintenance. Refer: Section 5.4.

Answer 8: Implement getters and setters as methods that validate and control access (for example `get_name()` and `set_salary()` with checks), and prefer the property decorator as a modern alternative so callers use attribute syntax while the class retains control. Refer: Section 5.2 (accessor and mutator methods).

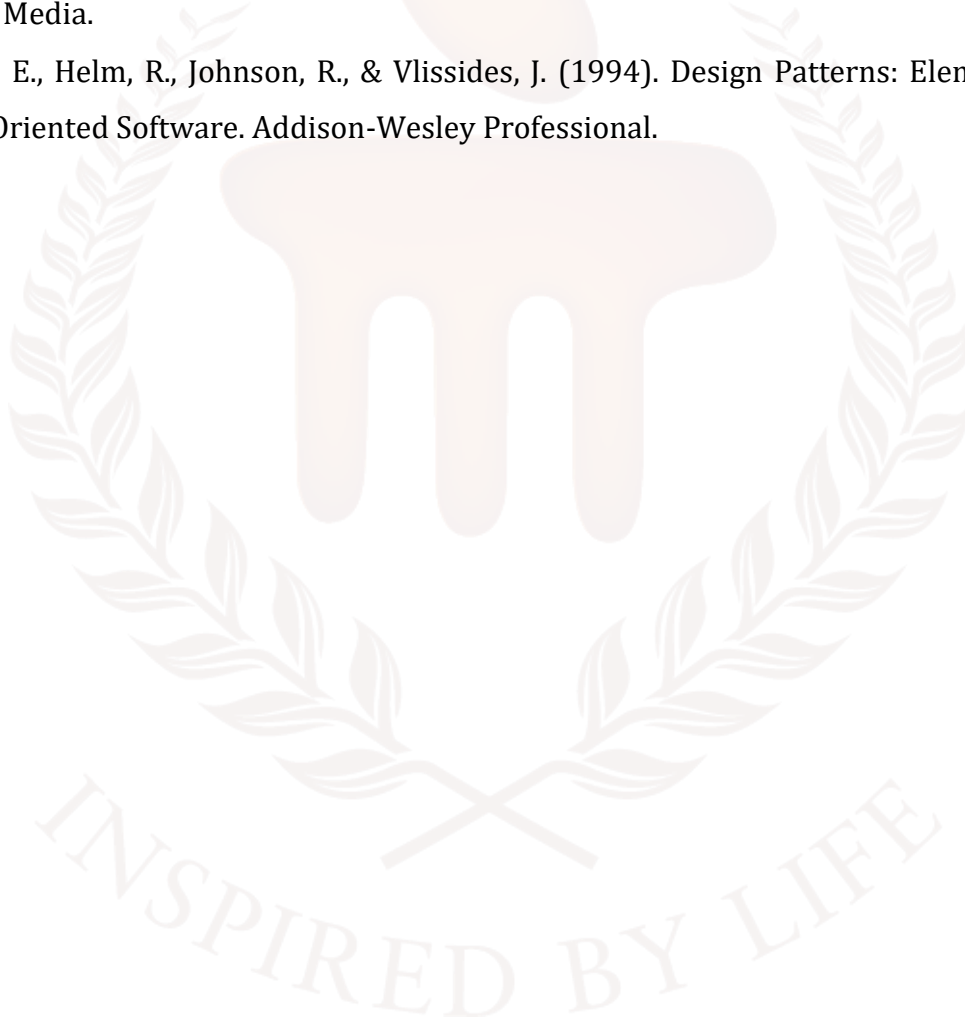
Answer 9: Best practices include clear `PascalCase` for class names and `snake_case` for methods/attributes, initialising all instance variables in `__init__`, keeping each class focused on a single responsibility, using docstrings, favouring composition over deep inheritance and avoiding excessive class variables. Refer: Section 5.4.

Answer 10: A simple design: a Book class (title, author, copies with borrow() and return_book()), a Member class (id, name) and a Library class that manages a catalogue and mediates borrow/return operations; a Member requests a borrow via Library, the Library checks Book.copies and calls book.borrow() to decrement copies and record the transaction. Refer: Section 5.3 (example: Book class and borrow/return methods).



10. REFERENCES

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- Ramalho, L. (2022). Fluent Python (2nd ed.). O'Reilly Media.
- Phillips, D. (2015). Python 3 Object-Oriented Programming (2nd ed.). Packt Publishing.
- McLaughlin, B., Pollice, G., & West, D. (2006). Head First Object-Oriented Analysis and Design. O'Reilly Media.
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley Professional.



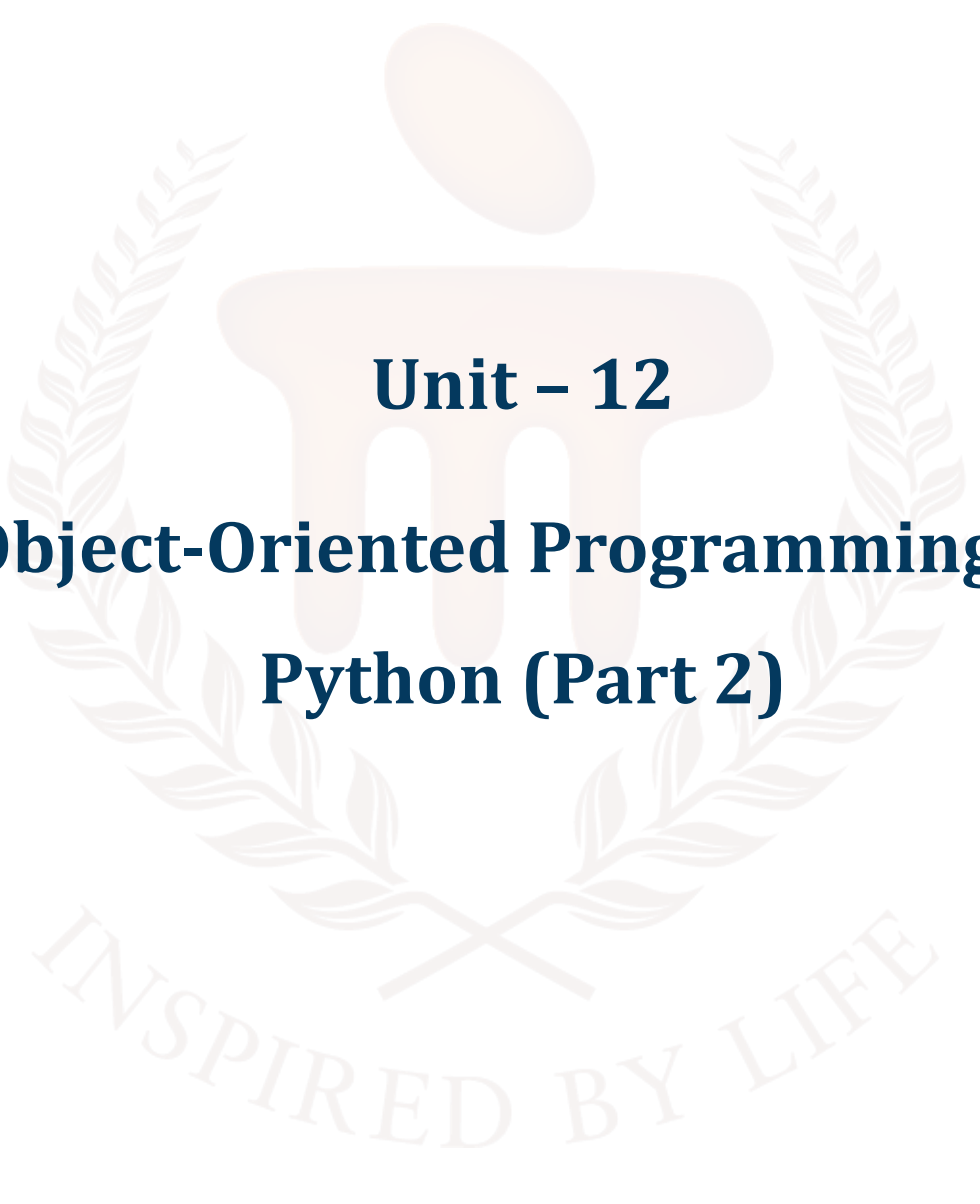
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 12

Object-Oriented Programming in Python (Part 2)

TABLE OF CONTENTS

Sl No	Topic	Fig No / Table / Graph	Saq / Activity	Page No
1	Introduction	-	-	4
	1.1 Learning Objectives	-	-	
2	Inheritance in Python	-	1	5-15
	2.1 Concept of Inheritance	1	-	
	2.2 Types of Inheritance	2 TO 6	-	
	2.3 Creating Base and Derived Classes	-	-	
	2.4 Method Overriding	-	-	
3	Polymorphism in Python	-	2	16-24
	3.1 Polymorphism in Python	7	-	
	3.2 Function Overloading Behaviour	-	-	
	3.3 Method Overriding in Polymorphism	-	-	
	3.4 Operator Overloading Basics	-	-	
4	Super Function, Inheritance and Polymorphism in Practice	-	3	25-34
	4.1 Purpose of super()	-	-	
	4.2 Method Resolution Order	8	-	
	4.3 Using super() in Inheritance	-	-	
	4.4 Code Reusability and Real-World Modelling	-	-	
	4.5 Best Practices in Object Oriented Design	-	-	
5	Summary	-	-	35
6	Glossary	-	-	36
7	Terminal Questions	-	-	37
	Answers	-	-	38-40
8	8.1 Self-Assessment Answers	-	-	
	8.2 Terminal Questions Answers	-	-	
9	References	-	-	41

1. INTRODUCTION

The previous chapter introduced the fundamentals of object oriented programming in Python, focusing on classes, objects, and the use of class and instance variables. You learnt how to define classes, create objects, and differentiate between shared data (class variables) and object-specific data (instance variables). These concepts provided the foundation for structuring programs in a modular and reusable way.

This chapter builds on that knowledge by exploring inheritance, which allows classes to extend the properties and behaviours of other classes. You will learn about different types of inheritance and how derived classes can override or extend the functionality of base classes. These techniques enable more efficient code reuse and hierarchical modelling.

The chapter also covers polymorphism, which allows the same operation to behave differently depending on the object it acts upon. Topics include method overriding, operator overloading, and using the `super()` function to access base class methods. These features enhance flexibility and enable developers to design programs that can adapt to different object types.

To study this chapter effectively, practise creating base and derived classes, and experiment with method overriding and polymorphic behaviour in small programs. Implement real-world examples to see how inheritance and polymorphism simplify code maintenance and extension. Regular coding exercises will reinforce the concepts and improve problem-solving skills.

1.1. Learning Objectives

By the end of this Unit, you will be able to:

- Apply inheritance to extend class functionality
- Implement method overriding in derived classes
- Use polymorphism to handle objects of different types
- Utilise the `super()` function for base class access
- Analyse and design class hierarchies for code reuse



2. INHERITANCE IN PYTHON

Inheritance is a fundamental concept in object-oriented programming (OOP) that allows a new class to acquire the properties and methods of an existing class. This mechanism promotes code reusability, reduces redundancy, and provides a clear hierarchical relationship among classes. Python, being a fully object-oriented language, supports multiple types of inheritance, enabling programmers to create complex relationships between classes efficiently.

Inheritance allows a derived class (also called a child or subclass) to inherit attributes and methods from a base class (also called a parent or superclass). This means that the derived class can use existing functionality while also introducing its own unique features or modifying inherited behaviour.

2.1 Concept of Inheritance

In essence, inheritance represents an “is-a” relationship between classes. The derived class is a specialised version of the base class, inheriting all its data members and functions. Inheritance promotes the following advantages:

- **Code Reusability:** Common functionality defined in a base class does not need to be rewritten for each subclass.
- **Extensibility:** New functionality can be added to subclasses without altering the base class.
- **Organisation and Hierarchy:** Classes can be structured in a hierarchical manner, reflecting real-world relationships.
- **Polymorphism Enablement:** Inheritance allows methods to be overridden in derived classes, enabling different behaviour while maintaining a consistent interface.

Consider a software system designed to manage different types of vehicles to understand how inheritance is applied in practice. A base class named `Vehicle` can be used to define common attributes such as `speed` and `fuel_type`, along with shared methods like `start()` and `stop()` that apply to all vehicles. From this base class, derived classes such as `Car` and `Motorbike` can inherit these shared features while also introducing their own specific attributes, such as `number_of_doors` for cars or `helmet_required` for motorbikes, allowing each vehicle type to be modelled accurately within the same system.

Example in Python:

```
class Vehicle:
    def __init__(self, speed, fuel_type):
        self.speed = speed
        self.fuel_type = fuel_type

    def start(self):
        print("Vehicle started")

    def stop(self):
        print("Vehicle stopped")

class Car(Vehicle):
    def __init__(self, speed, fuel_type, number_of_doors):
        super().__init__(speed, fuel_type)
        self.number_of_doors = number_of_doors

    def display_info(self):
        print(f"Car speed: {self.speed}, Fuel: {self.fuel_type}, Doors: {self.number_of_doors}")
```

In this example, the Car class inherits the attributes and methods defined in the Vehicle class, allowing it to reuse common vehicle-related functionality. At the same time, it extends the base class by introducing a new attribute, `number_of_doors`, along with a method `display_info()`, which is used to present details specific to a car.

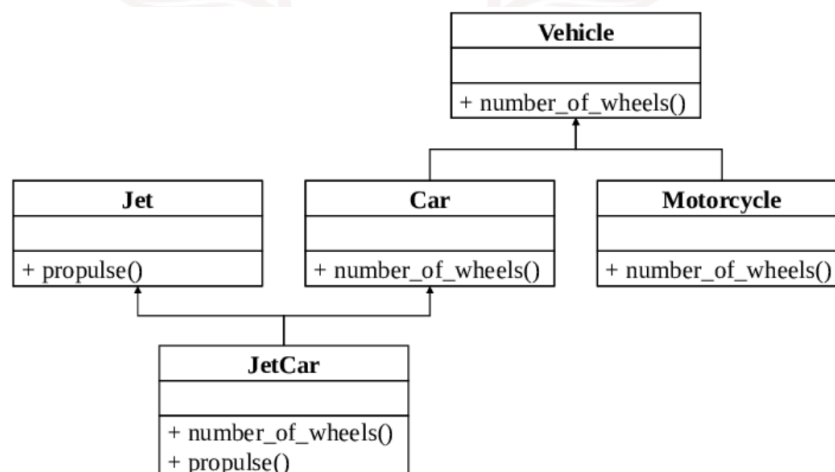


Figure 1: Inheritance

Source: <https://www.researchgate.net/profile/Felipe-R-Monteiro/publication/353053664/figure/fig1/AS:1043075489091586@1625699984441/ehicle-class-hierarchy-UML-diagram.png>

This figure illustrates a class hierarchy diagram in object-oriented programming using UML notation, demonstrating the principles of inheritance and multiple inheritance. The abstract class `Vehicle` acts as the base class, specifying the common method `number_of_wheels()`, which is inherited by its direct subclasses, `Car` and `Motorcycle`. The `Jet` class, which is not directly derived from `Vehicle`, introduces the `propulse()` method. Crucially, the `JetCar` class inherits from both `Car` and `Jet`, thus possessing both the `number_of_wheels()` method from `Car` (and ultimately `Vehicle`) and the `propulse()` method from `Jet`, showcasing how a composite object's functionality is aggregated through multiple inheritance.

2.2 Types of Inheritance

Python supports multiple types of inheritance, each with specific applications and rules. Understanding these types helps in designing classes that are structured, maintainable, and flexible.

- **Single Inheritance:** In single inheritance, a derived class inherits from a single base class. It is the simplest form of inheritance.

Example:

```
class Parent:
    def greet(self):
        print("Hello from Parent")

class Child(Parent):
    def greet_child(self):
        print("Hello from Child")

obj = Child()
obj.greet()           # Inherited method
obj.greet_child()    # Child method
```

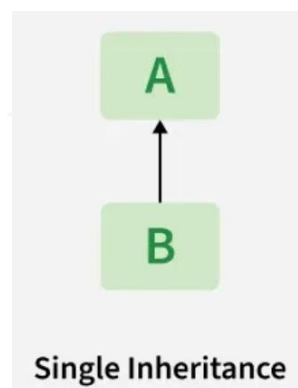


Figure 2: Single Inheritance

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20251009172725376159/SingleInheritance.webp>

The figure demonstrates the concept of single inheritance in object-oriented programming, which is a fundamental feature in Python. In this approach, a subclass such as Child inherits all attributes and methods from a single parent class, Parent, forming a simple and linear class hierarchy. This creates a clear is-a relationship, indicating that a Child is a type of Parent, and supports effective code reuse, as the Child object can directly use methods like greet() defined in the Parent class. Single inheritance forms the basis for well-structured class design and helps avoid the complexity and ambiguity that may arise in multiple inheritance scenarios.

- **Multiple Inheritance:** A derived class inherits from more than one base class. Python supports multiple inheritance directly, allowing a class to combine behaviours from multiple classes.

Example:

```
class Father:
    def skill_father(self):
        print("Father's skill: Gardening")

class Mother:
    def skill_mother(self):
        print("Mother's skill: Cooking")

class Child(Father, Mother):
    def skill_child(self):
        print("Child's skill: Drawing")

c = Child()
c.skill_father() # Output: Father's skill: Gardening
c.skill_mother() # Output: Mother's skill: Cooking
c.skill_child() # Output: Child's skill: Drawing
```

An important consideration in multiple inheritance is that when more than one base class defines a method with the same name, Python does not choose randomly. Instead, it follows the Method Resolution Order, which specifies the sequence in which classes are searched to determine which method should be invoked, ensuring consistent and predictable behaviour.

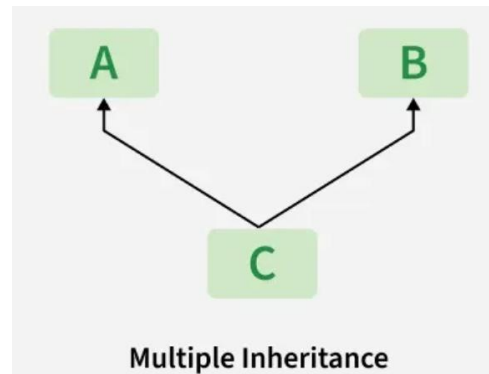


Figure 3: Multiple Inheritance

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20251009172626954371/MultipleInheritance.webp>

The figure demonstrates the concept of multiple inheritance in object-oriented programming, a feature that is fully supported in Python. In this approach, a single subclass such as Child can inherit attributes and methods from two or more separate superclasses, represented here by Father and Mother. As a result, an object of the Child class can access methods like `skill_father()` and `skill_mother()` in addition to its own method, `skill_child()`, bringing together functionality from both parent classes. Although multiple inheritance provides a flexible way to build complex and specialised classes by combining features, it is important to understand the Method Resolution Order, which defines the sequence Python follows to locate and execute methods when the same method name exists in more than one inherited class.

- **Multilevel Inheritance:** A class is derived from a base class, and another class is derived from the derived class, forming a hierarchy.

Example:

```
class Grandparent:
```

```
    def greet(self):
```

```
        print("Hello from Grandparent")
```

```
class Parent(Grandparent):
```

```
    def greet_parent(self):
```

```
        print("Hello from Parent")
```



```
class Child(Parent):  
  
    def greet_child(self):  
  
        print("Hello from Child")  
  
c = Child()  
  
c.greet()      # Inherited from Grandparent  
c.greet_parent() # Inherited from Parent  
c.greet_child() # Child method
```

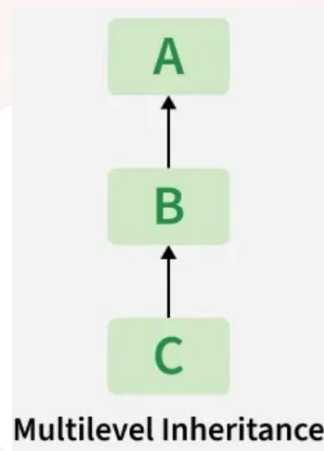


Figure 4: Multilevel Inheritance

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20251009172442620506/MultilevelInheritance.webp>

The figure illustrates the concept of multilevel inheritance in object-oriented programming, which is fully supported in Python. In this structure, the Child class inherits from the Parent class, and the Parent class in turn inherits from the Grandparent class, forming a sequential inheritance chain. As a result, an object of the Child class can access the attributes and methods defined in all preceding classes, such as calling `greet()` from the Grandparent class, `greet_parent()` from the Parent class, and its own `greet_child()` method. This approach allows developers to create increasingly specialised subclasses that build upon and extend existing functionality, leading to effective code reuse and well-organised class hierarchies.

- **Hierarchical Inheritance:** Multiple derived classes inherit from a single base class.

Example:

```
class Animal:
    def eat(self):
        print("Eating")

class Dog(Animal):
    def bark(self):
        print("Barking")

class Cat(Animal):
    def meow(self):
        print("Meowing")

dog = Dog()
cat = Cat()
dog.eat() # Output: Eating
cat.eat() # Output: Eating
```

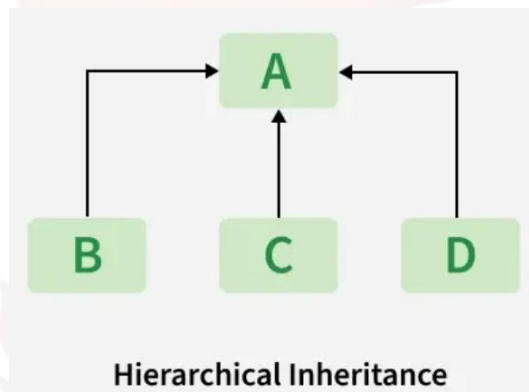


Figure 5: Hierarchical Inheritance

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20251009172317410936/HierarchicalInheritance.webp>

The figure demonstrates the concept of hierarchical inheritance in object-oriented programming, which is commonly used in Python. In this structure, a single base class, `Animal`, is inherited by multiple subclasses such as `Dog` and `Cat`. The base class defines common behaviour, for example the `eat()` method, which is inherited by all derived classes. At the same time, each subclass can introduce its own specific functionality, such as `bark()` for `Dog` or `meow()` for `Cat`, allowing shared behaviour to be reused while supporting specialised actions for each class within the same hierarchy.

- **Hybrid Inheritance:** Hybrid inheritance is a combination of two or more types of inheritance. For instance, a hierarchy that combines multilevel and multiple inheritance is hybrid.

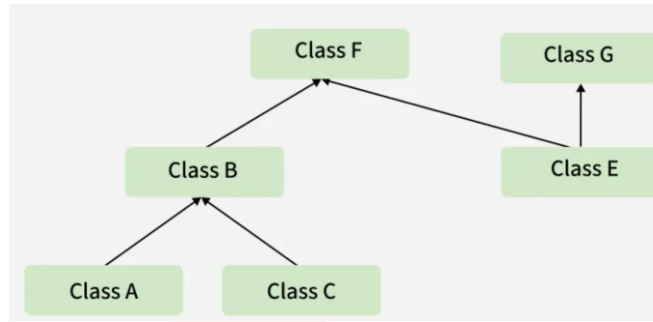


Figure 6: Hybrid Inheritance

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20251009172134171422/HybridInheritance.webp>

The figure represents a complex class structure known as hybrid inheritance, which combines multiple inheritance with other forms such as single or hierarchical inheritance within the same design. In this arrangement, Class B demonstrates hierarchical inheritance by acting as a parent to multiple classes like Class A and Class C, while Class F illustrates multiple inheritance by deriving features from both Class B and Class E. This blended structure makes it possible to achieve a high level of code reuse and to model complex real-world relationships where a class may need to inherit behaviour from different lineages. However, such complexity requires careful handling in Python, particularly with respect to the Method Resolution Order, to ensure that method calls are resolved in a clear and predictable manner.

2.3 Creating Base and Derived Classes

Creating base and derived classes in Python involves defining the parent class first and then creating subclasses using the syntax:

```
class DerivedClass(BaseClass):  
    # class body
```

Base Class Definition: A base class contains common attributes and methods that can be shared with derived classes.

Example:

```
class Employee:  
    def __init__(self, name, salary):  
        self.name = name  
        self.salary = salary  
  
    def display_info(self):
```

```
print(f"Name: {self.name}, Salary: {self.salary}")
```

Derived Class Definition: A derived class can inherit all attributes and methods of the base class. It can also introduce new attributes and methods or override existing ones.

Example:

```
class Manager(Employee):
    def __init__(self, name, salary, department):
        super().__init__(name, salary) # Call base class constructor
        self.department = department

    def display_manager_info(self):
        print(f"Name: {self.name}, Salary: {self.salary}, Department:
{self.department}")
```

Usage:

```
mgr = Manager("Alice", 90000, "HR")
mgr.display_info() # Inherited method
mgr.display_manager_info() # Derived class method
```

The `super()` function enables a derived class to invoke methods defined in its base class, making it easier to reuse existing functionality without rewriting code. Using inheritance, derived classes can safely extend or refine behaviour while keeping the original base class unchanged, which helps maintain clean and modular designs. Python also supports multiple inheritance, allowing a single derived class to inherit from more than one base class, though this flexibility requires careful design to avoid ambiguity in method resolution.

2.4 Method Overriding

Method overriding occurs when a derived class provides a new implementation for a method that is already defined in its base class. This allows the derived class to change or extend the behaviour of inherited methods.

Rules for method overriding:

1. The method name in the derived class must match the method name in the base class.
2. The derived class method is invoked when called on an object of the derived class.
3. The base class method can still be accessed using `super()`.

Example:

```
class Vehicle:
    def start(self):
```

```
print("Vehicle is starting")

class Car(Vehicle):
    def start(self):
        print("Car is starting")  # Overridden method

car = Car()
car.start()  # Output: Car is starting
```

Accessing base class method:

```
class Car(Vehicle):
    def start(self):
        super().start()  # Call base class method
        print("Car is starting")  # Additional functionality

car = Car()
car.start()
# Output:
# Vehicle is starting
# Car is starting
```

Method overriding enables polymorphism by allowing objects of different classes to respond to the same method call in their own specific way. It improves flexibility and extensibility, as new behaviour can be introduced in derived classes without altering existing code. By keeping the same method names across related classes, overriding also maintains a consistent interface while still supporting customised and class-specific behaviour.

Use cases:

- Implementing specific behaviour for subclasses while maintaining a general interface in the base class.
- Customising methods in GUI frameworks, where widgets inherit from a common base class.
- Defining specialised operations for different object types in games, simulations, or business applications.

Inheritance in Python is a powerful feature that allows classes to share data and behaviour, promoting code reuse, modularity, and maintainability. There are multiple types of inheritance, including single, multiple, multilevel, hierarchical, and hybrid, each useful in different programming contexts. Creating base and derived classes enables hierarchical modelling, where subclasses extend or modify the functionality of parent classes.

Method overriding further enhances inheritance by allowing derived classes to customise inherited methods while retaining a consistent interface. Together, inheritance and method overriding provide a strong foundation for object-oriented design, enabling programmers to build flexible, scalable, and real-world software systems.

SELF-ASSESSMENT QUESTIONS – 1

Multiple Choose Questions	
1	Which keyword is used in a subclass to call a method or constructor from its parent class? a) parent() b) super() c) base() d) init()
2	Which type of inheritance occurs when a class derives from more than one base class? a) Multilevel inheritance b) Hierarchical inheritance c) Multiple inheritance d) Single inheritance
3	What does MRO stand for in the context of Python multiple inheritance? a) Method Resolution Order b) Multiple Reachable Objects c) Method Return Optimisation d) Main Resolution Operator
4	If a subclass defines a method with the same name as one in its base class, which method is called when invoked on the subclass instance? a) The base class method b) The interpreter chooses at random c) The first method defined in the file d) The subclass method
5	Which inheritance type describes one base class being inherited by several derived classes? a) Hierarchical inheritance b) Hybrid inheritance c) Multiple inheritance d) Multilevel inheritance

3. POLYMORPHISM IN PYTHON

Polymorphism is a key concept in object-oriented programming that allows objects of different classes to be treated as objects of a common superclass. The term originates from Greek, meaning “many forms,” reflecting the ability of a single interface to represent multiple underlying forms. In Python, polymorphism enables functions, methods, and operators to operate on objects of different types while producing behaviour appropriate to the object’s actual class.

Polymorphism is central to the flexibility, maintainability, and scalability of Python programs. By enabling objects to respond differently to the same operation or method call, it supports code reuse and reduces complexity in large systems. This section explores the concept of polymorphism, function overloading, method overriding, and operator overloading in detail.

3.1 Concept of Polymorphism

At its core, polymorphism allows objects of different types to be used interchangeably while maintaining a consistent interface. It is closely linked to inheritance, as derived classes can override or extend the behaviour of base classes.

Key aspects of polymorphism in Python:

- **Object-level flexibility:** The same function or method call can produce different results depending on the type of the object.
- **Interface consistency:** Multiple classes can implement the same interface or method name, ensuring that code interacting with these objects does not need to know their specific types.
- **Reduced code duplication:** By designing functions and methods to work with multiple object types, the need for separate code for each type is eliminated.

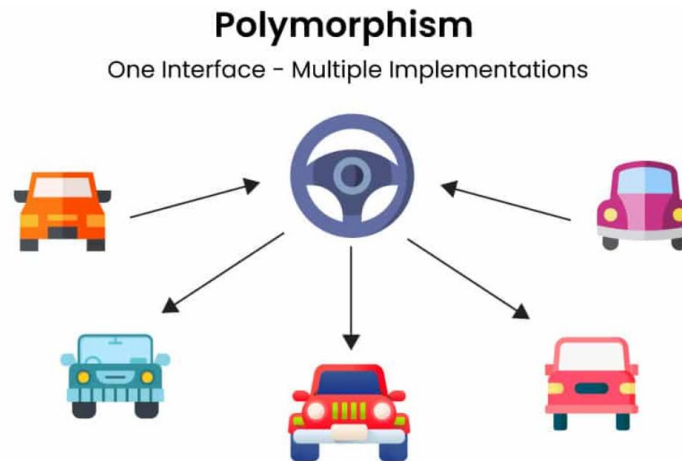


Figure 7: Polymorphism

Source: <https://intellipaat.com/blog/wp-content/uploads/2023/10/2-8.jpg>

The figure clearly illustrates polymorphism, a core concept in object-oriented programming that refers to the ability to take many forms. It represents a single interface, symbolised by the steering wheel and an action such as `drive()`, being implemented differently by various car objects. In Python, polymorphism allows the same function, method, or operator to work in different ways depending on the type of object it is applied to. This approach improves code reusability and flexibility, as developers can design general interfaces that work uniformly with objects from different classes without needing to know their internal implementation details.

In a drawing application, different shapes such as circles, squares, and rectangles can be represented using separate classes. A common method like `draw()` can be defined for all shape classes, allowing the same function call to be used for different objects. Each shape implements the `draw()` method in its own way, so when `draw()` is called, the object renders itself based on its specific shape, demonstrating polymorphism in practice.

Python example:

```
class Circle:
    def draw(self):
        print("Drawing a Circle")

class Square:
    def draw(self):
        print("Drawing a Square")

class Rectangle:
```

```
def draw(self):  
    print("Drawing a Rectangle")
```

```
shapes = [Circle(), Square(), Rectangle()]
```

```
for shape in shapes:  
    shape.draw()
```

Output:

Drawing a Circle

Drawing a Square

Drawing a Rectangle

In this example, the list shapes contains objects of different classes, yet each object responds to the same draw() method call. This is because each class defines its own implementation of draw(), allowing Python to decide at runtime which method to execute based on the object type. As a result, the same loop and method call produce different outputs, clearly illustrating polymorphism through dynamic method binding.

Benefits of polymorphism:

- Simplifies code by allowing one function to operate on multiple types.
- Enables dynamic behaviour in programs, adapting operations to the object type at runtime.
- Facilitates extensibility, as new classes can be added without modifying existing code.

3.2 Function Overloading Behaviour

Function overloading is a form of polymorphism where multiple functions share the same name but differ in the number or type of parameters. Traditional programming languages like C++ support function overloading directly. However, Python does not provide built-in support for function overloading in the conventional sense because Python functions are dynamically typed.

Simulating function overloading in Python:

- **Using default arguments:** Python allows functions to have default values, which enables a single function to handle different numbers of parameters.

```
def greet(name, message="Hello"):  
    print(f"{message}, {name}")
```

```
greet("Alice")           # Output: Hello, Alice
greet("Bob", "Good morning") # Output: Good morning, Bob
```

- **Using variable-length arguments:** Functions can accept any number of arguments using `*args` or `**kwargs`.

```
def add(*numbers):
    result = 0
    for num in numbers:
        result += num
    print(f"Sum: {result}")
```

```
add(2, 3)           # Output: Sum: 5
add(1, 2, 3, 4)     # Output: Sum: 10
```

- **Manual type checking:** Functions can determine the behaviour based on the type of input arguments.

```
def display(value):
    if isinstance(value, int):
        print(f"Integer: {value}")
    elif isinstance(value, str):
        print(f"String: {value}")
    else:
        print("Unknown type")
```

```
display(10)         # Output: Integer: 10
display("Hi")        # Output: String: Hi
```

Function overloading behaviour in Python is achieved through its dynamic typing system and the use of default values or variable-length arguments. Instead of defining multiple functions with the same name, Python allows a single function to handle different types or numbers of parameters at runtime. This form of polymorphism enables one function name to perform different operations based on the input provided, making the code simpler, more flexible, and easier to maintain.

Function overloading, or simulating it in Python using default or variable-length arguments, offers several advantages that enhance program design and readability. One major benefit is improved readability, as developers can use consistent function names for similar operations, making the code easier to follow and understand. Instead of creating multiple functions with different names for slightly different inputs, a single function can handle a variety of cases, providing a uniform interface.

This approach also increases flexibility, as the same function can process different types or numbers of arguments, adapting dynamically to the needs of the program. Additionally, function overloading reduces code duplication by consolidating related functionality into one function rather than

spreading similar logic across multiple functions. Overall, this technique supports clean, maintainable, and efficient coding practices, making Python programs more concise and easier to extend or modify.

3.3 Method Overriding in Polymorphism

Method overriding is a form of runtime polymorphism where a derived class provides a specific implementation of a method already defined in the base class. This allows objects of the derived class to respond differently to method calls while preserving a consistent interface.

Key rules:

- The method in the derived class must have the same name as in the base class.
- The derived class method is invoked for objects of the derived class.
- The base class method can still be accessed using `super()`.

Python example:

```
class Animal:
    def speak(self):
        print("Animal speaks")

class Dog(Animal):
    def speak(self):
        print("Dog barks")

class Cat(Animal):
    def speak(self):
        print("Cat meows")

animals = [Dog(), Cat(), Animal()]

for animal in animals:
    animal.speak()
```

Output:

Dog barks

Cat meows

Animal speaks

Each object calls the `speak()` method based on the definition within its own class, which shows how polymorphism works in practice. This approach allows the same method name to trigger different

behaviours depending on the object type, meaning the surrounding code does not need to know the exact class it is dealing with. As a result, programmes become easier to manage and scale, especially in large systems where many different object types must be handled in a consistent way.

Advanced use with `super()`:

```
class Dog(Animal):  
    def speak(self):  
        super().speak() # Call base class method  
        print("Dog barks")
```

```
d = Dog()  
d.speak()
```

Output:

Animal speaks

Dog barks

This allows the derived class to extend rather than completely replace the base class method, providing more flexible and layered behaviour.

Applications of method overriding:

- Implementing specific behaviour for different subclasses in GUI frameworks.
- Customising functionality in banking, e-commerce, or game systems.
- Enabling polymorphic function calls in collection processing or data pipelines.

3.4 Operator Overloading Basics

Operator overloading is another form of polymorphism where standard operators such as `+`, `-`, `*`, and comparison operators are redefined to perform class-specific operations. Python allows operator overloading by defining **special methods** (also called dunder methods) in classes.

Common special methods:

- `__add__(self, other)` for `+`
- `__sub__(self, other)` for `-`
- `__mul__(self, other)` for `*`
- `__str__(self)` for string representation

- `__eq__(self, other)` for equality ==

Example: Complex numbers

```
class ComplexNumber:
    def __init__(self, real, imag):
        self.real = real
        self.imag = imag

    def __add__(self, other):
        return ComplexNumber(self.real + other.real, self.imag +
other.imag)

    def __str__(self):
        return f"{self.real} + {self.imag}i"

c1 = ComplexNumber(2, 3)
c2 = ComplexNumber(4, 5)

print(c1 + c2)  # Output: 6 + 8i
```

In this example, operator overloading is used to define how the + operator behaves for objects of the ComplexNumber class. By implementing the add() method, Python knows how to combine two ComplexNumber objects by adding their real parts and imaginary parts separately. As a result, the expression `c1 + c2` works naturally and returns a new ComplexNumber object representing the correct mathematical sum, making the code intuitive and closely aligned with real-world complex number operations.

Other operator overloading examples:

- **Comparison operators:**

```
class Student:
    def __init__(self, marks):
        self.marks = marks

    def __gt__(self, other):
        return self.marks > other.marks

s1 = Student(85)
s2 = Student(90)

print(s1 > s2)  # Output: False
```

- **String representation:**


```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __str__(self):
        return f"({self.x}, {self.y})"

p = Point(3, 4)
print(p)  # Output: (3, 4)
```

Advantages of operator overloading:

- Makes class objects intuitive to use with standard operators.
- Enhances code readability and usability.
- Enables polymorphic behaviour for operators across custom classes.

When overloading operators in Python, it is important to ensure that the behaviour is logical and intuitive for the type of object being defined. Operators should not be overloaded in ways that could confuse users or produce unexpected results. Additionally, consistency should be maintained between related operators—for example, if **eq** is defined to test equality, **ne** should be defined to return the opposite result—to ensure predictable and coherent behaviour across the class.

Polymorphism in Python is a fundamental principle of object-oriented programming that allows objects of different types to be accessed through a common interface while exhibiting behaviour specific to their class. This capability enhances code flexibility, reduces duplication, and supports the development of scalable and maintainable programs. It can be achieved through various mechanisms, including method overriding, simulated function overloading with default or variable-length arguments, and operator overloading, all of which enable consistent interfaces and intuitive behaviour across diverse object types.

Polymorphism, together with inheritance and encapsulation, forms the cornerstone of object-oriented design in Python. It enables programmers to build modular, flexible, and reusable systems, where new classes or functionalities can be added with minimal changes to existing code. Real-world applications include GUI frameworks, banking systems, e-commerce platforms, game development, and simulation software, demonstrating the breadth and utility of polymorphism in Python programming.

SELF-ASSESSMENT QUESTIONS – 2

Multiple Choose Questions	
6	Which term describes using the same method name in different classes so each class provides its own behaviour? a) Function overloading b) Encapsulation c) Polymorphism d) Inheritance
7	Which technique is commonly used in Python to simulate function overloading? a) Default arguments b) Compile-time templates c) Static function signatures d) Type declarations
8	Which special method should be defined to customise the + operator for a class? a) add b) plus c) sum d) concat
9	How can a derived class call the implementation of a method from its base class? a) base.method() b) super().method() c) parent.method() d) self.parent.method()
10	Which of the following is a benefit of polymorphism? a) Increased code duplication b) Reduced flexibility c) Uniform interfaces for different object types d) Tighter coupling

4. SUPER FUNCTION, INHERITANCE AND POLYMORPHISM IN PRACTICE

Python is a versatile object-oriented programming language that enables programmers to model real-world entities using classes, objects, and relationships. The `super()` function, when used in combination with inheritance and polymorphism, allows developers to write efficient, modular, and maintainable code. Understanding the purpose, practical use, and best practices associated with `super()` and polymorphism is critical for implementing scalable software systems.

4.1 Purpose of `super()`

The `super()` function in Python provides a mechanism to access methods and attributes of a parent class from a derived class. It is particularly essential in multi-level and multiple inheritance hierarchies.

Key purposes of `super()`:

- **Base class access:** `super()` allows derived classes to call methods and constructors of their base classes without explicitly naming them.
- **Code reuse:** Methods defined in the base class can be reused in the derived class, reducing duplication.
- **Multiple inheritance support:** `super()` ensures that methods from all base classes are called in a predictable order defined by the Method Resolution Order (MRO).
- **Consistent initialisation:** When constructing objects of derived classes, `super()` ensures that the base class initialisation logic executes, maintaining object consistency.

Example: Basic use of `super()`

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound")

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)  # Call base class constructor
```

```
self.breed = breed
```

```
def speak(self):  
    super().speak() # Call base class method  
    print(f"{self.name} barks")
```

```
dog = Dog("Rex", "Labrador")  
dog.speak()
```

Output:

Rex makes a sound

Rex barks

Here, `super()` ensures that both the constructor and the `speak()` method of the base class are executed, demonstrating how derived classes can extend functionality without duplicating code.

Using `super()` in Python provides a clear advantage by simplifying code maintenance. By centralising shared functionality in base classes and allowing derived classes to call these methods, developers can avoid duplicating code. This not only reduces the potential for errors but also makes the program easier to read and maintain, as changes in the base class automatically propagate to all derived classes that use `super()`.

Another key benefit of `super()` is that it ensures a consistent object state across inheritance hierarchies. In multiple inheritance scenarios, `super()` works with the Method Resolution Order to guarantee that methods are called in a predictable sequence. This prevents conflicts and unexpected behaviour, making complex class structures more reliable and easier to manage.

4.2 Method Resolution Order (MRO)

The Method Resolution Order (MRO) is a rule Python uses to determine the sequence in which base classes are searched when a method is called on an object. Understanding MRO is essential in multiple inheritance to avoid ambiguity.

Python uses C3 linearisation to compute MRO. The rules ensure:

1. Each class appears before its base classes.
2. The order preserves the inheritance hierarchy.
3. Ambiguities are resolved in a predictable manner.

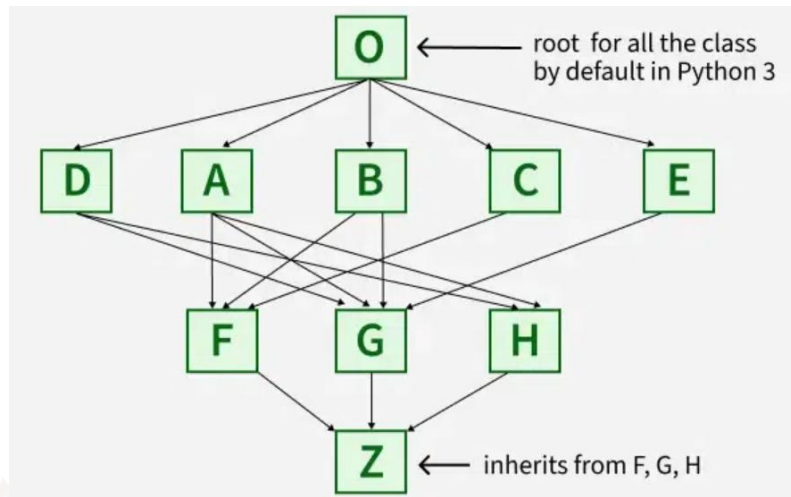


Figure 8: C3 Linearization Structure

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20241223120357694846/Class-structure.webp>

The figure illustrates the C3 Linearisation Structure, which defines the Method Resolution Order (MRO) in Python for classes involved in complex inheritance, especially multiple inheritance. MRO is the algorithm Python uses to determine the exact sequence in which base classes are searched when accessing an attribute or method, as shown by the class hierarchy descending from the base Object class, labelled 'O'. This linearisation ensures a predictable and consistent search path, following the principle that a class is always checked before its parents. In cases of multiple inheritance, it respects the order of parent classes specified in the class definition, effectively resolving ambiguities such as those arising from the Diamond Problem.

Example: Multiple inheritance and MRO

```

class A:
    def show(self):
        print("A")

class B(A):
    def show(self):
        print("B")

class C(A):
    def show(self):
        print("C")

class D(B, C):
    pass
  
```

```
d = D()
d.show()
print(D.__mro__)
```

Output:

B

```
(<class '__main__.D'>, <class '__main__.B'>, <class '__main__.C'>, <class '__main__.A'>, <class 'object'>)
```

The MRO shows the order Python uses to search for methods: $D \rightarrow B \rightarrow C \rightarrow A \rightarrow \text{object}$. This ensures that B's method is called first.

The Method Resolution Order (MRO) plays a critical role in Python's object-oriented programming, particularly when a class inherits from multiple base classes. When two or more parent classes define methods with the same name, MRO determines the sequence in which Python searches these classes to locate and execute the method. This prevents conflicts and ensures that the correct method is invoked, maintaining predictable and consistent behaviour across the program.

MRO also supports the safe use of the `super()` function in multiple inheritance scenarios. By following a well-defined order, `super()` can reliably call methods from parent classes without ambiguity, allowing derived classes to extend or modify functionality while preserving the integrity of the inheritance hierarchy. This makes complex class structures more manageable and reduces the risk of errors in method resolution.

4.3 Using `super()` in Inheritance

`super()` is crucial in inheritance for ensuring that base class methods and constructors are executed properly. It allows derived classes to extend, rather than replace, base class behaviour.

Real-world scenario – Employee Management System

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def show_info(self):
        print(f"Name: {self.name}, Salary: {self.salary}")

class Manager(Employee):
    def __init__(self, name, salary, department):
```

```
        super().__init__(name, salary)
        self.department = department

    def show_info(self):
        super().show_info()
        print(f"Department: {self.department}")

class Director(Manager):
    def __init__(self, name, salary, department, region):
        super().__init__(name, salary, department)
        self.region = region

    def show_info(self):
        super().show_info()
        print(f"Region: {self.region}")

director = Director("Alice", 120000, "HR", "North")
director.show_info()
```

Output:

Name: Alice, Salary: 120000

Department: HR

Region: North

This example illustrates how `super()` helps maintain a clear, modular hierarchy while enabling derived classes to extend base class functionality.

When using `super()` in inheritance, it is a best practice to always call the base class constructor within derived classes. This ensures that the base class is properly initialised and that any essential setup is executed, maintaining the integrity of the object. In overridden methods, `super()` should be used to extend the behaviour of the base class method rather than completely replacing it, allowing derived classes to add functionality while preserving the original logic.

It is also important to be mindful of the Method Resolution Order (MRO) when working with multiple inheritance, as it determines the sequence in which methods are searched and called. For maintainability and clarity, base class methods should generally not be invoked directly by name; using `super()` provides a safer and more consistent approach. This ensures that code remains predictable, easier to manage, and compatible with changes in the inheritance hierarchy.

4.4 Code Reusability and Real-World Modelling

Combining `super()`, inheritance, and polymorphism enables **code reuse** and efficient real-world modelling. By placing shared behaviour in base classes, derived classes can extend functionality without duplicating code.

Example: Banking System

- Base class: Account with attributes `account_number` and `balance`, and methods `deposit()` and `withdraw()`.
- Derived classes: `SavingsAccount` and `CheckingAccount` implement account-specific rules using `super()`.

```
class Account:
    def __init__(self, account_number, balance):
        self.account_number = account_number
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}, new balance: {self.balance}")

    def withdraw(self, amount):
        if self.balance >= amount:
            self.balance -= amount
            print(f"Withdrew {amount}, new balance: {self.balance}")
        else:
            print("Insufficient balance")

class SavingsAccount(Account):
    def withdraw(self, amount):
        if self.balance - amount < 1000: # Minimum balance restriction
            print("Cannot withdraw, minimum balance required")
        else:
            super().withdraw(amount)

sa = SavingsAccount(12345, 5000)
sa.withdraw(4500)
```

Output:

Cannot withdraw, minimum balance required

In this banking system example, the base class `Account` defines shared attributes such as `account_number` and `balance`, along with common methods like `deposit()` and `withdraw()`. These methods encapsulate standard functionality applicable to all account types, allowing code reuse in derived classes. The `SavingsAccount` class demonstrates how `super()` can be used to extend this behaviour without modifying the base logic. In its overridden `withdraw()` method, additional rules, such as enforcing a minimum balance, are applied before calling `super().withdraw(amount)`. This ensures that the fundamental withdrawal process defined in `Account` is preserved while allowing derived classes to implement account-specific constraints, promoting both code reuse and flexibility.

Example: E-commerce System

- Base class: `Product` with attributes `name` and `price`.
- Derived classes: `Electronics` and `Clothing` override methods for discounts or taxes.

```
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def display(self):
        print(f"{self.name}: ${self.price}")

class Electronics(Product):
    def display(self):
        super().display()
        print("Warranty: 2 years")

class Clothing(Product):
    def display(self):
        super().display()
        print("Return Policy: 30 days")

e = Electronics("Laptop", 1500)
c = Clothing("T-shirt", 25)
```

```
e.display()
c.display()
```

Output:

Laptop: \$1500

Warranty: 2 years

T-shirt: \$25

Return Policy: 30 days

This example illustrates polymorphism in Python by showing how the same method, `display()`, can behave differently depending on the type of object it is called on, while maintaining a consistent interface. The base class `Product` defines common attributes such as `name` and `price` and a standard `display()` method. The derived classes `Electronics` and `Clothing` override the `display()` method to include additional, type-specific information, such as warranty details for electronics and return policy for clothing. By using `super().display()`, the original behaviour of the base class is preserved, and the extended functionality is added, demonstrating how polymorphism enables flexible and reusable code.

Advantages of this approach:

- Avoids code duplication by centralising common behaviour in base classes.
- Facilitates maintainability, as changes in the base class propagate to derived classes.
- Supports polymorphism, allowing functions to operate on multiple object types.

4.5 Best Practices in Object-Oriented Design

When combining `super()`, inheritance, and polymorphism, the following best practices enhance maintainability and readability:

- **Single Responsibility Principle:** Each class should have a clear, focused responsibility.
- **Use `super()` for consistent initialisation:** Always initialise base class attributes to maintain object consistency.
- **Avoid deep inheritance hierarchies:** Deep hierarchies increase complexity and make code harder to maintain.
- **Leverage polymorphism:** Design functions to operate on base class objects while allowing derived classes to override behaviour.
- **Document class hierarchies:** Use UML diagrams and docstrings to clarify relationships and method behaviour.
- **Encapsulation:** Keep attributes private and provide getter and setter methods to control access.

- **Consistent operator behaviour:** When overloading operators, ensure behaviour matches real-world expectations.
- **Understand MRO:** Ensure predictable behaviour in multiple inheritance scenarios by understanding method resolution order.

Real-world example – Logistics Management System

- Base class: Vehicle with methods `start()`, `stop()`, and `fuel_check()`.
- Derived classes: Truck and Van override `fuel_check()` to implement vehicle-specific logic.
- `super()` is used to call base methods, ensuring consistent behaviour.
- Polymorphism allows a function `perform_check(vehicle)` to operate on both Truck and Van.

Combining `super()`, inheritance, and polymorphism in Python allows developers to write highly reusable and flexible code. By centralising common functionality in base classes, shared behaviour can be maintained in a single place, reducing redundancy. Polymorphism enables functions and methods to operate on different types of objects using a consistent interface, simplifying code interaction and supporting dynamic behaviour across diverse classes. These mechanisms together facilitate the creation of modular and well-organised class hierarchies that accurately represent real-world systems and their relationships.

The `super()` function is particularly useful for extending base class methods safely, while maintaining consistency and avoiding duplication. When working with multiple inheritance, understanding the Method Resolution Order ensures that method calls are resolved predictably and without conflicts. Following best practices while applying these concepts results in software that is maintainable, scalable, and adaptable. This approach is widely used in real-world applications such as banking systems, e-commerce platforms, logistics management, educational software, and enterprise resource planning solutions.

SELF-ASSESSMENT QUESTIONS – 3

Multiple Choose Questions	
11	What is the primary purpose of the <code>super()</code> function in a derived class? a) To create a new independent object of the base class b) To access methods and attributes of the parent class without naming it explicitly c) To enforce private access to base class attributes

	d) To convert instance methods into static methods
12	Which algorithm does Python use to compute the Method Resolution Order (MRO)? a) C3 linearisation b) Depth-first search only c) Round-robin scheduling d) Breadth-first traversal
13	In the SavingsAccount example, why is <code>super().withdraw(amount)</code> called after checking the minimum balance? a) To bypass the base class withdrawal checks entirely b) To log the transaction in a separate system c) To convert the amount into a different currency first d) To reuse the <code>Account.withdraw</code> implementation once the subclass-specific rule passes
14	Which of the following is a recommended best practice when using <code>super()</code> , inheritance and polymorphism? a) Build deep inheritance chains for every feature b) Call base class methods directly by name to ensure clarity c) Keep each class focused on a single responsibility and document hierarchies d) Make all attributes public to simplify access
15	Which benefit commonly results from combining <code>super()</code> , inheritance and polymorphism in real-world designs? a) Faster execution of all methods by default b) Consistent initialisation and reduced duplication across related classes c) Removal of the need to consider Method Resolution Order d) Automatic generation of UML diagrams

5. SUMMARY

- Inheritance allows one class to derive features and behaviour from another, reducing repetition and helping create structured, scalable code.
- Python supports several inheritance models, including single, multilevel, hierarchical, multiple, and hybrid, each suited to different design needs.
- Base and derived classes form the foundation of inheritance, with derived classes extending or modifying the behaviour of their parents.
- Method overriding lets a subclass redefine a method from its parent, enabling more specific or refined behaviour while keeping the same method name.
- Polymorphism allows the same method or operation to behave differently depending on the object type, improving flexibility and reducing complexity.
- Function overloading is not natively supported in Python, but similar behaviour is achieved using default arguments, variable-length arguments, or type checking.
- Operator overloading enables standard operators such as + or > to work meaningfully with user-defined classes by implementing special methods.
- The super function provides controlled access to parent class methods, supporting clean extension of inherited behaviour without naming the base class directly.
- Python's Method Resolution Order determines how classes are searched when resolving attributes, particularly important in multiple inheritance.
- Good object-oriented design emphasises readability, clear class responsibilities, appropriate use of inheritance, and predictable operator behaviour.

6. GLOSSARY

Inheritance	- A mechanism that allows one class to acquire attributes and methods from another class.
Base Class	- The parent class whose properties are inherited by another class.
Derived Class	- A class that inherits from a base class and may add or modify features.
Method Overriding	- Redefining a method in a derived class to change or extend its behaviour.
Polymorphism	- The ability of different classes to respond to the same method call in different ways.
Function Overloading	- A feature where multiple functions share a name but differ in parameters; simulated in Python using flexible argument handling.
Operator Overloading	- Customising how operators behave for user-defined objects by implementing special dunder methods.
super()	- A built-in function that allows a derived class to call methods from its base class safely and consistently.
Method Resolution Order (MRO)	- The sequence Python follows to determine which class method or attribute should be used, especially in multiple inheritance.
Code Reusability	- The practice of writing code that can be used across multiple classes or systems, improving efficiency and maintainability.

7. TERMINAL QUESTIONS

1. Explain the concept of inheritance in Python and give a concrete example showing how it reduces code duplication.
2. Compare and contrast single, multiple, multilevel, hierarchical and hybrid inheritance, and give one practical use case for each.
3. Describe how to create base and derived classes in Python, showing how to use `super()` in the derived class constructor.
4. Explain method overriding and show, with code, how a subclass can extend rather than replace a base class method.
5. Define polymorphism and give an example where polymorphism simplifies the design of a system that processes different object types.
6. Why does Python not support classical function overloading? Describe three Python techniques that simulate overloading and when you would use each.
7. Explain operator overloading: name at least three dunder methods commonly used and provide a short code example for one.
8. What is Method Resolution Order (MRO) and how does the C3 linearisation algorithm ensure predictable method lookup in multiple inheritance?
9. Discuss best practices for using `super()`, inheritance and polymorphism together when modelling a real-world domain (for example, a banking or logistics system).
10. Identify common pitfalls of deep or careless inheritance hierarchies and propose refactorings or design alternatives that reduce complexity.

8. ANSWERS

8.1. Self-Assessment Answers:

1. `super()`
2. Multiple inheritance
3. Method Resolution Order
4. The subclass method
5. Hierarchical inheritance
6. Polymorphism
7. Default arguments
8. `add`
9. `super().method()`
10. Uniform interfaces for different object types
11. To access methods and attributes of the parent class without naming it explicitly
12. C3 linearisation
13. To reuse the `Account.withdraw` implementation once the subclass-specific rule passes
14. Keep each class focused on a single responsibility and document hierarchies
15. Consistent initialisation and reduced duplication across related classes

8.2. Terminal Question Answers

Answer 1: Inheritance lets a new class (the derived class) obtain attributes and methods from an existing class (the base class), so common behaviour is written once and reused across related types. This creates an “is-a” relationship that organises code into clear hierarchies and reduces duplication; for example a `Vehicle` base class can provide `start()` and `stop()` methods reused by `Car` and `Motorbike`. A derived class may add or override features specific to its role while still using the base class implementation where appropriate. Refer: Section 2.1 (Concept of Inheritance).

Answer 2: Single inheritance involves one subclass inheriting from one parent (simple, linear reuse); multiple inheritance means a subclass inherits from several parents (combines behaviours but needs MRO); multilevel inheritance forms a chain of inheritance (specialisation through several layers); hierarchical inheritance has many subclasses inheriting from a single base (shared common behaviour); hybrid inheritance mixes these forms to model complex domains. Practical uses: single for a specialised widget subclass, multiple for combining mixins (behavioural traits), multilevel for `Employee→Manager→Director` chains, hierarchical for `Animal→Dog/Cat` families, and hybrid for

systems that combine component hierarchies with role mixins (for example, a JetCar combining Car and Jet features). Each style should be chosen to match design clarity and maintainability. Refer: Section 2.2 (Types of Inheritance).

Answer 3: To create base and derived classes in Python you define the parent first and then declare the child using `class Child(Base):`, placing shared attributes and methods in the base class. In the derived class constructor call `super().init(...)` to invoke base-class initialisation so common state is set consistently, then add subclass-specific attributes. Using `super()` avoids hard-coding parent names and supports predictable initialisation across multi-level or multiple inheritance. This pattern keeps code modular and prevents duplication of setup logic. Refer: Section 2.3 (Creating Base and Derived Classes).

Answer 4: Method overriding occurs when a subclass defines a method with the same name as one in its base class so calls on the subclass use the new implementation. A subclass can extend rather than replace the base behaviour by calling `super().method(...)` inside its override, for example `super().start()` followed by extra subclass steps. This preserves the original contract while allowing specialised behaviour and supports polymorphism when different classes respond to the same call. Overriding should follow the rule that method names match and, where needed, the base method can still be accessed via `super()`. Refer: Section 2.4 (Method Overriding).

Answer 5: Polymorphism allows different classes to present the same interface so code can operate on varied object types without knowing their concrete classes. For example, a `draw()` function can accept Circle, Square or Rectangle objects and call `shape.draw()` for each; each class implements `draw()` differently so the correct action occurs at runtime. This reduces branching logic and makes it straightforward to add new types without changing client code. Polymorphism therefore improves extensibility and simplifies collection processing and plug-in style designs. Refer: Section 3.1 (Concept of Polymorphism).

Answer 6: Python lacks classical compile-time function overloading because it is dynamically typed and functions are identified by name at runtime, so later definitions replace earlier ones. Overloading behaviour is simulated using default arguments (one function handles optional parameters), variable-length arguments (`*args` and `**kwargs`) to accept differing numbers or kinds of inputs, and manual type checking (`isinstance()` or similar) to adjust behaviour by argument type. Use default arguments for simple optional parameters, `*args/**kwargs` for flexible argument lists or forwarding, and type checks when behaviour must change based on input types. Refer: Section 3.2 (Function Overloading Behaviour).

Answer 7: Operator overloading lets classes redefine standard operators by implementing special dunder methods such as `add` (for `+`), `eq` (for `==`) and `str` (for string conversion). By providing these methods a custom object can behave intuitively with built-in operators; for example a `ComplexNumber` class can implement `def add(self, other): return ComplexNumber(self.real+other.real, self.imag+other.imag)`. Care should be taken to keep operator semantics logical and to implement complementary methods (for example `ne` when `eq` is defined). Refer: Section 3.4 (Operator Overloading Basics).

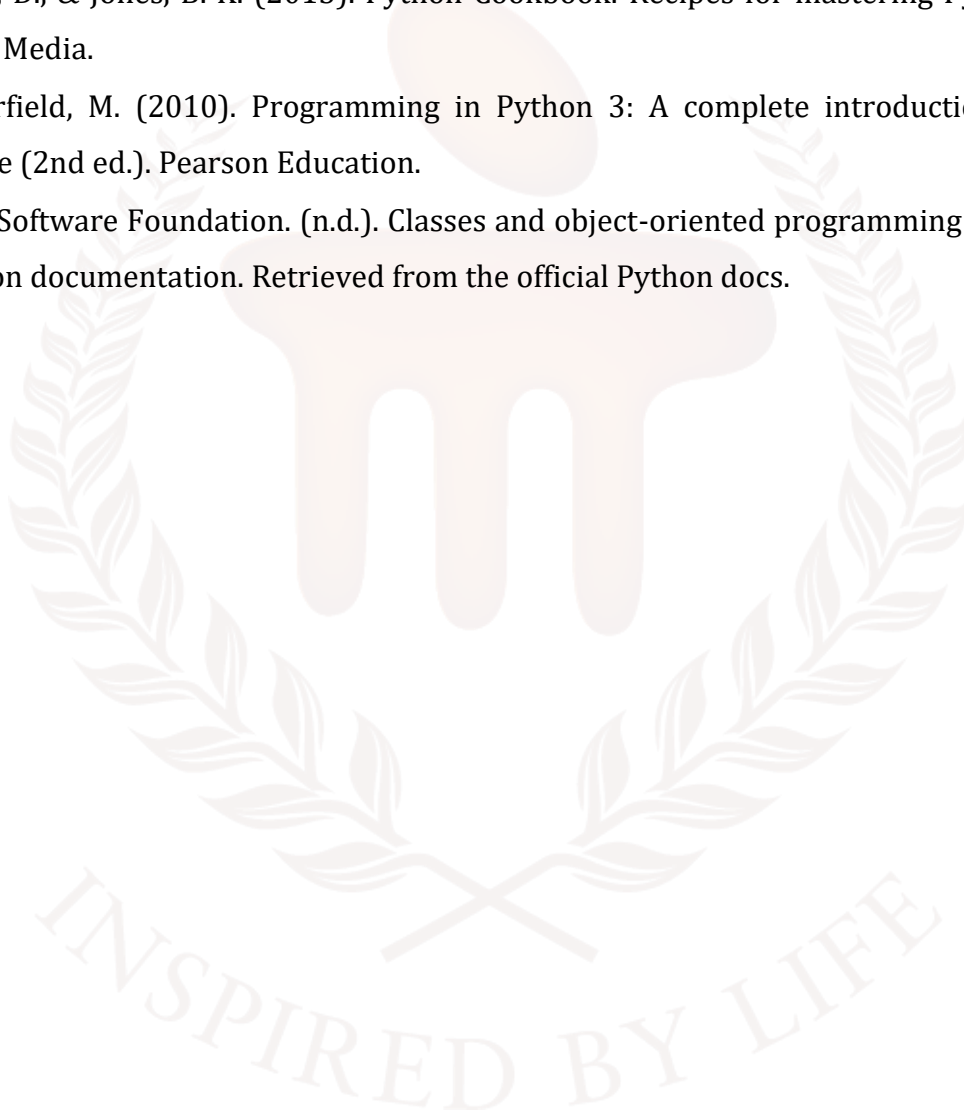
Answer 8: Method Resolution Order (MRO) is the sequence Python follows to search classes for a method or attribute, crucial in multiple inheritance to avoid ambiguity. Python computes MRO using the C3 linearisation algorithm which produces a consistent, monotonic ordering that preserves class precedence while ensuring each class appears before its parents. This predictable order lets `super()` and method lookups behave reliably even in diamond or complex inheritance graphs. Inspecting the `mro` attribute reveals the computed search path for any class. Refer: Section 4.2 (Method Resolution Order).

Answer 9: Best practice is to keep classes focused on single responsibilities, call `super()` in constructors and overridden methods to maintain base initialisation and behaviour, avoid deep hierarchies, and favour composition where appropriate. In a banking or logistics model place shared behaviour in a well-documented base class (for example `Account` or `Vehicle`), use polymorphism for operations that vary by type, and use `super()` to extend base logic safely. Document hierarchies with UML and prefer small, well-tested classes so the design remains maintainable and extensible. Refer: Section 4.5 (Best Practices in Object-Oriented Design).

Answer 10: Common pitfalls of deep or careless inheritance include tight coupling, fragile base-class problems, surprising method resolution, and difficulty of understanding or testing the hierarchy. Refactor by favouring composition over inheritance, extract interfaces or mixin classes for orthogonal behaviour, flatten hierarchies where specialisation is unnecessary, or apply design patterns (strategy, decorator) to reduce coupling. Regularly review and document class roles, and write unit tests that exercise subclasses independently to expose unexpected interactions. Refer: Section 4.5 (Best Practices) and Section 4.4 (Code Reusability and Real-World Modelling).

9. REFERENCES

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- Ramalho, L. (2015). Fluent Python: Clear, concise, and effective programming. O'Reilly Media.
- Beazley, D., & Jones, B. K. (2013). Python Cookbook: Recipes for mastering Python 3 (3rd ed.). O'Reilly Media.
- Summerfield, M. (2010). Programming in Python 3: A complete introduction to the Python language (2nd ed.). Pearson Education.
- Python Software Foundation. (n.d.). Classes and object-oriented programming (Python tutorial). In Python documentation. Retrieved from the official Python docs.



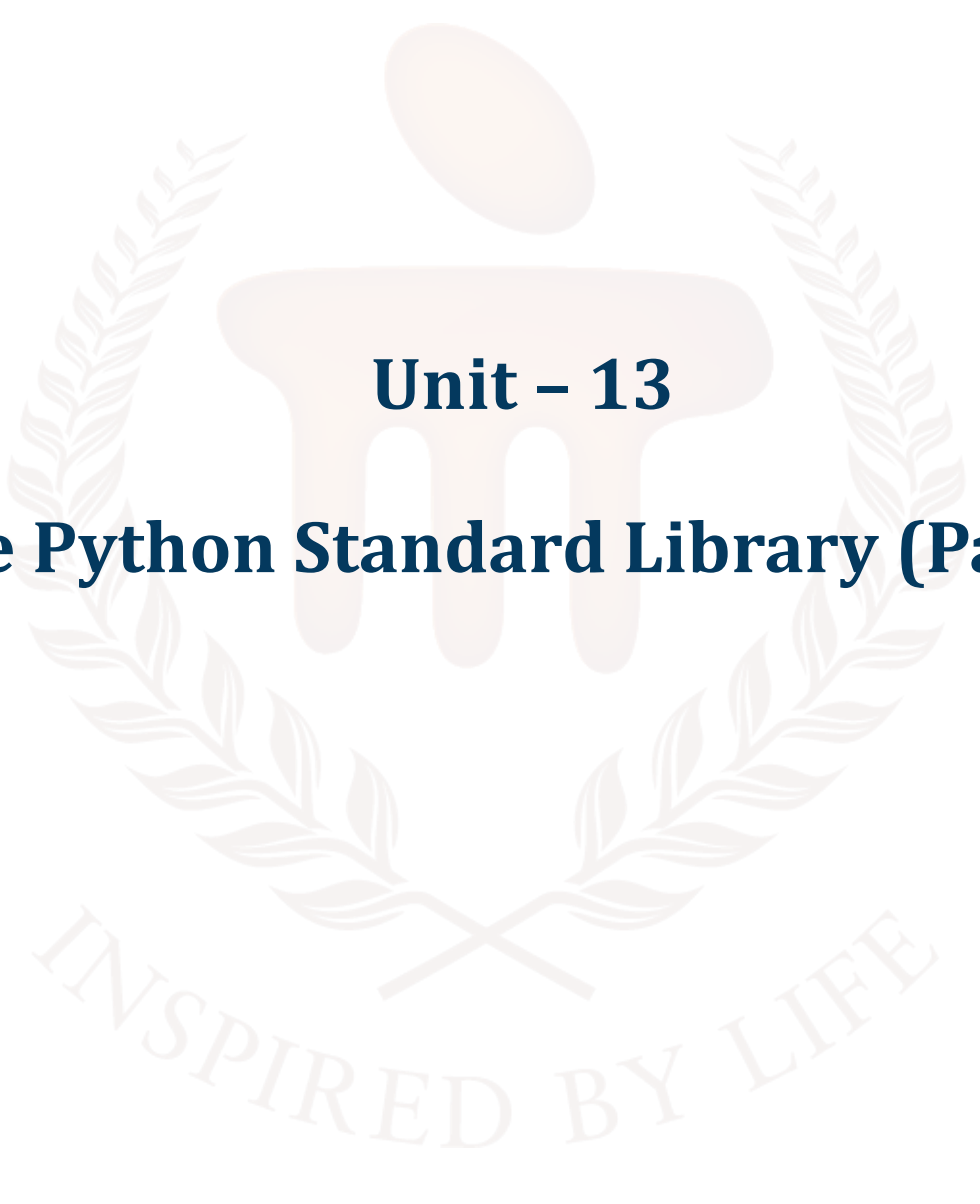
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 13

The Python Standard Library (Part 1)

TABLE OF CONTENTS

Sl No	Topic	Fig No / Table / Graph	Saq / Activity	Page No
1	Introduction	-	-	4
1.1	Learning Objectives	-	-	
2	Strings in Python	1	1	5-14
2.1	Overview of Strings	2	-	
2.2	String Creation and Indexing	-	-	
2.3	Common String Methods	-	-	
2.4	String Formatting	-	-	
3	Text Handling in Python	-	2	15-20
3.1	Working with Text Data	-	-	
3.2	Searching and Modifying Text	-	-	
3.3	Splitting and Joining Strings	3. 4	-	
3.4	Handling Whitespace and Case	-	-	
4	Networking in Python	-	3	21-28
4.1	Basic Networking Concepts	-	-	
4.2	Networking Modules in the Standard Library	5	-	
4.3	Establishing Network Connections	-	-	
4.4	Sending and Receiving Data	-	-	
5	Summary	-	-	29
6	Glossary	-	-	30
7	Terminal Questions	-	-	31
8	Answers	-	-	32-34
8.1	Self-Assessment Answers	-	-	
8.2	Terminal Questions Answers	-	-	
9	References	-	-	35

1. INTRODUCTION

The previous chapter focused on object-oriented programming concepts in Python, with emphasis on inheritance and polymorphism. You studied how classes can share and extend behaviour through inheritance and how polymorphism allows the same interface to work with different object types. These ideas supported the development of flexible and reusable program structures.

This chapter introduces selected components of the Python Standard Library related to strings and text handling. It explains how Python provides built-in tools to create, manipulate, and format strings, which are widely used in data processing and application development. Handling textual data efficiently is essential when working with user input, files, and system output.

The chapter also covers basic networking concepts using standard library modules. It introduces how Python supports communication over networks by allowing programs to establish connections and exchange data. These features form the foundation for building client-server applications and processing data received from external systems.

To study this chapter effectively, practise using string methods and text manipulation functions with small coding examples. Work through networking examples that demonstrate data transfer between systems. Combining practical exercises with concept revision will help in developing confidence in using the Python Standard Library.

1.1. Learning Objectives

By the end of this Unit, you will be able to:

- Apply string methods for text processing tasks
- Format and manipulate strings for practical applications
- Execute text handling operations using standard library functions
- Implement basic networking features in Python programs
- Analyse text and network data using built-in Python tools



2. STRINGS IN PYTHON

Strings play a central role in Python programming because almost all applications involve some form of text handling. Whether the task involves receiving user input, processing file content, preparing reports, handling communication between services, or working with structured data formats, strings remain essential. Python provides detailed support for working with text, offering a wide range of operations designed to make string handling consistent and reliable.

A string is a sequence of Unicode characters. This Unicode support allows Python programs to work comfortably with text in different languages and scripts, which is important in modern applications that must display or process international content. The important design feature of Python strings is that they are immutable. This means that once a string is created, its internal characters cannot be altered. Any operation that appears to modify a string, such as combining it with another string or replacing a portion of its content, actually produces a new string.

Strings are used across numerous areas:

- Receiving and processing input from the user
- Reading text from files
- Storing configuration data
- Displaying results to the user
- Working with structured data formats
- Sending and receiving information in network applications
- Managing log messages, labels, and system status output

Because strings are used so widely, understanding how they work is essential for mastering Python programming.

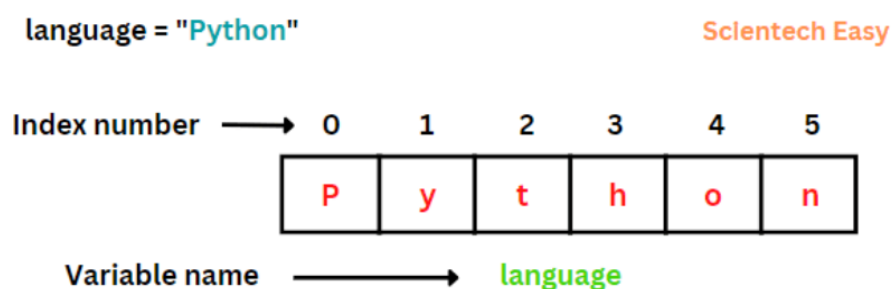


Figure 1: String Representation in Python

Source: <https://www.scientecheasy.com/wp-content/uploads/2023/01/python-strings.png>

This figure demonstrates the fundamental concept of string representation in Python, highlighting how strings are stored and accessed in memory. In Python, a string is treated as an ordered sequence of characters, where each character occupies a specific position within the sequence. For example, the variable `language` is assigned the string value `"Python"`, which is stored in memory in a sequential manner. Each character within this string can be accessed using a zero-based index, starting with 0 for the first character 'P' and continuing up to 5 for the last character 'n'. This indexing system allows programmers to retrieve or manipulate individual characters (though direct modification is not allowed due to immutability). It forms the basis for many string operations, including slicing, concatenation, and iteration, making it an essential concept for text handling in Python. Understanding this sequence and indexing is crucial for working effectively with strings in programming.

2.1 Overview of Strings

A string in Python is created by enclosing text within quotation marks. The type supports multiple quoting styles, which gives the programmer flexibility depending on the situation. Python accepts the following forms:

- Single-quoted strings, such as `'Hello Python'`
- Double-quoted strings, such as `"Hello Python"`
- Triple single-quoted strings, such as `'''A longer message covering several lines'''`
- Triple double-quoted strings, such as `"""A detailed explanation that spans multiple lines"""`

Triple-quoted strings are particularly useful for creating multi-line text without requiring special characters. These are often used when writing documentation lines or when storing longer blocks of descriptive information.

Python strings support a wide range of operations. Some of the commonly used ones include:

1. Concatenation using the plus sign
2. Repetition using the multiplication sign
3. Slicing to extract selected characters
4. Comparison using relational operators
5. Membership testing using the keyword `in`

6. Measuring length using the len() function

Strings in Python are immutable, which means that once a string is created, its content cannot be altered. Any operation that appears to change a string, such as concatenation, replacement, or trimming, does not modify the original string. Instead, it generates a new string with the desired changes while keeping the original intact. This immutability is a deliberate design choice, as it prevents accidental modification of string data, helping maintain consistency and stability within a program. By ensuring that strings remain unchanged unless explicitly reassigned, Python reduces the risk of unintended side effects, which is particularly important in larger applications or when passing strings between functions.

In addition, Python strings support Unicode, allowing them to represent a vast array of characters from different languages, symbols, and special characters. While Unicode is complex under the surface, Python handles the internal details, presenting each character as a simple, accessible unit. This abstraction allows programmers to focus on processing and manipulating text without worrying about encoding issues. It makes Python particularly suitable for applications that work with internationalised text, such as multilingual software, web applications, and data processing tasks, where handling diverse character sets is essential.

2.2 String Creation and Indexing

Creating strings in Python is straightforward. Any sequence of characters enclosed in quotation marks becomes a string:

```
text = "Python Programming"
```

```
note = 'A reminder for tomorrow'
```

```
paragraph = """This is a longer message  
that uses multiple lines for clarity."""
```

Once assigned, a string cannot be altered by assigning new characters to specific positions. For instance, the statement `name[0] = 'J'` will produce an error because Python does not permit modification of individual characters.

Indexing - Each character in a string has a position known as an index. Python supports two indexing approaches:

1. Positive indexing

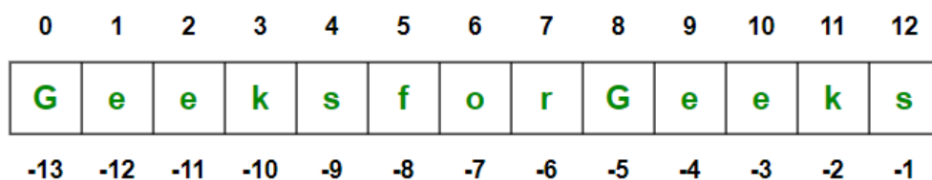
2. Negative indexing

Positive indexing begins at zero, meaning:

- Index 0 refers to the first character
- Index 1 refers to the second character
- Index 2 refers to the third character

Negative indexing begins from the end of the string:

- Index -1 refers to the last character
- Index -2 refers to the second last character
- Index -3 refers to the third last character



0	1	2	3	4	5	6	7	8	9	10	11	12
G	e	e	k	s	f	o	r	G	e	e	k	s
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Figure 2: Python Indexing

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20240118142121/Screenshot-2024-01-18-142040.png>

The figure demonstrates the concept of string indexing in Python, showing how a string is treated as an ordered sequence of characters stored consecutively in memory. In this example, the string "GeeksforGeeks" is used to illustrate how individual characters can be accessed by referring to their position through an index number. Python provides two main ways to index strings. Positive indexing starts at 0 for the first character and increases sequentially up to N-1, where N represents the total length of the string. Negative indexing, on the other hand, begins at -1 for the last character and decreases sequentially towards -N for the first character. These indexing methods allow programmers to retrieve, manipulate, or extract specific characters or substrings with precision. By combining positive and negative indexing, Python offers a flexible and efficient approach for working with strings, making it easier to perform operations such as slicing, searching, and modifying parts of a string without affecting the rest.

This dual indexing approach simplifies string handling because the programmer does not need to calculate the string length to reach the final few characters.

Example:

```
title = "Python"
```

```
title[0]    # P
```

```
title[3]    # h
```

```
title[-1]   # n
```

```
title[-4]   # t
```

Slicing - Slicing extracts specific parts of a string and uses the following notation:

```
string[start : end : step]
```

1. The start index marks the beginning of the slice
2. The end index marks where the slice stops, but this index is not included
3. The step controls how many characters to skip

Examples:

```
data = "Programming"
```

```
data[0:7]    # Program
```

```
data[3:10]   # gramm
```

```
data[:5]     # Progr
```

```
data[4:]     # ramming
```

```
data[::2]    # Pormig
```

```
data[::-1]   # reverse of the entire string
```

Slicing is a powerful part of Python's sequence rules and avoids the need for explicit loops to extract segments.

2.3 Common String Methods

Python includes a wide range of built-in string methods. These methods help perform tasks such as cleaning input, converting case, checking content, splitting text, and preparing formatted output. Since strings cannot be changed directly, each method returns either a new string or a different type of value.

Case Conversion Methods

- `upper()` - Converts every character to upper case.
- `lower()` - Converts every character to lower case.
- `title()` - Capitalises the first letter of each word.
- `capitalize()` - Capitalises the first character of the full string and lowers the rest.
- `swapcase()` - Switches upper case letters to lower case and lower case letters to upper case.

These methods are commonly used when normalising input or preparing labels for display.

Whitespace Removal Methods - Whitespace often appears when reading input from files or users, so Python provides tools to remove unwanted spaces:

- `strip()` removes leading and trailing spaces
- `lstrip()` removes spaces from the left side
- `rstrip()` removes spaces from the right side

These methods help ensure that extra spaces do not cause issues when comparing or storing text.

Searching and Counting Methods - Python includes methods for finding patterns inside a string:

- `find(substring)` returns the position of the first occurrence, or -1 if not found
- `rfind(substring)` returns the position of the last occurrence
- `index(substring)` similar to `find` but raises an error if not found
- `count(substring)` returns the number of times a substring appears

These methods are widely used when analysing content or preparing structured data.

Testing Methods - Several methods return True or False based on the characters in the string:

- `isalpha()` for letters
- `isdigit()` for digits
- `isalnum()` for letters and digits together
- `isspace()` for whitespace characters only
- `istitle()` to check title case
- `isupper()` and `islower()` to test case

These methods help verify whether input is suitable for further processing.

Splitting and Joining - Splitting breaks a string into a list of parts:

- `split()` divides the string at spaces

- `split(',')` divides the string at commas
- `rsplit()` behaves like `split` but starts from the right
- `splitlines()` separates text at line breaks

Joining combines items from a list into a single string using a separator:

```
items = ["Red", "Green", "Blue"]  
", ".join(items)  # Red,Green,Blue
```

Replacement and Alignment

- `replace(old, new)` substitutes all matching sections
- `zfill(width)` adds zeros to the left
- `ljust(width)` moves text to the left within a fixed width
- `rjust(width)` moves text to the right
- `center(width)` aligns text centrally

These methods are frequently used in report generation and formatted displays.

2.4 String Formatting

String formatting is important when producing readable and well-structured output. Python supports several formatting approaches.

Percent Formatting

This earlier form uses format codes:

```
name = "Arun"
```

```
age = 25
```

```
"Name: %s, Age: %d" % (name, age)
```

Common format codes include `%s` for strings, `%d` for integers, and `%f` for floating point numbers.

The `format()` Method

This method provides detailed control and uses braces as placeholders.

```
"Hello {}, you scored {}".format("Riya", 90)
```

The `format()` method in Python provides a flexible way to insert values into a string by using placeholders enclosed in curly braces. It supports positional arguments, where values are inserted based on their order, as well as named arguments, which improve readability by clearly identifying what each value represents. Width control allows developers to specify a minimum field width, helping align text neatly in outputs such as tables or reports. Alignment symbols can be used within the placeholder to position text to the left, right, or center within the given width, making formatted output easier to read and more organised.

Another important feature of the `format()` method is its support for numerical formatting. Decimal precision control allows floating point numbers to be rounded to a fixed number of decimal places, which is particularly useful when displaying measurements, financial values, or calculated results. The method can also be combined with other formatting options to ensure consistent presentation across large sets of output. Because `format()` returns a new formatted string, it works well with Python's immutable string design. This method is widely used in situations where structured, readable output is needed, such as logging, report generation, and data presentation.

Examples:

```
"{:>10}".format("Data")
```

```
"{:3f}".format(5.12389)
```

f-Strings

Introduced in Python 3.6, f-strings are now the most common approach. They support expressions inside braces, evaluated at runtime.

```
marks = 88
```

```
f"Student scored {marks} marks"
```

f-strings also support mathematical expressions:

```
x = 7
```

```
y = 4
```

```
f"Result is {x + y}"
```

Formatting instructions can be added:

```
value = 3.567
```

```
f"{value:.1f}"
```

Python offers advanced string formatting features that allow precise control over how values are displayed. One feature is padding numbers, which ensures numerical values align neatly in output. It also allows the inclusion of thousands separators, making large numbers easier to read. Programmers can control the display of positive and negative signs, ensuring clarity in financial or scientific data. Fixed-width formatting can be enforced to maintain consistent output across multiple lines or columns. Additionally, Python supports printing values as percentages, converting decimals into a more readable format. These features make Python formatting versatile and useful for professional, readable outputs in reports or applications.

Example:

```
value = 15000
```

```
f"{value:,}"
```

Strings represent one of the most frequently used data types in Python. Their versatility and broad set of built-in methods make them suitable for tasks ranging from simple input handling to complex text analysis. By understanding how strings are created, indexed, sliced, processed with methods, and formatted, programmers can write Python programs that handle text in a clear and structured manner.

Self-Assessment Questions – 1

Multiple Choose Questions

- | | |
|---|--|
| 1 | <p>What does it mean that Python strings are immutable?</p> <ul style="list-style-type: none"> a. Characters in a string can be changed directly by assigning to an index. b. Strings are stored only as ASCII bytes, so they cannot hold other characters. c. Once created, a string's characters cannot be altered; operations that seem to change a string produce a new string. d. Strings can be modified in place using the <code>replace()</code> method. |
| 2 | <p>Which index refers to the last character of a Python string?</p> <ul style="list-style-type: none"> a. 0 b. -1 |

	c. <code>len(string)</code> d. <code>1</code>
3	Which slicing expression produces the reverse of the entire string data? a. <code>data[::-1]</code> b. <code>data[::2]</code> c. <code>data[0:len(data)]</code> d. <code>data[-1]</code>
4	Which method removes both leading and trailing whitespace from a string? a. <code>lstrip()</code> b. <code>strip()</code> c. <code>rstrip()</code> d. <code>split()</code>
5	Which statement best describes f-strings in Python? a. They use percent-style format codes such as <code>%s</code> and <code>%d</code>. b. They are limited to formatting integers only. c. They require the <code>format()</code> method to insert values. d. They allow expressions inside braces that are evaluated at runtime.

3. TEXT HANDLING IN PYTHON

Text processing is one of the most frequent tasks carried out when working with Python. Almost every application involves some form of text input, storage, transformation, or display. Whether a program reads data from files, processes user input, communicates with web services, or prepares output for reports, text manipulation lies at the heart of these operations. Python provides a rich set of tools in its standard library for handling text in a clear and structured manner, reducing the need for external packages for common tasks.

Python treats text as a sequence of characters, offering numerous operations that allow programmers to search, modify, split, or combine strings. These operations are efficient and intuitive, which helps developers build reliable text-processing systems. The following subsections explore the core techniques used when working with text data in Python using standard library features.

3.1 Working with Text Data

Working with text data involves understanding how Python stores and processes characters. All text in Python is handled using string objects which represent sequences of Unicode characters. Unicode support allows Python to work with data from various languages, symbols, and formats without special handling.

When programmers begin working with text, they often need to read text from files, accept user input, interact with network responses, or manipulate output before displaying it. It is therefore essential to understand the common operations that apply to text objects.

Python strings are immutable. This means that once created, the value of a string cannot be changed directly. Any operation that appears to change a string actually produces a new string. This behaviour prevents accidental alteration of text and supports predictable behaviour in programs.

Key points when working with text data:

- Text in Python is stored as Unicode, which ensures consistency across languages.
- Strings cannot be changed in place. Each transformation returns a new text object.
- Common tasks include reading input, cleaning unwanted characters, transforming text, and generating structured output.
- Python provides built-in functions and methods for efficient handling of text.

In practice, working with text usually begins with a source such as a file. Once the data is read, the program applies transformations such as trimming spaces, checking whether certain words appear, converting text into different forms, or preparing structured output. Python's standard library supports these tasks through its rich set of methods on string objects.

3.2 Searching and Modifying Text

Many real-world tasks require searching through text to find certain words, characters, or patterns. Python offers simple methods that cover most basic search operations without requiring advanced pattern matching. These methods help programmers determine whether a string contains a particular value, locate where it appears, or count how many times it shows up.

Common search operations include:

- Finding the position of a substring using methods that return the index.
- Checking whether a substring exists within a larger text.
- Counting occurrences of a particular substring.
- Detecting prefixes or suffixes to match structured formats.

When modifying text, programmers often need to clean or restructure the content. This may involve replacing certain words, removing unwanted characters, or adjusting the spacing. Python's built-in methods handle these transformations without affecting the original string, due to immutability.

Examples of common modifying operations:

- Replacing one part of a string with another.
- Removing characters through substitution.
- Normalising formatting such as removing repeated spaces.
- Creating new versions of text based on transformed content.

Searching for text is frequently combined with modification. For instance, a program may search for a word, and if found, it may replace it with another. These simple combined tasks form the basis of more complex text-processing systems.

On a larger scale, text searching is often connected with data cleaning. For example, when processing information collected from web pages or user input, the text may contain extra symbols or incorrect spacing. The search and replace operations become core parts of the cleaning process.

3.3 Splitting and Joining Strings

Splitting and joining are central techniques when processing structured text. Many data formats rely on separators such as commas, spaces, or new lines. Python provides straightforward methods that convert a single string into a list of smaller components or merge a list of strings into a single output.

Splitting text usually happens when reading information that contains repeated separators. For example, information from a comma-separated file needs to be split into fields so the program can work with each part separately.

- Splitting converts the string into a list of elements based on a separator.
- If no separator is specified, Python uses whitespace by default.
- Splitting can be used to prepare data for analysis or storage.
- Programmers can work with each element of the resulting list independently.



```
'leaders and best'.split("e")  
['l', 'ad', 'rs and b', 'st']
```

Figure 3: Splitting in Python

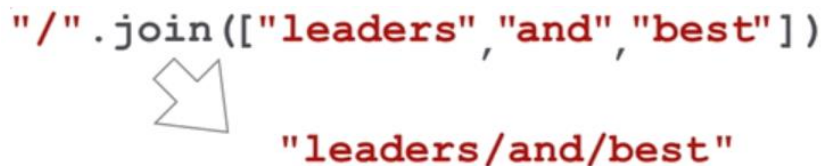
Source: https://runestone.academy/ns/books/published/fopp/images/split_on_e.jpeg

The figure illustrates the use of the `split()` method in Python, which divides a string into a list of substrings based on a specified delimiter. In the example, the string 'leaders and best' is called with `.split("e")`, using the character 'e' as the delimiter. The method separates the string at each occurrence of this character and returns a new list containing the resulting substrings, excluding the delimiter itself. The output in this case is `['l', 'ad', 'rs and b', 'st']`. This method is essential for tasks such as data cleaning, text parsing, and processing user input efficiently.

Joining text often comes after processing has taken place. Once a list of cleaned or modified elements has been handled, the program may need to convert it into a single string again. Joining is the reverse of splitting and is widely used in data formatting.

- Joining takes a list of text components and forms a new string.
- A chosen separator is placed between each element when joining.
- It is commonly used for sentence construction, file output, or formatting lists for display.

- Joining helps build structured content from processed elements.



```
"/" .join(["leaders", "and", "best"])
```

```
"leaders/and/best"
```

Figure 4: Joining in Python

Source: <https://runestone.academy/ns/books/published/fopp/images/join.gif>

The figure demonstrates the `join()` method in Python, which combines all string elements within an iterable, such as a list, into a single string. In this example, the method is called on the forward slash `'/'` as the delimiter and takes the list `["leaders", "and", "best"]` as its argument. The delimiter is inserted between each element, joining them into a continuous string. The resulting output, `"leaders/and/best"`, shows how `join()` efficiently constructs structured text, making it particularly useful for creating file paths, URLs, or other formatted outputs from sequential data in a clear and organised manner.

Splitting and joining strings are common operations in everyday programming tasks, providing a way to organise and manipulate text data efficiently. Splitting allows a string to be divided into smaller parts based on a specified delimiter, which is particularly useful when converting lines of text from a file into separate entries or when breaking user input into commands or fields. This operation helps programmers handle and process data in a structured manner, making it easier to extract meaningful information from raw text.

Joining, on the other hand, combines individual elements, such as words or values, into a single string using a chosen separator. It is often used to create readable sentences from separate words or to prepare data for storage or transmission in a standardised format. By using splitting and joining together, programmers can manipulate text efficiently, ensuring data is both organised and accessible for further processing.

3.4 Handling Whitespace and Case

Whitespace handling is a frequent requirement in text processing. Whitespace includes spaces, tabs, and new lines. These characters play a structural role in text but can also cause problems when placed unintentionally. Python provides reliable methods to trim or manage whitespace.

Whitespace handling tasks often include:

- Removing extra spaces from the beginning of text.
- Removing trailing spaces from the end of text.
- Removing both leading and trailing spaces.
- Normalising spacing within a string to improve readability.

Another crucial aspect of text handling involves case conversion. When processing user input or comparing values, differences in capitalisation can lead to mismatches. Python offers simple mechanisms to convert text to lower case, upper case, or title case. This helps maintain consistency across data sources.

- Converting text to a common case ensures reliable comparison.
- Lower case is often used for internal processing or searching.
- Upper case is useful for output formatting.
- Title case helps when preparing headings or names.

Whitespace and case operations often appear together, since many text-cleaning tasks require resetting both spacing and capitalisation to produce a consistent result. When handling large datasets or repeated user interactions, these methods ensure predictable behaviour.

Handling whitespace and case is a common requirement in programming to ensure data is clean and consistent. For example, preparing a list of user names often involves removing extra spaces and standardising the case for uniformity. Similarly, data from external sources may need validation and correction, including trimming leading or trailing whitespace and converting text to the desired case. Proper formatting of output is also important to meet organisational standards, ensuring readability and professionalism. Removing unnecessary spaces before storing text helps maintain database integrity and prevents errors during searches or comparisons, making whitespace and case management a key aspect of text handling.

Text handling in Python is an essential part of programming, supported by a comprehensive set of methods in the standard library. The immutability of strings provides reliability, and the built-in operations cover common needs such as searching, modifying, splitting, joining, and formatting. These operations allow programmers to handle text data efficiently without requiring external libraries for typical tasks. Through a combination of these techniques, Python applications are able to manage everything from simple user interaction to full-scale data processing pipelines.

Self-Assessment Questions – 2

Multiple Choose Questions	
6	Which statement correctly describes Python string immutability? a. A string's characters can be changed by assigning to an index. b. Once created, a string cannot be altered; operations that appear to change it return a new string. c. Strings are stored only as ASCII and cannot hold other characters. d. The replace() method edits the original string in place.
7	If no separator is provided, what does the split() method use by default? a. Comma b. Empty string between characters c. Any consecutive whitespace (spaces, tabs, newlines) d. Full stop (period)
8	Which of the following is the correct way to combine a list of strings into a single string with a chosen separator? a. concat(["a", "b"], "-") b. ["a", "b"].merge("-") c. combine("-", ["a", "b"]) d. "-".join(["a", "b"])
9	Which method raises an error if the specified substring is not found? a. index() b. find() c. count() d. rfind()
10	Which method would you use to remove both leading and trailing whitespace from a string? a. lstrip() b. strip() c. rstrip() d. split()

4. NETWORKING IN PYTHON

Networking forms the backbone of communication between devices in the modern world. Almost every digital system depends on some form of network interaction, whether it involves exchanging information on a local network or retrieving data from distant servers. Python, being a widely used programming language across different fields, offers extensive support for networking tasks through its standard library. These built-in modules allow programmers to write applications that can connect to servers, exchange messages, download content, or operate as servers themselves.

Python simplifies network programming by hiding many of the complex underlying details. While the language exposes fine-grained control for those who need it, it also provides higher-level tools that handle much of the low-level communication automatically. This makes Python a suitable choice for beginners exploring network concepts and professionals building robust, network-driven systems.

Networking in Python usually revolves around the processes of setting up connections, resolving addresses, exchanging data, and managing communication protocols. The most common protocol used is TCP, which ensures reliable data transfer. Another protocol often used is UDP, which offers speed advantages by not enforcing delivery guarantees. Python supports both, giving developers the flexibility to choose based on the requirements of each application.

4.1 Basic Networking Concepts

To understand how Python handles network communication, it is necessary to examine the core concepts that shape modern networking. These concepts describe how devices identify each other, how messages travel across networks, and how protocols ensure that data is exchanged reliably.

Networking uses structured rules and addressing systems. These provide predictable behaviour, even when thousands of devices communicate simultaneously. Python builds its networking tools on top of these universally recognised standards. Understanding these ideas helps developers create applications that behave consistently across different environments.

Key concepts include:

- A computer network is a system of devices connected to share information. This can include small local networks or global networks such as the internet.
- Communication is carried out using protocols. A protocol is a set of rules that describes how data must be formatted and transmitted.

- One of the most widely used protocol suites is TCP/IP. It governs how devices communicate across networks.
- TCP, or Transmission Control Protocol, ensures reliable delivery of data. When packets are lost or damaged, TCP resends them. This makes TCP suitable for tasks such as website interactions and file transfers.
- UDP, or User Datagram Protocol, sends messages without confirming delivery. It offers lower delay but does not guarantee that packets reach the destination. It is used in situations requiring speed, such as live audio or gaming.
- An IP address identifies a device on a network. Every device connected to a network must have an address that other devices can use to send messages.
- A port number identifies a specific application or service running on a device. For example, web servers often use port 80 or 443.
- A client is a programme that initiates a request. A server is a programme that responds to requests.
- DNS, or Domain Name System, translates human-readable addresses such as website names into numeric IP addresses.

To establish communication, two devices must agree on the protocol, the address, and the port number. Once these details match, communication can proceed. Networking problems often stem from incorrect addresses, blocked ports, or incompatible protocols. A clear understanding of these concepts helps developers diagnose issues and create stable network applications.

4.2 Networking Modules in the Standard Library

Python includes a range of modules that support network communication. Some are high-level, focusing on convenience, while others offer close control over packet transmission and reception. These modules cover tasks ranging from basic socket programming to secure communication and web requests.

The most important networking modules in the Python standard library include:

- **socket** - This is the foundation of network programming in Python. It allows developers to create network endpoints, establish connections, and manually transmit or receive data. It supports TCP, UDP, and raw sockets. It is suitable for programmes requiring detailed control or custom communication patterns.

- **socketserver** - This module builds on top of socket and offers a simpler way to implement network servers. It provides base classes for TCP and UDP servers, allowing the programmer to focus on request handling rather than connection management.
- **http** - This module contains tools for managing HTTP communication. It supports server-side functionalities such as processing HTTP requests and building simple web servers. It also includes classes for handling HTTP status codes and headers.
- **urllib** - This module provides a simple way to interact with URLs. It can fetch web content, handle query parameters, open connections, and process responses. It is commonly used in scripts that download online data.
- **ssl** - This module supports secure communication through encrypted channels. It allows programmes to establish secure connections using protocols such as HTTPS. It is essential when exchanging confidential information.
- **email** - Though not strictly a networking module, it works closely with networking functionalities. It helps programmes construct and parse email messages, which can then be sent using networking protocols such as SMTP.
- **smtplib** - This module enables sending email through the Simple Mail Transfer Protocol. It provides tools to communicate with email servers, authenticate users, and transmit messages.
- **poplib** and **imaplib** - These modules support receiving email using POP3 and IMAP protocols. They allow applications to connect to mail servers, retrieve messages, and manage inboxes.

These modules allow Python to handle tasks ranging from simple content retrieval to building complete communication systems. The programmer can choose the module that best fits the problem, whether it involves low-level control or convenient high-level abstractions.

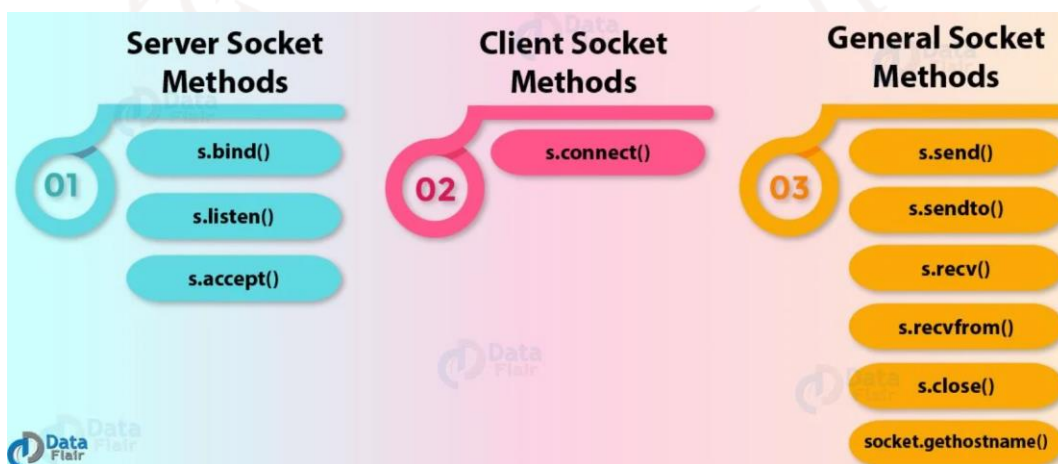


Figure 5: Python Socket Methods

Source: <https://data-flair.training/blogs/wp-content/uploads/sites/2/2018/05/Python-Socket-Methods-01.jpg>

The figure presents the key socket methods in Python, which are essential for network programming and can be grouped according to their function in communication. Server socket methods include `s.bind()`, which assigns an address to the socket, `s.listen()`, allowing the server to accept incoming connections, and `s.accept()`, which establishes a connection with a client. Client socket methods mainly involve `s.connect()`, used to initiate a connection to a server. General socket methods cover data transmission through `s.send()` and `s.recv()`, closing connections with `s.close()`, and utility functions such as `socket.gethostname()`, enabling effective management of network communication.

4.3 Establishing Network Connections

Networking involves more than sending and receiving information. Before any data can move between systems, a connection must be established. Python supports different ways of forming these connections depending on the protocol and application requirements.

The most common method is through sockets. A socket acts as a communication endpoint. A pair of sockets, one on each end of a connection, allows two programmes to exchange data.

The steps to establish a TCP connection are as follows:

1. Create a socket object using the socket module.
2. Choose address family (usually IPv4 or IPv6).
3. Select the socket type, typically a stream socket for TCP.
4. Provide the target server address, either as an IP or a domain name.
5. Specify the port number that the server listens on.
6. Use the connect method to establish the connection.
7. Once connected, the programme can send or receive data.
8. Close the socket when communication is complete.

The steps for working with UDP in Python reflect the fact that UDP is a connectionless protocol. The process begins by creating a socket object using the socket module and selecting the appropriate address family, such as IPv4 or IPv6, along with a datagram socket type. Unlike TCP, there is no need to establish or maintain a persistent connection with the server. Instead, the server's address and

port number are specified each time data is sent. The `sendto` method is used to transmit messages directly to the target address, making the communication quick and efficient. This approach is suitable for scenarios where fast data transfer is more important than guaranteed delivery.

Receiving data in UDP communication is handled using the `recvfrom` method, which reads incoming packets and also provides information about the sender's address. Since UDP does not guarantee that messages will arrive, programmes often include logic to manage missing or out-of-order packets at the application level. Once data exchange is complete, the socket should be closed to release system resources. UDP's simplicity and low overhead make it effective for applications such as live streaming, voice communication, or online gaming, where occasional data loss is acceptable and low latency is a priority.

Python programmes can also act as servers. The steps to create a TCP server include:

1. Create a socket object.
2. Bind the socket to an IP address and a port number.
3. Put the socket into listening mode.
4. Wait for incoming connections.
5. Accept a connection from a client.
6. Exchange data using the new connection socket.
7. Repeat the process for other clients.
8. Close the socket when shutting down the server.

When using `socketserver`, many of these steps become automated. Developers only need to create a request handler class and pass it to the server class. The framework handles binding, listening, and connection management.

Network connections in Python can fail for a variety of reasons, often related to configuration or accessibility issues. One common cause is specifying an incorrect address or port, which prevents the client from locating the intended server. Firewalls can also block requests, either on the client or server side, restricting network traffic and stopping connections from being established. Additionally, if the server is unavailable due to maintenance, crashes, or overload, clients will be unable to connect. These scenarios highlight the importance of verifying network addresses, ports, and server status before attempting communication.

Other factors that can lead to connection failure include network congestion and poor connectivity. Heavy traffic on a network can delay or drop packets, resulting in timeouts or incomplete transmissions. Similarly, unstable or weak internet connections may prevent a socket from maintaining a reliable link. Handling these situations often requires implementing error detection, retries, or alternative connection strategies to ensure robust network communication. Python programmes often include error handling to manage these situations gracefully.

4.4 Sending and Receiving Data

Once a connection is established, the most important task is exchanging data. Python offers several ways to handle this, depending on whether the connection is TCP, UDP, or a higher-level protocol.

For TCP:

- Data is transmitted using send and recv methods.
- The programme must convert text into bytes before sending.
- Recipients decode the bytes back into text.
- TCP ensures complete and ordered delivery.
- Large messages may arrive in segments, so programmes must handle partial reads.

For UDP:

- The sendto method transmits packets to a destination address.
- recvfrom receives packets and includes the sender's address.
- UDP does not guarantee message delivery, so reliability must be handled at the application level.
- Packets may be lost or arrive in the wrong order.

Python's higher-level modules simplify communication by implementing these steps internally. For example, urllib:

- Opens a connection to a server.
- Sends an HTTP request automatically.
- Receives a structured response.
- Provides functions to read the returned content.

Working with web servers often involves dealing with structured formats such as JSON or XML. Python includes modules to parse these formats so that the received data can be converted into usable structures.

Sending binary data, such as images or files, involves reading the file in binary mode and sending the bytes through the socket. Receiving binary data requires similar handling, with buffering used to ensure that large files are received in full.

Programmes must also consider when to close connections. Leaving sockets open may drain system resources. Python's context managers help ensure proper cleanup.

Data transmission in Python often requires additional considerations to ensure secure and reliable communication. One important aspect is authentication, which is necessary when connecting to servers that restrict access, ensuring only authorised users can establish a connection. Encryption, often implemented using SSL, protects sensitive information from interception during transmission, maintaining confidentiality and integrity.

Timeouts are another key factor, preventing programmes from waiting indefinitely when a server fails to respond or a network issue occurs. Similarly, handling slow or unreliable connections is essential to maintain stability and prevent interruptions in service. Addressing these considerations is crucial for developing dependable applications that can manage data securely and effectively, even under varying network conditions or potential threats.

Networking is an essential part of modern applications, and Python provides a solid foundation for handling different network interactions. The standard library includes modules that address low-level communication, high-level web requests, secure connections, email transmission, and server construction.

By understanding basic networking concepts and practising with Python's tools, developers can build applications that communicate reliably across networks. Learning to establish connections, manage sockets, handle data formats, and deal with common networking challenges forms an important part of becoming skilled in Python programming.

Self-Assessment Questions – 3

Multiple Choose Questions	
11	Which protocol provides reliable, ordered delivery of data in network communications? a. TCP b. UDP c. HTTP d. FTP

12	Which Python module provides the low-level socket interface for creating network endpoints? a. socketserver b. ssl c. urllib d. socket
13	Which socket method is used by a client to establish a TCP connection to a server? a. bind() b. sendto() c. recv() d. connect()
14	Which statement best describes UDP compared with TCP? a. UDP guarantees ordered delivery and retransmits lost packets. b. UDP is connectionless and does not guarantee delivery, often used for low-latency tasks. c. UDP is only used for secure connections. d. UDP automatically retries failed transmissions until successful.
15	Which module should you use when you need to add encryption to a network connection in Python? a. http b. ssl c. smtplib d. poplib

INSPIRED BY LIA

5. SUMMARY

- Strings are one of the core data types in Python and are used whenever programs handle text, such as user input, files, reports, or communication between systems.
- A Python string is a sequence of Unicode characters, which allows applications to support multiple languages and symbols without extra configuration.
- Strings in Python are immutable, meaning their content cannot be changed after creation; any modification results in a new string object.
- Python supports different ways to create strings, including single quotes, double quotes, and triple quotes for multi-line text.
- Each character in a string can be accessed using indexing, with positive indexing starting from zero and negative indexing starting from the end.
- Slicing allows specific sections of a string to be extracted efficiently using start, end, and step values.
- Python provides a wide range of built-in string methods for case conversion, searching, testing content, replacing text, and alignment.
- String formatting methods, including percent formatting, the `format()` method, and f-strings, help produce clear and well-structured output.
- Text handling builds on string features and focuses on searching, modifying, splitting, joining, and normalising text data for consistency.
- Python networking uses standard protocols such as TCP and UDP and provides modules like `socket`, `urllib`, and `ssl` to establish connections and exchange data securely and reliably.

6. GLOSSARY

String	-	A sequence of Unicode characters used in Python to represent text..
Unicode	-	A standard that allows text to represent characters from many languages and symbols consistently
Immutability	-	A property of strings where their content cannot be changed once created.
Indexing	-	Accessing individual characters within a string using their position number.
Slicing	-	Extracting a portion of a string using start, stop, and step values.
split()	-	A method that divides a string into a list based on a specified separator.
join()	-	A method that combines a list of strings into a single string using a chosen separator.
f-string	-	A modern string formatting technique that allows expressions inside curly braces.
TCP	-	A networking protocol that ensures reliable and ordered delivery of data.
Socket	-	A communication endpoint used in Python networking to send and receive data between systems.

7. TERMINAL QUESTIONS

1. Explain what string immutability means in Python and describe two practical consequences for how you write code.
2. Compare single-quoted, double-quoted and triple-quoted strings. Give an example use case where a triple-quoted string is the best choice.
3. Show with code how to extract the substring "gram" from the string "Programming" using slicing, and explain each slice parameter used.
4. Describe the difference between `find()` and `index()`, and give an example where using one is preferable to the other.
5. Write a short Python function that normalises user input by trimming whitespace, converting to lower case and collapsing repeated internal spaces to a single space. Explain why each step is useful.
6. When processing CSV data, why might simple `split(',')` be insufficient? Describe a safer approach using the standard library.
7. Explain the difference between `str` and `bytes` in Python and outline a scenario where explicit encoding and decoding is necessary.
8. List the steps needed to establish a TCP client connection with the `socket` module, and sketch minimal example code to connect to `example.com` on port 80 and then close the socket.
9. Compare TCP and UDP in terms of reliability, latency and typical use cases. Give two examples for when you would choose UDP over TCP.
10. Describe how you would fetch JSON from a web API using `urllib`, and show how to convert the response into a Python dictionary.

8. ANSWERS

8.1. Self-Assessment Answers:

1. c) Once created, a string's characters cannot be altered; operations that seem to change a string produce a new string.
2. b) -1
3. a) data[::-1]
4. b) strip()
5. d) They allow expressions inside braces that are evaluated at runtime.
6. b) Once created, a string cannot be altered; operations that appear to change it return a new string.
7. c) Any consecutive whitespace (spaces, tabs, newlines)
8. d) "-".join(["a", "b"])
9. a) index()
10. b) strip()
11. a) TCP
12. d) socket
13. d) connect()
14. b) UDP is connectionless and does not guarantee delivery, often used for low-latency tasks.
15. b) ssl

8.2. Terminal Question Answers

Answer 1: A string's immutability means its contents cannot be changed after it is created; any operation that seems to alter a string actually returns a new string object. This prevents accidental in-place modifications and makes passing strings between functions safer, since callee code cannot alter the caller's string. Practically, this means you reassign a variable when you want an updated value and prefer methods that return new strings rather than attempting index assignment. Refer Section 2.1.

Answer 2: Single-quoted and double-quoted strings are equivalent for simple text and allow convenient inclusion of the other quote type without escapes, while triple-quoted strings span multiple lines and preserve line breaks. Triple quotes are ideal for long explanatory text or documentation strings where line breaks and indentation matter, for example embedding a multi-

line help message or an email template. Use triple quotes when you want readable multi-line literals without explicit newline characters. Refer Section 2.1.

Answer 3: To extract "gram" from "Programming" use slicing:

```
s = "Programming"

substr = s[3:7] # 'gram'
```

Here 3 is the start index (inclusive), 7 is the end index (exclusive) and the omitted step defaults to 1, so characters at positions 3,4,5,6 are returned. This follows the string[start:end:step] rule explained in the document. Refer Section 2.2.

Answer 4: find() returns the lowest index of a substring or -1 if not found, while index() returns the index but raises an exception when the substring is absent. Use find() when you want to test for presence without handling exceptions, and use index() when a missing substring should be treated as an error that stops normal flow. Both are useful depending on whether you prefer error handling or conditional checks. Refer Section 2.3.

Answer 5: A short normalising function is:

```
def normalise(text):

    return ' '.join(text.strip().lower().split())
```

This trims leading and trailing whitespace with strip(), converts to lower case for consistent comparisons, and split() plus join() collapses runs of internal whitespace to a single space. These steps ensure stored or compared values are uniform and reduce mismatches caused by stray spaces or capitalisation. Refer Section 3.4.

Answer 6: Using split(',') is often insufficient for CSV because fields can contain commas inside quoted values or embedded newlines, which a simple split will mis-handle. The safer approach is to use the csv module from the standard library, which honours quoting rules, escape sequences and different delimiters, returning parsed rows as lists or dictionaries. That module handles edge cases reliably and should be preferred for any non-trivial CSV data. Refer Section 3.3.

Answer 7: str represents text (Unicode characters) while bytes represent raw binary data; they are distinct types and cannot be mixed directly. Encoding converts str to bytes (for example using UTF-8) before sending over a network or writing binary files, while decoding converts received bytes back into str for processing. A common scenario is sending an HTTP request or socket message: you encode the string to bytes before send and decode bytes after receive. Refer Section 2.1 and 4.4.

Answer 8: Steps for a TCP client are: create a socket, choose address family and socket type, resolve the server address and port, call `connect()`, exchange data with `send()/recv()`, and close the socket. Minimal example:

```
import socket

s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

s.connect(('example.com', 80))

s.close()
```

Include error handling and timeouts in real code to manage failures and resource cleanup. Refer Section 4.3.

Answer 9: TCP provides reliable, ordered delivery with retransmission, so it is suitable where correctness matters but introduces extra latency; UDP is connectionless and does not guarantee delivery, so it offers lower latency and is used when speed or simplicity is prioritised. Choose UDP for real-time audio/video streaming or multiplayer gaming where occasional packet loss is acceptable and latency is critical. For precise guarantees or file transfers prefer TCP; for low-delay, tolerant streams prefer UDP. Refer Section 4.1 and 4.4.

Answer 10: Use `urllib` to open the URL, read and decode the response, then parse JSON with the `json` module. Example:

```
from urllib import request

import json

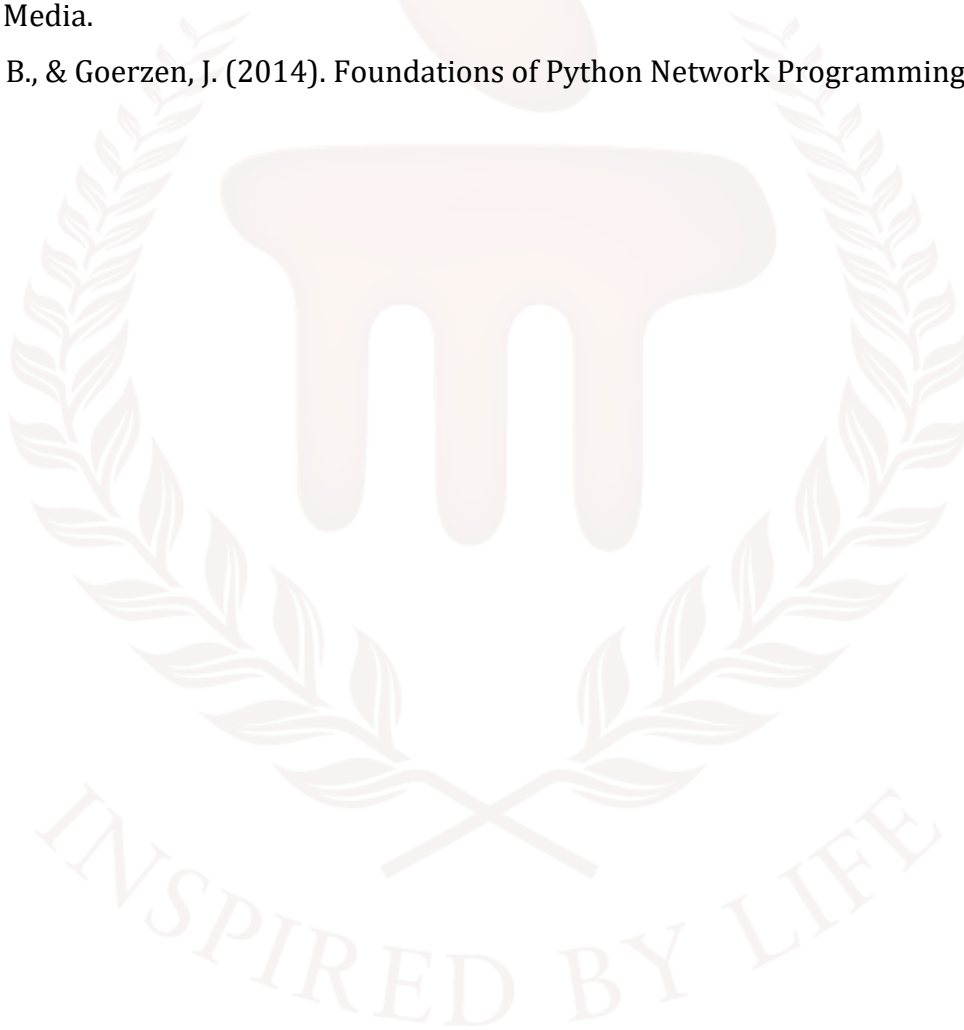
with request.urlopen('https://api.example.com/data') as r:
    body = r.read().decode()

data = json.loads(body)
```

This opens a connection, reads bytes, decodes to text and converts the JSON text into a Python dictionary for further processing. Refer Section 4.2 and 4.4.

9. REFERENCES

- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- Ramalho, L. (2015). Fluent Python: Clear, concise, and effective programming. O'Reilly Media.
- Sweigart, A. (2019). Automate the Boring Stuff with Python (2nd ed.). No Starch Press.
- Beazley, D. M., & Jones, B. K. (2013). Python Cookbook: Recipes for mastering Python 3 (3rd ed.). O'Reilly Media.
- Rhodes, B., & Goerzen, J. (2014). Foundations of Python Network Programming (3rd ed.). Apress.



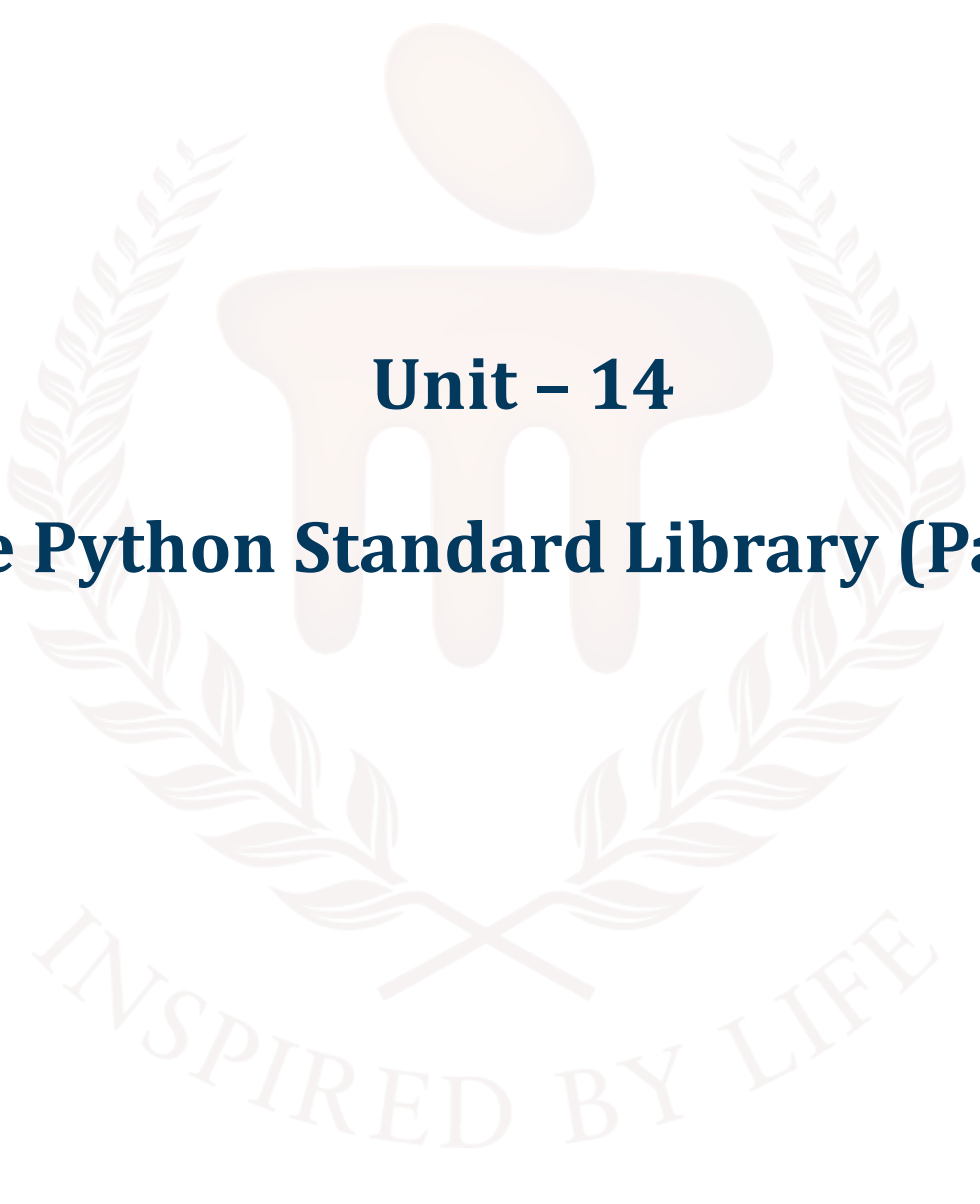
BACHELOR OF COMPUTER APPLICATIONS

SEMESTER 4



DCA2205

PYTHON PROGRAMMING



Unit – 14

The Python Standard Library (Part 2)

TABLE OF CONTENTS

Sl No	Topic	Fig No / Table / Graph	Saq / Activity	Page No
1	Introduction	-	-	4
1.1	Learning Objectives	-	-	
2	Database Access in Python	-	1	5-14
2.1	Overview of Database Connectivity	-	-	
2.2	Python Database API	1	-	
2.3	Creating Database Connections	2	-	
2.4	Executing SQL Queries	-	-	
3	Working with Database Records	-	2	15-25
3.1	Inserting Data into Tables	-	-	
3.2	Retrieving Records	-	-	
3.3	Updating and Deleting Records	-	-	
3.4	Using Cursors	3	-	
3.5	Example: Using sqlite3 for Basic Database Operations	-		
4	Error Handling and Best Practices	-	3	26-33
4.1	Handling Database Exceptions	4	-	
4.2	Transaction Management	-	-	
4.3	Securing Database Access	-	-	
4.4	Performance Considerations	-	-	
5	Summary	-	-	34
6	Glossary	-	-	35
7	Terminal Questions	-	-	36
8	Answers	-	-	37-39
8.1	Self-Assessment Answers	-	-	
8.2	Terminal Questions Answers	-	-	
9	References	-	-	40

1. INTRODUCTION

The previous unit introduced key components of the Python Standard Library related to strings, text handling, and networking. You learnt how to create and manipulate strings, process textual data, and perform basic networking tasks such as establishing connections and transferring data. These features highlighted Python's ability to handle communication and data representation efficiently using built-in libraries.

This unit focuses on database access using the Python Standard Library. It introduces the concept of connecting Python applications to databases and explains how the standard database modules support interaction with relational databases. The unit explains how Python acts as a bridge between applications and stored data.

The unit also covers performing common database operations such as inserting, retrieving, updating, and deleting records. It explains the use of database connections and cursors, along with basic error handling and transaction control. These topics are important for building reliable applications that work with persistent data.

To study this unit effectively, practise setting up simple database connections and executing queries using Python programs. Work through examples that involve real data operations and observe how changes are reflected in the database. Regular hands-on practice will help in gaining confidence with database-driven applications.

1.1. Learning Objectives

By the end of this Unit, you will be able to:

- Apply Python database modules to establish database connections
- Execute SQL queries using Python programs
- Manage database records through insert, update, and delete operations
- Implement transaction control in database applications
- Handle database errors within Python programs



2. DATABASE ACCESS IN PYTHON

Databases play a central role in software systems because they store, organise, and protect data used by applications. Python provides several built-in and standardised tools that allow programs to interact with both relational and non-relational databases. The focus in this unit is on relational databases, where information is arranged in tables consisting of rows and columns. Python's approach to database access is based on defined standards that make it possible for developers to write portable code without changing the structure of their applications when switching between different database systems.

2.1 Overview of Database Connectivity

Python interacts with databases by using modules that follow a standard pattern known as the Python Database API. This pattern ensures that regardless of the chosen database, from small systems such as SQLite to large enterprise databases such as PostgreSQL or MySQL, the code used to send commands and retrieve results stays generally consistent. Understanding how Python communicates with a database helps develop robust programs that can store, retrieve, filter, and update information.

A database system usually contains four main elements:

- A database server, which stores the actual data.
- A database driver, which allows Python to communicate with the database.
- A connection object, which opens a link between Python and the database.
- A cursor object, which is used to send commands and receive results.

These elements work together to allow applications to run SQL operations such as creating tables, inserting records, updating information, deleting unwanted entries, or selecting information based on certain properties.

When learning database access in Python, it is also important to understand the types of databases you may interact with:

- Embedded databases, such as SQLite. These do not require a separate server and store data in a single file on disk.
- Client-server databases, such as PostgreSQL, MySQL, MariaDB, Oracle, and Microsoft SQL Server. These systems require a running server process that handles requests.

- Cloud-hosted databases, where the server runs on remote infrastructure and Python connects through a network. Examples include Amazon RDS or Google Cloud SQL.

Python can access all these varieties as long as an appropriate module or driver is available.

Database communication in Python is built on several central ideas that shape how programs interact with stored data. SQL acts as the language that defines table structures and carries out operations, whether inserting new information or retrieving records. Before any command can be issued, a connection must be opened to serve as the entry point into the database. Within this connection, a cursor functions as the channel through which commands are passed and results are gathered.

When updates involve several related actions, databases rely on transactions to ensure that changes occur safely and consistently, preventing partial updates if something goes wrong. Since errors can arise from issues such as incorrect SQL statements or interruptions in connectivity, careful handling of exceptions is essential. One of the strengths of Python's approach is the consistent pattern it provides: create a connection, run commands through a cursor, process results, and close the connection when finished.

2.2 Python Database API

The Python Database API, often referred to as DB-API, is a specification that describes how Python modules should interact with relational databases. This standard ensures that the same general approach applies regardless of the specific database that is being accessed.

The DB-API provides a consistent set of objects, methods, and behaviours. Each database module that follows this specification provides several features:

- A connect function that creates a connection object.
- A connection object that can produce cursor objects and manage transactions.
- A cursor object that handles SQL execution and fetches results.
- Standard exceptions that help identify and handle database errors.
- Type conversions for converting Python data types to database types and back.

The DB-API is formalised in a document known as PEP 249. This document outlines the expectations for database modules so that developers can produce software that works across different systems with minimal change.

The main components of the DB-API are described below.

Connection Object - The connection object represents the active link between the Python program and the database. Its responsibilities include:

- Holding authentication details such as username and password.
- Managing the communication channel with the database server.
- Providing methods for beginning, committing, and rolling back transactions.
- Producing cursor objects when required.

Cursor Object - A cursor object represents a temporary workspace where SQL commands can be issued. It allows the following operations:

- Executing SQL statements.
- Retrieving results from select queries.
- Managing iteration over records.
- Holding information about the most recent command.

Cursors make it easy to separate the communication process from result processing. This simplifies code organisation and makes data retrieval more structured.

Parameter Binding - The DB-API encourages parameterised queries. Instead of placing values directly in SQL strings, placeholders are used. This approach:

- Improves safety by preventing unwanted SQL behaviour.
- Makes code easier to maintain.
- Allows the database engine to optimise repeated queries.

Standard Exceptions - The API defines several exception classes such as:

- DatabaseError
- OperationalError
- IntegrityError
- DataError

These help identify whether the issue arises from connection problems, invalid SQL statements, or issues with the underlying database.

Type Conversions - Many modules automatically convert Python types such as strings, integers, dates, and floating point numbers into the appropriate database types. This allows programmers to focus on logic rather than formatting values manually.

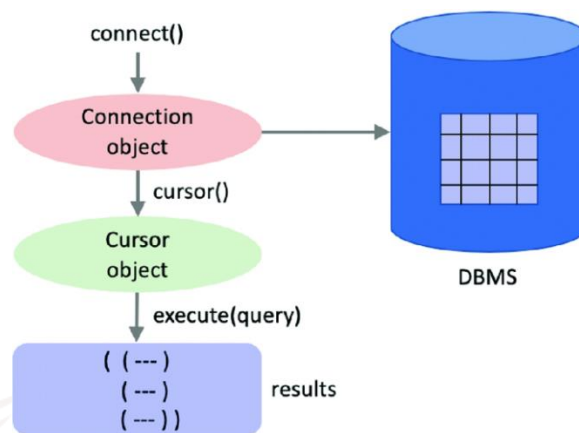


Figure 1: Python DB-API

Source: https://link.springer.com/unit/10.1007/978-3-030-25943-3_24

This figure, illustrating the typical Python DB API interaction, outlines the standardised procedure for communicating with a Database Management System (DBMS). Initially, the application establishes a connection by calling the `connect()` method, which successfully returns a `Connection` object representing the active link to the database. This object is subsequently utilised to create a `Cursor` object via the `cursor()` method; the cursor acts as a control structure that manages the context of a fetch operation and facilitates the execution of database commands. Finally, the cursor's `execute(query)` method sends the specified SQL query to the DBMS, which processes the request and returns the resulting data set, designated here as 'results'. The entire sequence represents the essential three-stage workflow for performing database operations in Python.

The Python Database API helps maintain a consistent structure while allowing flexibility. Whether you use SQLite for a small project or PostgreSQL for a large corporate system, the core workflow behaves predictably, which simplifies both development and learning.

2.3 Creating Database Connections

Creating a connection is the first step in any database operation. Once a connection is established, the program can create cursors, perform transactions, and execute SQL statements. Python uses a similar connection pattern across different modules, though the details vary depending on the database.

General Steps in Creating a Connection

1. Import the database module.
2. Call the connect function with the required parameters.
3. Store the returned connection object.

4. Use the connection object to create a cursor.
5. Begin working with SQL commands.
6. Close the connection when the program finishes.

Connection parameters differ depending on the database engine being used, but most systems rely on a similar set of essential details. A program typically needs the hostname or server address to locate the database, along with a valid username and password to authenticate the connection. The specific database name must also be provided so that the system knows which dataset to access. Many servers operate on defined port numbers, which must be included as part of the connection setup. Some environments also allow optional settings such as timeouts or SSL configurations, which help control performance and security.

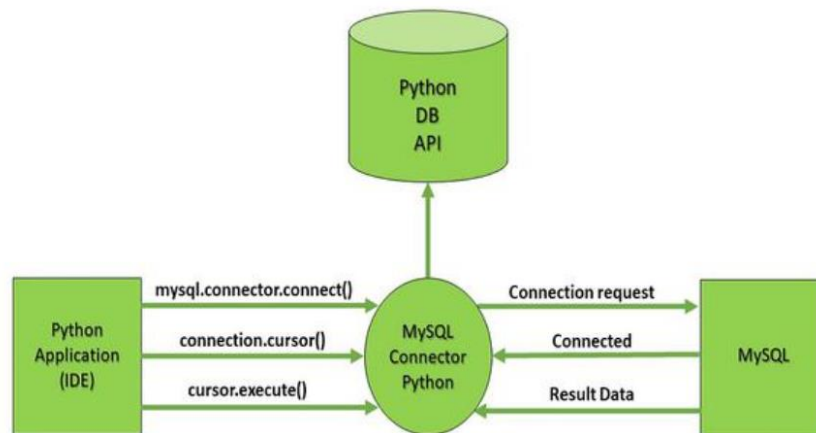


Figure 2: SQL Connection with Python

Source: <https://media.geeksforgeeks.org/wp-content/uploads/20210907134817/SQLConnectionpython-660x432.jpg>

The figure illustrates the client-server architecture for establishing a MySQL connection using a Python application. The Python Application acts as the client, utilising the `mysql.connector.connect()` method to initiate a Connection request to the MySQL Connector Python, which handles the low-level communication. Upon successful establishment, a Connected message is returned from the MySQL database server. The application then creates a cursor object using `connection.cursor()`, which is essential for executing SQL queries via `cursor.execute()`. This command is translated by the Connector and sent to the MySQL server, which processes the request and returns the Result Data. The overall process is often managed through a Python DB API layer, ensuring a standardised interface for database operations.

SQLite Connections

SQLite is unique because it does not require a separate server. The connection is created by specifying a file path.

1. Import the `sqlite3` module.
2. Use `sqlite3.connect`.
3. Provide the path to the database file.

If the file does not exist, SQLite creates it automatically. This makes SQLite convenient for learning or small applications that do not require a full server.

PostgreSQL and MySQL Connections

These systems require a running server and use modules such as `psycopg2` or `mysql.connector`. Typical parameters include host, user, password, and database.

When creating connections, it is important to consider several factors that commonly affect reliability. Connection failures often arise from incorrect authentication details, which can prevent the system from validating user credentials. In some cases, servers may block requests coming from unfamiliar machines, especially when strict security rules or IP whitelisting are in place. Network instability can also disrupt long-running operations, causing timeouts or dropped sessions. Taking these aspects into account helps ensure smoother and more predictable connection behaviour.

Connection Lifecycle

Once a connection is established:

1. Cursors are created from the connection.
2. SQL operations are performed through the cursor.
3. Changes are saved using `commit`.
4. Errors may require a `rollback`.
5. When the work is complete, the connection must be closed.

Proper connection handling avoids resource problems and prevents unfinished transactions from affecting the database.

Transaction Management

Relational databases operate through the use of transactions, which treat a sequence of instructions as one complete unit of work. In Python, a transaction usually begins automatically as soon as database commands are issued, allowing all subsequent operations to be grouped together. When the work is complete, a commit is used to save the changes permanently, ensuring that the data reflects the intended modifications. If any issues arise, a rollback cancels all actions performed since the last commit, restoring the database to a stable state.

This form of transaction control is crucial because it protects the integrity of the data. Without it, unexpected disruptions such as connection loss, invalid SQL commands, or server failures could leave the database in an incomplete or inconsistent condition. By ensuring that updates occur in whole or not at all, transactions help maintain reliable operation and prevent errors from propagating through the system.

Error Handling

Connections should always be wrapped in error-handling code. Otherwise, unexpected issues can leave the database in an inconsistent state. Typical problems include:

- Wrong database credentials.
- Database server not running.
- Incorrect SQL syntax.
- Network timeouts.

Using try and except blocks ensures that connections are properly closed even when errors occur.

2.4 Executing SQL Queries

Once a connection and cursor are ready, SQL queries can be executed. Python's DB-API provides a structured way to send SQL commands and fetch results. Understanding how SQL execution works is essential for working effectively with relational databases.

Types of Queries

SQL commands generally fall into four categories:

- Data definition queries, which create or modify tables.
- Data manipulation queries, which insert, update, or delete records.
- Data retrieval queries, which extract information.
- Transaction control statements such as commit and rollback.

Executing SQL Statements

To execute a query:

1. Create a cursor object.
2. Call the execute method on the cursor.
3. Optionally pass parameters in place of direct values.
4. For select queries, use fetch methods to retrieve results.
5. For update queries, commit changes to make them permanent.

Fetching Results

When retrieving data, the cursor provides several methods:

1. `fetchone()` retrieves a single row.
2. `fetchmany()` retrieves a specified number of rows.
3. `fetchall()` retrieves all remaining rows.

These options allow the program to manage memory efficiently when working with large datasets.

Parameterised Queries

Parameterised queries replace direct values in SQL statements with placeholders, which keeps commands safe and avoids unexpected behaviour. In practice, `sqlite3` uses question marks to stand in for values, while modules such as `psycopg2` use `%s` for the same purpose. By ensuring that data is handled through parameters rather than being inserted directly into the SQL text, these queries prevent issues caused by special characters and reduce the risk of mistakes or malicious input.

Transactions and SQL Execution

After performing operations that modify data, `commit` must be called. Failing to commit:

- Leaves the changes in an unconfirmed state.
- May cause the database to discard the work when the connection closes.
- Can create inconsistencies if other programs expect updated information.

Error Handling During Query Execution

SQL processing can fail for various reasons, including situations where a table does not exist or the SQL statement sent to the database contains mistakes. Problems also arise when the data being inserted or updated conflicts with expected types, or when integrity rules such as unique or foreign

key constraints are broken. These issues prevent the database from carrying out the requested operation.

When something goes wrong, Python raises exceptions that describe the nature of the failure. These exceptions give the programmer clear information about where the issue occurred, making it easier to correct the SQL statement, fix the data being passed in, or adjust the database structure.

When processing a large amount of data, it is often best to retrieve records in batches. `fetchmany()` provides a controlled method of reading results in manageable sections. This approach avoids excessive memory usage.

Processing Results

Result processing may involve:

- Looping through rows.
- Converting data types.
- Building data structures.
- Passing results to other parts of the program.

This separation of concerns keeps code organised and readable.

Some databases allow stored procedures. Python can call them using specific cursor methods, though their availability varies depending on the database engine. Stored procedures are used when complex operations need to be encapsulated on the server side.

Closing Cursors

Once work is complete:

1. Close the cursor.
2. Close the connection afterwards.

This ensures that resources are returned and prevents unwanted locking in the database.

Database access in Python is organised around a well-defined model known as the Python Database API. This structure ensures that programs maintain consistency whether they use SQLite for small applications or client-server systems such as PostgreSQL for larger projects. Understanding how connections, cursors, SQL commands, transactions, and error handling operate helps develop reliable software that manages data safely.

Self-Assessment Questions – 1

Multiple Choose Questions	
1	Which element in Python database access handles the low-level communication between the application and the database? - Database driver - Cursor object - Connection object - Commit method
2	What is the main benefit of using parameterised queries (parameter binding) in the DB-API? - Faster network transfers - Reduced risk of SQL injection and safer handling of values - Automatic transaction rollback - Enabling stored procedures
3	Which cursor method retrieves a specified number of rows from the result set? - fetchone() - fetchall() - fetchmany() - fetchrows()
4	How is a connection to an SQLite database typically created in Python? - Using host and port with <code>mysql.connector.connect</code> - Through a network URL to a server process - By calling <code>psycopg2.connect</code> with database name - By calling <code>sqlite3.connect</code> with the database file path
5	After performing insert, update or delete operations, which action makes those changes permanent in the database? - Closing the cursor - Calling <code>fetchall</code> - Rolling back the connection - Calling <code>commit</code> on the connection

3. WORKING WITH DATABASE RECORDS

Working with database records is a major part of most applications that store or manage information. Once a connection to the database is established, the next task is interacting with the tables that hold real data. This includes inserting new entries, retrieving existing values, making updates to correct or revise information, and removing records that are no longer required. Python provides a structured way to perform these actions using its database modules, most of which follow the guidelines set out in the Python Database API (DB-API) standard.

3.1 Inserting Data into Tables

Insertion is the first and most fundamental action when working with records. It involves adding new rows to a table so that fresh information can be stored and made available for future retrieval. Python's database modules follow a predictable pattern for insertion, allowing programmers to reuse the same approach across different database systems.

To understand insertion properly, it helps to begin with the underlying principles.

Key Principles of Data Insertion

- Databases expect values to match the structure of the table, which means the number of columns, the order of fields, the type of each value, and any constraints must be respected.
- Insert operations should use parameterised queries rather than direct string formatting. This prevents accidental errors and protects against harmful input.
- Every insert operation occurs inside a transaction. Unless the change is committed, the new record does not become permanent.
- Database constraints such as uniqueness rules, primary keys, and foreign keys may either allow or block an insertion, depending on whether the incoming values meet the defined requirements.
- Insert statements must be executed through a cursor, which acts as the link between the Python program and the database engine.

Typical Steps in the Insert Process

The insertion workflow in Python generally follows a sequence similar to the one below:

1. The program establishes a connection to the target database.
2. A cursor object is created from the connection.
3. A parameterised INSERT statement is prepared, containing placeholders for the values.

4. A set of values is provided, usually in a tuple or dictionary.
5. The cursor executes the insertion command.
6. The connection commits the transaction, making the change permanent.
7. The cursor and connection are closed when all tasks are complete.

Although each database module has its own pattern for placeholders, the steps remain consistent enough that a programmer who understands one module can easily adapt to another.

Practical considerations during data insertion revolve around ensuring that every value entering the database meets the rules set by the table structure and the database engine. The data types, lengths, and any constraints such as uniqueness must be respected so that records are accepted without errors. When working with date and time information, the format should match what the database recognises, otherwise the insertion will fail. In cases where large volumes of data are added, grouping insert operations often improves overall performance by reducing repeated processing overhead. Programmes should always check conditions that might cause conflicts, such as existing records that violate uniqueness constraints. It is also important to wrap insert operations in suitable exception-handling code so that problems like missing tables or invalid inputs can be handled without disrupting the entire application.

3.2 Retrieving Records

Retrieval refers to the process of reading information from a database. It is one of the most frequently performed tasks in Python database programming. Retrieval allows programs to display results to users, process stored information, run reports, and analyse data stored over long periods.

Retrieving data involves sending SQL `SELECT` statements to the database. Depending on the requirement, a `SELECT` query might request all records in a table or only those matching certain conditions.

Key Principles of Retrieval

- Retrieval always begins with a `SELECT` statement, which may include clauses such as `WHERE`, `ORDER BY`, and `LIMIT`.
- Python cursors carry the `SELECT` command to the database engine and receive the resulting rows.
- Results may be fetched one row at a time or collected as a group using methods such as `fetchone()`, `fetchmany()`, or `fetchall()`.
- Retrieval does not alter the database, so no commit is required.

- Retrieved values are typically returned in tuples, though some modules offer dictionary-style access.

Steps in Retrieving Data

1. A cursor is created from an active database connection.
2. A SELECT statement is prepared, often using parameters if filtering is involved.
3. The cursor executes the SELECT statement.
4. The program requests rows using `fetchone()`, `fetchmany()`, or `fetchall()`.
5. The returned data is processed according to the program's needs.
6. The cursor is eventually closed.

Programmers often need to choose between different fetch methods depending on the amount of data being retrieved.

Three Main Retrieval Patterns

- **Single-Record Retrieval** - This approach is used when only one record is expected, such as when searching by a primary key.
- **Multi-Record Retrieval** - This method is used when many rows match the condition. The program may display them, analyse them, or use them to produce reports.
- **Conditional Retrieval** - This involves using WHERE clauses to restrict results, which prevents unnecessary data transfer and improves performance.

Retrieval errors occur when a SELECT command cannot be carried out as expected, often due to issues in the structure or execution of the query. A common cause is the use of a table name that does not exist, which prevents the database engine from locating the required data. Errors also arise when column names are entered incorrectly, since the database cannot match them to the table structure. Problems in the WHERE clause may occur when values of the wrong type are used, causing the comparison to fail. At times, filters may be so restrictive that no records meet the conditions, resulting in an empty output rather than an actual error. Retrieval may also fail if the database connection expires during the operation, leaving the query without a valid channel to complete the request. Careful planning and error handling help avoid problems, especially in long-running applications.

3.3 Updating and Deleting Records

Updating and deleting records allow a program to modify or remove information that already exists in the database. These operations must be handled with care because they alter the stored data.

Mistakes made during updates or deletions can have long-lasting consequences, especially if transactions are not used properly.

Updating records follows principles similar to insertion, but the focus is on modifying rows that already exist rather than creating new entries. An UPDATE statement typically includes a WHERE clause so that only the intended records are changed, preventing accidental modification of the entire table. Parameterised statements are recommended because they reduce the risk of errors and ensure that values are handled safely, even when they contain special characters. Since updates occur within transactions, a commit is required for the changes to become permanent. Programmes should also be written carefully to prevent unintended updates to large portions of a table, especially when conditions are missing or too broad.

Typical Update Steps

1. Create a cursor from the active database connection.
2. Write a parameterised UPDATE statement containing the new values.
3. Provide the values to replace placeholders.
4. Include a WHERE clause to specify the target rows.
5. Execute the UPDATE statement.
6. Commit the transaction.
7. Close the cursor when finished.

Deleting records removes rows that are no longer required within the system, and it is considered a sensitive operation because once the deletion is committed, recovery is difficult unless the system has a suitable backup process. A DELETE statement should always contain a WHERE clause so that only the intended rows are removed. Since deletions occur within a transaction, a commit is needed for the changes to take effect. Parameterised statements help ensure that the deletion conditions are applied correctly and safely. When a row is linked to other tables through foreign keys, the deletion may fail unless suitable cascading rules exist. Exception handling is essential so that the programme can respond properly to issues such as constraint violations or incorrect deletion conditions.

Steps in Deleting Records

1. Create a cursor.
2. Prepare a DELETE statement with placeholders.
3. Supply the filter values.
4. Execute the DELETE statement.

5. Commit the transaction.
6. Close the cursor.

Practical concerns arise when carrying out updates and deletions, mainly because these operations alter existing data and can affect the integrity of the system. The WHERE clause must be checked thoroughly to ensure that only the intended rows are targeted. Updates should be written with awareness of constraints such as unique values or foreign-key relationships so that the database does not reject the changes. Deletions often depend on cascading rules when related records must be removed at the same time; without these rules, the operation may fail. Using transactions allows mistakes to be reversed before they become permanent, making the process safer. Logging is also useful because it provides a record of changes applied to important tables, helping administrators track system behaviour and identify problems.

3.4 Using Cursors

Cursors form the bridge between Python and the database engine. Every operation—whether inserting, retrieving, updating, or deleting—runs through a cursor. Understanding how cursors work is essential for writing efficient and safe database programs.

A cursor performs several essential roles in the interaction between a Python program and a database. It sends SQL commands from the application to the database engine and receives results produced by SELECT queries. During these operations, it maintains internal state so that the program can move through the data in an orderly manner. It also manages parameter binding, which ensures that values are passed safely and correctly into SQL statements. A cursor supports iteration through result sets as well, allowing the program to process rows one at a time or in groups, depending on what the task requires.

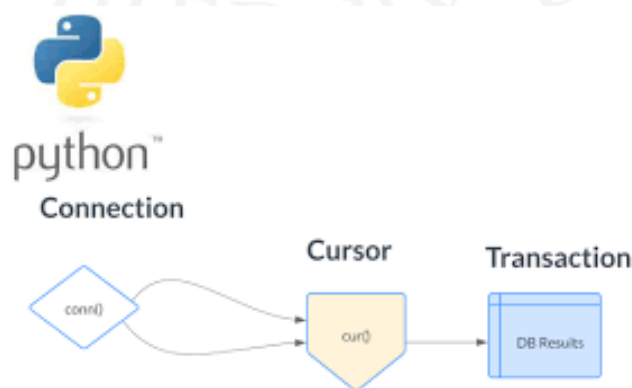


Figure 3: Cursor

Source: <https://dataengineeringcentral.substack.com/p/a-treatise-on-database-connections>

The figure illustrates the essential components and workflow for managing a database connection in a Python application, specifically referencing the steps for interacting with a MySQL database. The process begins with establishing a connection object using a function like `conn()` or, in the MySQL example, `mysql.connector.connect()`, which handles the initial request and receives a confirmation from the database server. Once connected, a cursor object is created via a method such as `cur()` or `connection.cursor()`, acting as a control structure to execute SQL queries. The cursor's `execute()` method passes the query to the underlying DB API and connector, which then retrieves the DB Results (or Transaction) from the database server, completing the standard transactional workflow.

Types of Cursor Operations

- Executing SQL statements.
- Fetching command results.
- Binding values for parameterised queries.
- Managing multi-row operations.
- Reporting metadata such as column names.

Steps When Using a Cursor

1. Acquire a cursor through the active connection.
2. Prepare and execute an SQL command.
3. Fetch results if required.
4. Commit changes if the operation modifies data.
5. Release resources by closing the cursor.

Cursors operate within the boundaries of a transaction, meaning every command they execute becomes part of the current transactional block until a commit or rollback is issued. This approach ensures that all changes made through the cursor remain temporary and reversible until the programmer decides to make them permanent. It also protects the database from accidental partial updates, since any failure allows the system to roll back the entire set of operations. By linking cursor activity to transactional control, Python maintains consistency, prevents data corruption, and supports reliable multi-step operations where several commands must succeed together.

Common Cursor Pitfalls

- Forgetting to close cursors may lead to resource leaks.

- Using string formatting instead of placeholders may create unsafe SQL.
- Fetching large datasets without using limits can slow down the program.
- Running long-running queries without setting timeouts may freeze applications.
- Mismanaging transactions may cause unexpected results.

Working with database records forms the core of data-driven Python applications. Whether inserting new values, retrieving existing ones, modifying stored details, or removing outdated entries, each operation relies on clear patterns provided by the Python Database API. Cursors act as the primary tool for communication, while parameterised statements protect the system from errors and unsafe input. Proper transaction handling ensures that changes are made safely and consistently. As developers gain familiarity with these operations, they can build robust applications that treat stored data with care and precision.

3.5 Example: Using sqlite3 for Basic Database Operations

The sqlite3 module in Python provides a simple way to work with relational databases without requiring a separate database server. SQLite stores the database in a single file, making it suitable for learning, testing, and small applications. The following example demonstrates how basic database operations such as creating a table, inserting records, selecting records, updating records, and deleting records can be carried out using Python and sqlite3.

Example:

```
import sqlite3

# Step 1: Create or connect to the database

conn = sqlite3.connect("student.db")
cursor = conn.cursor()

# Step 2: Create a table
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER,
    course TEXT
```

```
)  
"""
```

Step 3: Insert records

```
cursor.execute("INSERT INTO students (name, age, course) VALUES (?, ?, ?)",  
              ("Asha", 20, "Python"))  
cursor.execute("INSERT INTO students (name, age, course) VALUES (?, ?, ?)",  
              ("Ravi", 21, "Data Science"))  
conn.commit()
```

Step 4: Select records

```
print("Records after insertion:")  
cursor.execute("SELECT * FROM students")  
for row in cursor.fetchall():  
    print(row)
```

Step 5: Update a record

```
cursor.execute("UPDATE students SET course = ? WHERE name = ?",  
              ("Advanced Python", "Asha"))  
conn.commit()
```

```
print("\nRecords after update:")  
cursor.execute("SELECT * FROM students")  
for row in cursor.fetchall():  
    print(row)
```

Step 6: Delete a record

```
cursor.execute("DELETE FROM students WHERE name = ?",  
              ("Ravi",))  
conn.commit()
```

```
print("\nRecords after deletion:")
```

```
cursor.execute("SELECT * FROM students")
for row in cursor.fetchall():
    print(row)
```

Step 7: Close the connection

```
cursor.close()
conn.close()
```

The program begins by importing the `sqlite3` module and creating a connection to a database file named *student.db*. If the file does not already exist, SQLite creates it automatically. A cursor is then created to execute SQL commands.

A table named *students* is created using the `CREATE TABLE IF NOT EXISTS` statement. This ensures that the table is created only if it is not already present in the database.

Next, two student records are added using the `INSERT INTO` statement. The values are passed using placeholders (`?, ?, ?`) to ensure safe parameter handling. After insertion, `commit()` is called so that the changes are permanently stored.

The `SELECT * FROM students` query is then used to retrieve all rows from the table. The `fetchall()` method returns all matching records, which are printed one by one.

The `UPDATE` statement changes the course value of the student named Asha. Again, `commit()` is required to save the modification.

The `DELETE` statement removes the record of the student named Ravi from the table. After committing, another `SELECT` query shows the remaining records.

Finally, the cursor and connection are closed to release resources properly.

Basic `sqlite3` Commands Used in the Example

- `sqlite3.connect()` – Opens or creates an SQLite database
- `cursor()` – Creates a cursor object for executing SQL
- `CREATE TABLE` – Creates a new table
- `INSERT INTO` – Adds new rows into the table
- `SELECT` – Retrieves data from the table
- `UPDATE` – Modifies existing data

- DELETE – Removes rows from the table
- commit() – Saves changes permanently
- close() – Closes the cursor and database connection

Basic SQLite Shell Commands

When working directly in the SQLite command-line environment, some commonly used basic commands are:

.tables

.schema students

SELECT * FROM students;

.exit

Where,

- .tables displays all tables in the database
- .schema students shows the structure of the students table
- SELECT * FROM students; retrieves all rows
- .exit closes the SQLite shell

Self-Assessment Questions – 2

Multiple Choose Questions	
6	Which sequence best describes the typical steps to insert a new row into a relational database using Python DB-API? - Create cursor, open connection, execute INSERT, commit, close connection - Open connection, create cursor, execute INSERT with parameters, commit, close cursor and connection - Open connection, execute INSERT directly on connection, close connection, commit - Create cursor, execute INSERT, fetchall(), commit, close cursor
7	What is the main advantage of using parameterised queries when inserting or updating records? - They automatically compress data to save space - They reduce the risk of SQL injection and ensure values are handled safely - They force the database to always use cached plans

	d. They skip transactional control so operations run faster
8	Which cursor method should you use when you need just a single row from a SELECT query? a. fetchmany() b. fetchone() c. fetchall() d. fetchlimit()
9	When performing an UPDATE or DELETE, which precaution is most important to avoid changing or removing unintended rows? a. Using ORDER BY in the statement b. Ensuring the statement includes an appropriate WHERE clause c. Calling fetchall() before executing the statement d. Using print statements to display affected rows
10	Which of the following is a common pitfall when working with cursors in Python database code? a. Closing the cursor too soon after a commit b. Using parameter placeholders instead of string formatting c. Forgetting to close cursors, which can cause resource leaks d. Always using fetchone() for large result sets

4. ERROR HANDLING AND BEST PRACTICES

Working with databases through Python requires careful handling of exceptions, transactions, security precautions, and performance concerns. Although databases are designed to be reliable and stable, many things can go wrong during normal use. A program might attempt to run an incorrect SQL statement, a connection might fail midway through an operation, or a data value might not meet the rules expected by the table. For these reasons, robust error handling is not something to add at the end of the project; it must be an essential part of the database design and code structure.

4.1 Handling Database Exceptions

Database operations may fail for many reasons, ranging from simple mistakes in SQL syntax to larger system faults. Python database modules raise exceptions to indicate that something has gone wrong. Understanding how to catch and respond to these errors is crucial for building resilient applications.

Why Database Exceptions Occur

Database errors often arise due to predictable causes. Some relate to the structure of the database, while others involve user input or network behaviour. Common reasons include:

- A table or column referenced in the SQL command does not exist.
- The SQL statement contains mistakes such as missing commas or incorrect keywords.
- A data value does not match the required type, for example inserting text where a number is expected.
- Constraint rules are broken, such as inserting a repeated value into a unique field.
- The connection to the server is lost during a transaction.
- Permissions are insufficient for the requested operation.

Each situation requires a different response, and Python provides useful exception classes in line with the guidelines of the Database API. By catching these exceptions, the program can either correct the issue or fail gracefully instead of crashing.

Approaches to Handling Exceptions

Python's try-except structure makes it easy to manage errors. A good design ensures that no database operation is executed without proper error handling. A typical structure includes the following steps:

1. The operation is placed inside a try block.
2. The specific exception classes are listed in except blocks.

3. Appropriate messages are presented when errors occur.
4. Rollback is issued if the failure happened during a modifying operation.
5. The system logs the error for later review.

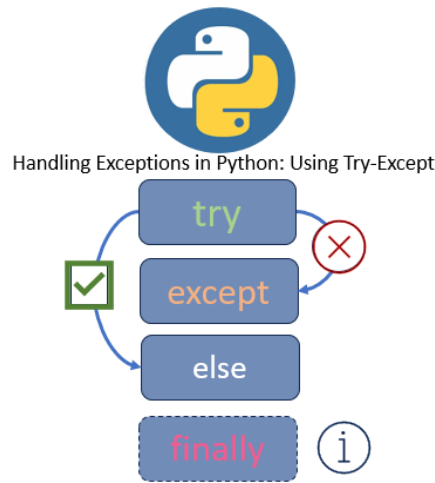


Figure 4: Handling Exceptions in Python: Using Try-Except-Finally

Source:

https://miro.medium.com/v2/resize:fit:1100/format:webp/1*jJPb4sOG90LKmhY_yiB00g.png

The figure illustrates the flow control for exception handling in Python using the try-except-else-finally block structure. The code that might raise an error is placed within the try block. If an exception occurs during the execution of the try block (indicated by the red 'X'), the execution is immediately passed to the except block, which contains the code for handling the error. However, if the try block executes successfully without any exceptions (indicated by the green checkmark), the code in the optional else block is executed. Crucially, the code in the finally block is *always* executed regardless of whether an exception occurred or not, making it suitable for necessary clean-up actions like closing files or database connections. This mechanism is fundamental for creating robust and reliable Python programmes.

This approach ensures that the program can respond intelligently rather than simply stopping execution.

Suitable Exception Categories

Most Python database modules support the exception types suggested by the DB-API. These categories make it easier to group related error types:

1. **InterfaceError** for problems with the database interface rather than the database itself.

2. **DatabaseError** for any error received from the database engine.
3. **OperationalError** for issues with connections or transactions.
4. **IntegrityError** for violations of constraint rules.
5. **ProgrammingError** for incorrect SQL, wrong parameters, or misuse of the API.
6. **DataError** for problems with data values, such as type mismatches.

A well-written program catches these exceptions and responds in a way that protects the reliability of the stored data.

A common pattern for managing errors during database operations involves placing the sensitive commands inside a controlled structure. The process usually begins with a try block where the SQL statement is executed through a cursor. If something goes wrong, the programme catches the specific exception raised by the database engine and records or displays a clear explanation of the problem. When the failure concerns an operation that alters data, a rollback is used to undo any changes that were attempted. Once the issue has been handled, the rest of the programme can continue without causing further disruption or leaving the database in an inconsistent state.

4.2 Transaction Management

Transactions form the heart of reliable database operations. A transaction is a controlled group of actions that either all succeed together or fail together. This prevents partial updates from damaging the integrity of the data.

Key Principles of Transactions

- A transaction groups several related commands into a single logical unit.
- Commit confirms all actions within the transaction.
- Rollback cancels all uncommitted changes.
- Many Python modules start a transaction automatically when the first SQL command is executed.
- Each connection manages its own transactions.

Why Transactions Matter

Without transactions, an application could leave the database in an inconsistent state. Imagine an operation that inserts information into one table and then updates a related table. If the second action

fails, the first should not remain in place. Transactions ensure that the database always returns to a safe state when something unexpected happens.

Types of Operations That Require Transactions

Not all SQL commands need transaction management. SELECT queries do not change data, so they do not require commits. However, the following commands must always be treated as part of a transaction:

- INSERT, which adds new rows.
- UPDATE, which modifies existing rows.
- DELETE, which removes data.
- CREATE TABLE, DROP TABLE, and ALTER TABLE.

Since all of these actions change the organisation or contents of the database, they must be grouped into safe units.

Transaction Errors and Their Effects

Problems can occur during a transaction:

- A network failure interrupts communication.
- A constraint is broken by one of the commands.
- The SQL statement contains an error.
- The server becomes unavailable.

If any of these situations arise, the program must call rollback to undo changes. Failure to do so can leave the database in a problematic state.

Good transaction management relies on keeping modifying operations inside a controlled structure so that the database remains safe and consistent. Each group of related changes should be placed within a try-except block, allowing the programme to commit only when every step has succeeded without errors. If any exception occurs, a rollback should be issued immediately to undo partial updates and prevent inconsistent states. Transactions should also remain short so that they do not block other users or delay system activity, and developers should avoid running several long-lasting transactions at the same time. This approach keeps the system responsive while ensuring that updates are carried out safely.

4.3 Securing Database Access

Security is a critical part of database programming. A database often contains personal records, financial information, or other sensitive details. Without proper precautions, this data may be exposed to misuse. Python applications must therefore use safe methods when connecting, authenticating, and working with SQL commands.

Principles of Database Security

Security begins with an understanding of the main dangers. These include unauthorised access, accidental leaks, harmful commands, and insecure communication. The following principles guide safe development:

- Never store plain text passwords inside programs.
- Always use parameterised queries instead of combining text manually.
- Restrict permissions so that the user account used by the program cannot perform unnecessary actions.
- Apply encryption for connections that travel across networks.
- Avoid sharing the same database credentials across different services.

Dangers of Unsafe SQL

The most common threat arises from constructing SQL statements by joining strings. When user input is added directly to the SQL command, there is a risk of harmful content causing unwanted behaviour. Parameterised queries remove this risk because the values are not mixed with the SQL text.

Managing Database Credentials

Proper storage of credentials is essential. Some practical guidelines include:

- Store passwords in environment variables, not in source code.
- Use configuration files with restricted access rights.
- Avoid using accounts with full privileges.
- Change credentials regularly as part of routine maintenance.
- Never display database passwords in logs or error messages.

When a database is accessed across a network, the communication should be protected with encryption so that no one can intercept or read the information being transferred. Secure links usually rely on SSL or TLS, which ensure that data travels through an encrypted channel rather than plain text. Certificates play an important role by confirming that the server is genuine and not an imitation. Some database modules also support encrypted tunnels that provide another layer of protection for

sensitive workloads. Many servers can be set to refuse any unencrypted requests, ensuring that all connections follow strict security requirements.

Access Control and Permissions

A database user account used by the application should have only the permissions required for its tasks. Best practices include:

- Create a separate user account for each application.
- Allow only the required commands such as SELECT, INSERT, UPDATE, DELETE.
- Avoid giving structural permissions like DROP TABLE unless absolutely necessary.
- Track login attempts using server logs.

4.4 Performance Considerations

Performance is a major concern when applications grow, data increases, or many users access the database at the same time. Poorly designed queries can slow a system down, create bottlenecks, or overload the server. Python programs must use efficient methods when working with records, especially in environments where speed matters.

Factors That Affect Performance

Database performance depends on several aspects, such as the structure of the tables, the way queries are written, and the behaviour of network connections. Key influences include:

- Poorly designed SQL statements.
- Missing indexes for columns used in filters.
- Large result sets fetched at once.
- Long-running transactions that lock tables.
- Repeated opening and closing of connections.
- Inefficient loop structures inside Python code.

Understanding these influences helps programmers identify the right improvements.

Efficient query writing has a strong influence on how quickly a Python programme can retrieve or modify information in a database. Selecting only the required columns instead of using a broad `SELECT *` reduces the amount of data transferred and processed. Well-designed WHERE clauses help narrow results so that the database engine can focus on a smaller, more relevant set of rows. Searches run more smoothly when the columns involved have proper indexes, as this allows the engine to locate records without scanning entire tables. Complex joins should be kept to a minimum, since they

increase processing time and may lead to unnecessary overhead. When only part of a dataset is needed, using a LIMIT clause keeps workloads small and helps the programme run more efficiently.

Efficient Handling of Large Data

When retrieving many rows, performance suffers if the entire dataset is fetched in one step. Better methods include:

- Using `fetchmany()` to retrieve a fixed number of rows at a time.
- Processing results in batches before requesting more.
- Using server-side cursors in modules that support them.
- Reducing the number of round-trips between application and database.

Managing database connections is important because each connection consumes resources on both the client and the server. Programmes should avoid opening and closing connections inside loops, since this creates unnecessary overhead and slows down the system. When a large number of operations must be performed, connection pools offer a practical way to reuse existing connections rather than creating new ones for every task. It is also important to release each connection as soon as the work is finished so that other parts of the system can use it. Unused cursors should be closed as well, as they hold resources that the database engine must maintain until they are released.

Reducing Transaction Overheads

Long transactions keep locks in place, blocking other users. Improving performance involves:

- Keeping modifying operations short.
- Performing large updates in controlled batches.
- Using commits at suitable points.
- Avoiding unnecessary locks.

Error handling and best practices are essential elements of professional database programming in Python. Exception management ensures that failure does not bring down an application. Transaction control provides a safety net that protects stored data from accidental damage. Security precautions safeguard information from misuse or exposure, while performance optimisations ensure that the system remains responsive even under demanding conditions. When these principles are applied consistently, a Python application becomes reliable, safe, and capable of supporting long-term growth.

Self-Assessment Questions – 3

Multiple Choose Questions

11	Which exception type is most appropriate for catching violations of database constraints, such as a duplicate key error? a. InterfaceError b. IntegrityError c. OperationalError d. ProgrammingError
12	What is the main reason to call rollback inside an except block when a modifying operation fails? a. To make the pending changes permanent on the server b. To undo all uncommitted changes and restore the database to a safe state c. To log the error and continue without altering the transaction state d. To close the cursor and free resources immediately
13	Which practice is recommended for keeping database credentials secure in a Python application? a. Storing passwords in source code for easy access b. Saving credentials in plain text configuration files shared across services c. Placing passwords in environment variables and restricting file access d. Writing credentials into application logs for troubleshooting
14	When an application must retrieve a very large result set, which technique helps reduce memory usage? a. Using fetchall() and processing every row in memory b. Selecting all columns with SELECT * to avoid extra queries c. Using fetchmany() to process rows in manageable batches d. Opening and closing a new connection for each row
15	Which practice helps prevent long-running transactions from blocking other users and degrading performance? a. Keeping transactions short and committing at appropriate points b. Performing all updates inside a single, long transaction to ensure consistency c. Avoiding indexes so updates run faster d. Fetching complete tables into memory before making changes

5. SUMMARY

- Python accesses relational databases through modules that follow the DB-API standard, allowing similar code to work with SQLite, PostgreSQL, MySQL and other engines.
- A typical Python database workflow uses a connection object to open a link, a cursor to send SQL and fetch results, then commits or rolls back transactions before closing resources.
- Connections require parameters such as host, user, password, database name and sometimes port or SSL options; SQLite instead opens a file path and needs no server.
- The DB-API defines common objects, methods and exceptions (for example connect, cursor, execute, commit and DatabaseError) so code remains portable across adapters.
- Parameterised queries should be used in place of manual string interpolation to prevent SQL injection and to allow safe handling of special characters and data types.
- Insert, update and delete operations occur inside transactions; commit makes changes permanent while rollback undoes uncommitted changes if an error occurs.
- Retrieval is performed with SELECT statements and the cursor's fetch methods (fetchone, fetchmany, fetchall); choosing the right fetch method helps control memory use.
- Cursors maintain state for executing statements, binding parameters, iterating results and reporting metadata; they must be closed to free server-side resources.
- Robust programmes wrap database operations in try-except (and often finally) blocks, catching DB-API exceptions and issuing rollbacks where required to keep data consistent.
- Security and performance best practices include storing credentials outside source code, using least-privilege accounts, encrypting network connections, indexing relevant columns and keeping transactions short.

6. GLOSSARY

DB-API	-	A Python specification (PEP 249) that defines a common interface for database modules so client code can remain largely unchanged across different database engines.
Connection object	-	An object returned by a module's connect function that represents an active link to the database and manages transactions and cursor creation.
Cursor	-	A temporary workspace produced by a connection used to execute SQL statements, bind parameters, fetch results and iterate over rows.
Parameterised query	-	An SQL statement that uses placeholders for values, letting the database adapter safely substitute parameters to avoid injection and formatting errors.
Transaction	-	A logical group of database operations that either all succeed when committed or all fail when rolled back, ensuring atomicity and consistency.
Commit	-	The operation that makes all changes in the current transaction permanent and visible to other database sessions.
Rollback	-	The operation that reverts all uncommitted changes in the current transaction, restoring the database to its prior state after an error.
Integrity constraint	-	A rule defined by the schema (for example primary key, unique or foreign key) that enforces correctness of stored data and may block invalid insertions or updates.
Fetch methods	-	Cursor methods for retrieving query results: fetchone returns a single row, fetchmany returns a set number of rows, and fetchall returns all remaining rows.
Connection pooling	-	A technique for reusing a set of open connections across multiple operations or threads so that the overhead of repeatedly opening and closing connections is reduced.

7. TERMINAL QUESTIONS

1. Explain the role of the DB-API in making Python database code portable across different database engines.
2. Describe the lifecycle of a database connection in Python, from opening to closing, and explain why each step matters.
3. Outline the steps required to insert multiple rows efficiently into a table and explain why batching helps.
4. Compare fetchone, fetchmany and fetchall in terms of memory usage and typical use cases.
5. Explain how parameterised queries prevent SQL injection and give an example of placeholder syntax for SQLite and PostgreSQL.
6. Describe a safe pattern for performing an update that affects multiple tables and explain how transactions are used to preserve consistency.
7. Discuss common cursor-related mistakes programmers make and how to avoid them.
8. List at least four categories of DB-API exceptions and explain what types of errors each category signals.
9. Explain three practical measures a Python application should use to secure database credentials and connections when deployed to production.
10. Identify three query- or schema-related factors that most frequently cause poor database performance and suggest one remedy for each.

8. ANSWERS

8.1. Self-Assessment Answers:

1. Database driver
2. Reduced risk of SQL injection and safer handling of values
3. fetchmany
4. By calling sqlite3.connect with the database file path
5. Calling commit on the connection
6. b) Open connection, create cursor, execute INSERT with parameters, commit, close cursor and connection
7. b) They reduce the risk of SQL injection and ensure values are handled safely
8. b) fetchone
9. b) Ensuring the statement includes an appropriate WHERE clause
10. c) Forgetting to close cursors, which can cause resource leaks
11. b) IntegrityError
12. b) To undo all uncommitted changes and restore the database to a safe state
13. c) Placing passwords in environment variables and restricting file access
14. c) Using fetchmany to process rows in manageable batches
15. Keeping transactions short and committing at appropriate points

8.2. Terminal Question Answers

Answer 1: The DB-API defines a standard interface (PEP 249) that database modules implement so client code can create connections, issue SQL through cursors and handle results in the same way across engines; this standardisation is what makes Python database code portable. By exposing common functions such as connect(), cursor(), execute() and shared exception classes, adapters for SQLite, PostgreSQL or MySQL behave predictably and allow the same application logic to work with little or no change. Refer: Section 2.2 Python Database API.

Answer 2: A connection lifecycle begins with importing the adapter and calling connect() to obtain a connection object, then creating cursors from that connection to execute SQL, committing or rolling back as needed, and finally closing cursors and the connection to free resources. Each step matters because authentication and network parameters are established at connect time, cursors provide the execution context, commits make changes permanent, rollbacks avoid partial updates on error, and

closing prevents resource leaks. Refer: Section 2.3 Creating Database Connections (Connection lifecycle).

Answer3: Efficient multi-row insertion is done by preparing a parameterised INSERT, supplying groups of value tuples (batched rather than one row at a time), executing these through the cursor and committing once the batch succeeds. Batching reduces repeated parsing and network overhead, lowers transaction costs and therefore improves throughput when inserting large volumes. Refer: Section 3.1 Inserting Data into Tables.

Answer 4: fetchone returns a single row and uses minimal memory, fetchmany returns a fixed block of rows so you can process results in controlled batches, and fetchall loads all remaining rows into memory which can exhaust resources for large sets. Use fetchone for single-record queries, fetchmany or server-side cursors for large result sets, and reserve fetchall for small queries where simplicity matters. Refer: Section 3.2 Retrieving Records.

Answer 5: Parameterised queries keep data values separate from SQL text so user input is passed as parameters rather than being concatenated into statements; this prevents attackers from injecting SQL fragments and avoids errors with special characters. In practice the DB-API uses placeholders (SQLite commonly uses ? while adapters like pycpg2 use %s) and the adapter safely substitutes values into those placeholders. Refer: Sections 2.2 and 2.4 (parameter binding and executing queries).

Answer 6: For multi-table updates, wrap all related UPDATE statements inside a single transaction and use a try-except block: acquire a cursor, execute each parameterised update, and commit only if every step succeeds; on any exception call rollback to undo partial work. This pattern preserves consistency because the transaction ensures either all modifications apply together or none do, avoiding orphaned or inconsistent rows across tables. Refer: Section 3.3 Updating and Deleting Records and Section 4.2 Transaction Management.

Answer 7: Common cursor mistakes include forgetting to close cursors (causing resource leaks), using string formatting instead of placeholders (creating SQL injection risks), fetching huge result sets with fetchall, and running long queries without timeouts; all of these can harm stability or security. Avoid them by always closing cursors, using parameterised statements, fetching in batches or using server-side cursors, and keeping queries and transactions short. Refer: Section 3.4 Using Cursors.

Answer 8: Typical DB-API exception categories include InterfaceError (adapter/interface problems), DatabaseError (errors reported by the DB engine), OperationalError (connection or transaction issues), IntegrityError (constraint violations), ProgrammingError (bad SQL or API misuse) and

`DataError` (invalid data types/values). Catching and handling these specific classes lets a programme respond appropriately (for example rollback on `OperationalError` or report `IntegrityError` to the user). Refer: Section 4.1 Handling Database Exceptions and Section 2.2 (standard exceptions).

Answer 9: Practical measures are: store credentials outside source code (for example in environment variables or protected configuration files), use least-privilege accounts so the application has only needed permissions, and encrypt network connections with SSL/TLS so credentials and data are not sent in plain text. Additional steps include rotating credentials periodically and never logging secrets. Refer: Section 4.3 Securing Database Access (access control and credential management).

Answer 10: Three common causes of poor performance are poorly written SQL (remedy: rewrite queries to select only needed columns and simplify joins), missing indexes on filter or join columns (remedy: add appropriate indexes), and fetching large result sets at once (remedy: use `fetchmany`, `LIMIT` clauses or server-side cursors and process in batches). Also avoid long transactions and repeated open/close of connections; use short transactions and connection pooling where appropriate. Refer: Section 4.4 Performance Considerations.

9. REFERENCES

- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.
- Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.
- Beazley, D., & Jones, B. (2013). Python Cookbook (3rd ed.). O'Reilly Media.
- Python Software Foundation. (1999). PEP 249 — Python Database API Specification v2.0.
- Python Software Foundation. (n.d.). sqlite3 — DB-API 2.0 interface for SQLite databases (Python Documentation).

